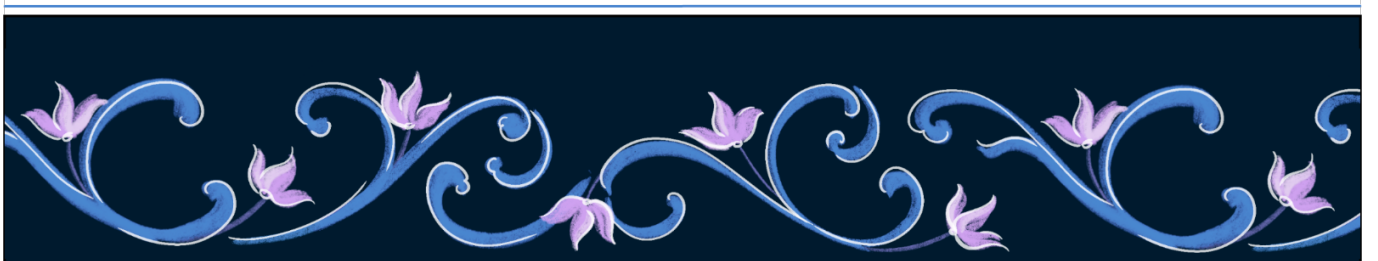
Maximum Efficiency with Minimum Learning Curve

LazyVim

For Ambitious Developers



Dusty Phillips

LazyVim for Ambitious Developers

Dusty Phillips

LazyVim for Ambitious Developers

Book cover by Jennifer Phillips
Proofread by Jennifer Phillips

Interior Fonts are Quicksand, Lora, and Source Code Pro.
Cover Fonts are Lora and Academy Engraved LET.
Screenshot Fonts in the screenshots are VictorMono Nerd Font.
Screenshots were taken with the Catpuccin-mocha colour scheme enabled.

First Edition 2024

http://lazyvim-ambitious-devs.phillips.codes/

ISBN: 978-1-0689808-2-4

Table of Contents

Chapter 1. Introduction and Installation

This book is meant for all the developers out there who are intrigued by the power modal editing can bring, but are either intimidated or bored by the sheer amount of configuration that even introductory tutorials on the topic force upon them.

I'm guessing you're a Visual Studio Code user (a great editor that I used for a few years) but you may be coming to modal editing from numerous other integrated development environments and code editors. You've probably heard of Vim and now you want to try it out.

1.1. Why Vim

Vim has a *very* ancient history. It was created in the early '90s as an improvement on the (much) older Vi editor, written in the 1970's. The name "Vim" actually stands for "Vi, improved". Its predecessor, Vi is short for "Visual", as it was an iteration on an even earlier (1971) non-visual line editing experience called `ed`.

Interestingly, `ed` is somehow still available on modern Unix systems, and if you use a MacOS or Linux environment, you can probably access it by typing `ed` in any terminal. (If you made the mistake of trying it, `Control-D` will get you out again). You now know as much as I do about `ed`, and trust me, you don't want to know more.

You may also be familiar with another iteration of `ed` called `sed`, the stream editor. To this day it is still used for modifying text in a shell pipeline.

The `ed` command was also **ex**tended to create another line editor called `ex`, which isn't really used anymore, except (extensively) as a submode of Vim. In fact, if you install Neovim and type `ex` on your command line, you will get a very crippled instance of Neovim that only supports `ex` commands.

So that's quite the family tree, but ancestry alone is not a great predictor of quality, as any fourth generation somebody can attest.

There are many reasons to use any one editor, but, the ones that stand out for Vim are:

Health Benefits

This is the big one for me. I used Vim a lot through my early career, though, like many developers, I switched to VS Code when it came out in 2015. I spend a lot of time at my keyboard, and by 2020, I was so crippled by RSI that I spent six months exclusively coding by voice (a blog article on the topic has made me more famous than I expected). A friend suggested I switch back to modal editing, and it made a huge difference for me. The vast majority of Vim keystrokes do not require holding multiple keys with the same hand, something that really aggravates carpal tunnel syndrome.

Performance

Most IDEs have some sort of "vi emulation" layer or plugin that allows you to reap some of the health benefits of modal editing without fully switching to a new editor. But waiting for VS Code to start up and load all the extensions you know and love has become a meditative exercise for many (or an opportunity to return to doom scrolling). In contrast, when I load my LazyVim configuration, it helpfully tells me how long it took to load: 56.98 milliseconds. To my slow, human eyes, it's instant.

Developer Velocity

This is debatable and subjective. Any tool is only as good as the trades-person who wields it, and I certainly know excellent coders who can really fly with their VS Code, Emacs, or JetBrains configurations. Still, I really believe that a finely-tuned Vim configuration can outpace any of them.

Open Ecosystem

I have a lot of respect for how Microsoft treats open source software since Nadella took the reigns, but VS Code is showing increasing signs of proprietary lock-in (especially with its AI integrations). In contrast, the Neovim ecosystem is a vibrant open community, and innovative plugins arrive every week that would never be attempted in VS Code.

1.2. Why Neovim, Specifically

If you decide to use Vim, you'll quickly discover that there are two modern variations: Vim, and Neovim. Both are direct descendants of that original open source '90s Vim code. Neovim was forked from Vim in 2014 with a vision of refactoring and modernizing the codebase and feature set.

Both are actively maintained and both have the potential to feel polished and modern, though it takes some configuration to reach that potential. They each have

extremely strong plugin ecosystems, but Neovim has one killer feature: It has introduced the Lua programming language for plugin development (and configuration) in addition to the native VimScript that its sibling uses.

By itself, Lua doesn't make Neovim inherently better than Vim. However, where most Vim plugins also run on Neovim, the inverse is not always true, and there are best-in-class Lua plugins that only work with Neovim.

For the purposes of this book, your decision is already made for you: The LazyVim distribution this book is about only works with Neovim.

1.3. Introducing LazyVim

The chief drawback of both Vim and Neovim is that while they have the capability to have the same (or better) modern editing experience as any other IDE, they don't start out that way. Vim has a long tradition of maintaining compatibility with the outdated `vi` tool and Neovim has only marginally diverged from that tradition. When you first install Neovim, you get a pretty bland code editing experience.

When I switched back to Vim in 2020, after becoming used to the great functionality VS Code brought to the editor ecosystem, it took me **two weeks** to get my configuration where I wanted it and I tweaked it incessantly for months after. It came in at about 300 lines of Vimscript, which I later ported to around 250 lines of Lua code. To be fair, I'm a heavy customizer in general, and my VS Code configuration was actually longer than either of those! But I'll happily admit that the VS Code out-of-the-box experience was much better.

I looked at several so-called Neovim "distributions" (preconfigured setups). I won't go into details of the comparison, but LazyVim is the clear winner by far, mostly because it balances a dead simple out-of-the-box experience with relatively easy customization and configuration.

To be clear, LazyVim is Vim. The editing experience is identical. It's not a new iteration or version of Vim in the way that Neovim is. Instead, LazyVim is a mindset; it starts with an agreement on what constitutes the best plugin configurations for modern development, and a configuration that makes them work well together (keeping different plugins' keybindings from conflicting with each other is one of the major painpoints when managing editor configurations manually).

But if you go into using LazyVim with the mindset that this experience will be better than any other editing experience you've worked with before, and accept that you will be retraining some keyboard muscle memory, you will find that it is extremely

well thought out. In my experience, it is *exactly* what a 2020s-era modal editing experience should be.

1.4. Choosing a Terminal

If you've made it this far, you probably think you're ready to install Neovim. Actually, you still have one decision to make.

Neovim can run in a lot of different contexts (you can even run it inside Visual Studio Code!) By default it is a terminal program, but there are also tons of GUIs available. I have tried almost all of them, and, honestly, I don't think they have any inherent advantage over running Neovim directly in the terminal.

We will briefly discuss hooking up Neovim to a few GUIs much later in the book, but for starting out, I recommend running Neovim in a terminal. A very good terminal, to be specific.

To get the best Vim editing experience, you want a GPU accelerated terminal. What's that mean? Basically that you will be using the chip designed to render photo-realistic video for rendering source code. Makes as much sense as using it for artificial intelligence, right?

You will need to do your own research on the following options, so ask your favourite search engine or the AI Chat bot de jour to help you decide:

Kitty Terminal **[https://sw.kovidgoyal.net/kitty/]**

is my personal preference. I find it well-documented, easy to configure, and fully featured.

Alacritty **[https://alacritty.org]**

is probably the winner for raw speed, but configuration is awkward, and it is less featureful.

Wezterm **[https://wezfurlong.org/wezterm/index.html]**

has some very nifty features, but I found the documentation to be lacking and had trouble getting some aspects to work.

Windows Terminal **[https://github.com/microsoft/terminal?tab=readme-ov-file]**

if you are a Windows user, does claim to be GPU accelerated, though I found that Neovim was occasionally unresponsive in it.

Warp Terminal [https://www.warp.dev]

> If you're already on the Warp train and you just can't live without it, Neovim *will* work inside it. I found the experience a little choppy, and I didn't enjoy the look and feel of Warp (or the fact that I needed to sign in to use it).

So install one or more of those until you find one that you like. You *can* use other terminal emulators if you want to. You probably won't even notice that the experience is inferior, but I can promise that if you later switch to a GPU-accelerated experience, you'll notice the *improvement*.

1.5. Setting Up Your Terminal Font

LazyVim and its plugins look beautiful in a terminal, and you would almost not believe that they are not a GUI application. To do this, they depend on special fonts that have a TON of glyphs for coding related things. Most notably, this gives you access to icons representing the type of file you have open, but also provide nice frames and window-like behaviour in the terminal.

To get the best LazyVim experience, you will need to install one of these special fonts and configure your terminal to use it. Indeed, you should really be using one of these fonts in your terminal even if you aren't a heavy Neovim user. A lot of modern terminal apps (there's a phrase, eh?) look better if you have them installed.

Visit Nerd Fonts [https://nerdfonts.com] for more information. If you use Kitty, you simply need to install the `NerdFontsSymbolsOnly` font, and it will automatically pull symbols from that font if it can't find them in your configured monospace font. For other terminals, you will likely need to install and configure a "patched" font as discussed on the Nerd Fonts site. There are many to choose from.

You can choose from many of the most popular programming fonts. Downloading and installing them is very much operating system dependent, so I'll leave the guide on the Nerd Fonts website to explain it to you.

1.6. Install Neovim

Neovim works pretty much anywhere software can be installed, and it only relies on standard system dependencies. The chief problem is that no matter which operating system you use, you are spoiled for choice!

You can visit the Neovim home page [https://neovim.io/] and click the "Install Now" button to get the latest instructions for your operating system of choice (or of

necessity) from the Neovim developers.

1.6.1. Which Version Should I Install?

Neovim development happens at a super fast pace compared to their release cycle, so it is not uncommon for folks to run the latest nightly build. I have only rarely encountered bugs in builds cut from the master branch on Github, so it's generally safe. I usually run off the latest stable release when it comes out, and then when some new plugin update says "here's a cool feature if you use Neovim nightly," I'll install the latest Neovim build instead.

I suggest starting with the latest stable version of Neovim for now, which at time of writing is 0.10.0. Avoid older instances if possible. LazyVim does tend to add features from the nightly release, so if you start to get as excited about this distro as I am, it will be a perfectly natural progression to switch to the pre-release.

1.6.2. Windows

On Windows, I recommend using the Windows Subsystem for Linux (WSL) and doing all development in there. WSL is way outside the scope of this book, but it is well-documented by Microsoft and many online tutorials. Once you have chosen a WSL-compatible Linux distribution, set it up, and have it running in your chosen terminal, you can install Neovim using the Linux instructions below.

If you have a reason—or preference—to develop on native Windows, the easiest thing to do is grab the MSI installer from the Releases section of the neovim/neovim [https://github.com/neovim/neovim/] repository on GitHub.

If you already use Winget, Chocolatey, or Scoop to manage packages on your Windows machine, there is a Neovim package in each of them.

Note that if you use Windows without WSL, you will also need to install a C compiler in order to get treesitter support (this basically means better syntax highlighting and code navigation support). This is, sadly, not a trivial task. It is documented in the nvim-treesitter/nvim-treesitter [https://github.com/nvim-treesitter/nvim-treesitter] GitHub repo, so I won't go into detail here.

1.6.3. MacOS

Most developers on MacOS use Homebrew so if you don't have it already, I recommend installing it by following the instructions at brew.sh [https://brew.sh/].

Once you have brew up and running, the command `brew install neovim` will

install Neovim.

If you want to live on the edge, `brew install --HEAD neovim` will install the latest nightly version of Neovim, which is probably, but not guaranteed to be, stable.

I find the brew experience to be much kinder than other MacOS installation options for Neovim, so if you aren't already a Homebrew user, I strongly recommend setting it up. There are other open source tools that you will want to install as we get deeper into the LazyVim journey, and brew will be the easiest way to get them all.

If you don't want to use Homebrew, things are a bit more annoying. The Neovim dev team doesn't maintain a MacOS installer, so you'll have to download a tarball and extract it, then link to the binary from somewhere on your system path. If you don't know what any of that means, honestly, use Homebrew, it's easier!

1.6.4. Linux

If Neovim isn't available using your distribution's default package manager, you have a very strange Linux distribution, indeed!

So just run `sudo pacman -S neovim`, `sudo apt install neovim`, `sudo dnf install neovim`, or the appropriate command for whichever more esoteric package manager you prefer.

Some distros ship with painfully outdated Neovim versions, though. If you want a nightly version, you may find the instructions on the `neovim/neovim` GitHub Releases page, or will have to dig into your distro's documentation.

You will also need to install a C compiler in the unlikely event that your Linux distribution didn't come with one. For most distros, just install the `gcc` package and you should be good to go.

1.7. Try Neovim Raw (If You Dare)

Once you have Neovim installed, you can try it out by simply typing `nvim` (or `nvim <filename>` to open a specific file) into the terminal you installed a few sections ago. If Neovim is installed correctly and on your path, you'll get a visually unappealing editor that looks like it was forked from something written in the 90s and had the express intent of looking like it was written in the 70s.

So, at least it's honest?

Unfortunately, you're now trapped. To save you the frantic "how do I exit Vim" Google search, the command to quit is `Escape` followed by the three characters `:q!` followed by `Enter`. The whole sequence will therefore be `<Escape>` `<Colon>` `q` `<Exclamation>` `<Enter>`.

Seriously, "How do I exit Vim" is one of the top three autocompletes on Google for "How do I exit...". Apparently only a Samsung TV plus and full screen mode on MacOS are less intuitive to get out of! We'll understand why this incantation works after we dig deeper into the vim mental model and command mode.

If you want to, you can run the command `<Escape>:Tutor<Enter>` to open an interactive text file that you can read through and edit while learning the basics of Neovim. I do recommend doing this at some point, but now may not be the right time. A lot of things that are "normal" in the Vim tutor are different (better!) using LazyVim. The rest of this book does **not** assume you have gone through the tutor.

1.8. Install LazyVim

Now that you have Neovim up and running, let's get it configured to look like it was developed this century.

Installing LazyVim requires a bit of work with `git`. Since you are reading this book, I am assuming you are a software developer and therefore somewhat familiar with `git`.

The git commands to install LazyVim are more or less the same for the various operating systems, though paths and environment variables are slightly different.

1.8.1. Start With a Clean Slate

If you already have a Neovim config and you want to try LazyVim without losing your existing configuration, set the `NVIM_APPNAME=lazyvim` environment variable. Skip to the `Clone the starter template` section below, and clone into `~/.config/lazyvim` instead of `~/.config/nvim`. If you want to make the changes permanent, either set `NVIM_APPNAME` in your shell's startup file or rename that `lazyvim` config folder to `nvim`.

First, remove or back up all existing Neovim state. This step is largely optional if you've never used Neovim before, but I recommend making sure the following directories have been removed or moved:

Clean up: Windows with WSL, MacOS, and Linux

Remove or back up (with `mv` instead of `rm`) the following directories:

```
$ rm -rf ~/.config/nvim
$ rm -rf ~/.local/share/nvim
$ rm -rf ~/.local/state/nvim
$ rm -rf ~/.cache/nvim
```

Listing 1. Unix Cleanup Commands

Clean Up: Windows Without WSL

The location of the config and data folders is a little bit different on Windows, but the idea is the same as for the Unix systems. Just use Powershell commands instead of the Unix core-tools:

```
$ Move-Item $env:LOCALAPPDATA\nvim $env:LOCALAPPDATA\nvim.bak
$ Move-Item $env:LOCALAPPDATA\nvim-data $env:LOCALAPPDATA
\nvim-data.bak
```

Listing 2. Powershell Cleanup Commands

1.8.2. Install Other Recommended Dependencies

I strongly recommend installing `fzf`, `lazygit`, `ripgrep` and `fd`, which are used by LazyVim to provide enhanced git, string searching, and file searching behaviours. Most operating system package managers will have these available for trivial installation. You can find more specific installation instructions on their respective GitHub repositories:

- junegunn/fzf [https://github.com/junegunn/fzf]
- jesseduffield/lazygit [https://github.com/jesseduffield/lazygit]
- BurntSushi/ripgrep [https://github.com/BurntSushi/ripgrep]
- sharkdp/fd [https://github.com/sharkdp/fd]

1.8.3. Clone The LazyVim Starter Template

You'll use a `git clone` command to download the starter template and copy it into the user config directory for Neovim, then remove the `.git` folder.

The starter is just that: a starter. So you won't ever need to pull changes from this repo. Instead, LazyVim will manage updating itself and all its plugins for you. The only reason the starter is a git repo is that it's easy for the LazyVim maintainers to maintain. From your point of view you're just downloading the current state of the repo and don't need to know about the past or future state.

Clone Starter Repository: WSL, MacOS, and Linux

On Unix systems, use these commands:

```
$ git clone https://github.com/LazyVim/starter ~/.config/nvim
$ rm -rf ~/.config/nvim/.git
```

Listing 3. Unix Clone Starter

Clone Starter Repository: Windows without WSL

On native Windows, use these commands:

```
$ git clone https://github.com/LazyVim/starter
$env:LOCALAPPDATA\nvim
$ Remove-Item $env:LOCALAPPDATA\nvim\.git -Recurse -Force
```

Listing 4. Powershell Clone Starter

1.9. The Dashboard

Ok, you have completed the most difficult section of this book and you're finally ready to start LazyVim! Use the same terminal command as before: `nvim`.

You'll see a flurry of activity as LazyVim sets everything up and downloads the plugins it thinks are essential. You may see it compile and install a bunch of treesitter grammars; if you see a message to "Show More" use `G` (i.e. `Shift+g`) to skip to the end.

Once everything is installed, you'll see a summary of the plugins that were installed

inside a window managed by a plugin called Lazy.nvim that we will discuss shortly. For now, once you get to the Lazy.nvim screen, you can press the `q` key. The plugin will interpret this as "quit Lazy.nvim" and the window will close.

Now you can see the LazyVim dashboard, which is the first thing you'll see every time you start LazyVim. It's a little more friendly than the out of the box Neovim experience:

Figure 1. LazyVim Dashboard

As you can see, there are several commands that allow you to interact with the dashboard via a single keystroke. Most importantly, of course, is the `q` keystroke to quit!

Most of these options are self-explanatory, but we'll discuss a few of them more deeply in later chapters.

1.10. Lazy.nvim Plugin Manager

When you first open LazyVim, it checks for any plugins that are available to be updated, and gives you an overview in a message notification that will look something like this:

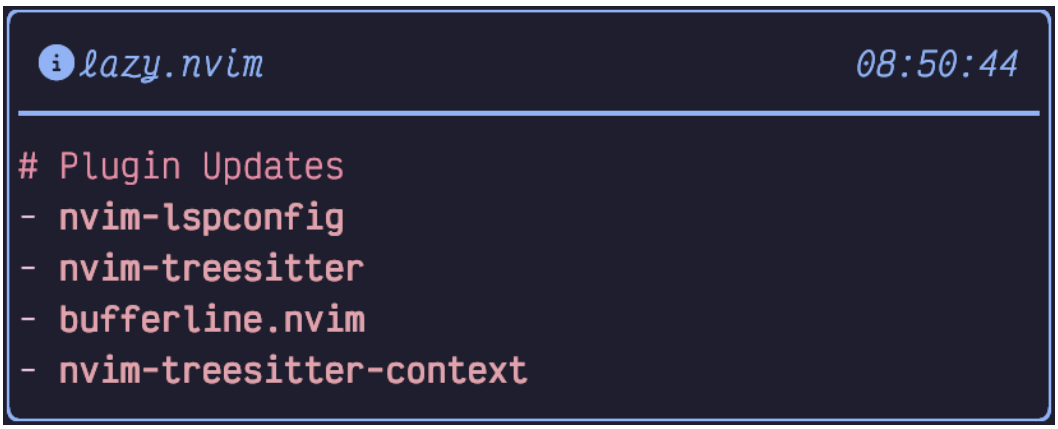


Figure 2. Plugin Update Notification

Because Neovim is pretty barebones by default, LazyVim ships with a ton of useful plugins ready to go. And there's a good chance they are out of date because plugin development in the Neovim world happens at a ridiculously fast pace.

In the old old days, plugin management was a completely manual process. In the less old, but still old days, it was managed by a variety of plugins that did the job but felt like they were lacking something.

Then came the plugin manager called Lazy.nvim, created by the same person that later created LazyVim. The Lazy.nvim plugin manager should not be confused with LazyVim itself, though both are maintained by the same person. Lazy.nvim is strictly a plugin manager, whereas LazyVim is a collection of plugins and configurations that ship together. One of those plugins is Lazy.nvim.

Lazy.nvim has a ton of slick features, most notably loading plugins only when needed (hence the name "Lazy") so that your editor is lightning fast to start up. It also has a nice UI for managing plugin installation and updates.

You can access this UI from the dashboard simply by pressing the `l` key, which is labelled in the dashboard as `Lazy`. The label should probably be `Lazy PLugin`

`Manager` to make it a bit clearer, but now you know what `Lazy` means so you won't forget.

If you are not actively displaying the dashboard, you can show the plugin manager at any time by entering Space mode. We'll cover Space mode in detail in the next chapter, but for now: First make sure you are in Normal mode by checking the lower left corner of the active window. If not, press `Esc` to enter Normal mode. Then press `Space` to enter Space mode, followed by `l` to bring up the Lazy.nvim plugin manager.

(Don't worry, those keybindings will all be second nature within a week.)

The Lazy.nvim plugin manager interface looks like this:

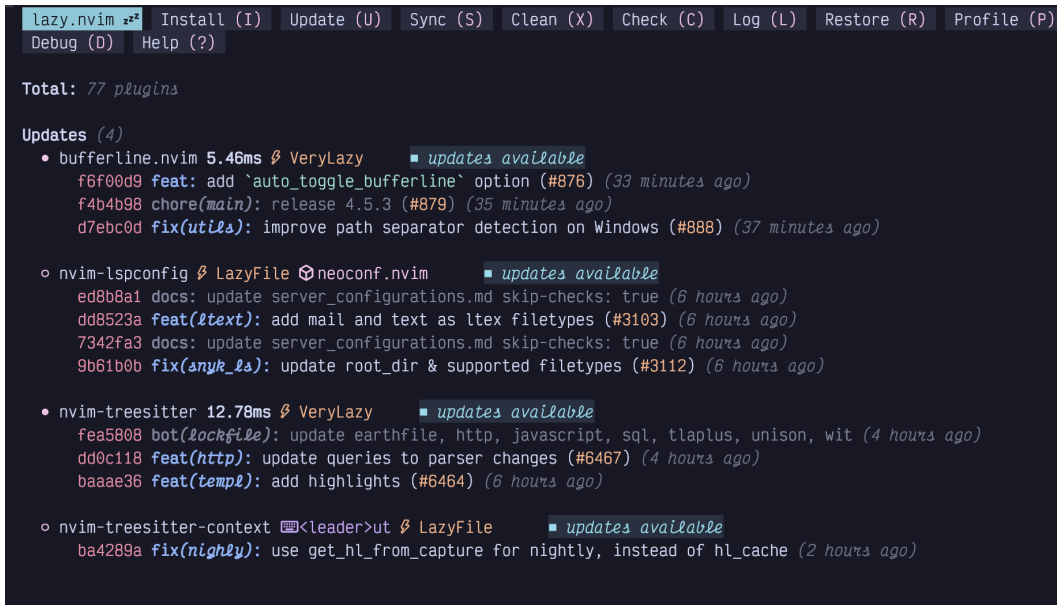


Figure 3. Lazy.nvim Plugin Manager

The window that has popped up is called a floating window. You'll see these in a few different situations, usually when there is interactive data that you need to work with. This particular floating window comes with its own set of keybindings. The keybindings are listed across the top, and pay attention to the fact that all of them are capitalized, so you need to use the `Shift` key when invoking them.

Typically, the only Lazy.nvim keybinding I use is `S`, for `Sync`. This is equivalent to running install, clean, and update in a single action. It guarantees that the versions of plugins that are actually installed are exactly consistent with the ones specified in the LazyVim configuration.

So when the "Plugin Updates" notification pops up, just press `Space-l` and then `S` and wait for the sync to complete. Then press `q` to close the Lazy.nvim Plugin mode and floating window and return to what you were doing.

1.11. A Note on Managing Dot Files

If you work on multiple different computers, you'll quickly find that you don't want to set up your LazyVim configuration separately on all of them. LazyVim does not have the equivalent of VS Code's "settings sync", though such plugins exist.

The alternative I recommend is to store your config files in a git repository. You'll probably find there are a few other files you want to keep in there such as your terminal's config file, your `.gitconfig` and `.zshrc` / `.bashrc` / `.config/fish/config.fish`.

If you use GitHub Codespaces, you may already manage some dot files with git. If not, my personal recommendation is to follow the advice in the excellent blog article Dotfiles: Best way to store in a bare git repository [https://www.atlassian.com/git/tutorials/dotfiles] from the Atlassian blog.

Before distributions like LazyVim came along, it was very common for people to store their Vim configuration in a public repository, and borrow ideas from each other. This practice is not quite dead, and you can find my own dot files on GitHub in the dusty-phillips/dotfiles [https://github.com/dusty-phillips/dotfiles] repository.

1.12. Summary

In this chapter, we briefly discussed the history of Vim, Neovim, and LazyVim, and why they are still relevant today. Then we covered the importance of GPU accelerated terminals and Nerd Fonts.

We figured out how to install Neovim and its dependencies under whichever operating system(s) you use, and finally, installed LazyVim from its starter template.

In the next chapter, we'll discuss Vim's core feature: Modal Editing, and dig into the many things you can do with your keyboard in LazyVim.

Chapter 2. What is Modal Editing, Anyway?

As the letters on the dashboard suggest, LazyVim is keyboard-centric. As many actions as possible can be performed without moving your hands between mouse and keyboard. Of course, it's still possible to use the mouse. You can click anywhere in the editor, interact with buttons and modals when they pop up, use the scroll wheel or touchpad gestures to scroll, and resize editor panes by dragging their borders. But you can also do all of these things using the keyboard, and usually more efficiently.

More importantly, you can do most things by holding at most two keys, and usually just one. You will only rarely have to contort your hands into painful (and dangerous) positions to `Control + Shift + Alt + <some key>`.

How does Vim do this? Modal editing.

2.1. Introduction to Modal Editing

"Modes" in LazyVim simply mean that different keystrokes mean different things depending on which mode is currently active. For example, when you start the editor up, you are in a "Dashboard Mode", and the most common interpretation of each keystroke in that mode are listed right on the dashboard. This discoverability of keybindings in a given mode is a common theme in LazyVim, and a huge improvement over the opaque default behaviour of Neovim itself.

To see what I mean, press the spacebar to enter "Space mode". Space mode is a LazyVim concept; it does not exist in a raw Neovim installation (though you could install various plugins to recreate the effect if you want Space Mode without LazyVim).

Entering Space mode pops up a menu along the bottom of your screen. It will look something like this (my menu contains some customizations, so yours won't be identical):

Figure 4. Space Mode

That's a big menu. The important thing to focus on right now is the `f` key, which we will use to understand modal editing.

If you are in Dashboard mode and press the `f` key, you will open the `Find file` dialog using a picker plugin we'll discuss later. However, now that you are in Space mode, if you press the `f` key, it will instead open the `file/find` Space mode submenu.

This is the crux of what modal editing means: The behaviour of a given key depends on the current mode. As indicated by the line at the bottom of the Space mode menu, you can press the `Escape` key to exit Space mode and return to the dashboard. Go ahead and do that.

Now you're back in Dashboard mode, and you can press the `n` key to create a new, empty buffer.

Pay close attention to the lower left corner of that buffer, where you'll see the word `INSERT`:

Figure 5. Insert Mode

Remember how I said Space mode is a LazyVim concept? Insert mode is an original Vi concept, that the successors Vim, then Neovim, and now LazyVim have all inherited. In Insert mode, the vast majority of keystrokes do what you would expect in any editor: they insert text. So you can touch type as usual.

You can access *some* keyboard shortcuts in Insert mode using `Control` and `Alt` keys. For example, you can hit `Control-r` to enter the "Registers" mini-mode, which pops up a list of "registers" you can paste from. We'll cover registers in detail in Chapter 8. For now, it is enough to know that `Control-r` followed by the plus key (i.e. `Shift-=`) will paste text from the clipboard when in Insert mode.

However, you will much more often change to *Normal* mode to perform any non-text-entry operations, including pasting text. Normal mode is another major Vim mode that has been around since the days of Vi.

To get into Normal mode from Insert mode, hit the `Escape` key. The cursor will change from a bar to a block and the indicator in the lower left corner will change to a blue `NORMAL`:

Figure 6. Normal Mode

In Normal mode, pressing various keyboard characters will **not** insert text like it does in Insert mode. For example, pressing `p` will, rather than inserting a literal `p` character into the document, instead paste from the system clipboard.

Vim and Neovim aren't very discoverable, but they ARE generally memorable. As often as possible, the keyboard shortcuts to perform an action start with a letter that makes sense for the action being performed. You might think `p` stands for "**p**aste", but in fact the concept has been around for longer than the clipboard abstraction. You are welcome to think of it as "paste" if that's easier for you, but in Vim parlance, it actually stands for "put", and we'll use that word throughout the book.

For some contrast, the `Control-r` key that pops up the list of registers in Insert mode does **not** pop up a list of registers in Normal mode. Instead, `Control-r` means "redo" (i.e. undo an undo). In order to enter the Registers mini-mode from Normal mode, you would press the `"` (quote, as in `<Shift>-apostrophe`) key instead.

If that sounds confusing, don't worry. Your brain and muscle memory will adapt more quickly than you expect and you'll always understand that behaviours in Normal mode are not the same as in Insert mode.

I hardly ever use non-text-entry commands in Insert mode. I find it is easier to switch back to Normal mode and then perform the command from Normal mode. It doesn't usually take a higher number of keystrokes to do so and I don't have to hold down multiple keys at once.

The universal key to exit Insert mode and return to Normal mode is `Escape`. And that brings us to an important point: You will be using this key a lot, but moving your hands from the home row to the Escape key in the upper left corner and back again is somewhat inefficient.

You can choose from a few common solutions to this situation:

- If you have a customizable keyboard you can put the `Escape` key in a more accessible location. This is what I do. I have a Kinesis Advantage 360, and I remapped the keys so that escape is in the "thumb key" section of this admittedly bizarre keyboard. It's as easy to hit as `Enter`, `Space`, and `Backspace`, other keys that I use very frequently.
- Your operating system is probably also capable of remapping keys. A lot of modal users replace the useless `Capslock` with the `Escape` key. (For non-modal paradigms it can be more comfortable to remap `Capslock` to the commonly-held `Control` key, especially on laptop keyboards).
- Neovim itself is also able to remap keys. We'll discuss how to do this in LazyVim later. One common pattern is to map a series of uncommon keystrokes that you wouldn't likely type together when inserting text to the escape key. So you can set it up to map something like `jk`, `jj` or `;;` in Insert mode to switch to Normal mode. I've tried this and don't care for it as it introduces a timing thing when you hit the first character and Neovim is waiting to see if you're going to type a command or let text insertion continue, but you might like it.
- The `Control-C` keyboard combination also works to exit Insert mode, with no remapping required. I don't like this because it's two keystrokes and on my Dvorak keyboard, `Control-C` is harder to hit than on a qwerty keyboard where C is on the bottom row near the `Control` key.

Don't worry about actually changing it for now; just start getting used to using `Escape` where it is and see if you find it annoying.

Once you're in Normal mode, you'll obviously want to get back to Insert mode to enter text at some point! There are several different ways to do this. Here are a couple of the most common ones:

The `i` key always inserts text *before* the current cursor position. This means that you could (very clumsily) move your cursor left by pressing `i <Escape> i <Escape>` repeatedly. When you press `i`, you insert text before the current position, and then `Escape` takes you out of Insert mode at that new "before" position.

Commonly, you want to enter Insert mode *after* the current cursor position. To do that, use the `a` key instead (mnemonic: i for **I**nsert Before, a for **A**ppend, although I usually think of it as **A**fter).

You'll find that you need to alternate between these a lot as you are navigating a document because the various navigation commands we'll cover later will often put you just before or just after the position you need to insert at. So it's important to remember both of them.

Two other very common operations are to insert at the very beginning or the very end of the current line. You *could* use navigation commands to move to the start or end and then use `i` and `a`, but it's easier to use the commands `I` and `A` instead (The difference is that they are capitalized, so you need the `Shift` key with them).

2.1.1. A Note on Keybinding Mnemonics

It is very common for related keybindings like these to be assigned to the lowercase and uppercase versions of the same key. You will often find that the lower case version means "do something" and the uppercase version means either "do the same thing only BIGGER" or "do the OPPOSITE thing", depending on the situation. In this case, `i` and `a` mean "insert one character before or after the cursor" and `I` and `A` are "insert before or after the cursor, only BIGGER (i.e. at the beginning or end of the line)".

To illustrate the "do the opposite thing" situation, consider the `o` and (shifted) `O` keys, which are two more ways to get into Insert mode.

The `o` key is used to enter Insert mode on a new line *below* the current one. For the "do the opposite thing" scenario, the shifted `O` means "create a new line *above* the current one and enter Insert mode on it".

The mnemonic "**O**pen a new line above/below" can help you remember the otherwise not terribly memorable `o` command.

One final useful command that takes two keystrokes is `gi`. That is a single press and release of `g` followed by `i`. It means "Go to the last place you entered Insert mode, and enter Insert mode again". In this case, the `g` key is actually switching to a new mini-mode I call "Go To" mode, though not all the commands accessible from it are strictly related to going places. You can see the entire list of commands available in "Go To" mode by pressing the `g` key in Normal mode and waiting for the menu to pop up at the bottom of the window:

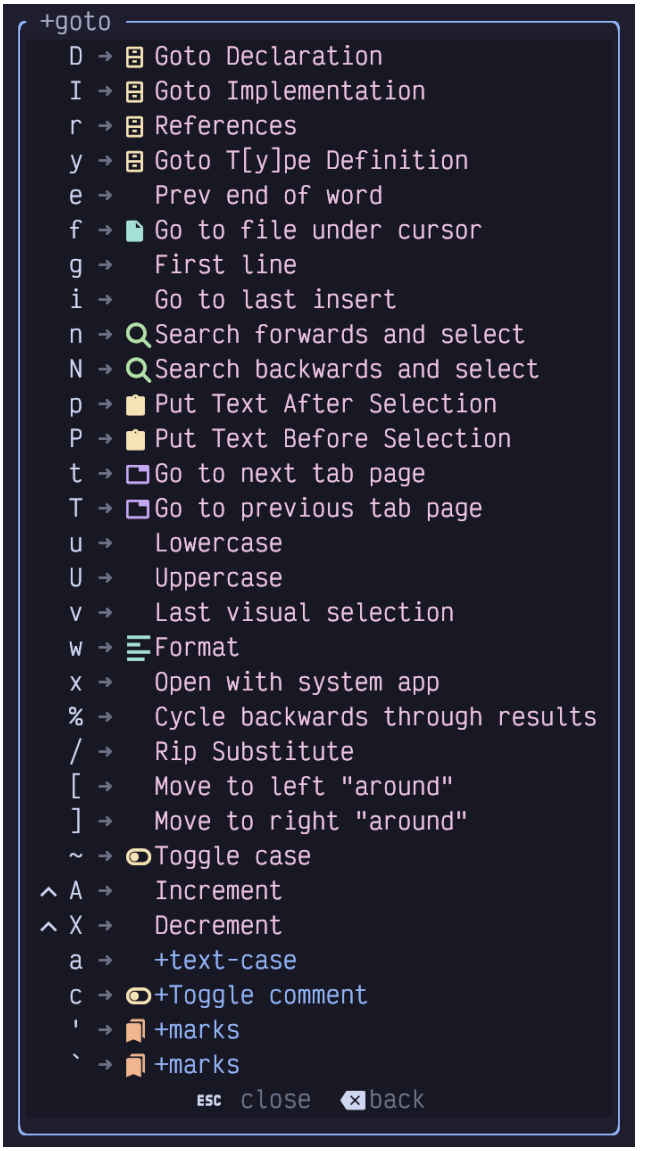


Figure 7. Go To Mode

We'll cover most of them later, but notice that the `i` key is in there labelled "Move to the last insertion and INSERT". So if you forget how to go to the last insertion point, you can enter Go To mode and scan the menu to find the `i` again.

Try all of those commands (`a`, `i`, `o`, `A`, `I`, `O`, and `gi`) repeatedly, entering some text and pressing `Escape` to return to Normal mode. Then try it again. Move your cursor around the text using the mouse (we'll get to keyboard navigation soon, I promise), and try using the commands again to see how they behave in new locations.

Get *really* comfortable with switching between Normal and Insert mode. You might think you'll spend most of your time in Insert mode, but the truth is code is edited far more often than it is written afresh, and you'll be alternating between them constantly.

2.2. Visual Mode

The other major mode that LazyVim inherits from its ancestors is "Visual" mode. Visual mode is used to select text. In general, you can enter Visual mode and then use many of the same navigation keys you would use in Normal mode to move your cursor around. Since we haven't covered those navigation keystrokes yet, I'm going to defer a detailed discussion of Visual Mode until Chapter 8, when we will have the necessary foundation.

2.3. Command Mode

Command mode is different from the other modes we've seen, which were mostly either submenus or the editor-level "major" modes. You can get into command mode from Normal mode by using the `:` (i.e. `Shift-<semicolon>`) command. In LazyVim, this will pop up a little widget where you can type what is known as an "Ex Command." This name comes from `vi`'s predecessor, `ex`, which hasn't really been used (other than as part of Vim) in decades.

Essentially, you can enter a wide variety of commands into this widget and expect certain behaviours to happen as a result. It is actually more similar to the command palette popularized by Sublime Text and VS Code than anything else, though it is quite a different experience.

You already know one ex command from Chapter 1! Remember `<Escape><Colon>q!<Enter>` the command to exit the editor? You now know that the `Escape` is to enter Normal mode from whatever mode you are in. The colon is used to switch to Command mode, and the `q` is short for quit (You could type the full word `quit` if you didn't feel the need to conserve keystrokes). The exclamation point

says “without saving" and the `Enter` means "submit the ex command”.

As another example, let’s consider the `write` ex command. Type `:` followed by `write myfile.txt` like this:

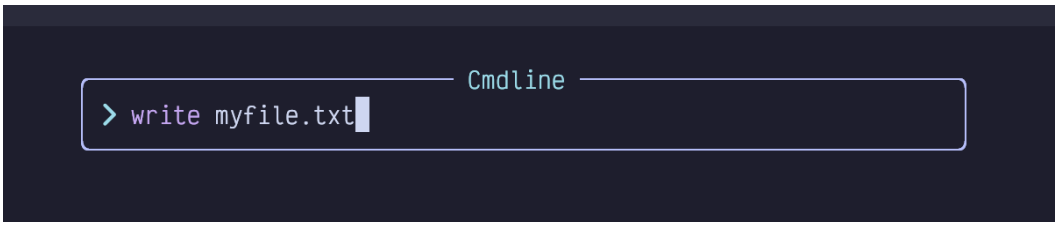


Figure 8. Write Command

Press `Enter` to confirm and execute the command, which will save the file with the given name.

Most commands can be shortened to their shortest unique common prefix. You usually type `:w myfile.txt` instead of `:write myfile.txt`. The most popular commands even have special combined commands, so `:wq` will save and exit, although you’ll probably prefer `:x` as it’s even shorter.

Command mode is kind of weird. It’s like an Insert mode in the sense that you can type text into it, and some of the keybindings that work in Insert mode also work in Command mode (including `Control-r` to paste from a register). But other keybindings work differently in Command mode. The most important one is the `Tab` key, which will do a sort of “tab completion” on the command. For example, `:q<Tab>` pops up a menu like this:

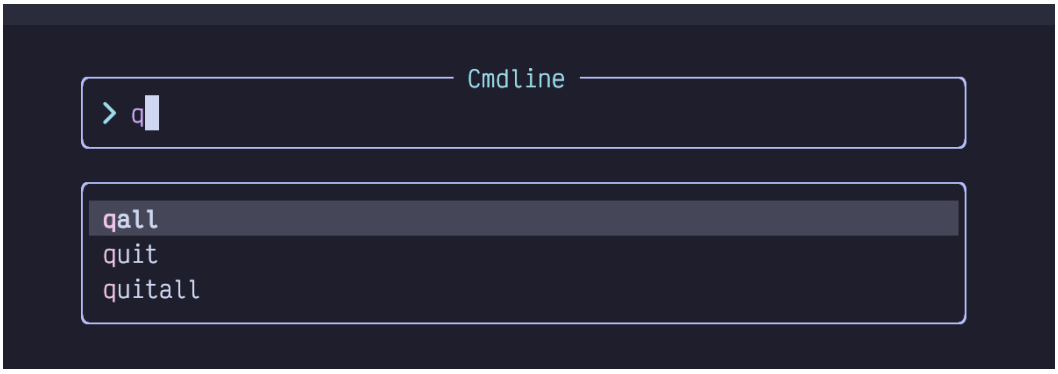


Figure 9. Tab in Command

This completion menu is disturbingly unintuitive to navigate. You’re probably going to want to bookmark this section or take some notes to refer to until you get used to

it!

First, if you want to select a different entry in the menu, you would surely think you can use the arrow keys, right? Well, you can, but it's a mind-mess because you need to use `Left` and `Right` to move the cursor `Up` and `Down`. I know! WTF, right?

This is mostly because the menu looks different in LazyVim than it did in the original Vim, but the keys haven't been remapped. So instead, I suggest using `Tab` and `Shift-Tab` to select different entries from the menu. It's easier to remember and much easier on the muscle memory: Tab once to show the menu, tab again to cycle through other entries in the menu.

Second, there is some nuance around *confirming* one of those menu entries. In the above example, you can just press `Enter` to confirm the selection **and execute it**. However, there are often cases where you want to confirm the selection and then continue editing the command. An excellent example is the `:e` or `:edit` command.

This command is used to open a file on your filesystem, but you have to type the entire path to the file. For example, if you have the following directory structure:

```
.
└── foo
    ├── bar
    └── baz
    │   └── fizz.txt
```

Listing 5. Nested Directory Structure

...and you have Neovim open, you would have to type the following to open the `fizz.txt` file:

```
:e foo/baz/fizz.txt
```

Listing 6. Edit File Command

That's a lot of typing if you need to get to deeply nested directories. Luckily, you *can* use tab completion for this. You can type `:e f<tab>b<tab><tab><tab>` to select `foo/baz`, but at this point the menu is still open:

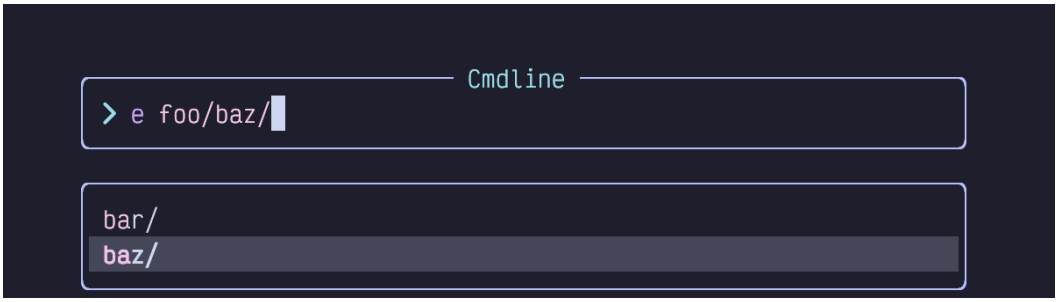


Figure 10. Edit Command with Completion Menu

If you press `Enter` now, it's going to open the `baz` *folder* instead of just confirming the selection, which is not what you want. And if you press `Tab` again it will cycle through the menu some more.

Instead, you have a couple of options. The `Down` arrow key will (unintuitively) move "into" the selected directory, allowing you to tab through the files inside it. Alternatively, use the `Control-y` (`y` for "**y**es") key combination. This will confirm the `baz` selection and close the menu but leave you in Command mode. Now you can press tab again to complete the `fizz.txt` portion of the command.

It **is** possible to remap these keys to be more like other software, and I honestly think this is one thing LazyVim should do by default. I haven't found a combination that I like, though, so I just stick with the default keybindings.

You probably won't spend a lot of time in Command mode. There are easier ways to open files in LazyVim, for example, as well as to quit the editor. And if you need to do something more complex with command history, there is a special window you can use to edit commands with Insert and Normal mode that we will cover later.

For now, remember `<Tab>` and `Control-y` and you'll be able to navigate the Command menu when you need to.

The most important command, by the way, is `:help`. Vim was created before folks had ready access to the Internet, so it has a tradition of shipping all of its documentation with the editor. So for example, if you can't remember the keyboard shortcut to `put` text, try `:help put`. Or, if you want to know what the `Control-R` keyboard shortcut does, try `:help CTRL-R`. Of course, the Vim help documents have also been indexed by your favourite search engines and AI chat bots, so you can go all new-school and ask them if you prefer.

2.4. Summary

In this chapter, we became comfortable with the concept of modal editing and the most important LazyVim modes. There are other mini-modes that will come up as we progress through this book, but becoming comfortable with Normal, Insert, and Command mode (and how to switch between them) will take you a long way on your LazyVim journey.

In the next chapter, we'll learn a whole bunch of different ways to move the cursor around inside a document.

Chapter 3. Getting Around

Software developers spend far more time editing code than we do writing it. We're always debugging, adding features, and refactoring.

Indeed, the most common thing I ever do is add a print/printf/Println/console.log at some specific line in the codebase.

If you are coming from the more common word processing or text editing ecosystems, navigating code is the thing that is most different in Vim's modal paradigm. Even if you're used to Vim, some of the plugins LazyVim ships by default provide different methods of code navigation than the old Vim standbys.

In VS Code, often the quickest way to get from one point in the code to another is to use the mouse. For minor movements, the arrow keys work well, and they can be combined with `Control`, `Alt`, or `Cmd/Win` to move in larger increments such as by words, paragraphs, or to the beginning or end of the line. There are numerous other keyboard shortcuts to make getting around easier, and the Language Server support allows for easy semantic code navigation such as "Go to Definition" and "Go to Symbol".

Vim also supports mouse navigation, but you'll likely reach for it less often once you train up on the navigation keymappings. LazyVim has keybindings for the same Language Server Protocol features that VS Code has, and they are often more accessible. The big difference with Vim is the entire keyboard's worth of navigation commands that are opened up to you when your editor is in Normal mode.

3.1. Seeking Text

LazyVim ships with a plugin called flash.nvim, which was created by the maintainer of LazyVim and integrates very nicely with it.

This plugin provides a code navigation mode that has been available in various vim plugins for many years, and has historically been quite controversial. A lot of long-time Vim users think it breaks the Vim paradigm. I won't go into the details as to why, but I will acknowledge that this was true in older iterations of the paradigm but is somewhat less true in modern versions such as flash.nvim.

If you can see the code you want to navigate to (i.e. because the file is currently open and the code is scrolled into view), flash.nvim is almost always the fastest way to move your cursor there. It admittedly takes at least three keystrokes, but those

three keystrokes require no mental math or incrementally "moving closer" to the target until you get there, which are two of the less efficient problems that come up with certain other Vim navigation techniques (as well as in non-modal editing).

To invoke `flash`, press the `s` key in Normal mode. My mnemonic for `s` for "**s**eek", although I've also heard it referred to as "sneak" or "search" mode. Searching in LazyVim is a different behaviour (it doesn't care if the text is currently visible or not), and "sneaking" sounds a little too dishonest, so I use "Seek".

The first thing to notice when you press `s` is that the text fades to a uniform colour and there's a little lightning symbol in the status bar indicating that Flash mode is active:

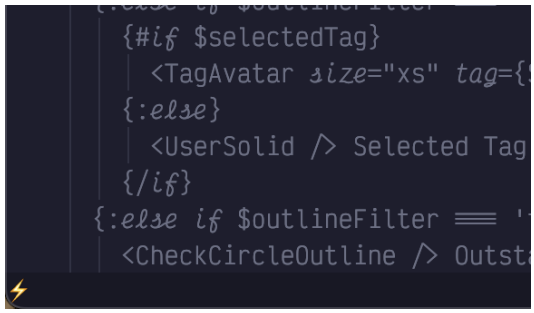


Figure 11. Flash Mode Active

Since you know where you want the cursor to be, your eyes are probably looking right at it, and you know exactly what character is at that location. So after entering Seek mode, simply type the character you want to jump to.

For example, in the following screenshot, I want to fix the typo in the heading of this section, changing `Test` to `Text`.

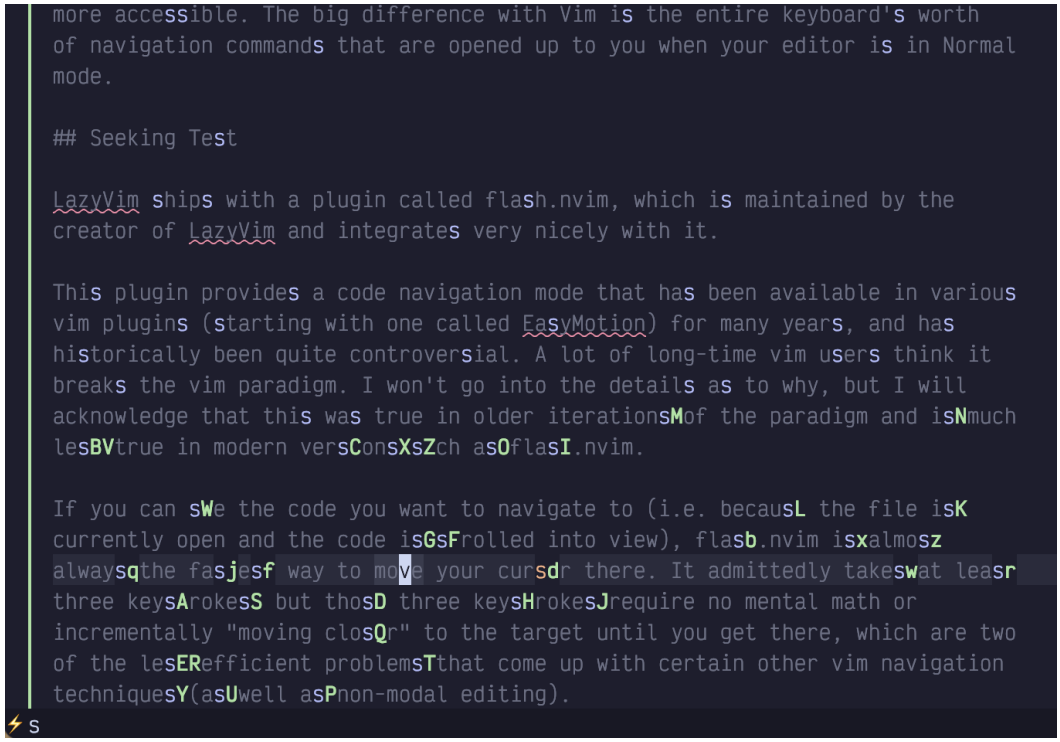


Figure 12. Seek `s` characters

I have hit `ss`, and every single `s` in the screenshot has turned blue, including capitals. There is an `s` character beside the flash icon in the status bar indicating that I have seeked an `s`.

In addition, all the `s` characters nearest to the cursor (which is in the bottom paragraph) have a green label beside (to the right of) them. If I wanted to jump to any of those `s` characters, I would just have to type that label and boom, I'd be there.

However, the character I want to hit is too far away to have a unique label, as there are a lot of `s` characters in my text. No matter! I just have to type the character to the right of the target `s` character, which is a `t`. Now my screen looks like this:

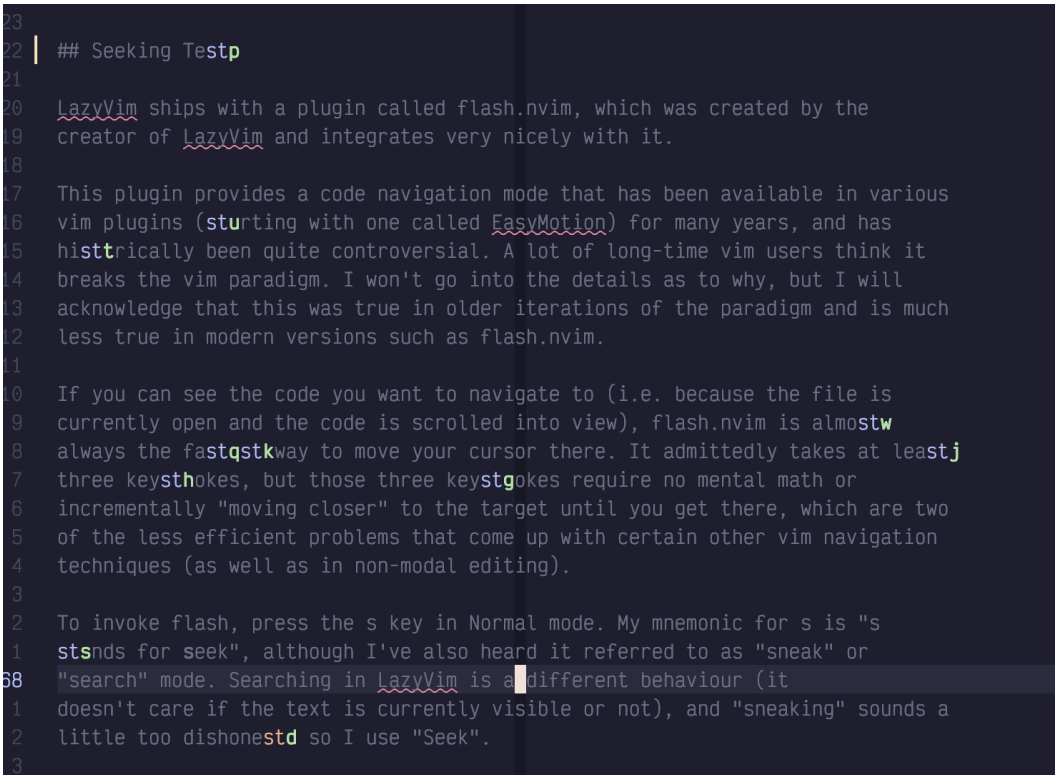


Figure 13. Seek `st` characters

Now, all instances of `st` in the file are highlighted in blue, and since there aren't as many `st` as `s`, all of those instances have a label beside them. The text I want to move to is labelled with a `p`, so I press `p` and my cursor is moved to the `s` character I wanted to change. Now I can type `rx` to replace the `s` with an `x` (we'll discuss *editing* code in chapter 6, but now you've had a taste of it).

If you have multiple files open in split windows (which we'll discuss in Chapter 9), Seek mode can be used to move your cursor *anywhere* on the screen, not just in the currently active split.

Seek mode does have drawbacks however, at least the way flash.nvim implements it. There are some characters you can't move to directly because you run out of text to search for before a labelled match targets that location. For me this happens most often when I want to edit the end of a line. If I type `sn` because I want to edit a line that has `n` as the last character, but there are a bunch of `n` characters closer to my cursor than the one I want to move to, flash may not label the `n` I want to move to, and it won't accept a carriage return as a "next character" input.

For this reason, I don't seek near ends of lines. Instead, I'll seek to a word somewhere in the middle of the same line and then use `A` which, as you may recall, will put me in Insert mode at the end of the line. Alternatively, if I don't want to

enter Insert mode, I will use the `$` symbol (`Shift+4`), which is the Normal mode command for "Move cursor to end of current line".

3.2. Scrolling the Screen

Seek mode only works if the text you want to jump to is visible on the screen. You can't label something you can't see! Often, this means you want to use search or one of the larger or more specific motions discussed later, but there are also a few keybindings you can use to scroll the screen so you can see your target before you jump to it.

These keybindings are a little unusual by Vim standards because they mostly involve using the control key. How anti-modal! In my experience, these keybindings don't actually get a ton of use. Indeed I've forgotten some of them and had to look them up to write this chapter.

The scrolling keys I use the most are definitely `Control-d` and `Control-u`, where the mnemonics are **d**own and **u**p. They scroll the window by half a screen's worth of text. The cursor stays in the same spot relative to the **window**, which means that it is moved up or down by half a screen's worth of text relative to the **document**.

If you need to move even further, you can use the `Control-f` and `Control-b` keybindings (**f**orward and **b**ackward), which scroll by a full page of text. I don't like these ones because I never quite know where the cursor is going to end up and I become disoriented. But it can be handy if you need to scroll something into view quickly to use Seek mode on it. Unlike `Control-d` and `Control-u`, `Control-f` and `Control-b` can be prefixed with a count, so you can type `5<Control-f>` if you need to scroll ahead by `5` pages.

To scroll the window by a single line, use `Control-y` and `Control-e`. I have no idea why these keybindings were chosen. There is no mnemonic. I easily forget them, and so I never use them. These keybindings accept a count, so if you can remember them, they are useful for subtle repositioning of the text. The main advantage of these keybindings is that they don't move the cursor unless it would scroll off the screen, so if you are working on a line and need more visibility but don't want to move the cursor, you could use `Control-y` and `Control-e` to do it.

The reason I don't use these keys (other than lack of a decent mnemonic) is that I prefer to do relative cursor positioning using `z` mode.

3.2.1. Z Mode

The `z` menu mode is an eclectic mix of cursor positioning, code folding, and random commands. You can see a list by pressing the `z` key while in Normal mode:

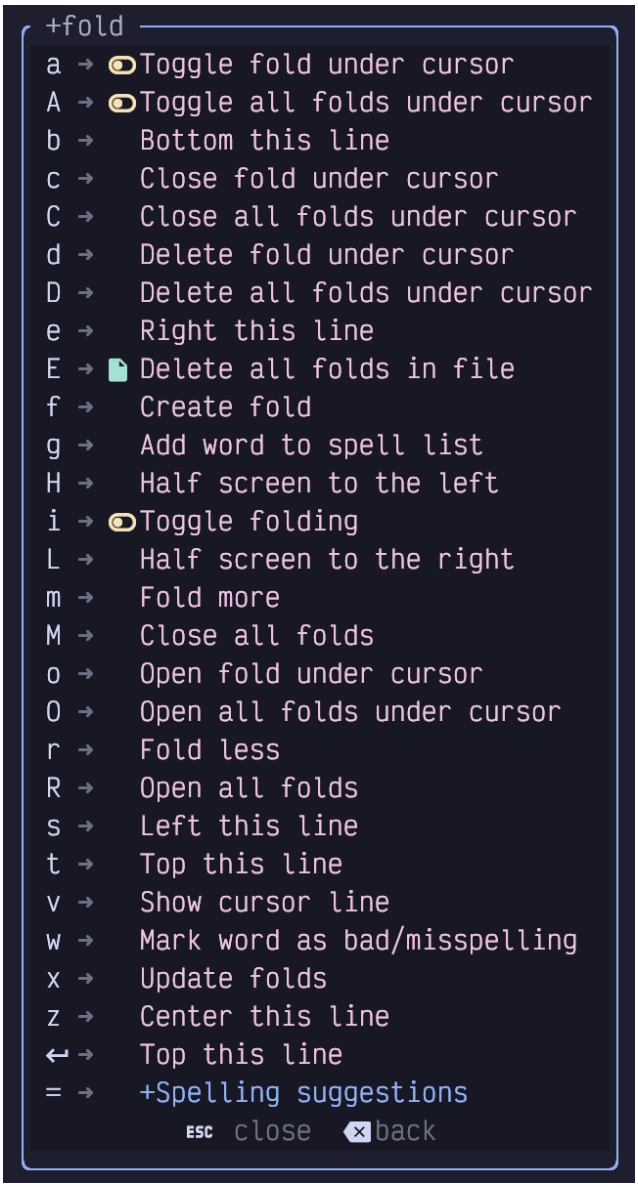


Figure 14. The `z` Menu

If that looks like a big menu, you don't know the half of it! There are a ton of other z-mode keybindings that are obscure enough that the which-key plugin doesn't bother to list them in the menu! I'll cover the three most useful scrolling related ones here and we'll discuss others later.

The relative cursor keybindings I use exclusively are `zt`, `zb`, and `zz`. These move the

line that the cursor is currently on to the **t**op, **b**ottom, or **m**iddle of the screen, respectively. When moving to the top or bottom it will leave a few lines of context above or below the cursor.

There are others that will also move the cursor to the first column of the window, but instead of memorizing those shortcuts, I recommend combining the previous commands with `0` instead, as in `zt0`, `zb0`, and `zz0`. The `0` command just means "Go to the start of the line". You can also use *home* if your keyboard has a *home* key, but `0` is easier to hit on many keyboards.

You can find other scrolling keybindings in the Neovim documentation by typing `:help scrolling`, but the ones I just mentioned will probably more than cover your needs as you learn far more nuanced methods of navigating code.

3.3. The First Rule of Vim

So there is a holy rule in Vim that I constantly break for valid reasons. Unless you are the very strange combination of weird that I am, you probably should not break it quite so often:

Never use the arrow keys to move the cursor.

The background behind this rule is that it takes a tenth of a second or so to move your hand to the arrow keys on most keyboards, and another tenth of a second to move it back to the home row. I'm not convinced these tenths of a second add up to an appreciable amount of time, even considering the millions of characters I have typed in my lifetime. (Yes, millions. I did the math once).

But I do think the arrow keys on most keyboards can do nasty things to the long term health of your hands, and honestly, the more you get used to the alternative Vim keybindings, the more you'll prefer to use them.

The Vim keybindings for arrow keys seem rather unintuitive when you first look at them: `h`, `j`, `k`, and `l`. These map to the directions, left, down, up, and right. If it seems weird that `l` means `right` instead of `left`, or you're wondering why they skipped `i` since that appears to be an alphabetic sequence, look at your keyboard.

If you are an English-speaker with a standard Qwerty keyboard, the letters `h`, `j`, `k`, and `l` are on the home row under your right hand, in that order, and are therefore the easiest keys to hit on the entire keyboard.

Open a largish file in Neovim (you can use `:e path/to/filename`) and experiment

with moving the cursor left, right, up and down using the home row keys. While you do that, I'll tell you why I don't use them because I'm triply abnormal.

First, I'm left handed, so the right hand home row is slightly less accessible feeling. Second, I've been a Dvorak user for two decades. The `j`, `k`, and `l` keys are not on my home row. Third, I use a Kinesis Advantage 360 keyboard, which, among other bizarre layout features, places the arrow keys within reach of my fingers so I don't have to move my hand to hit them.

By a strange twist of fate, these weirdnesses kind of cancel each other out. The `j` and `k` keys happen to be directly above the left and right arrow keys under my dominant left hand. So that's what I use for navigation: `Left` `Right`, `j` `k`. If you are less weird than me, you should probably use the right-hand home row keys the way Vim was designed.

Vim, Neovim, and LazyVim are all *really* good at reusing motions, so you will find that `h`, `j`, `k`, and `l` are used for a lot of different navigation sequences as you progress through this book. Take enough time to really get used to them. But recognize that if you ever have to push these keys more than twice in succession to move the cursor, you're wasting keystrokes.

3.4. Counting

The vast majority of commands in Vim can be prefixed with a *count* to repeat the motion multiple times. The count is typically entered as a sequence of digits before the command you want to repeat.

So, for example, to move the cursor up 15 lines, you would enter Normal mode and hit the keys `15k`. To move it five characters to the right, use `5l`.

This is why LazyVim has such weird line numbering by default. Consider the following screenshot:

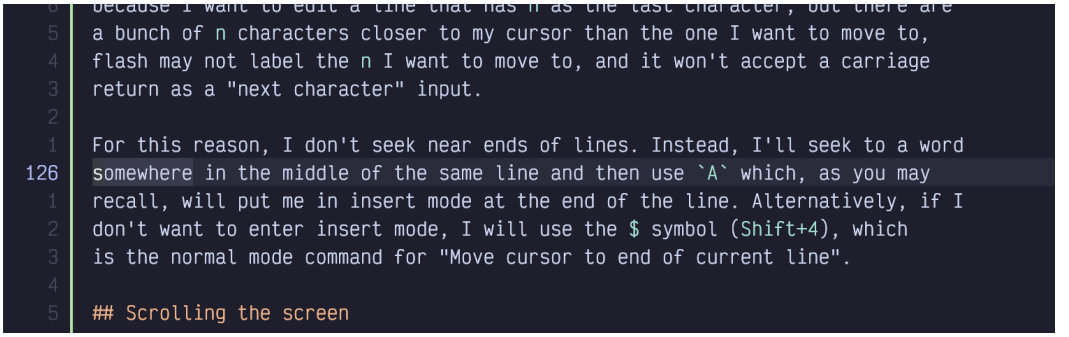


Figure 15. Relative Line Numbering

My cursor in this screenshot is on line 126, which is highlighted in the left gutter. It's also is also visible in the lower right corner of my window, though I cropped it out in this screenshot. But directly above line `126` we see the line number 1, and directly below it we also see the line number 1.

Let's say I want to move my cursor to the `Scrolling the screen` heading.

This line has the number `5` beside it, so I don't have to count lines or do any mental arithmetic to figure out the count to use to move my cursor. I just type `5j` and my cursor moves to the desired line.

Now that you know what they are for, I suggest leaving relative numbers on until you get used to them. If you find them distracting or just don't use them, you can change to normal line numbers by editing your LazyVim configuration. Open the file `~/.config/nvim/lua/config/options.lua`, which should have been created for you by LazyVim but currently won't have anything in it other than a comment describing what it is for.

You can use the Space mode command `<Space>fc` to quickly find files in the LazyVim configuration directory. This will pop up one of the file pickers that we'll discuss in detail in the next chapter. Type `options` and press `<Enter>` to open the file.

To disable relative file numbers by default, add this line to the file and save it:

```
vim.opt.relativenumber = false
```

Listing 7. Disable Relative Line Numbers

Then reopen Neovim, and you should see the absolute value of line numbers in the left column.

Personally, I find line numbers to not be very useful and I don't like wasting valuable screen width on displaying those characters. As has become a running theme, I recognize that I am somewhat odd! But if you also want to disable line numbers altogether, you'll need a second line in `options.lua`:

```
vim.opt.number = false
vim.opt.relativenumber = false
```

Listing 8. Disable All Line Numbers

3.5. Find Mode

If you need to move your cursor to a position that is relatively close to its current position, you may want to use LazyVim's Find mode instead of the Seek mode we described earlier. The default Find mode in Neovim is rather limited, but the flash.nvim plugin that enables Seek mode makes it much nicer to use.

To enter Find mode, press the `f` key. Like Seek mode, a portion of your screen will dim, indicating that you should type another character. After you do so, all instances of that character *after the cursor* will be highlighted. For example, `fs` will highlight all instances of the letter `s` after the current cursor position.

This is where the similarities between `Find` mode and `Seek` mode end, however. Instead of showing a label, the cursor immediately jumps forward to the first matching character after the cursor. You'll also notice that none of the text before the cursor has been dimmed, and that none of the matching characters in the lines before the cursor are highlighted.

Instead, we need to use counts to jump to later instances of the character. If I want to jump ahead to the third highlighted `s`, I type `3f` and my cursor will move there. However, if you want to jump to a much later `s`, you probably don't want to individually count how many `s` keys there are. Luckily, after you use a count, LazyVim leaves you in Find mode, so you can just guess how many `s` characters there are, and then once you are closer, repeat with a new count. If you only want to jump ahead by one `s` character, you don't need to enter a count, just press `f` by itself and you'll move ahead.

If you miscounted or misguessed and jump too far, don't worry! You can take advantage of the fact that (Shifted) `F` means "find backwards", and can also be counted. So if you need to move to the 15th highlighted `s`, it's totally fine to guess `18f`, realize you've gone three too far, and use `3F` to jump back to the previous

character.

Moreover, if you know that the character you are looking for is behind or above your cursor in the document, you can enter Find mode with `F` instead of `f` in the first place. This will immediately start a backwards find operation instead of a forward one. And if you know right off the bat that you want to jump back or ahead by three instances of the given character, you can use a count when you first enter Find mode.

There is also a subtle variation of Find mode that I call "**T**o" mode, although the official Vim mnemonic is actual "'**T**il" mode. You enter it with a `t` or `T` depending on what direction you want to go.

"To" mode behaves identically to Find mode except that it jumps to *just before* the target character.

You might think that To mode is kind of redundant because you could fairly easily use Find mode followed by a single `h` to move the cursor left. But "To" mode is extremely useful when you are combining it with operations to edit the text, which we will discuss later. As a taste, if you use the command `d2ts`, it will delete all text between the cursor and the second `s` it encounters, but leave that `s` alone. This is much easier than the `d2fsis<Escape>` that would be required if you used a find command and then had to enter Insert mode to add the `s` back.

3.6. Moving by Words

When `f` or `t` feels too big, and cursors with counts feel too small, you'll most likely want to use the word movement commands. In other editors and IDEs you might be used to getting this functionality by holding `Control`, `Alt`, or `Option` while using the arrow keys.

Neovim is easier; you don't have to move your hands to the arrow key section of the keyboard and you don't have to hold down multiple keys at once.

Instead, you can just enter Normal mode and press the `w` key to move to the beginning of the next word. If you instead want to move to the *end* of the current word, use the `e` key. If you are *already* at the end of the current word, `e` will go to the end of the *next* word.

This is useful when you want to combine it with counts: If you need to move to the end of the word that is two words after the current word, press `3e`. This is the same as pressing `e` three times, which would move to the end of the current word, then

the next word, and finally to the end of the word you want to hit. `w` can also be prefixed with a count if you need to move to the beginning of a word that is a certain number of words after the current one.

Use the `b` key if you want to move backwards instead. This will move you to the beginning of the current word, or if you are already at the beginning of the word, it will move to the beginning of the previous word. As before, use a count to move to the beginning of even more words.

Surprisingly, it takes a bit more work to move to the end of the *previous* word, as you need to press two keys: `g` followed by `e`. The mnemonic for this is "**g**o to **e**nd of previous word". In practice, you'll find that you hardly ever need this functionality for some reason, and the honest truth is I usually use `be` (`b` to move to beginning of previous word, then `e` to go to end of that word) to move to the end of the previous word. If you do use `ge`, however, it can be combined with a count as well. You'll need to type something like `4ge`, depending on the count. The command `g4e` wouldn't do anything useful.

Collectively, you may occasionally hear the `w`, `e`, and `b` commands referred as the "web" words. It just means "moving by words". These are probably the most common movements you will use, more than individual cursor positions, simply because most editing actions tend to involve changing or deleting a word or sequence of words.

3.7. Moving by Words, Only BIGGER

The "shifted" form of the `web` words also move by words, but the definition of "word" is subtly different. Specifically, a capital `W` will move to just after the next whitespace character, where a lowercase `w` will use other forms of punctuation to delimit a word. Consider a method call on an object that looks something like this in many languages:

```
myObj.methodName('foo', 'bar', 'baz');
```

Listing 9. Example Method Call

If your cursor is currently at the beginning of that line, a `w` will move your cursor to the period on the line, a second `w` will move you to the `m`, and subsequent `w` presses will stop at the paren and quotes as well.

On the other hand, if your cursor is at the beginning of the line, a `W` will move you all the way to the first quote in the `"bar"` argument, since that is where the first

whitespace character is.

As a visualization, here are all the stops on that line of code when you use `w` compared to when you use `W`:

```
myObj.methodName('foo', 'bar', 'baz')
-----ww---------w-w--w--ww--w--ww--w---->
------------------------W------W-------->
```

Listing 10. Behaviour of `w` and `W`

The `B`, `E`, and `gE` motions behave similarly, moving in the appropriate direction by whitespace-delimited words instead of punctuation ones.

One thing that is kind of annoying both in Vim and the way LazyVim is configured is that there's no way to navigate between the individual words of `CamelCaseWords` or `snake_case_words`. You can use `fC` or `t_` and similar if you want to, but I will later show you up how to set up the `nvim-spider` plugin that makes navigating these common programming constructs simpler.

3.8. Line Targets

Very frequently, you need to move to the beginning or end of the line you are currently editing. Often you can use `I` or `A` for this if your goal is to move to that location and enter Insert mode, but if you need to move there and stay in Normal mode (e.g. for other purposes such as to delete or change a word) you can use the `^`, `$`, and `0` commands.

If you are familiar with regular expressions, you might know that `^` is used to match the start of text or start of the line and that `$` is used to match the end, so the mnemonic of using these two keybindings to match the beginning and end of the current line will hopefully be less unmemorable than they seem at first.

There is a certain lack of symmetry between the two, however. The `$` (`Shift-4`) command simply means "go to the end of the line", as in the last character before the ending newline, no matter what that character is. The `^` or caret (`Shift-6`) means "go to the beginning of the text on this line". The "of the text" there is important: if your line has whitespace at the beginning (e.g. indentation), the `^` caret will **not** go to the very first column, but will instead go to the first non-whitespace character.

To move to the very beginning of the line, use the `0` key. `0` is the **only** numeric key that maps to a command because the others all start a count. But it wouldn't make sense to start a count with `0`, so we get to use it for "move to the zeroth column".

There is also a command to go to the end of the line excluding whitespace, but I have never used it, probably because I usually have formatters configured to trim trailing whitespace so it doesn't come up.

The two character combination `g_` (g underscore) means "go to the last non-blank character". I guess `_` kind of looks like "not a space", so it's kind of mnemonic? I include it to be comprehensive, but you'll likely not use it much. You also have the option of combining other commands you've learned so you don't have to memorize this one-off. For example, you can use the three character `$ge` (combining "end of line" with go backwards to end of word) or `$be` to move to the last non-blank on the line. You have options; pick the one that you find is easiest to remember or type!

3.9. Jumping to Specific Lines

If you compile some code or run a linter, you will invariably be given a line number where the error occurred (unless the compiler is particularly useless).

You can jump to a specific line by entering the line number as a count, followed by (shifted) `G`. So `100G` will move your cursor to line 100. Alternatively, you can use the `:100<Enter>` ex command.

`G` can also be issued without a count, in which case the `G` command will always take you to the end of the file.

You can go to the top of the file with `1G` if you want, but since this is such a common operation, you can instead use `gg` (two lower-case `g`s). The mnemonic for `g` in all cases is "Go to", and there are a lot of things that can come after a `g` (`:help g` will introduce you to the ones I don't cover, although be aware that LazyVim has overridden some of them).

Since the most common place you are likely to want to "go to" is a line number, the easiest to type `G` and `gg` commands are used for line number navigation.

3.10. Jump History

All this jumping around can make you feel a little lost. Luckily, there are two super-useful keybindings for going back to places you previously jumped.

`Control-o` is the non-modal control-based keybinding that I use most often. I should honestly bind it to something more accessible, I use it so much. It basically means "Go to the place I jumped from".

This is super handy when you're editing code deep in a file or module and realize you need to import a library at the top of the file. You can use `gg` to jump to the top of the file, `s` to seek to the line you want to add the import on, and then enter Insert mode to add the import. Now you want to go back to the code you were working on so you can actually use the import. `Control-o` a couple times will take you there.

Neovim keeps a history of *all* your jumps, so you can jump between several locations (perhaps to look up documentation or the call signature for a function) and always find a way back.

If you jump too far, you can use the `Control-i` keybinding to jump *forward* in history. It's just the opposite of `Control-o`. I don't know why `i` and `o` were chosen for these; maybe because they are side-by-side on a Qwerty keyboard? They are used commonly enough that once you learn them, you won't forget.

3.11. Summary

Navigating code is a huge topic in Vim. You've already learned enough commands that you can navigate a Vim window more efficiently than most non-modal editors can dream of. But we've barely scratched the surface, and we'll be covering a bunch of even more useful code navigation commands later.

We covered the LazyVim `Seek` mode to jump anywhere in the visible window, and then the scrolling commands to make sure the thing you want to jump to is visible. Then we covered moving the cursor with the home row keys and multiplying them with counts.

We learned how Find mode differs from Seek mode, even though they are superficially similar. Then we covered some standard keybindings for moving by words and to key places on a line before jumping to specific lines. We wrapped up by covering how to navigate to places you have jumped before.

In the next chapter, we'll learn more about opening files and navigating the filesystem.

Chapter 4. Opening Files

In the previous chapter, as a side-effect of learning about Command mode, we saw how to open files the old-fashioned Vim way, using the `:edit` command. Another old-school alternative is to open them directly from the terminal shell command line, using `nvim filename`.

Both of these are occasionally handy, but LazyVim pre-configures more modern ways of navigating and opening files.

4.1. Introducing File Pickers

LazyVim ships with the `snacks.nvim` plugin, which includes a bunch of quality-of-life improvements from a modern age. It's maintained by Folke Lemaitre, the creator of LazyVim, so we have some confidence that they integrate nicely. It is really a collection of random small plugins that perform a huge variety of tasks.

The first snacks integration we'll discuss is a high-performance "picker" utility for selecting items from a list. The picker interface includes preview and fuzzy search capabilities. If you've used the command menu in many modern editors (or even Github or Slack), you may know what I'm talking about. The picker itself doesn't care what you are picking, and it is used for a bunch of different tasks built into LazyVim or as third-party plugins, including opening files, selecting open buffers, project-wide search, and more.

The most common picker task you will perform is to open a file using fuzzy search. I use this command dozens, maybe hundreds of times per day, so it's a good thing it's got a really accessible keybinding.

The file picker is best illustrated while working in a code repository with a lot of files. So close Neovim with `Space q q` and use the `cd` command in your terminal to change to the directory of a project you've been working on recently (If you don't have one close to hand, clone your favourite open source project and use that instead). Then type `nvim` to open Neovim again.

I had you exit to the terminal above because it's easy to reason about, but it is also possible to change directories from inside LazyVim using the `:cd` command. Type `:cd the/path/to/the/directory` and hit `Enter`, remembering that you can use the `Tab` key to autocomplete the path. Now if you use `:e` to open files, they will be relative to the directory you specified. If you are using a file picker,

they may be relative to that working directory or to the project containing the current file, as discussed shortly. Use `:pwd` to see what the current directory is.

Ok, so you're in the root directory of a large project and you want to open an arbitrary file. Simply press `Space` twice (i.e. `Space Space`) to pop up the "Files In Current Project" picker. As I mentioned, this is the easiest keybinding to type on your entire keyboard. The Space bar on most keyboards is big, and you're hitting it with your strongest digit: the thumb. As usual, just one `Space` will pop up the Space mode menu, and you can see that a second `Space` will present you with "Find Files (Root Dir)".

For the project containing the current state of this book, the picker looks like this:

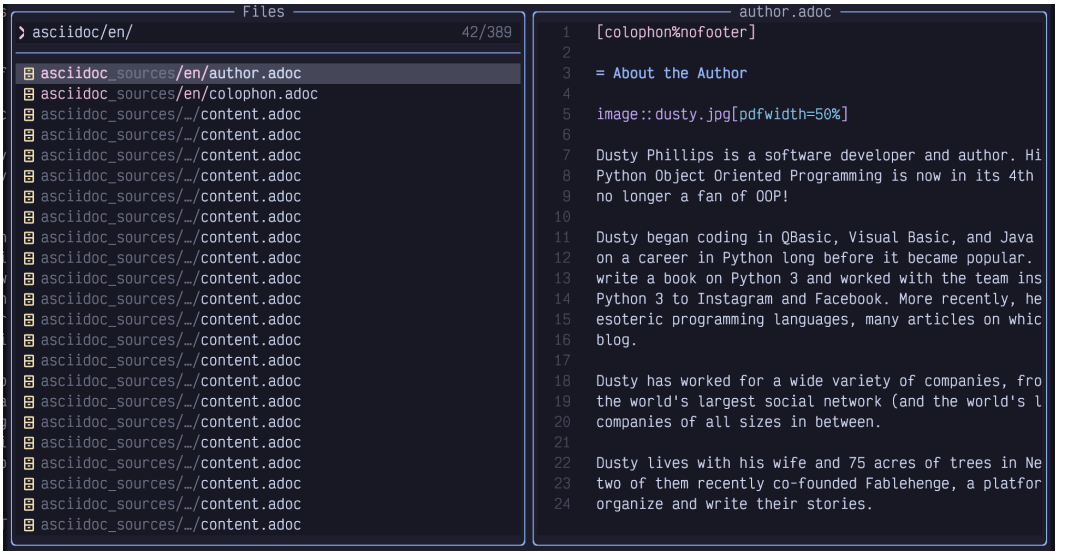


Figure 16. Snacks picker

The picker is divided into three main areas: The input area, in this case labelled "Files" in the upper left, the results list in the lower left, and a preview of the currently selected file on the right.

The input area is actively focused and currently in Insert mode, so you can just start typing the name of whatever file you want to open. This is a "fuzzy search", (a concept popularized by Sublime Text) which means you can skip letters, saving oh-so-precious milliseconds. For example, if I type `ch3`, my list gets filtered down to the following files:

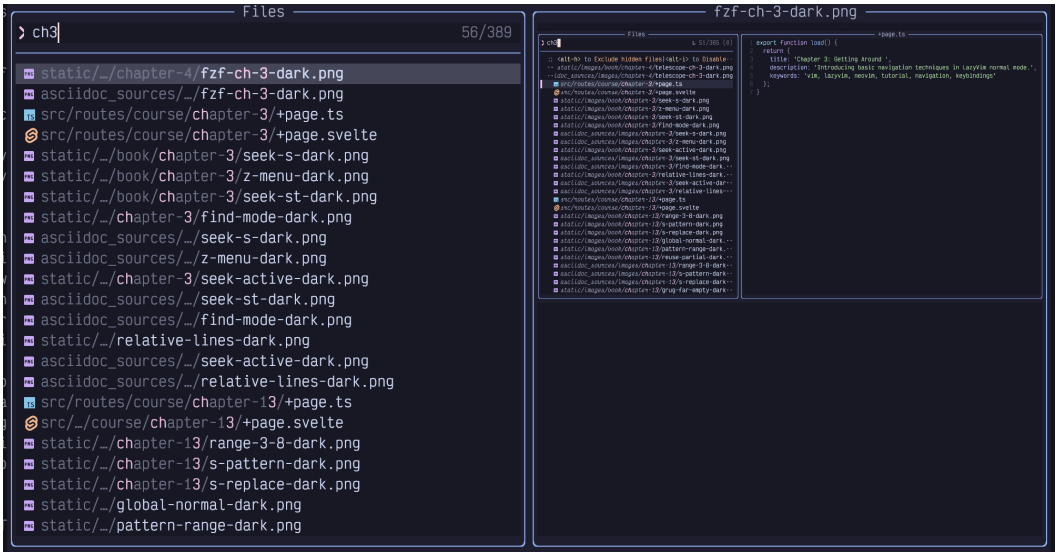


Figure 17. Pick Chapter 3

Only files whose paths contain those three characters in order, with possibly other characters in between, are visible. The picker has helpfully highlighted those three letters in the results so you can easily see why it matched.

Also notice that by default, the match is case **in**sensitive. I typed the lowercase letter `c`, but it matched the uppercase `C` in the filename. This is usually sufficient to narrow the search results to what you need. However, if you *do* use **any** capitalized letters in your search, it switches to a case sensitive mode (this is sometimes referred to as "smart case").

That means that `Ch` will match all the `Chapters`, but `cH` will not match anything. More interesting, `chF` will *also* not match anything at all because the presence of the capitalized `F` makes the whole thing case sensitive, and the chapters are all named with a capital `C`, so the lowercase `c` is not able to match them.

Sometimes you will start typing a word and realize you need to match something *earlier* in the path to distinguish it. For example, I started typing `outline` in these source files from Fablehenge [https://www.fablehenge.com]:

Figure 18. Outline in Picker

"Outline" is a common word in this app. There are 243 matching files, and I realize I should probably have typed `comp` in front to narrow it to just files in the `component` directory. I *could* switch to Normal mode and edit the beginning of the line, but it's faster to just type `<space>comp`. The picker will interpret the space as "filter the lines again, fuzzy matching this new word from the beginning". Here we can see that only `comp…outline` files have been matched:

Figure 19. Narrow picker to "comp"

This image might be surprising; the most promising match is obviously the selected one at the top of the list. The other 35 matching lines contain all the letters of the word "outline" **and** all the letters of the word "comp" in order from left to right. However, because of the fuzzy matching algorithm, the two can actually overlap! So on e.g. the `outline1.png` entry, the `c` of the matching "comp" is in the `src` part of the path, **before** the word `outline`, the `o` is **in** it, and the `m` and `p` both come **after**

the word `outline`. The picker doesn't care, though it will rank matches with the matching letters closer together as more important, so they'll be visible at the top of the results.

You can use the up and down arrow keys to select a different file in the search results, and its preview will show up in the right-hand window. Once you find the file you want to open, press the `Enter` key to open it in the currently active Neovim window.

You can even use a sort of Seek mode, as we discussed in Chapter 3, though it works a bit differently in the picker. Press the `Alt-s` keys while in the picker's input area. You'll see a label show up beside every line in the picker:

Figure 20. Seek Labels

These characters are labels for each line in the picker. Simply press one of the shown letters on your keyboard, and whichever line the label associated with that letter is on will be selected. Then press `Enter` to actually open the file (or, if it is not a file picker, perform the default action for that picker).

If you want to open multiple files from the picker, instead of pressing `Enter`, press `Tab` to **select** the file. Navigate to other lines and press `Tab` to select them as well. Press `Enter` to confirm your selection. We'll discuss other things you can do with a group of selected files later.

If you need to scroll the *results* window to see something lower down in the list, use the `Control-d` and `Control-u` keys. If you want to scroll the **preview** window, use `Control-f`, and `Control-b` instead.

Finally, if you are in the picker window and decide you don't want to open any files after all (or you got the information you needed from looking at the preview), press `Escape` **twice**. Why twice? The first time you press escape will put you into normal mode so you can use all the usual normal mode commands to edit your filter.

4.2. The Difference Between "Root" and "Cwd"

The `<Space><Space>` command is mapped to "Find Files (Root Directory)". Two other ways to open the file picker are to use `<Space>f` to open the "file/find" menu, and follow it with either `f` again or `F`.

`<Space>ff` is the same as `<Space><Space>`. It opens "Find Files (Root Directory)" and is just another longer way to get there. I assume it exists in both places so that users can choose to map some other action to to the super-accessible `<Space><Space>` and still be able to access the picker functionality through `<Space>ff`.

`<Space>fF`, where the second `F` is shifted, is slightly different; it is mapped to an action called "Find Files (cwd)". If you run it in your project, you'll *probably* find that it appears to do the exact same thing as "Find Files (Root Directory)" (depending on how your project is set up), so the purpose of two separate keybindings may be confusing.

4.2.1. Current Working Directory

"Cwd" stands for "Current Working Directory", and by default, it refers to whatever directory your terminal was in when you typed `nvim` to open the editor. You can change the `cwd` for the entire editor by entering Command mode with `:` and then typing `cd path/to/directory` (remember, all commands are followed by a carriage return, so press `Enter` or `Return` afterwards). Now if you use `<Space>fF`, the list of files will be shown relative to the new directory you have changed into.

If you are unsure what directory you are in, you can use the `:pwd` (short for "print working directory") command to have it pop up in a little notification window. `cd` and `pwd` are the same commands used by `bash`, `zsh`, and many other shells for changing and printing the working directory, so they may already be familiar to you.

We haven't discussed splitting your editor or opening new tabs yet, but for future reference: It is actually possible to have *different* working directories for different windows. The command to change just the current window's directory is `:lcd`, short for "local change directory". This can be a powerful way to work on multiple

projects at the same time (for example, if you are a full stack developer working on backend and frontend projects). However, the LazyVim concept of a "Root" directory can semi-automate this.

4.2.2. Root Directory

The root directory is not a Vim concept, but is instead a Language Server Protocol (LSP) concept. LSPs are the reason that VS Code became so popular so quickly; the idea was that the editor could call out to an external service running on your computer to find out useful things about the codebase. The LSP powers a lot of useful stuff such as go to definition and references, highlighting errors in your code, and showing documentation for a variable or class. It can even help with formatting and syntax highlighting.

The root directory is the directory that the LSP infers is the "home" directory of the currently open file. How the LSP does this is language (and language server) dependent. For example, in Javascript or Typescript projects it probably searches parent directories for the presence of a `package.json` or `tsconfig.json` file to detect the root directory. whereas in a Python project it might instead look for things like `pyproject.toml` or `poetry.lock`. and Rust projects use the directory that contains a `Cargo.toml`. Alternatively, some LSPs might just use the presence of a `.git` folder as the "root" of the project's workspace.

The only reason this root directory is "often the same as your `cwd`" is that this is usually the folder you want to work from when you are working on a project, so it's the one you `cd` into before you open Neovim.

This automatic root directory thing can be super useful if you are working on multiple projects. Instead of using `lcd` as discussed in the previous section, you can just open a file in a different project using `:e` or one of the file finding extensions we'll discuss next. Then if you invoke the "Find files (root dir)" command using `<Space><Space>` or `<Space>ff`, it will look for other files in the same root directory as the one you just opened.

However, it can sometimes be confusing, especially if you are working in a monorepo or if you have root directories in places you don't expect. For example, I have a fairly normal Svelte project that has a `package.json` file in it. This project uses Cypress for testing, and the Cypress folder contains a `tsconfig.json` file that causes the Typescript language server to interpret that as a separate root. So if I am working on one of the cypress test files and press `<Space><Space>`, the root directory is considered the Cypress folder and I can only open other Cypress tests. But often the thing I *wanted* to do was open a source file in the main folder to see why a test is failing. In this case, I have to press `<Escape>` to exit the picker, then

`<Space>fF` to open the picker in current working directory mode instead.

The snacks picker is inspired by the command line tool fzf [https://github.com/junegunn/fzf] which means "fuzzy find". This tool allows you to quickly access files and open directories from your shell, and I highly recommend it.

4.3. The Snacks Explorer Plugin

Snacks.nvim also has an "explorer" a left-sidebar file explorer experience that should be familiar to users of many modern IDEs and editors. While, like many of those environments, the explore works with the mouse, it is optimized for keyboard interactions, making it faster to work with once you learn “Explorer mode”.

I want to be upfront and honest here: I don’t personally use the Explorer. I find that the file pickers we just discussed are the fastest way to open files, and when I need to manipulate the filesystem, I prefer to use `mini.files`, which we will discuss later in this chapter. The primary reason I prefer `mini.files` is that it uses the same keybindings as Vim Normal mode instead of having a custom "explorer mode" that I have to memorize. Modes are great, but having more of them than necessary is not!

However, I suspect that many readers will prefer the familiar tree view experience the explorer provides, and since this plugin ships with LazyVim by default, I want to make sure it gets fair coverage in this book.

Let’s start by opening an explorer using the `<Space>-e` keybinding, where the mnemonic is “**e** for Explore”. If you pop up the Space mode menu, you’ll see that, as with the picker, there are two ways to open the explorer: `<Space>-e` for `Explore Snacks (root directory)` and `<Space>-E` for `Explore Snacks (cwd)`.

“Root directory” and “cwd” have the same meanings we discussed in the previous section, and you will notice the consistent relationship between lowercase and uppercase letters: `<Space>ff` and `<Space>e` both open the root directory, and `<Space>fF` and `<Space>E` both open the current working directory.

To hide the explorer window, just press `<Space>e` again while it is visible, or press `q` or `Escape` while it is focused.

When the explorer is opened, it shows all the files and folders in the relevant

directory, with all the folders collapsed, except for the one containing the currently active file, if there is one. For example, while editing this file, my explorer looks as follows:

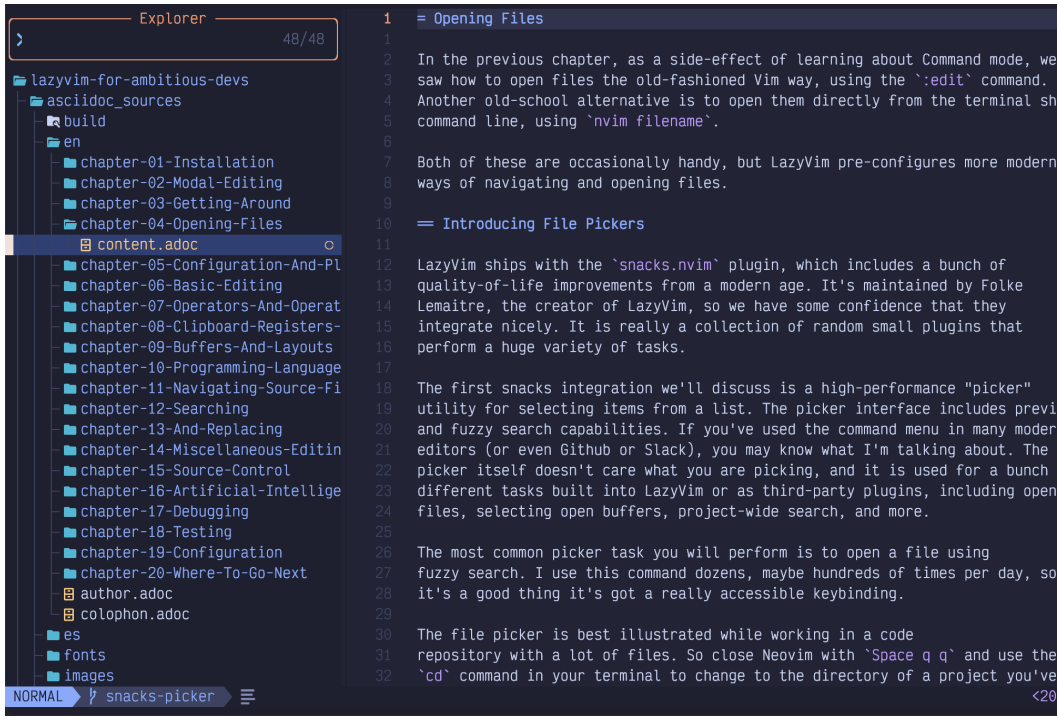


Figure 21. Explorer

The cursor is on the file I'm currently editing. I can move that cursor up and down using the ubiquitous `j` and `k` keys.

Folders are collected to the top of the view. If you move the cursor to one of these folders, you can press the `Enter` key to see the files inside the folder. And if you move it to a file, you can open the file in the current Vim window with `Enter` as well.

You can also select multiple files to manipulate using `Tab`, similar to the picker window (In fact, the explorer is just a fancy picker window in disguise).

You can also expand and collapse folders and open files by double clicking with the mouse, but my guess is you won't want to do that once you learn proper keyboard navigation.

Speaking of keyboard navigation, yes, `j` and `k` to move up and down can be super slow if there are a lot of files to navigate. All of the commands that we discussed in Chapter 3 can be used to move faster. For example, `10j` will move the cursor 10 lines down with just three keystrokes compared to pressing `j` 10 times, and `Control-d`

or `Control-u` can be used to scroll the tree down or up.

Use `i` to enter Insert mode while the explorer is focused to search for a specific file. Since this is a picker under the hood, `Alt-s` can be used to Seek to any line in the picker view. You can also use the normal mode `s` command to seek to text in any window, including the explorer.

The explorer will show either the root or cwd as the topmost directory. If you need to navigate "up" the tree to a higher-level directory, you will need to use the `Backspace` key.

Backspace is often coded as `<BS>` in Vim, so if you see a keybinding or instructions telling you that `<BS>` does something, they aren't full of (bull)! It just means Backspace.

In addition to navigating and opening files, you can even make changes to the file system using the explorer. For example, to delete a file, you can move the cursor over that file and hit the `d` key. You'll be prompted with a popup window asking if you are sure. Hit `y` and then `Enter` to confirm it:

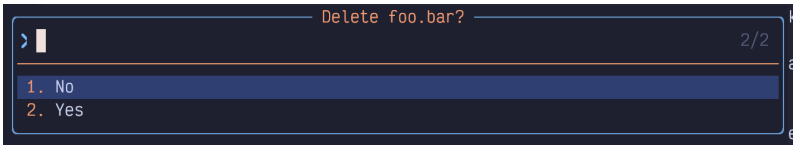


Figure 22. Delete Confirmation in Explorer

To add a file or folder/directory, use the `a` key and enter a new name. Use a trailing slash (`/`) to indicate a folder.

The `r` key can be used to rename the file or folder under the cursor.

To copy or move a file, you can use the explorer's pseudo-clipboard. I say "pseudo-" because you can't use this to copy a file to be pasted in e.g. MacOS Finder or Windows Explorer; only to other places in the explorer.

If you want to *copy* the file, use `y`. The mnemonic for `y` is `yank`, and is actually the same key you would use to copy text in the normal editor. To complete the copy, you'll need to navigate to the destination folder and use the `p` key (which you may recall means "put" or "paste").

Use the `m` keybinding to move a file to a new location or name.

There is a *ton* of other cool stuff that the explorer can do. In the meantime, you can use the `?` (mnemonic "ask question for help") key while the explorer window is focused to get an overview.

4.4. The Mini.files Alternative

As I mentioned, I don't actually use the explorer for file navigation. I find that it feels kind of "foreign and un-vim-like". To me, it is a completely separate experience that just happens to be embedded in a Neovim window. That said, I *also* don't like the tree view sidebar experience in VS Code and the editors it emulates / is emulated by, so it's possible that tree views just aren't right for me.

These are just **my** opinions, and one of the golden rules of text editors is "all opinions are valid" (otherwise there would be war). A large number of Neovim users love the explorer, and you should use it if it matches your mental model.

That said, I'm clearly not alone in these opinions, because LazyVim optionally provides a different file management experience with a plugin called mini.files. It is disabled by default.

Mini.files is part of a suite of fairly random Neovim packages known as mini.nvim. These plugins are independent from each other and provide a lot of common features that in many cases ought to ship with Neovim. Occasionally, the mini.nvim plugins are inferior to other plugins that they clone, but many are best in class. Mini.files is not the only mini.nvim plugin that ships with LazyVim, and we'll touch on others later.

The mini.files file manager is kind of like a Neovim-native experience of the columnar view that is popular in MacOs finder, among other file managers. The main reason I like it is that editing the directory listing is just like editing a normal text buffer. I don't have to remember that `a` means "after" in Normal mode, but it means "add file/folder" in Explorer mode. Instead, in mini.files, I use the `o` key to "create a new line below the current line", and then enter a new file name in Neovim Insert mode. Later, I tell mini.files to sync my changes and it will create the file for the new row.

In order to use mini.files, you have to enable it as a Lazy Extra. We'll go into this in more detail in the next chapter, but for now, these steps should be sufficient:

- Type `:LazyExtras<Enter>`

- Move your cursor to the line that contains mini.files (Seek mode is fastest)
- Press `x` to install the e**X**tra
- Wait a moment for the plugins to install
- Restart Neovim

4.4.1. Using Mini.files

Once installed, you can show the mini.files view using `<Space>fm` and `<Space>fM`. By default, these are *not* quite the same as the `cwd/root` structure we've seen in the picker and explorer. Instead, they are listed in the `<Space>f` menu as follows:

```
m -> Open mini.files (Directory of Current File)
M -> Open mini.files (cwd)
```

Listing 11. Mini.files keybindings

The default mini.files configuration doesn't have an open in root option. I like having the ability to open the directory of the currently open file, but I don't like *losing* the ability to open the root of the current project. I show how to address this in Chapter 5.

Instead of a sidebar, the mini.files menu shows up as columns of windows (known as Miller columns) side-by-side. For example, here's what happens when I open mini.files to the current working directory of this book:

Figure 23. Mini.files

The left-hand panel shows the current working directory, and the middle column shows the contents of the `book` directory, where my cursor is currently on chapter 4. The right column shows the preview of that chapter 4 directory, which contains only one file.

Interacting with mini.files is *very* similar to interacting with a standard vim window. You can use the `j` and `k` keys to move the cursor up and down. If this places your cursor over a folder, the contents of that folder will immediately show up to the right, and if it is over a file, you will see a preview of the file.

If you want to move "into" a folder to interact with the contents of that folder instead, simply press the `l` key to move "right".

Similarly, pressing `h` will move "out" of the current folder. If the cursor is in the left-most column, moving left will open a new left-most column, so you can navigate right up to the root of your file-system if you need to.

To open a file in the currently active Neovim window, press `l` on that file again. The behaviour here may be a bit surprising; the file will open *under* the mini.files view, but it won't hide the file menu. This allows you to open multiple files before closing the navigator, which can be done with the `q` key.

The beautiful thing about mini.files compared to the explorer is that the little windows act like normal editors, and all the navigation features you have become used to are available. For example creating a file or folder is done with the `o` command, which is the same command to open a new line in a normal editor.

We haven't really covered editing yet (I'm just as surprised as you are), but here's a quick overview:

- To rename a file or folder, navigate to the line that has it, and enter Insert mode to change or add text.
- Deleting a file or folder uses the command `dd` which is the keybinding to delete an entire line of text in normal Neovim windows.
- Copy a file or folder with `yy`, the command to copy ("**y**ank") a line of text.
- Put/paste a deleted or yanked file with `p`.

We'll discuss these commands and more in Chapter 6. The main point is that pretty much any navigation or editing command you learn in the future will work with mini.files.

Saving Filesystem Changes

Any modification that you make using these keybindings will not actually be saved on the filesystem until you type the `=` key, which is a (rare) mini.files specific keybinding. I think of it as meaning "make the filesystem **equal** to what I've typed". This will pop up a little window telling you what actions mini.files wants to take on your behalf, such as deleting, moving, renaming, or copying files. You can confirm or decline the changes with a `y` or `n` (**y**es or **n**o, of course).

I encourage you to play with both the Snacks explorer and mini.files until you can make a decision as to which of the two you prefer. Eventually, you will arrive at one of the following conclusions:

- You prefer the explorer and don't need mini.files. In this case, revisit the LazyExtras mode and disable mini.files with the `x` key.
- You use explorer for some interactions (possibly things we haven't covered yet, such as navigating git, buffers, or symbols) and mini.files for others. In this case, you are probably content with the default LazyVim configuration of the mini.files extra.
- You are *my kind of weird* and don't want to use the explorer at all, preferring only mini.files. Disabling plugins is discussed in the next chapter.

4.5. Summary

In this chapter, we learned not one, but three different ways to open files and interact with the filesystem in LazyVim: the Snacks picker, explorer, and mini.files. Each provides a different mechanism for opening and managing files, and you will find some of them more comfortable than others.

As a side-effect of studying these filesystem tools, we got a tiny preview of configuring plugins and installing LazyVim extras. We will go into more detail on this in the next chapter.

Chapter 5. Configuration and Plugin Basics

I've talked about plugins several times and you even got to see the Lazy.nvim plugin manager in action back in Chapter 1. LazyVim has a unique multi-layered approach to managing plugins that requires a bit of description, but is quite elegant in practice.

Installing plugins allows you to configure Neovim to do things it can't do by default. Plugins are typically written in either Lua or VimScript, though other languages are supported with Neovim's remote plugin architecture.

5.1. The Three Categories of Plugins in LazyVim

The simplest plugins to use in LazyVim are pre-installed by LazyVim itself. You've used many of them already. Some, such as Snacks.nvim's picker and explorer windows, and Lazy.nvim provide custom UI components to interact with them. Others, such as flash.nvim and which-key provide new commands or modes to work with. Still others operate quietly in the background auto-matching parentheses or tags and drawing indent guides.

These plugins are preconfigured in LazyVim with (generally) sane defaults. Because they are so well integrated, customizing those defaults is doable, but sometimes requires a few tricks that we will cover in this and later chapters.

The second category of plugin in LazyVim are the "Lazy Extras". These plugins are **not** enabled by default, but can be enabled with just a couple of keystrokes if you want them. Lazy Extras exist to make it easy to install popular plugins with a configuration that is guaranteed to play nicely with the other plugins that ship with LazyVim.

The third category includes third-party plugins that LazyVim has no awareness of. You will have to configure these plugins from scratch and do your own due diligence to ensure that keybindings and visual artifacts don't conflict with the plugins that LazyVim manages. In a non-LazyVim configuration, all plugins fell in this category, and it could be a headache to maintain as plugins evolved and fell out of use over time. In LazyVim, this category includes relatively few plugins, so the whole experience is much more pleasant.

As some specific examples consider these three Neovim plugins for file management, two of which we discussed in the previous chapter:

Snacks explorer

ships with LazyVim and is active by default. The LazyVim configuration for the Explorer does not conflict with other LazyVim plugins by default. However, if you want to tweak that configuration, there may be some hoops to jump through.

Mini.files

ships as a Lazy Extra, and is basically a "one click" (or, since this is Vim we're talking about, one keypress!) install that is expected to cooperate well with LazyVim.

Oil.nvim

is an alternative plugin for filesystem management that LazyVim does not explicitly support. You can install it in LazyVim with a few lines of configuration, but it's not quite as easy to set up as mini.files and there is no guarantee it won't have command or keybinding conflicts you need to sort out yourself.

From Neovim's point of view, all these plugins are exactly the same, as Neovim only knows about third-party plugins. LazyVim just comes with a bit of extra structure that you need to think about when using plugins. Usually this structure simplifies things, but occasionally it adds extra hassle.

5.2. Lazy Extras

In the previous chapter, I covered how to enable and use `mini.files`, but I was pretty terse on the installation instructions. Now we'll get to dive deep.

The Lazy Extras mode can be accessed by pressing `x` from the dashboard. If you aren't on the dashboard, you'll need to enter Command mode with `:` and type `LazyExtras` followed by the usual `Enter` to confirm a command (Incidentally, you can also show the dashboard at any time by typing the command `:lua Snacks.dashboard()` or binding that to a keypress).

Either way, you'll be presented with a list of possible plugins to install. On my setup this looks as follows:

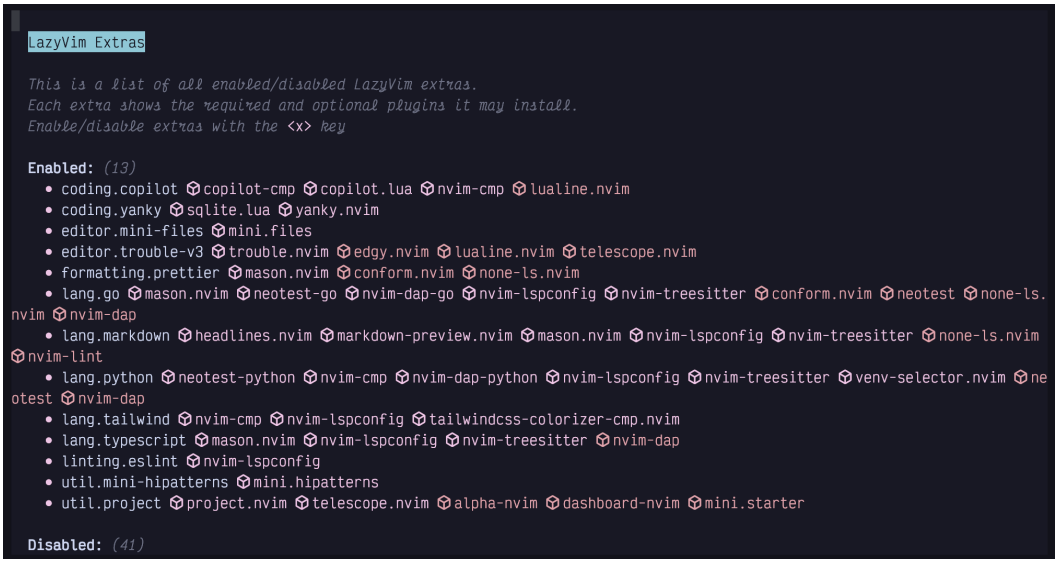


Figure 24. Lazy Extras

I've installed over a dozen extras at the moment, mostly for the various programming languages I dabble in. You can navigate this file using all the standard navigation commands such as `j`, `k`, or `s`.

No matter how you get there, once your cursor is on the extra you want to install (such as `editor.mini-files`) line, just hit the `x` key to install the extra. If you want to uninstall it, do the same thing; move to the appropriate line (now under the list of `Enabled` extras), and hit `x` to disable the extra. The mnemonic here is that `x` means "E**x**tra".

You may need to quit and restart Neovim for Lazy.nvim to pick up that the extra has been installed and sync its dependencies.

While we're in the `LazyExtras` screen, I recommend enabling the `lang.*` extras for whichever programming languages you use most frequently. You should also install all the plugins in the "Recommended plugins" section (they have star icons beside them).

I wouldn't install any other non-recommended extras until you've either encountered them later in this book or had a chance to research them after you finish the book. Otherwise, they may change behaviours in ways that I won't have the foresight to write about.

You can find more information on each extra by visiting https://lazyvim.org and clicking the "Extras" menu item on the left menu bar. It includes links to the list of plugins each extra installs as well as the configuration LazyVim brings for that extra.

5.3. Disabling a Built-in Plugin

Sooner or later, you're going to want to edit your LazyVim configuration. The out-of-the-box defaults are wonderful, but the odds are that they don't 100% exactly match your personal needs.

While the vast majority of LazyVim's default plugins are no-brainers that you want to keep, you may find there are one or two plugins that you just don't need. In most cases, it doesn't matter, since LazyVim only loads plugins when you actually use them, so you can just ignore the ones that aren't relevant to you.

The only LazyVim plugin I have disabled is the bufferline. I'll show how to do that and you can adapt it to any other built-in plugins you want to disable.

First I want to give an introduction to the LazyVim configuration directory. You can open the config directory from the dashboard by simply pressing the `c` key. Or you can use Space mode to access the configuration files at any time using `<Space>fc` for "Find Config File".

This will load the LazyVim config folder in your file picker. This folder is typically `$HOME/.config/nvim`. Neovim loads `$HOME/.config/nvim/init.lua` by default, and if you weren't using LazyVim, this is where you would do all your configuration.

With LazyVim, `init.lua` just uses the Lua `require` statement to include the LazyVim configuration infrastructure. **You will normally not have to touch this file**, even though most third party plugins have installation files that presume your configuration is in that file. Instead follow the "LazyVim way" as outlined in this chapter.

In addition to a barebones `init.lua`, LazyVim has put a few configuration files and a bit of folder structure in the configuration directory.

For now, the main thing we need to know is that *any* Lua files inside the `lua/plugins` subdirectory will automatically be loaded by LazyVim, no matter what their name is. I have a number of different files in this folder for my custom configurations.

I call the one that holds my disabled plugins `disabled.lua`. The easiest way to create this file is to open one of the existing config files and use either the explorer or mini.files to create a new file in the same folder, as described in Chapter 4.

When I created my `disabled.lua` file in the `lua/plugins` directory, my intention

was to collect all the LazyVim plugins I don't want in it because I assumed LazyVim wouldn't quite match my needs. In reality, it's a really short list! The contents of this file is simply:

```
return {
  { "akinsho/bufferline.nvim", enabled = false },
}
```

Listing 12. Disabling LazyVim Plugins

If there are any other plugins that LazyVim enables by default that you don't want to use, just follow the same syntax. The first argument in each Lua table is a string containing the github repo (with owner) you want to disable. The second argument is to set `enabled = false`. That's it!

You will inevitably forget the `return` statement at the beginning of a plugins file at some point. Now you know to watch out for it.

If you don't know the Lua language... honestly, don't worry about it. I've never formally studied it, but I've picked up enough by osmosis to easily maintain my Neovim configuration.

If you're less foolish than me, you might want to type `:help lua` and read the official Neovim docs on the topic. Then check out `:help lua-guide-api` to learn about the vim-specific APIs.

5.4. Modifying Keybindings (Example)

Keybindings are one of the few things I don't love about working with LazyVim, although it's not strictly LazyVim's fault. I just never quite know **where** to define them!

There are three possible places to configure keybindings, depending on how any one plugin is configured:

- In `.config/nvim/lua/config/keymaps.lua`. This is typically where you configure or modify keybindings that are not specific to plugins, but rather modify core Neovim or LazyVim functionality.
- In the `keys` field of the Lua table (in Lua, a "table" is like a combination of an array and a record or dict in many other dynamic languages) passed to a plugin.

This is typically where you map global Normal mode keybindings to set up a plugin. This is what we will do with mini.files.

- In the `opts` (options) argument passed into a plugin's configuration. The format of the options for any one plugin are plugin-specific, but many plugins prefer to set up keymaps on your behalf through options instead of having you do the mapping yourself. This is especially true if the keymaps define a different "mode" or only apply if the plugin is currently open or active. I'll give an example of this with mini.files as well.

To demonstrate, I want to "fix" the fact that mini files doesn't have a "open in root" option. I like the "open in directory of current file" option, but I also want to be able to open in the root directory.

Remember that the root directory is the top level directory of the current project according to the existence of some language-specific file such as `package.json` or `Cargo.toml`. The `cwd` is the current working directory of the editor.

Since I don't use the explorer, I'm going to steal the `<Space>e` and `<Space>E` keybindings and use them for mini.files instead, then I'll remap the existing `<Space>fm` keybinding to open the root so I can access all three commands. You can, of course, choose different keybindings if they map better to your mental model or you like the explorer for some things.

I used mini.files to create a new file named `extend-mini-files.lua` in my `.config/nvim/lua/plugins/` directory. As with the `disabled.lua` file, this file can be named anything so long as it's in the `plugins` directory.

I have a habit of prefixing any configuration that I am using to change the defaults provided by LazyVim with the word `extend`. This makes it easy to distinguish it from non-LazyVim plugins I've installed when I'm listing the directory using mini.files or a picker.

Inside this new file, I used this code:

```
return {
  "echasnovski/mini.files",
  keys = {
    {
      "<leader>e",
      function()
        require("mini.files").open(vim.api.nvim_buf_get_name(
0), true)
      end,
      desc = "Open mini.files (directory of current file)",
    },
    {
      "<leader>E",
      function()
        require("mini.files").open(vim.uv.cwd(), true)
      end,
      desc = "Open mini.files (cwd)",
    },
    {
      "<leader>fm",
      function()
        require("mini.files").open(LazyVim.root(), true)
      end,
      desc = "Open mini.files (root)",
    },
  },
}
```

Listing 13. Mini.files Custom Keymaps

I constructed this by pulling relevant function calls from the default configuration for the Snacks.nvim configuration conveniently provided on the LazyVim website.

To satisfy the contract with Lazy.nvim, we need to return a Lua table, wrapped in curly braces. Lua tables can act like an array and a dictionary at the same time. In this case, the first element in the table is the string `"echasnovski/mini.files"`. It doesn't have a named key, so it's kind of like a "positional argument".

The second element in the table is more like a "named argument" in that it is indexed with the name `keys`, and the value is another Lua table. However, that second table acts more like an "array" of three values (three more separate Lua tables) because it doesn't have named indices.

It is important to understand that the `keys` field is **merged** with the keys that are provided by the default LazyVim (extras) configuration for mini.files. If there are conflicts (such as with `<space>fm`), my values take precedence over the defaults.

This is a powerful feature of LazyVim that allows you to use hosted configuration provided by LazyVim but override it as needed. Older Neovim distros tended not to have this much flexibility, so you were either stuck with their configuration or had to copy the whole thing and edit it, which made updates a nightmare.

To be clear, `keys` is a LazyVim concept (technically, it's actually part of the underlying Lazy.nvim plugin manager). Any plugin configuration can have a `keys` array table, and those keybindings will be merged with the default Neovim keybindings, the LazyVim keybindings, your custom global keybindings, AND any other plugin keybindings.

Yes, that's a lot of potential for conflicts, which is why I'm so glad LazyVim has done most of the configuration for me!

5.4.1. Structure of a Keys Entry

Each item in the `keys` table is another Lua table with (in this case) three fields. The first two fields are positional and represent the keybinding name and the Lua callback function that gets called whenever that keybinding is invoked. The third field is a named field, `desc` and provides a string description that will be shown in the Space mode menu.

The keybinding sequence in the first entry is using a standard syntax that comes from Vim. Note that `<leader>` is an old Vim concept that allows you to configure which key is used as the prefix for custom keybindings. In LazyVim (and indeed, for most modern Neovim users), the leader is `<Space>`. Special keys are indicated to Vim's keybinding engine using angle brackets, so you will often see notations such as `<Space>`, `<Right>`, `<Left>` or `<BS>`.

After the `<leader>` string, we include any additional keys that need to be pressed. For the simple ones, we have `e` and `E` to replace the explorer keybindings we disabled with new mini.files keybindings. The third one is a bit more complicated, as the `f` indicates that this action will be available under the `file/find` submenu in Space mode, and the `m` indicates which letter will be in this menu.

For the callbacks, we use Lua functions, which always start with `function` and end with `end`. These are anonymous (unnamed) functions, and they don't accept any parameters inside the parentheses. In the function bodies, we call specific code to

open mini.files the way we want. In two cases I just copied this code from LazyVim's default mini.files configuration, and in the third, I cobbled it together by combining code from the Snacks.nvim and mini.files configurations. The LazyVim global is a handy library with a collection of utility functions to aid with configuration. The `LazyVim.root` function is used to find the root of a project and returns a string that we pass to `mini.files.open`.

5.4.2. Customizing Mini.files Options

As I mentioned, the `keys` table is merged with the default `keys` table that LazyVim has configured for mini.files. Similarly, most Neovim plugins can be configured with an `opts` table that contains custom configuration specific to that plugin. If you supply an opts `table`, it will be *merged* with the default LazyVim one (if there is one).

You'll need to read each plugin's documentation (often available on Github, and usually available with `:help plugin-name`) to know exactly what options are available for it. You'll also need to review the default configuration that LazyVim sets up for that plugin so you understand how it will merge.

In my case, I pass the following `opts` array to mini.files:

```
return {
  "echasnovski/mini.files",
  keys = {
    -- the keybindings from above
  },
  opts = {
    mappings = {
      go_in = "<Right>",
      go_out = "<Left>",
    },
    windows = {
      width_nofocus = 20,
      width_focus = 50,
      width_preview = 100,
    },
    options = {
      use_as_default_explorer = true,
    },
  },
}
```

Listing 14. Mini.files Custom Opts

The mappings table in mini.files is used to override the default keymappings that are active *while* the mini.files view is open. This is different from the *global* keymaps we defined earlier to open mini.files. In my case, I have mapped `go_in` and `go_out` to use the arrow keys instead of `h` and `l` because of the left-handed Dvorak Kinesis weirdness I described previously. I don't recommend you make this change; `h` and `l` will work better for most anybody who isn't me.

The window options are there because I have a 32-inch 6k monitor, which means I can afford to have larger-than-normal explorer columns. Refer to `:help mini.files` for more information on these and other options.

So now you know a little bit about configuring plugins in LazyVim. It is both a little bit easier and a little bit harder than configuring plugins from scratch:

- It is easier because you only need to change the values that are non-default, instead of setting up an entire configuration, and LazyVim comes with sane defaults.
- But it is harder because you sometimes have to think about how the option and

keybinding merging happens, which wouldn't be necessary if you just had one great big configuration object to begin with. This merging can get quite tricky for plugins that have complicated default LazyVim configurations. We'll see some examples later.

5.5. Modifying Existing Options

Sometimes the "merging" behaviour LazyVim uses to overwrite options with the ones you provide in your plugin overrides is too simplistic. This most often happens when you are modifying a plugin that calls or defines a function for options behaviour instead of customizing it.

To support this situation, the `opts` entry in a Lazy.nvim plugin's configuration table can be a function instead of a static table. The function accepts the previous `opts` table as it was configured by LazyVim as an argument. Your function needs to *modify* this table to suit your desired behaviour.

The function based version of `opts` **does not** return a new `opts` table; it needs to **modify** the one that was passed in.

For example, the default configuration for the LazyVim dashboard, which is configured as part of the `snacks.nvim` suite of plugins is a table that contains several entries in the keys array. The dashboard already allows you to restore the most recent session using the `s` key, but if you want to select from a list of recent sessions, you have to use the `<Space>qs` keybinding. (Sessions are discussed further in Chapter 9).

Let's say you want to add a "select session" entry to the Dashboard. You can use the following structure to **modify** the existing keys array, as configured by LazyVim and add a new entry:

```
return {
  "snacks.nvim",
  opts = function(_, opts)
    table.insert(
      opts.dashboard.preset.keys,
      7,
      { icon = "S", key = "S", desc = "Select Session", action
= require("persistence").select }
    )
  end,
}
```

Listing 15. `snacks dashboard` *Options Function*

You may want to replace the icon with something from the nerd fonts suite.

This `opts` function accepts the LazyVim-defined `opts` table as its second parameter. My code changes those `opts` using the `table.insert` function provided by Neovim. I add a new entry that is positioned at index 7 in the list, just after the existing `Restore Session` entry.

This is harder to maintain than if I just had the whole configuration the way I wanted it in the first place, but easier to maintain than if I had to write that entire configuration from scratch. I am willing to accept that tradeoff for all the places that LazyVim configures things better than I would have done on my own.

5.6. Installing Third-Party Plugins

Installing a third-party plugin is little different from configuring a LazyVim provided plugin, except that you don't have to worry about how the keys and opts are merged with a default config.

Simply create a new Lua file in the `plugins` directory (named appropriately for the plugin). Inside the file, return a Lua table where the first entry is the GitHub repo and name of the plugin, with other configuration (`opts` and `keys`, among others) after that name.

For example, I like the guess-indent.nvim plugin to set my shift width based on the contents of the file I am currently editing. It is maintained by the github user

`nmac427`, so my `plugins/guess-indent.lua` file looks like this:

```
return {
  "nmac427/guess-indent.nvim",
  opts = {
    auto_cmd = true,
    override_editorconfig = true
  },
}
```

Listing 16. Guess-indent.nvim Third Party Plugin

The `opts` table depends entirely on what the plugin expects. In this case, I read the guess-indent.nvim README and found two options that I wanted to set.

Most modern Lua plugins will be documented as having to call a `setup` function with a Lua table containing the configuration. If the plugin you are trying to set up does not have explicit Lazy.nvim instructions, don't worry: Whatever gets passed into that `setup` function is what you need to include in the `opts` passed to the LazyVim plugin manager.

For example, another third-party plugin I recommend is `chrisgrieser/nvim-spider`, which subtly changes the `w`, `e`, and `b` commands to support navigating within CamelCase and snake_case words. I have a file named `nvim-spider.lua` in my `plugins` directory as follows:

```lua
return {
  "chrisgrieser/nvim-spider",
  opts = {},
  keys = {
    {
      "w",
      "<cmd>lua require('spider').motion('w')<CR>",
      mode = { "n", "o", "x" },
      desc = "Move to start of next of word",
    },
    {
      "e",
      "<cmd>lua require('spider').motion('e')<CR>",
      mode = { "n", "o", "x" },
      desc = "Move to end of word",
    },
    {
      "b",
      "<cmd>lua require('spider').motion('b')<CR>",
      mode = { "n", "o", "x" },
      desc = "Move to start of previous word",
    },
  },
}
```

Listing 17. `nvim-spider` *Third Party Plugin*

This plugin doesn't automatically set up keybindings, so I pass a `keys =` table to the plugin configuration. This array is **not** passed to the plugin. Rather, the keys are parsed by the Lazy.nvim plugin manager and added to the global keybindings. It is convenient to keep the keys with the plugin so all the configuration is in one place.

I am satisfied with the default options that `nvim-spider` passes to its `setup` function (after reading the README), so I pass an empty `opts` array.

The best resource for finding third-party plugins is the github repository rockerBOO/awesome-neovim [https://github.com/rockerBOO/awesome-neovim]. The list is well-maintained and (most importantly) pruned regularly, so there are few outdated or unmaintained plugins on the list.

In practice, LazyVim already ships with the best-in-class versions of most plugins (built-in or as extras), so you won't have to add too many. But if you come across any

"I wish LazyVim could..." scenarios, the answer is probably "it does and the plugin to do it is listed in the Awesome Neovim repo".

5.7. Summary

In this chapter, we learned about how LazyVim integrates with the wider Neovim plugin ecosystem. It provides sane default plugins and configuration, but makes it easy to customize that configuration for your own needs.

We learned that built-in, extras, and unknown third party plugins are all treated slightly differently (though consistently), and saw examples of how to install some third-party plugins.

Now that you know how to open files and configure plugins, we can get back to some of the nuts and bolts of modal editing. You already know how to switch between Normal and Insert mode and you can navigate around your code. In the next chapter, we'll cover some basic editing features that blur the line between navigating and inserting text.

Chapter 6. Basic Editing

Armed with the navigation keybindings you've already learned and the ability to enter and leave Insert mode at will, your Vim editing experience is getting pretty close to on par with what you might be used to in non-modal editors.

However, moving around and inserting text is a very small part of the life of a software developer. More often, you need to *edit* text. Deleting code, changing code, refactoring code, moving code around. It's the majority of what we do.

Yes, you can do all of these things by navigating to where you want to, and entering Insert mode. The delete and backspace keys do the same thing in Insert mode that they do in other editors. But there are far more efficient tools available.

The best part is that you already know most of what you need to take advantage of very powerful editing commands!

6.1. The Vim Command Mental Model

The navigation commands such as `s` and `f` and `hjkl` and `web` that you already know are collectively known as *motion* commands. They move the cursor from its current location to a new location.

Most motion commands can be prefixed with a count, so the navigation model is always `<count><motion>`. A count usually repeats the motion a certain number of times, although some commands such as `Shift-G` for "Go to line" will use the count as an absolute value instead. If the count is blank, the "default" count is typically `1`. Even a Seek command which uses labels is allowed to be prefixed with a count... although the count will be ignored!

The `<count><motion>` commands are great for navigation, which is all we've used them for so far, but they can also be combined with a *verb* to do something to the text between the cursor and the location the motion would move you to.

Verbs come first, so the structure is always `<verb><count><motion>`. Navigation is the "default" verb, so if you leave the verb blank (i.e. skip it), your cursor moves to the location indicated by the motion. We'll discuss various important verbs in this chapter.

But the model keeps growing! It turns out, verbs can *also* be counted. The syntax becomes `<count><verb><count><motion>`. I have never in my life used all four of

those in one command, however. Typically you would either do `<count><verb><motion>` **or** `<verb><count><motion>`.

This model is nice because it allows you to divide and conquer your learning strategy, and reuse knowledge as you study. First you learned motion commands. Then you learned counts. Now you will learn verbs. If you learn new motion commands or new verbs in the future, you can mix them with all the verbs and motions you already know and they should behave in a predictable way.

Various plugins try to mimic this strategy, and, well, most are successful. My main complaint with the exploer is that it doesn't operate with the `<verb><motion>` mental model, while mini.files does. Similarly, some folks argue that Seek mode violates the Vim mental model because counts don't make sense with it. My opinion is that Seek mode simply transcends counts, but it still combines cleanly with verbs so it is a valid Vim model.

6.1.1. A Note on Insert Mode

Like all models, this one is not perfect. For example, you can use counts with the `i`, `I`, `a`, and `A` commands, but it's clear that "enter Insert mode" is neither a motion nor a verb.

For example, if you type `5ifoo<Escape>`, Neovim will insert `foofoofoofoofoo` for you. That may not seem very useful, but if you ever want an 80 character `*` ruler to underline a heading, `80i*<Escape>` is pretty nifty!

But the `<count>i` "not-motion" commands *cannot* be combined with verbs like the navigation commands you've learned. So it's important to know the limits of the model.

So now that you understand how the motions you already know can combine with verbs to perform actions other than navigation, you just need to learn some verbs.

6.2. Deleting Text

I've previewed this a couple times already, and even if I hadn't, you can probably guess that the verb for deleting text is `d`.

So where `<motion>` will take you to a specific location in the code, `d<motion>` will delete all the text between the cursor and that location. Here are some examples:

- `dh` to delete the character to the left of the cursor.
- `d3w` to delete three words.
- `3dw` to delete one word, three separate times.
- `d^` to delete from the cursor to the beginning of the line.
- `d2fe` to delete all text between the cursor location and the second `e` after the cursor, including that second `e`.
- `d2Ta` to delete all text between the cursor and the second `a` *behind* the cursor, *not* including that second `a`.
- `dsfoos` to delete text between the current cursor position and the label `s` that pops up when you use Seek mode to seek to `foo`. Note that Seek mode **always** jumps to the beginning of the word you searched for. This means that if the `foo` you jump to is *after* the current cursor location, the `oo` will not be deleted, but the `f` will. But if the `foo` you jump to is *before* the current cursor location, all three letters of `foo` will be deleted.

If any of those are surprising, ignore the `d` and refer back to chapter 3 to refresh your memory of the motions.

So `d` will work with all the motion commands you know, as well as all the motion commands you don't know yet, **and** all the motion commands that are defined by plugins you haven't yet installed.

When the delete command is completed, Neovim will still be in Normal mode, and you can immediately perform any other `<verb><motion>` combination.

6.3. Changing Text

Sometimes you just want to delete text, but another common task is editing text. Replace a word with another word, change spelling, delete the rest of the paragraph and replace it with something new, etc.

This *could* easily be handled by combining the delete verb with Insert mode (e.g. `dwi` will delete a word and enter Insert mode.) However, you can save a keystroke by using the `c` verb, which means "**c**hange". If you replace the `d` in each of the examples I outlined above with a `c`, you will effectively get "delete the text and immediately enter Insert mode."

6.4. Operating to End of the Current Line

It is very common to want to delete or change from the cursor position to the end of the current line, leaving the beginning of the line intact. These actions happen more often than you would expect in source code editing, so there is a shortcut for them.

Yes you *could* `d$` and `c$` to delete or change to the end of the line, since `$` is the "jump to end of line" motion. That is the "correct" format for the mental model. However, because this is such a common operation, you can "cheat" with one fewer keystrokes and just use the capitalized `D` or `C` instead.

There is no inverse shortcut verb for "delete to the beginning of the line", so you'll have to use `d^` or `d0` instead, where `^` is the motion to jump to the first non-blank character and `0` is the motion to jump to the first column regardless of whether it is blank.

6.5. Operating on Entire Lines

Another common action is to change or delete an *entire* line of text. So much so, in fact, that there are special motions for "the whole line". These motions are accessed by duplicating the verb. This is another place where the mental model kind of breaks down; the interpretation of the motion *depends* on the verb.

In practice, this just means that `dd` deletes an entire line and `cc` deletes it and enters Insert mode. These are nice and easy to type, so it makes for a nice shorthand.

You can combine these bespoke motions with counts. `d3d` will delete three lines, and `3dd` will delete one line three times (which is faster to type because you don't have to move your finger off of `d` to hit it twice). Yes, that has the same outcome either way, but the model is such that you can use either of them. Note that there are situations where the two formats may have subtly different behaviours, although in practice I have never encountered surprises.

6.6. Some Shortcuts for Modifying Individual Characters

Another common operation is to perform a delete or change operation on a single character or specific number of characters. You could do this using `dl` to delete the character under the cursor or `4dl` to delete that character and the three characters that come after it. However, because you do this so often, there is a shorthand verb

that doesn't have a motion (or rather, the motion is implied): `x`. For example, you can use `x` to delete an extraneous `u` in words like "behaviour" if you prefer American spelling. The single letter will be deleted, and you'll be back in Normal mode ready to proceed.

The `x` command can be used with a count, so if you want to delete five characters starting with the one under the cursor, just use `5x`.

If you need to go the other direction and delete characters *before* the cursor, use a capital `X`. This, too, can have a count, and it will basically delete that many characters to the left. I rarely use this, since the shift brings us up to two keystrokes anyway, and `hx` or `d4h` is no harder.

If, instead of deleting, you need to replace a character with a different character, use the `r` command. This command will briefly enter Insert mode while you type one character, then immediately return to Normal mode. Much fewer keystrokes for a common operation (spelling errors are common, right? It's not just me?) than something like `cle<Escape>`. Using `r` with a count is possible, but the behaviour is kind of unhelpful: it will replace the character under the cursor and the `<count>` number of characters after that character with the same letter. The only place I can imagine this being helpful is when you copy-paste a password prompt from somewhere and need to replace all the characters in the password with `*`.

Another common operation is deleting the newline at the end of the current line. Use the `J` ("**J**oin Lines") command from anywhere in the line. I use this one a lot. If you need to merge multiple consecutive lines together, `J` takes a count. It generally does the right thing around whitespace (replacing indentation with a single space), but if you need to do a join without modifying whitespace, use the two-character verb `gJ`.

6.7. Manipulating Case

If you need to convert a character or sequence of characters to uppercase, use the verb `gU` (that's a shifted `U` for the second character) followed by any standard navigation motion. I find this particular verb frustrating because `g` is normally assigned to the `Go To` motions. In this case, (as with `gJ` above) it is a verb instead.

I guess you can think of it as "Go To and Convert to Uppercase" where `U` is short for `Uppercase`.

The inverse function to convert all text between the current cursor position and the motion destination is to use a lowercase `gu` before the motion. Kind of weird to

remember, but it does match the common Vim idiom of `gu` means an action and `gU` means the same action BUT BIGGER.

I don't find these commands very useful. I more frequently use the `~` command, which inverts the case of the character under the cursor.

The duplicate commands `gUU` and `guu` do the same thing as other duplicate verbs, applying the upper/lower case operation to the entire line.

If you find yourself doing a lot of case switching work, see Chapter 13 for a discussion of the `text-case` plugin.

6.8. Repeating Commands

LazyVim doesn't have a multiple cursor mode. There *are* plugins to support multiple cursors, but in my experience they don't work very well. The Neovim developers have multiple cursors on their roadmap, so I am hoping they will come up with a paradigm that integrates nicely with the Vim mental model.

In the meantime, Neovim provides several different tools available for performing an action in multiple places in your code. We'll cover basic repetitions here, and other useful techniques in later chapters.

Once you have performed any verb, you can navigate to another place in the document and repeat that verb with a single keypress: `.` (That's a period, although you will usually hear it referred to as "dot repeat" in this context).

This highlights why `d` and `c` need to be separate verbs, as opposed to using something like `d<motion>i`. When you use `c`, the delete motion **and** the text you inserted is remembered, so you can repeat the entire change with a `.` command. For example, if you want to replace all instances of a variable named `i` with a much better name of `index`, you could jump to the first instance of `i` and type `clindex<Escape>` to "change one character to index". Then you can use navigation commands to go to the next use of `i`. Now just type `.` to repeat the change and continue to the next instance.

Like motions and verbs, the `.` command can be given a count. However, counts with `.` are a little bit nuanced. Rather than blindly repeating the command `<count>` number of times, it will instead *replace* the count of the command being repeated.

This means that if you use the verb `3dd` to delete three lines, and the next operation

you perform is `2.` ("2 dot"), the second operation will delete *two* lines, rather than six.

6.9. Recording Commands

Vim's command recording and playback system is extremely powerful. You can trivially record an arbitrary sequence of navigation, editing, and insertion commands, then repeat that sequence on demand at any location you want.

To start a recording, press `qq`. Sorry, but I have no mnemonic to remember `q`. I have a feeling it was just the last available key on the keyboard!

After that, type whatever sequence of navigation, editing, and insertion commands you want to record. Delete words, insert text, change text, search for words (don't use Seek mode, as the replay mechanism will have no idea what label to jump to). Virtually anything you can do in Vim (even `:` commands) can be recorded and replayed later.

When you are finished recording, just press `q` again. The recording will be stored ready for replay whenever you desire.

6.9.1. Appending to a Recording

If you partially complete your recording and then realize you need some more information or need to make an edit before completing the recording, you can stop the recording using `q` as usual and do the thing you need to do.

When you are ready to continue recording, use `qQ` to record in *append* mode instead. The main tip here is that you need to make sure your cursor is in a location such that the merged recording will make sense. This usually means the same place it was when you stopped recording, although it may depend on what changes you made in the meantime.

6.9.2. Playing Back a Recording

The easiest and fastest way to play back your most recently saved recording is with a capital `Q`.

It is possible to store and replace multiple recordings at once using *registers* (a stupid name for a storage location that harkens back to humanity's dark days of assembly programming). We will go into

more detail about registers in chapter 8.

6.10. Undo and Redo

Obviously, these are the most important operations in the whole book! Use the `u` key to undo your most recent change. Note that "most recent change" can be a pretty big whack of text, especially if you haven't exited Insert mode for a while. For example, I wrote this entire paragraph in one Insert session. If I press `u` the entire paragraph will be lost.

That's ok, though, because I can redo using `Control-r`. Like most developers, I use both of these extensively. (Did you know that in the old days of typewriters, secretaries had to get 100% accuracy scores on their typing tests? There was no backspace or delete key, you see).

Neovim actually does an amazing job of keeping track of your entire history, rather than just the most recent suite of changes. So if you make a bunch of changes to get to state B, then undo to state A, and then make a bunch more changes to get to state C, it is still possible to get back to state B (ie: back out of the C changes to state A and go back up the B changes to state B).

It's kind of the same concept as git branches, except your history is *automatically* tracked for every keystroke you make. Working with branches of undo history using raw Neovim commands can feel pretty clumsy, though (read through `:help undo-branches` if you're brave). Instead I recommend configuring and installing the undotree [https://github.com/jiaoshijie/undotree] plugin.

About 99.9% of the time, `u` and `Control-r` will be all you need, but that remaining 0.1% can make `undotree` a godsend when you need it.

6.11. Summary

In this chapter, we expanded our understanding of the Vim mental model, and then introduced several verbs that can be combined with the navigation motions we were already familiar with.

We discussed an assortment of other editing commands before covering how to repeat motions using `.` and command recordings. Finally, we covered undo and redo.

In the next chapter, we'll learn about text objects and some additional nuances of

the Vim mental model with operator-pending mode. Combined, these allow us to very quickly perform actions on a huge variety of code constructs.

Chapter 7. Objects and Operator-Pending Mode

The navigation and motion commands we've learned so far are invaluable, but Neovim also comes with several more advanced motions that can supercharge your editing workflow. LazyVim further amends this collection of motions with other powerful navigation capabilities powered by a variety of plugins.

For example, if you are editing text rather than source code, you will find it useful to be able to navigate by sentences and paragraphs. A sentence is basically anything that ends with a `.`, `?`, or `!` followed by whitespace.

The sentence keybindings are two of the hardest for me to remember. I use them rarely enough that it hasn't become muscle memory, and it doesn't have a good mnemonic I can remember.

Have I built enough suspense? Pay attention, because you will forget this. To move one sentence forward (to the first letter after the whitespace following sentence ending punctuation), type a `)` (right parenthesis) command in normal mode. To move to the start of the current sentence, use `(`. Press the parenthesis again to move to the next or previous sentence or a add count if you want to move by multiple sentences.

I hate that the command is `(` since that feels like it should be moving to, you know, a parenthesis! But it's not; it's moving by sentences. Since the `.`, `!`, and `?` characters rarely mean "sentence" in normal software development, I just don't use it that much until I start writing a book (something I keep telling myself I won't commit to again, but it never lasts).

In addition to stopping after punctuation followed by whitespace, navigating by sentence also stops on "paragraph boundaries", which is to say "blank lines". If you are using sentences with counts, this can throw your navigation off because you need to add an extra step for each paragraph, as well as the punctuation that normally defines a sentence.

I do use the paragraph motions all the time, though. A paragraph is defined as all the content between two empty lines, and that is a concept that makes sense in a programming context. Most developers structure their code with logically connected statements separated by blanks. The commands to move up or down by

one “paragraph” are the curly braces, `{` and `}`. If you need to jump multiple paragraphs ahead or back, they can, as usual, be prefixed by a count.

Again, you might expect `{` to jump to a curly bracket, so it is a bit annoying that it means “empty line” instead, but once you get used to it, you’ll probably reach for it a lot.

7.1. Unimpaired Mode

LazyVim provides a bunch of other motions that can be accessed using square brackets. It will take a while to internalize them all, but luckily, you can get a menu by pressing a single `[` or `]`. Like the sentence and paragraph motions, the square brackets allow you to move to the previous or next *something*, except the *something* depends what key you type after the square bracket.

Collectively, these pairs of navigation techniques are sometimes referred to as “Unimpaired mode”, as they harken back to a foundational Vim plugin called vim-unimpaired [https://github.com/tpope/vim-unimpaired] by a famous Vim plugin author named Tim Pope. LazyVim doesn’t use this plugin directly, but the spirit of the plugin lives on.

Here’s what I see if I type `[` and then pause for the menu:

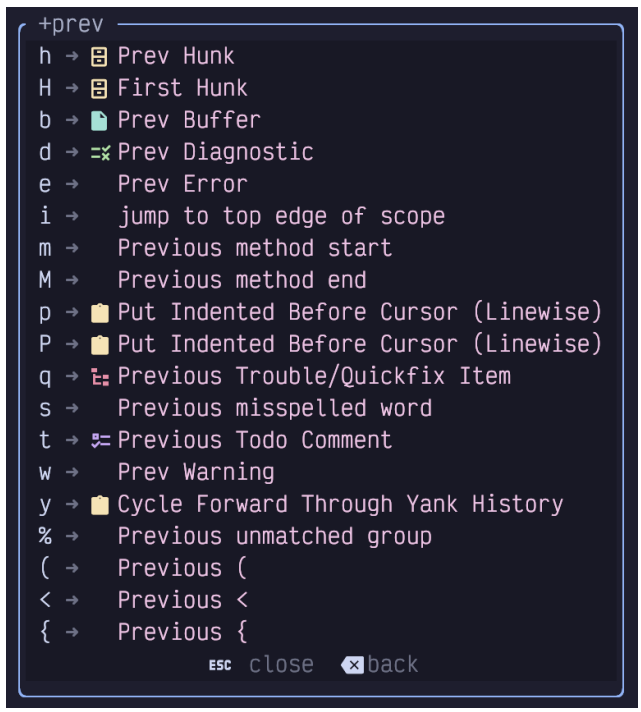


Figure 25. Unimpaired Mode Menu

Not all of these are related to navigation, and one of them is only there because I have a Lazy Extra enabled for it. We'll cover the motion related ones here and most of the others in later chapters.

First, the commands to work with `(`, `<`, and `{` are quite a bit more nuanced than they look. They **don't** blindly jump to the *next* (if you started with `]`) or *previous* (if you used `[`) parenthesis, angle bracket, or curly bracket. If you wanted to do that, you could just use `f(` or `F(`.

Instead, they will jump to the next **unmatched** parenthesis, angle bracket, or curly bracket. That effectively means that keystrokes such as `[(` or `]}` mean "jump out". So if you are in the middle of a block of code surrounded by `{}` you can easily jump to the end of that block using `]}` or to the beginning of it using `[{`, no matter how many other curly-bracket delimited code blocks exist inside that object. This is useful in a wide variety of programming contexts, so invest some time to get used to it.

As a shortcut, you can also use `[%` and `]%` where the `%` key is basically a placeholder for "whatever is bracketing me." They will jump to the beginning or end of whichever parenthesis, curly bracket, angle bracket, or square bracket you are currently in.

That last one (square bracket), is important, because unlike the others, `[[` and `]]` do *not* jump out of square brackets, so using `[%` and `]%` is your only option if you need to jump out of them.

7.1.1. Jump by Reference

Instead of jumping out of square brackets as you might expect, the easy to type `[[` and `]]` are reserved for a more common operation: jumping to other references to the variable under the cursor (in the same file).

This feature typically uses the language server for the current language, so it is usually smarter than a blind search. Only actual uses of that function or variable are jumped to instead of instances of that word in the middle of other variables, types, or comments as would happen with a search operation.

As you move your cursor, LazyVim will automatically highlight other variable instances in the file so you can easily see where `]]` or `[[` will move the cursor to.

7.1.2. Jump by Language Features

The `[c`, `]c`, `[f`, `]f`, `[m`, and `]m` keybindings allow you to navigate around a source code file by jumping to the previous or next class/type definition, function definition, or method definition. The usefulness of these features depends a bit on both the language you are using and the way the Language Service for the language is configured, but it works well in common languages.

By default, those keybindings all jump to the *start* of the previous or next class/function/method. If you instead want to jump to the *end*, just add a `Shift` keypress: `[C`, `]C`, `[F`, `]f`, `[M`, and `]M` will get you there.

Note that these are *not* the same as "jump out" behaviour: if you have a nested or anonymous function or callback defined inside the function you are currently editing, the `]F` keybinding will jump to the end of the nested function, not to the end of the function after the one you are currently in.

I personally don't use these keybindings very much as there are other ways to navigate symbols in a document that we will discuss later. But if you are editing a large function and you want to quickly jump to the next function in the file, `]f` is probably going to get you there faster than using `j` with a count you need to calculate, or even a `Control-d` followed by `S` to go to seek mode.

7.1.3. Jump to End of Indention

If you are working with indentation-based code such as Python or deeply nested tag-based markup such as HTML and JSX, you may find the `[i` and `]i` pairs helpful.

These are provided as part of the `snacks` suite of plugins, via `snacks.indent`. This plugin helps visualize the levels of indentation in a file. Here's an example from a Svelte component I was working on recently:

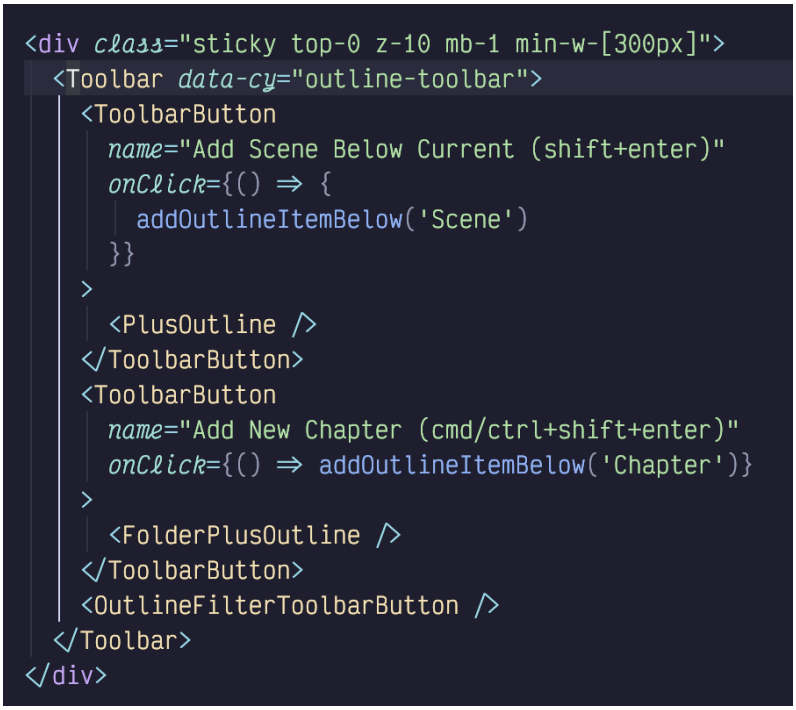


Figure 26. Indent Guides

This Svelte code uses two spaces for indentation. Each level of indentation has a (in my theme) grey vertical line to help visualize where that indentation level begins and ends, and the "current" indentation level is highlighted in a different colour.

In addition, the plugin adds the unimpaired commands `[i` or `]i` to jump out of the current indentation level; it will go either to the top or the bottom of whichever indentation line is currently highlighted.

I use this functionality all the time when editing Python code and Svelte components. I use it less often in other languages where `[%` and `]%` tend to get me closer to where I need to go next. But the visual feedback of indent guides can be super helpful, even in bracket-heavy languages; I may be surprised by which curly bracket I will "jump out" to, but the indent guides are always obvious.

7.1.4. Jumping to Diagnostics

I don't know about you, but when I write code, I tend to introduce a lot of errors in it. Depending on the language, LazyVim is either preconfigured or can be configured to give me plenty of feedback about those errors, usually in the form of a squiggly underline

If you aren't seeing squiggly underlines, go back to Chapter 1 and pick

a better terminal.

These squiggly underlines are usually created by integration with compilers, type checkers, linters, and even spell checkers, depending on the language. Some of them are errors, some are warnings, some are hints. Some are just distractions, but most of them are opportunities to improve your code.

Because I am so incredibly talented at introducing problems in my code, a common navigation task I need to perform is "jump to the next squiggly line". Collectively, these are referred to as **d**iagnostics, so the key combinations are `[d` and `]d`. If you only want to focus on errors and ignore hints and warnings, you can use `[e` and `]e`. Analogously, the `[w` and `]w` keybindings navigate between only warnings.

If you are editing a file in a language that enables spellcheck, or you have enabled it explicitly with `<Space>us`, misspellings can be jumped to with `[s` and `]s`. This tripped me up when I started this book because I expected the `]d` to take me to the squiggly underlines under misspelled words, but it doesn't. I need `]s` instead.

Finally, if you use `TODO` or `FIXME` comments in your code, you can jump between them using `[t` and `]t`.

Note that unlike most of the previous `]` and `[` keybindings, it is not possible to combine diagnostic jumps with a verb. So `d[d` will **not** delete from the current location to the nearest diagnostic. This is (probably) just an oversight in how LazyVim defines the keybindings.

7.1.5. Jumping to Git Revisions

This is actually my favourite of the square bracket pairs: `[h` and `]h` allow you to jump to the next git "hunk". If you aren't familiar with the word (or if you're from a generation that thinks it means a gorgeous man), a "git hunk" just refers to a section of a file that contains modifications that haven't been staged or committed yet.

A lot of my editing work involves editing a large file in just three or four places. For example, I might add an import at the top of the file, an argument to a function call somewhere else in the file, and change the function that receives that argument in a third place. Once I've started editing, I may have to jump back and forth between those locations. `]h` and `[h` are *perfect* for this, and I don't need to remember my jump history or add named marks (essentially bookmarks) to do it.

Even better, LazyVim gives you a simple visual indicator as to which lines in the file have been modified, so you have an idea where it's going to jump. Have a look at this

screenshot:

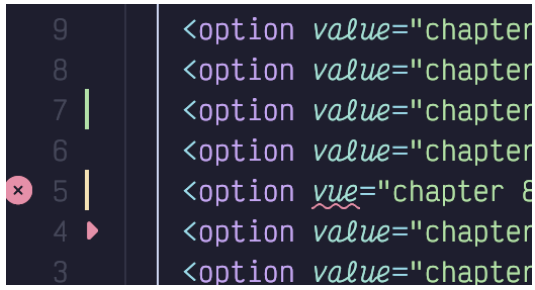


Figure 27. Git Hunk Indicators

On the left side, to the right of the line numbers, you can see a green vertical bar where I inserted two lines, an orange bar where I changed a line, and a small red arrow indicating that I deleted a line. (As a bonus, it also shows a diagnostic squiggly and a red x-in-circle to the left of the line numbers on the line where my modification introduced the error). If I place my cursor at the top of the file and type `]h` three times, I will jump between those three places.

Like diagnostics, `[h` and `]h` cannot be combined with a verb.

7.2. Text Objects

Combining verbs with motions is very useful, but it is often more helpful to combine those same verbs with *objects* instead of motions. Vim comes with several common objects, such as words, sentences and the contents of parenthesis. LazyVim adds a ridiculous pile of other text objects.

The grammar for objects is `<verb><context><object>`. The verbs are the same verbs you have already learned for working with motions, so they can be `d`, `c`, `gU`, etc.

The context is always either `a` or `i`. As you know, these are two commands to enter Insert mode from Normal mode. But if you have already typed a verb such as `d` or `c`, you are technically not in Normal mode anymore!

You are in the so-called “Operator Pending Mode”. The navigation keystrokes you are familiar with are generally also allowed in Operator-Pending mode, which is the real reason you can perform a motion after a verb. But if a plugin maintainer neglects to define the operator-pending keymaps, you end up with situations where you can navigate but not perform a verb.

It doesn’t make sense to switch to Insert mode after an operator, so the `a` and `i`

keystrokes mean completely different things. Typically, you can think of them as `around` and `inside` (though in my head I always just pronounce them as "a" and "in"). The difference is that `a` operations tend to select everything that `inside` selects **plus** a bit of surrounding context that depends on the object that is defined.

For example, one common object is the parenthesis: `(`. If you type the command `di(`, you will delete all the text inside a matched pair of parenthesis. But if you instead type `da(`, you will delete all the text inside the parenthesis as well as the `(` and `)` at each end.

To see a list of many possible text objects in LazyVim, type `da` and pause. Here's what I see:

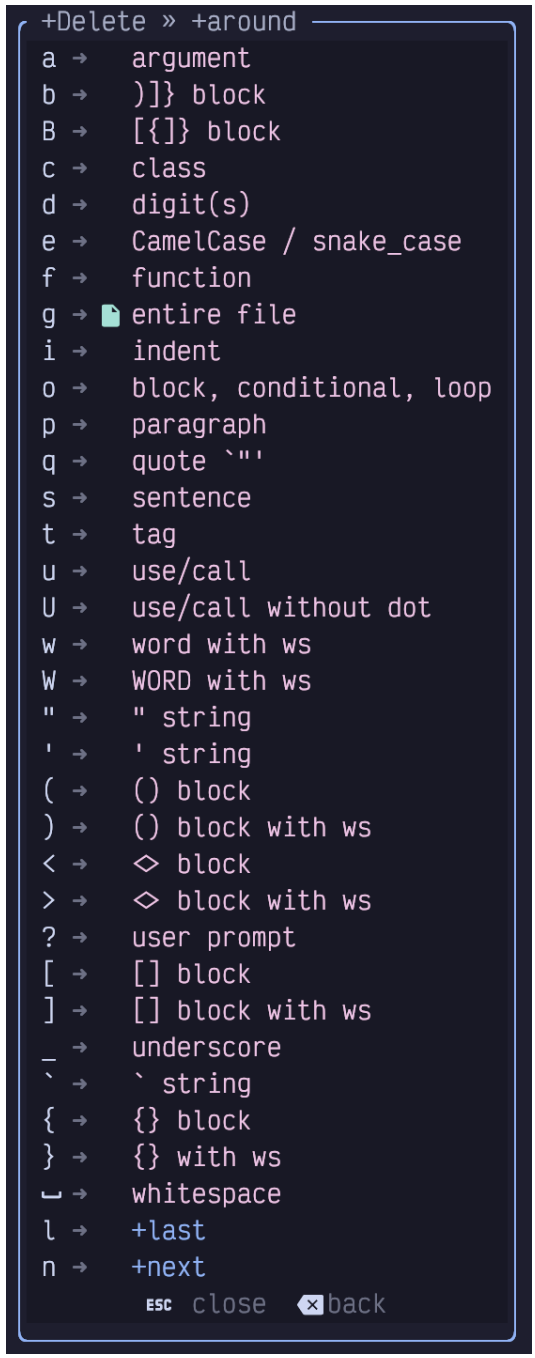


Figure 28. Operator Pending Menu

Let's cover most of these in detail next.

7.2.1. Textual Objects

The operators `w`, `s`, and `p` are used to perform an operation on an entire word, sentence, or paragraph, as defined previously: word is contiguous non-punctuation, sentence is anything that ends in a `.`, `?`, or `!`, and paragraph is anything separated by

two newlines.

The difference between `around` and `inside` contexts with these objects is whether or not the surrounding whitespace is also affected.

For example, consider the following snippet of text and imagine my cursor is currently at the `|` character in the middle of the word `handful` in the second sentence:

```
This snippet contains a bunch of words. There are a hand|ful
of
sentences.

And two paragraphs.
```

Listing 18. A Plain Text Paragraph

If I want to delete the word `handful` while I'm at that location, I *could* type `bde` to jump to the back of the word, then delete to the end of the word. Or I can use the `inside word` text object and type `diw`.

Either way, I end up with an extra space between `a` and `of` because `diw` is `inside` the word and doesn't touch surrounding whitespace.

If I instead type `daw`, it will delete the word and *one* surrounding space character, so everything lines up correctly afterward with a single space between `a` and `of`.

There is also a `W` (capitalized) operator that has a similar meaning to the captial `W` when navigating by words: it will delete everything between two whitespaces instead of interpreting punctuation as a word boundary.

Similarly, I can use `dis` and `das` from that same cursor position to delete the entire "There are a handful of sentences." sentence. The former won't touch any of the whitespace before `The` or after the `.`, while the latter will sync up the whitespace correctly.

Finally, I can delete the entire paragraph with `dip` or `dap`. The difference is that in the former case, the blank line after the paragraph being deleted will still be there, but in `around` mode, it will remove the extra blank.

Typically, I use `i` when I am changing a word, sentence or paragraph, with a `c` verb,

since I want to replace it with something else that will need to have surrounding whitespace. But I use `a` when I am deleting the textual object with `d` because I don't intend to replace it, so I want the whitespace to behave as if that object never existed.

7.2.2. Quotes and Brackets

The objects `"`, `'`, and `` ` `` operate on a string of text surrounded with double quotes, single quotes, or backticks. If you use the command `ci"`, you will end up with your cursor in Insert mode between two quotation marks, where everything inside the string was removed. If you use `da"`, however, it will delete the quotation marks as well.

As a shortcut, you can use the letter `q` as a text object and LazyVim will figure out what the nearest quotation mark is, whether single, double, or backtick, and delete that object. I don't use this, personally, but I guess it would save a keypress on double quotes.

Similarly if you want to apply a verb to an entire block contained in parentheses or curly, angle, or square brackets, you just have to type one of those bracketing characters. Consider, these examples: `di[`, `da(`, `ci{` or `ca<`. As with quotes, the `i` versions will leave the surrounding brackets intact, and the `a` version will delete the whole thing.

The shortcut to select whatever the nearest enclosing bracket or parenthesis type is the `b` object. (Mnemonic is "**b**racket").

These actually work with counts so you can delete the "third surrounding curly brackets" instead of the "nearest surrounding curly brackets" if you want to. I can never remember where to put the count, though! If your memory is better than mine, the syntax is to place the count **before** the `a` or `i`. So for example, `d2a{` will delete everything inside the second-nearest set of curly brackets. I'm not sure if that makes sense, so here's a visual:

```
class Foo {
    function bar() {
       let obj = {fizz: 'buzz'}
    }
}
```

Listing 19. An Odd Little Class

If my cursor is on the colon between `fizz` and `'buzz'` then you can expect the following effects:

- `di{` will delete `fizz: 'buzz'` but leave the surrounding curly brackets.
- `c2i{` will remove the entire `let obj =` line and leave my cursor in Insert mode inside the curly brackets defining the function body.
- `c2a{` will do the same thing, but *also* remove those curly brackets, so I'm left with a `function bar()` that has no body.
- `d3i{` will remove the entire function and leave me with an empty `Foo` class.

You can also delete things between certain pieces of punctuation. For example, `ci*` and `ca_` are useful for replacing the contents of text marked as bold or italic in Markdown files.

If you want to operate on the entire buffer, use the `ag` or `ig` text object. So `cag` is the quickest way to scrap everything and start over and `yig` will copy the buffer so you can paste it into a pastebin or chatbot. The `g` may seem like an odd choice, but it has a symmetry to the fact that `gg` and `G` jump to the beginning or end of the file. If you need a mnemonic, think of `yig` as "yank in **g**lobal".

7.2.3. Language Features

LazyVim adds some helpful operators to perform a command on an entire function or class definition, objects, and (in HTML and JSX), tags. These are summarized below:

- `c`: Act on **c**lass or type
- `f`: Act on **f**unction or method
- `o` Act on an "**o**bject" (the mnemonic is a stretch) such as blocks, loops, or conditionals
- `t` Act on an HTML-like **t**ag (works with JSX)
- `i` Act on a "scope", which is essentially an **i**ndentation level (only available if the aforementioned mini.indentscope extra is installed)

7.2.4. Git Hunks

Remember the git hunks we discussed with Unimpaired mode? You can similarly act on an entire hunk with the `h` object. So one way to quickly revert an addition is to just type `dih`. But you probably won't do this much as there are better ways to deal with git, as we will discuss in Chapter 15.

7.2.5. Next and Last Text Object

The text object feature is great if you are already inside the object you want to operate on, but LazyVim is configured (using a plugin called mini.ai) so that you can even operate on objects that are only *near* your cursor position.

Once installed, the next and last text objects can be accessed by prefixing the object you want to access with a `l` or `n`.

Consider the `Foo.bar` Javascript class again:

```
class Foo {
    function bar() {
       let obj = {fizz: 'buzz'}
    }
}
```

Listing 20. That Odd Class Again

If my cursor is on the `{` of the `function bar` line, I can type `cin{` to delete the contents of the `fizz: 'buzz'` object and place my cursor there in insert mode. I can save myself an entire navigation with just one extra `n` keystroke. I think this is a really neat feature, but I tend to forget it exists... hopefully writing about it here will help me remember!

7.3. Seeking Surrounding Objects

The flash.nvim plugin that gave us `Seek` mode, has another trick up its sleeve: the holy grail of text objects. After specifying a verb, you can use the `S` key (there is no `i` or `a` required) to be presented with a bunch of paired labels around the primary code objects surrounding your cursor.

As an example, I'm going to lean on that `Foo` class again. I have placed my cursor on the `:` and typed `cS`. The plugin identifies the various objects surrounding my cursor and places labels at both ends of each object:

Figure 29. Seek Surrounding Object

The labels in this image are in green, and (typically) go in alphabetical order from "innermost" to "outermost". The primary difference from Seek mode is that each label comes in pairs; there are two `a` labels, two `b` labels, and so on. The text object is whatever is between those labels.

If the next character I press is `a` (or enter, to accept the default), then I will change everything inside the curly brackets defining the `obj`. If I press `b`, it will also replace those curly brackets. Pressing `c` will change the entire assignment and `d` will change the contents of the function. Hitting `e` replaces the curly brackets as well, and `f` changes the full function definition. The `g` label is the contents of the class, while `h` changes the entire class.

This is a super useful tool when you need to change, delete, or copy a complex structure that does not immediately map to any of the other objects.

7.3.1. Seeking Surrounding Objects Remotely

The `S` Operator-pending mode is useful for acting on objects that surround the cursor, but if your cursor isn't currently within the object you want to select, it won't suffice. You could use `s` to navigate to inside the object followed by `S` to select it, but you can save yourself a few keystrokes by instead using the `R` operator.

With a mnemonic of "**R**emote", `R` is easy to use, but hard to explain. It is an operator-pending operation, so you need to type a verb first, followed by `R` (as with `S`, there is no `i` or `a` required).

At this point, LazyVim is essentially in Seek mode, so you can type a few characters from a search string to find matches anywhere on the screen. However, instead of showing a single label at any matches for the string you searched for, flash.nvim will automatically switch to surrounding object mode, and show pairs of labels of all constructs that surround the matching locations.

To put the icing on the cake, you can also perform a remote seek on any kind of object without using the surround mode. In this case, you would type a verb

followed by a lowercase `r` (it still means "**r**emote"). This will also put you in Seek mode, and you can start typing the matching characters. Single (normal Seek mode, rather than Surround Seek mode) labels will pop up, and you can enter a character to temporarily move your cursor to that label, just like normal Seek mode. But when your cursor arrives there, it is automatically placed in Operator-pending mode again. So you can now type any other operator such as `aw` or `i(`. Once the operation completes, your cursor will move back to where it was before you entered the remote Seek mode.

As a specific example, the command `drAth2w` will delete two words starting at the word "At" that gets the label `h`, then jump your cursor back to the position it was at before you started the delete. In other words, it is the same as the command `sAthd2w<Control-o>`, which will seek to the word "At" at label h, then delete two words, and use `Control-o` to jump back to your previous history location. The remote command is a little shorter, but it's another one that I tend to forget to use. My brain goes into "move the cursor" mode before it figures out "delete" mode, so by the time I realize I could have done it remotely, it's too late.

7.4. Operating on Surrounding Pairs

We've already seen the text objects to operate on the *contents* of pairs of quotation marks or brackets, but what if you want to keep the content but change the surrounding pair?

Maybe you want to change a double quoted string such as `"hello world"` to a single quoted `'hello world'`. Or maybe you are changing a `obj.get(some_variable)` method lookup to a `obj[some_variable]` index lookup, and need to change the surrounding parentheses to square brackets.

LazyVim ships with the mini.surround plugin for this kind of behaviour, but it's not installed by default. It is a recommended extra, so if you followed my suggestion to enable all the recommended plugins, you may have it already.

7.4.1. Add Surrounding Pair

The default verb for adding a surrounding pair is `gsa`. That will place your editor in operator-pending mode, and you now have to type the motion or text object to cover the text you want to surround with something. Once you have finished inputting that object, you need to type the character you want to surround it with, such as `"` or `(` or `)`. The difference between the latter two is that, while both will surround the text with parentheses, the `(` will also put extra spaces *inside* the parentheses.

That may sound complicated, but it should make sense after you see some examples:

- `gsai[(` will select the contents of a set of square brackets (using `i[`) and place parentheses separated by spaces inside the square brackets. So if you start with `[foo bar]` and type `gsai[(`, you will end up with `[( foo bar )]`.
- `gsai[)` does the same thing, except there are no spaces added, so the same `[foo bar]` will become `[(foo bar)]`.
- `gsaa[)` will place the parentheses *outside* the square brackets, because you selected with `a[` instead of `i[`. So this time, our example becomes `([foo bar])`.
- `gsa$"` will surround all the text between the current cursor position and the end of the line with double quotation marks.
- `gsaSb'` will surround the text object that you select with the label `b` after an `S` operation with single quotation marks.
- `gsaraa3e*` will surround the remote object that starts with `a` that is labelled with an `a` followed by the next three words with an asterisk at each end of the three words.

Depending on the context it can be a lot of characters to type, but it's typically fewer characters than navigating to and changing each end of the pair independently.

7.4.2. Delete Surrounding Pair

Deleting a pair is a little easier, as you don't need to specify a text object. Just use `gsd` followed by the indicator of whichever pair you want to remove.

So if you want to delete the `[]` surrounding the cursor, you can use `gsd[`.

If you want to delete deeply nested elements, you need to put a count *before* the `gsd` verb. So use `2gsd{` to delete the second set of curly braces outside your current cursor position. For example, if your cursor is inside the `def` of the string `{abc {def}}`, typing `2gsd{` results in `abc {def}`, leaving the "inner" curly braces around `def`, but removing the second outer set around the whole.

7.4.3. Replace Surrounding Pair

Replacing is similar to deleting, except the verb is `gsr` and you need to type the character you want to replace the existing character with *after* you type the existing character.

So if you have the text `"hello world"` and your cursor is inside it, you can use `gsr"'` to change the double quotes to single quotes: `'hello world'`.

7.4.4. Navigate Surrounding Characters

Performing operations on surrounding pairs or on the entire contents of the pair is convenient, but sometimes you just want to move your cursor to the beginning or end of the pair. You can often do this using Seek mode, Find mode, or the Unimpaired mode commands such as `[(`, but there are other commands that are more syntax aware if you need them.

The easiest one has been built into Vim for a long time. If your cursor is currently on the beginning or ending character of a parenthesis, bracket, or curly brace pair, just hit `%` to jump to its mate at the other end of the parenthetical. If you use `%` in Normal mode when you aren't on a pair, it will jump to the nearest enclosing pair-like object. This only works with brackets, though, so arbitrary pairs including quotes are not supported.

The mini.pairs plugin comes with `gsf` and `gsF` keybindings, which can be used to move your cursor to the character in question. I don't use these, because the mini.ai plugin provides a similar feature using the `g[` and `g]` shortcuts. These shortcuts both need to be followed by a character type, so e.g. `g[(` will jump back to the nearest surrounding open parenthesis, and `g]]` will jump to the nearest closing square bracket. If you give it a count, it will jump out of that many surrounding pairs.

7.4.5. Highlighting Surrounding Characters

If you just need to double check where the surrounding characters are, you can use something like `gsh(`, where `h` would mean "**h**ighlight". This can sometimes be used as a dry run for a delete or replace operation that is using counts so you can double check that you are operating on the pairs you think you are.

7.4.6. Bonus: XML or HTML Tags

The mini.surround plugin is mostly for working with pairs of *characters*, but it can also operate on html-like tags.

Say you have a block of text and you want to surround it with `p` tags. String together the command `gsaapt`. That is `gsa` for "add surrounding" followed by `ap` for "around paragraph". So we're going to add something around a paragraph. Instead of a quote or bracket, we say the thing we are going to add is a `t` for **t**ag.

mini.surround will understand that you want to add a tag, and pop up a little prompt

window to enter the tag you want to add. Type the `p` for the tag you want to create. You don't need angle brackets; just the tag name:

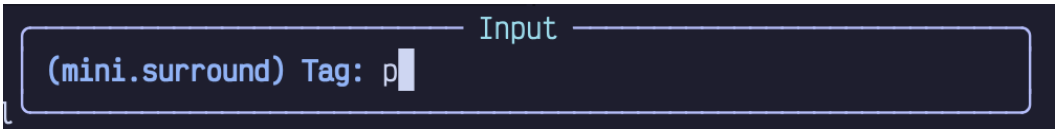


Figure 30. Surround With Tag

If the tag you want to add has attributes, you can add them to the prompt. mini.surround is smart enough to know that the attributes only go on the opening tag.

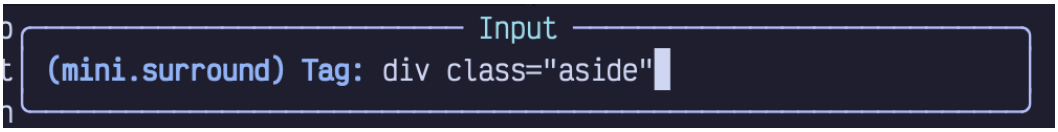


Figure 31. Suround Tag Attributes

7.4.7. Modifying the Keybindings

I love the mini.surround behaviour. I use it a lot. So much that I quickly got tired of typing `gs` repeatedly. I decided to replace the `gs` with a `;` so I can instead type `;d` or `;r` instead of `gsd` or `gsr`. For adding surrounds, I decided to leverage the fact that double keypresses are easy to type, so I used `;;` for that action instead of `gsa` or even `;a`.

In order for this to work correctly, I also had to modify the flash.nvim configuration to remove the `;` command. (By default the `;` key can be used as a "find next" behaviour for the `f` and `t` keys, but the way flash is designed, you don't need a separate key for it; just press `f` or `t` again).

If you want to do the same thing, just create a new Lua file named whatever you want (mine is `extend-mini-surround.lua`) inside the `config/plugins` directory.

The contents of the file will be:

```
return {
  {
    "echasnovski/mini.surround",
    opts = {
      mappings = {
        add = ";;",
        delete = ";d",
        find = ";f",
        find_left = ";F",
        highlight = ";h",
        replace = ";r",
        update_n_lines = ";n",
      },
    },
  },

  {
    "folke/flash.nvim",
    opts = {
      modes = {
        char = {
          keys = { "f", "F", "t", "T" },
        },
      },
    },
  },
}
```

Listing 21. Configuring Mini.surround And Flash

Since we are modifying *two* plugins, I put two Lua tables inside a wrapping Lua table, which Lazy.nvim is smart enough to parse as multiple plugin definitions. The first passes `mappings` to the `opts` that are passed into mini.surround. These will replace the default keybindings that LazyVim has defined for that table (the ones that start with `gs`).

The second definition also passes a custom `opts` table. It replaces the default keys, which include `;` and `,` with a new table that only defines `f`, `F`, `t`, and `T`.

If I had known that `;` was being rebound by flash.nvim, I could have found this solution by reading the config for flash.nvim on the

LazyVim website [https://www.lazyvim.org] and seeing what needed to be overwritten. However I wasn't able to figure out where the `;` was being defined, and ended up asking for help on the LazyVim GitHub Discussions. People are really helpful there, and I encourage you to come say hello if you have any questions.

7.5. Summary

In this chapter, we learned a bunch of advanced code motion techniques that LazyVim gives us by its reimplementation of Unimpaired mode. Then we learned what text objects are and took in a crash course on the many, many text objects LazyVim provides.

We then covered the exceptionally useful `S` motion, which can be used to pick text objects on the fly, as well as the remote variations of `R` and `r`.

We wrapped up by going over several new verbs that can be used to work with surrounding pairs of characters such as parentheses, brackets, and quotes.

In the next chapter, we'll cover clipboard interactions and registers, as well as an entire new Visual mode that can be used for text selection.

Chapter 8. Clipboard, Registers, and Selection

Vim has a robust copy and paste experience that predates the operating system clipboard you are used to in other editors. Happily, the LazyVim configuration sets up the Neovim clipboard system to work with the OS clipboard automatically.

In fact, you already know how to cut text to the system clipboard: Just delete it.

That's right. Any time you use the `d` or `c` verb, the text that you deleted is automatically cut to clipboard. This is usually very convenient, and occasionally somewhat annoying, so I'll show you a workaround to avoid saving deleted text later in this chapter.

8.1. Pasting Text

Pasting (typically referred to as "putting" in Vim) text uses the `p` command I mentioned briefly in Chapter 1. In Normal mode, the single command `p` will place whatever is in the system clipboard at the current cursor position. This is usually the text you most recently deleted, but it can be an URL you copied from the browser or text copied from an e-mail or any other system clipboard object.

The position of the text you inserted can be somewhat surprising, but it usually does what you want. Normally, if you deleted a few words or a string that is not an entire line, it goes immediately after the current cursor position. However, if you used a command that operates on an entire line, such as `dd` or `cc`, the text will be placed on the next line. This saves a few keystrokes when you are moving lines around, a common task in code editing.

The `p` command can be used with a count, so in the unlikely event you want to paste 5 consecutive copies of whatever is in the clipboard, you can use `5p`.

When you paste with `p`, your cursor will stay where it was, and the text is inserted after the cursor. If you want to instead paste the text *before* the current cursor position, use a capital `P`, where the shifting action is interpreted as "do `p` in the other direction." As with `p`, the text will be inserted directly before the cursor position unless it was a line-level edit such as `dd`, in which case it will be placed on the previous line.

If you are already in Insert mode and need to paste something and keep typing, you can use the `Control-r` command, followed by the `+` key. The `r` may be hard to

remember, but it stands for "**r**egister." We'll go into more detail about what registers really are, soon.

8.2. Copying Text

Copying text requires a new verb: `y`. It behaves similarly to `d` and `c`, except it doesn't modify the buffer; it just copies the text defined by whatever motion or text object comes after the `y`.

"Why `y`?" you might ask? It stands for "yank", which is Vim speak for "copy." I have no idea why `vi` called it "yank," but my guess is that it was a reverse acronym. The original authors probably noticed that `y` was currently free on the keyboard and decided to come up with a word that matches it. The concept of a clipboard or copy/paste had not been standardized yet, so they were free to use whatever terminology worked for them.

The `y` command works with all of the motions and text objects you already know. It is especially useful with the `r` and `R` Remote Seek commands. If you need to copy text from somewhere else in the editor (even a different window) to your current cursor location, `yR<search><label>p` is the quickest way to be on your way without adding unnecessary jumps to your history.

The `yy` and `Y` commands yank an entire line, and from the cursor to the end of the line, analogous to their counterparts when deleting and changing text.

LazyVim will briefly highlight the text you yanked to give a nice indicator as to whether your motion command copied the correct text.

8.3. Selecting Text First

Your Vim editing experience so far has not involved the concept of selecting text. Isn't that weird? We're 8 chapters in! In normal word processors and VS Code-like text editors, you have to select text before you can perform an operation such as deleting, copying, cutting, or changing it. Considering how awkward text selection is in those editors (you have to use your mouse or some combination of shift, and cursor movements, with extra modifier keys to make bigger movements), it's amazing anyone gets anything done!

In Vim-land, you normally perform the verb first and follow up with a text motion or object to implicitly select the text before manipulating it. This is *usually* the most effective way to operate, but in some situations it is convenient to invert the mental model and highlight text *before* operating on it.

This is where Visual mode comes in. Visual mode is a Vim major mode, like Normal and Insert mode. Technically, there are three sub-modes of Visual mode. We'll start with "Visual Character mode" and dig into the other two shortly.

You might think that it makes sense to always select text first so you can see what you are acting on. Two newish editors called Kakoune and Helix have been experimenting with this paradigm. They are pretty cool, but I found that the "select text first" model was kind of a let-down. The editor isn't able to determine whether any given motion is meant to **move** the selection or to **extend** the selection, so you end up with an extra keypress to tell it to do an extension. At that point, it's no different from pressing `v` to enter Visual mode in Neovim. After using Helix for several months, I determined that it actually required more keystrokes on average than Neovim and I switched back.

To enter Visual Character Mode, use the `v` command from Normal mode. Then move the cursor using most of the motions that you are used to from Normal mode. I say "most" only because Visual mode keymaps are independent of Normal mode keymaps, and plugins occasionally neglect to set them up for both modes. LazyVim is really good about keymaps, though, so you will rarely be surprised.

You can also get into Visual mode by clicking and dragging with your mouse.

Once you have text selected in Visual mode, you can use the same verbs you usually use to delete, change, or yank the selection. The difference is they will happen instantly to the selection rather than requiring a motion. You can even use single character verbs like `x` (which does the same thing as `d`) or `r` to replace all the characters with the same character. After you complete the verb, the editor automatically switches back to Normal mode. You can also exit visual mode without performing an action using either `Escape` or another `v`.

You can exit Visual mode temporarily without completely losing your selection. From Normal mode, use the `gv` ("**g**o to last **v**isual selection") command to return to the selection. This is useful if you are about to perform a visual operation and realize you need to look something up, make an edit, or copy something from elsewhere in the file, then go back to the selection.

Use the `o` (for "**o**ther end") command to move the cursor to the opposite end of the block. Useful if e.g. you've selected a few words, and realize you forgot one at the

other end of the block. You can't get into Insert mode from Visual mode, so the `o` command gets reused for this purpose.

8.3.1. Line-wise Visual Mode

The `v` command is useful for fine-grained selections, but if you know that your selection is going to start and end on line boundaries, you can use a (shifted) `V` instead, to get into Line-wise Visual mode. Now wherever you move the cursor, the entire line the cursor lands on will be selected.

Other than selecting entire lines, the main difference with Line-wise Visual mode is that when you apply a verb that manipulates the clipboard, (including `d`, `c`, and `y`), the lines will be cut or copied in Line-wise mode. When you put them later they will show up on the next or previous line instead of immediately after the cursor.

8.3.2. Block-wise Visual Mode

Block-wise Visual mode is a neat feature that is kind of unique to Vim. It allows you to visually select and manipulate a block of text that is vertically, but not horizontally contiguous. For example, I have selected several characters on each of four separate lines in the following screenshot:

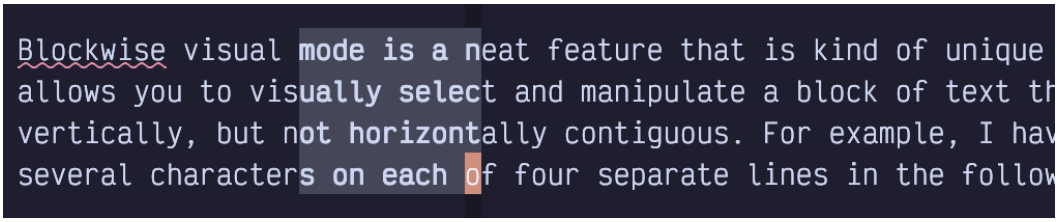


Figure 32. Block-wise Visual Mode

To enter block-wise Visual mode, use `Control-v` instead of `v` or `V` for Visual and Line-wise Visual mode.

In plain text like this, Block-wise Visual mode doesn't appear to be very useful, but it is handy if you need to cut and paste columns of tabular data in a csv file or markdown table, for example. I don't use it for that functionality terribly often, but when I need it, I know there is no other way to efficiently perform the action I need.

If you use `Control-V$`, you will get a slight adaptation of Block-wise Visual Mode where the block extends to the end of each line, in the block. This is handy if you need the block to extend to the longest line as opposed to the line your cursor is currently on.

Block-wise Visual mode can also be used as a (poor) imitation of multiple cursors. If you use the `I` or `A` command after selecting a visual block, and then enter some text followed by `Escape`, the text you entered will get copied at either the beginning or end of the visual block. A common operation for this feature is to add `*` characters at the beginning of Markdown ordered lists or a block comment that needs a frame.

8.4. Registers

Registers are a way to store a string of text to be accessed later (so think of the Assembly-language definition of the word). In that regard, they are no different from a clipboard. In fact, the system clipboard in Vim **is** a register that LazyVim has set up as the default register.

But Vim has dozens of other registers. This means you can have *custom clipboards* that each contain totally different sequences of text. This feature is pretty useful, for example, when you are refactoring something and need to paste copies of several different pieces of code at multiple call sites.

There are several different types of registers, but I'll introduce the concept with the named registers, first. There are over two dozen named registers, one for each letter of the alphabet.

To access a register from Normal mode, use the `"` character (i.e, `Shift-<Apostrophe>`) followed by the name of the register you want to access. Then issue the verb and motion you want to perform on that register.

So if I want to delete three words and store them in the `a` register *instead* of the system clipboard, I would use the command `"ad3w`. `"a` to select the register, and `d3w` as the normal command to delete three words. And if I later want to put that same text somewhere else, I would use `"ap` instead of just `p`, so the text gets pasted from the `a` register instead of the default register.

`"ad<motion>` will always *replace* the contents of the `a` register with whatever text motion or object you selected. However, you can also build up registers from multiple delete commands using the capitalized name of the register. So `"Ad<motion>` will *append* the text you deleted to the existing `a` register.

I find this useful when I am copying several lines of code from one function to another but there is a conditional or something in the source function that I don't need in the destination. Copy the text before the conditional using `"ay` and append the text after the conditional using `"Ay`, and then paste the whole thing in one operation with `"ap`.

I can copy totally different text into the `b` register using e.g `"byS<label>`. Now I can paste from either the `a` or `b` register at any time using `"ap` and `"bp`.

If you forgot which register you put text in, just press `"` and wait for a menu to pop up showing you the contents of all registers. If that menu is too hard to navigate, you can instead use the `<Space>s"` command to open a picker dialog that allows you to search all registers. Just enter a few characters that you know are in the register you want to paste, use the usual picker commands to navigate the list, and hit `Enter` to paste that text at the last cursor position.

To show the same menu from Insert or Command mode, use `Ctrl-r` instead of `"`.

If you're in the `<Space>s"` picker dialog, you'll notice a bunch of other registers besides the named alphabet registers. I'll discuss each of those next.

8.4.1. Clipboard Registers

In LazyVim, by default, the registers named `*` and `+` are always identical to the default (unnamed) register, and represent the contents of the system clipboard.

To understand why, we need some history: vi had registers, and then operating systems got excited about the ideas of clipboards and vi users wanted to copy stuff to the system clipboard. A default (non-lazy) Vim configuration means that if you want to copy text to the system clipboard, you have to always type `"+` before the `y`. The three extra keystrokes (`Shift`, `'`, and `+`) can get pretty monotonous in modern workflows where you're copying stuff into browsers, AI chat clients, and e-mails on a regular basis.

On top of that, some operating systems (Unix-based, usually) actually have *two* Operating System clipboards, an implicit one for text you select, and one for text you explicitly copy with `Control-c` (in most programs). This text would be stored in the `"*` register and the OS lets you paste it elsewhere with (typically) a middle click.

I recommend sticking with LazyVim's synced clipboard configuration, but if you already have muscle memory from using Vim the old way or you're tired of deleted text arbitrarily overwriting your system clipboard, you can disable this integration so that the three registers behave as described above instead of being linked together. To do so use `space f c` to open the `options.lua` configuration file and add the following line:

```
vim.opt.clipboard = ""
```

Listing 22. Disable Clipboard Sync

Speaking of having your clipboard contents randomly overwritten, if you know in advance that you don't want a specific delete or change operation to overwrite the clipboard contents, use the "Black Hole" register, `"_` (that's an underscore). So type `"_d<motion>` to delete text without storing it in any registers including the system clipboard.

If you want to copy the *contents* of one register to another register, you can use the ex command `:let @a = @b` where `a` and `b` are the names of the registers you want to copy to and from. The most common use of this operation is to copy the contents of the system clipboard (which may have come from a different program) into a named register so it doesn't get lost the next time you issue a verb. For example, `:let @b = @+` will copy the system clipboard into register `b`.

8.4.2. The Last Yanked or Last Inserted Text

Whenever you issue a `y` command without specifying a destination register, the text will always be stored in the `"0` register *as well as* the default register. And it will stay in `"0` until the next yank operation, no matter how many deletes or changes you do to change the default register.

So if you yank the text `abc` and then delete the text `def`, the `p` command will paste the text `def`, but you still can paste `abc` using `"0p`.

You can also use the `".` (period) register to paste a copy of the text that was most recently inserted. So if you type the command `ifoo<Escape>` somewhere in the document and move somewhere else in the document and type `".p`, it will insert the word "foo" at the new cursor position. `".` is a register that you may occasionally want to copy into a named register if you have inserted text you want to reuse. Use the previously discussed `:let @c = @.` command to do this.

8.4.3. The Delete (Numbered) Registers

The numbered registers *should* be really useful, but I find them rather confusing. The registers `"1` through `"9` always contain the text that you most recently changed or deleted, in ascending order. So after a delete operation, whatever was in `"1` gets moved to `"2`, `"2` moves to `"3` and so on, and whatever is in `"9` gets dropped.

I can *never* remember the order of my recent deletes, so I would normally have to use the `"` menu to see the contents of the numbered registers. It's handy that my recently deleted text is stored and I can find it this way. However, I usually use the yanky.nvim plugin (discussed later in this chapter) instead, so the numbered

registers are not that useful to me.

There is also a "small delete register" that can be accessed with `"-`. Whenever you delete any text, it will be stored in the numbered registers, but if that text is less than one line long, it will *also* be stored in this minus register. I have little use for this feature, as the majority of my changes are smaller than one line. That means it gets cleared before it drops out of the numbered registers.

8.4.4. The Current File's Name

The name of the file that you are currently editing is stored in the `"%` register. It is always relative to the current working directory of the editor (usually the folder you were in when you started Neovim). The only time I ever want to access this register is to copy the filename to the system clipboard with `:let @+ = @%` so I can paste it into a GUI app or my terminal.

8.5. Recording to Registers

Remember the recording commands I told you about in Chapter 6: `qq` to record and `Q` to play back the recording? Turns out I was a little overly simplistic there.

Recorded commands are actually stored in a named register. In that chapter, I arbitrarily chose the `q` register when I said to use `qq` to start recording. But you can just as easily store it in the `a` register using `qa` or the `f` register using `qf`.

The `qQ` command to "append to recording" operation is analogous to the capitalized `"A<command>` used to append to a register. In this case, `Q` is still an arbitrary name. You can append a recording to a different named register besides `q`, using `qA` or `qZ`, for example.

Having multiple sets of recordings can be really handy when you are performing a complex refactoring that requires you to make one of several different repetitive changes in different locations across your codebase.

The `Q` command to play back a recording always plays back the most recently recorded command, regardless of register. If you want to play back from a different register, you would use the `@` command, followed by the name of the register. So if you recorded using `qa`, you would play it back with `@a`. As a shortcut, `@@` will always replay whichever register you most recently *played* (which is different from `Q` which always plays back the most recent *recording*).

8.5.1. Editing Recordings

To be clear, recordings are placed in normal registers. So if you record a sequence of keystrokes to a register using `qa` and then put the register using `"ap`, you will actually see the list of Vim commands you recorded.

This can be useful if you mess up while recording and need to modify the keystrokes. After recording the keystroke, paste it to a new line using e.g. `"a]p`. At this point it's just a normal line of text that happens to contain vim commands. You can modify it to add other Vim commands, since they are all just normal keystrokes.

For example, let's say I recorded a command as `qadw2wdeq`, which records to the `a` register (`qa`), deletes a word (`dw`), skips ahead two words (`2w`), and then deletes the next word (`de`), then ends the recording with `q`. But too late, I realize I should have skipped over 3 words, not two words.

I can use `"ap` to paste the contents of the recording, which will look like this: `dw2wde`. Then I can use `f2` to jump to the `2` digit, followed by `r3` to replace it with a `3`. Now I can use `"ayiw` to replace the contents of the register with `dw3wde`.

Now if I want to play back that modified command, I can just use `@a` as usual.

8.6. The Yanky.nvim Plugin

Yanky.nvim has some niceties such as improving the highlighting of text on yank and preserving your cursor position so that you can keep typing after pasting, but its primary feature is better management of your clipboard history. LazyVim also configures it with several new keybindings to make putting text more pleasant.

The plugin is not enabled by default, but it is a recommended extra, so if you followed my suggestion of installing all recommended extras back in Chapter 5, you may have it enabled already. If not, head to `:LazyExtras`, find yanky.nvim and hit `x`. Then restart Neovim.

Now that Yanky is enabled, the easiest interface to see your clipboard history can be accessed with `<Space>p`. It pops up a picker menu of all your recent clipboard entries. Up to a hundred entries are stored, which is a lot more than you get in the numbered registers, and it stores your yanks, not just your deletes and changes. If you need to paste something that is no longer in the clipboard, `<Space>p` is probably the quickest way to find it.

Another super useful keybinding is `[y`. If you invoke this command *immediately* after

a put operation, the text that was put will be replaced with the text that was cut or copied prior to the most recent yank. And if you press it again, it will go back one more step in history, up to 100 steps. So if you aren't sure exactly which numbered register a delete operation is in, or you want to access text that was yanked but is no longer in the `"0` register, you can use `p[y[y[y…` until you find the text you really wanted to pasted. If you go too far, you can cycle forward with `]y`.

LazyVim also creates some useful keybindings to improve how text is put, especially with respect to indentation. The two most useful are `[p` and `]p`.

These commands will paste the text in the clipboard on the line above or below the current line, depending on whether you used `[` or `]`. You may think this would be identical to the automatic Line-wise pasting described above, but it's slightly different for two reasons:

- It pastes on a new line regardless of what command was used to cut or copy the text that is in the clipboard.
- It automatically adjusts the indentation of the text on the new line to match the indentation of the current line.

So if you're moving code into a nested block and need to change indentation, use `]p` instead of relying on Line-wise paste. Then you don't have to format it afterwards. (Not that formatting is hard in LazyVim; it happens on save).

You can also use `>p`, `<p`, `>P`, and `<P` to automatically add or remove indentation when you put code.

8.7. Summary

This chapter was all about selecting and copying text. We learned the `yank` verb for copying text and then dug into the various Visual modes that can be used for selecting text.

Then we learned that Vim has multiple "clipboards" called registers, and how to cut and copy to or paste from those registers. We even went into more detail about using registers to record multiple separate command sequences before discussing the yanky.nvim plugin to make your pasting life a little easier.

In the next chapter, we'll learn about various ways to view and work with multiple open files as well as how to show and hide code with folding.

Chapter 9. Buffers and Layouts

No matter which programming language you are working with, it is inevitable that you will be working on multiple files at a time. And in multiple areas within the same file.

Like all coding editors (other than Notepad), Neovim has a robust system for working with multiple files. LazyVim is configured with a powerful buffer, file, and window management system that may feel familiar at first, but is actually far more powerful than your average editor.

9.1. Some Terminology

Sometimes it seems like every window management system uses the same words for different things. If you read the documents for e.g. tmux, emacs, kitty, vim, and i3, you'll end up with multiple definitions for words like "window", "pane", "tab", and "layout".

I'll stick with the Vim definitions of these words so that you can switch between this book and most Vim and Vim plugin help files, tutorials, and documentation, without getting confused. Unfortunately, this may mean you get confused when interacting with any other software!

This list goes roughly from least to most specific, though understand that the relationship between most of these elements is a graph rather than a tree; it's not a strict hierarchy.

Server

Neovim can run in a server mode and can have multiple clients attached. This means you can have multiple views into the same Neovim instance, and those views could be from different terminals or GUI software, web browsers or even a VS Code Extension. You will probably not need to think about the Neovim server, and I won't mention it again in this book. But if you want to do something interesting such as connect to an existing Neovim instance to open a commit message rather than opening Neovim in a new window, you now know that this is possible.

Client

A Neovim application that you are actually running. Normally connected to its own independent server but can be configured to connect to an existing or remote one. A terminal client starts up when you type `nvim`, but other clients

include GUIs such as Neovide or VimR.

Tab

One client can have multiple tabs. Each tab is a full screen layout that is more or less independent from the other tabs. You can have different buffers visible and different configurations of window splits on each tab. Only one tab is visible at any one time. This is a much different paradigm from VS Code and many other environments where each split has its own set of tabs.

Window

Also known as a "pane" or a "split", a window is a section of the screen that is dedicated to viewing a buffer. Every tab has one or more windows on it. Every window is normally entirely visible; there is no overlap between windows. The excepton is floating windows such as the ones that pop up when you open a picker or Lazy Extras. If a buffer's contents don't fit in a window the window can be scrolled.

Buffer

This is Vim's word for a file that is currently open and available to be viewed/edited. One buffer can be displayed in multiple windows, which means you can have two side-by-side views into the same file at different scroll positions or you can view the same buffer in multiple tabs. If a buffer is visible in two places, they will have the exact same contents (other than scrolling position). There is only ever **one** underlying buffer open for each file, no matter how many views of the buffer are visible in different windows or tabs.

Fold

Within any one view of a buffer, it is possible to "collapse" a section of that file (for example a function, class, or indentation level) into a single line, effectively hiding the contents. This allows you to view two disjointed sections of the same file at the same time while keeping the unrelated information between those two sections hidden.

File

A file that exists on disk. Each buffer is linked to at most one file, though it is possible to have buffers with no file (sometimes called "scratch" buffers, a word borrowed from Emacs parlance). The contents of a buffer may not be the same as the contents of the file on disk if the buffer has not been saved.

So far in this book, all your interactions have happened in a single window in a single tab, with possibly multiple buffers open. Now, things are about to get much more

interesting.

9.2. Buffers

If you've used a picker, the explorer, or mini.files to open multiple files, you may well think that a buffer is a tab. In this view, I have three buffers open, only one of which is currently visible:

Figure 33. Bufferline With Three Buffers

Yes, I know they look like tabs in any other software, ever, but that is because LazyVim has configured the buffer line to look like tabs. With the buffer line visible, you may actually not need to use (real) Vim tabs very often, but in Vim, tabs are a completely different concept.

No matter how many **windows** you have open, there is only one buffer line. In the following screenshot, I have three buffers open, and two of them are visible in separate windows, side by side. But there is still only one buffer line along the top of the editor.

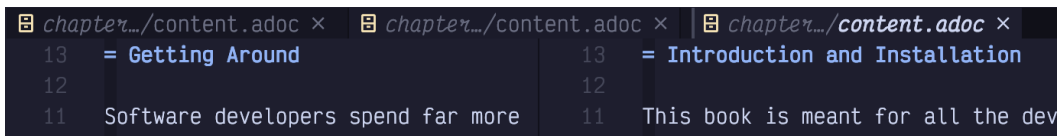


Figure 34. Only One Buffer Line

This implies that buffers are a "global" concept. There is one collection of buffers for the entire Neovim client, and you can access any of those buffers from any window (or tab).

You can, of course, use the mouse to select different buffers from the buffer line with a single click. But why would you do that when there are **so many** ways to access buffers with the keyboard in LazyVim?

9.2.1. Navigating Between Open Buffers

The absolute easiest way to switch between buffers is using the `H` and `L` (i.e. `Shift-h` and `Shift-l`) keys. By this point you are hopefully intimately familiar with the fact that `h` means left and `l` means right for cursor movement. If you press the shift key with these, you will switch the buffer visible in the currently active window to whichever buffer is to the left or right of the current buffer in the buffer line.

Alternatively, you can use the `[b` and `]b` commands which map to the same thing.

Annoyingly, you will find that these keybindings do not accept counts. So you cannot, by default, use `2L` to jump two tabs to the right. This frustrated me because I know the underlying `:bnext` and `:bprev` commands *do* accept a count.

It turns out that LazyVim maps these to a `BufferLineCycleNext` command provided by the underlying plugin, bufferline.nvim, and that plugin doesn't, as far as I can tell, support counts.

Upon investigation I learned that the `BufferLineCycle*` commands exist because the plugin can configure some kind of sorting mechanism on the buffer list. But LazyVim isn't configured to use that mechanism. So we can use the old-fashioned commands instead. To do so, create a new file in your `plugins` configuration folder named (something like) `extend-bufferline.lua`:

```lua
return {
  "akinsho/bufferline.nvim",
  keys = {
    {
      "L",
      function()
        vim.cmd("bnext " .. vim.v.count1)
      end,
      desc = "Next buffer",
    },
    {
      "H",
      function()
        vim.cmd("bprev " .. vim.v.count1)
      end,
      desc = "Previous buffer",
    },
    {
      "]b",
      function()
        vim.cmd("bnext " .. vim.v.count1)
      end,
      desc = "Next buffer",
    },
    {
      "[b",
      function()
        vim.cmd("bprev " .. vim.v.count1)
      end,
      desc = "Previous buffer",
    },
  },
}
```

Listing 23. Simplify Buffer Navigation Keybindings

If you have disabled buffreline as I have, you don't need the above. `]b` and `]b` are already mapped to `bnext` and `bprev` in the default LazyVim keybindings.

The `vim.v.count1` variable is set whenever a keybinding is called with a count, so it

can be accessed inside the callback and passed to the Vim command using string concatenation (the `..` operator). Restart Neovim and you can do things like `3L` to jump three buffers to the right on the buffer line.

Another keybinding you will want to reach for when jumping between buffers is `<Space><Backtick>`. This one simply jumps between the current file and the file that was most recently opened in the current window. In Vim parlance, this is referred to as "the alternate file."

If you have a large number of buffers open, the buffer line can get awfully crowded. At some point, it will show two arrows to the left and/or right of the buffer bar so you can tell that there are "hidden" buffers. When you navigate through buffers, it will always ensure the active buffer is visible. Here's a very full buffer line with four buffers hidden to the left and two hidden to the right:

Figure 35. Full Buffer Line

If you have this many buffers open, you may find it easier to use a picker to search through the open buffers. The keybinding to pop up a filterable, scrollable list of currently open buffers is `<Space><comma>`. It has the exact same contents as the buffer line, but it's a different interaction effect.

This is useful if you are working on large projects that have so many files that searching through them with `<Space><Space>` is difficult. Open the relatively low number of files that you actually need to access as active buffers, so they are easy to filter in the `<Space><comma>` buffer list.

Personally, I have disabled the Bufferline plugin altogether, and `<Space><comma>` is the primary interface through which I manage my buffers.

9.2.2. Closing Buffers

You will often want to close the current buffer without closing the split(s) it is currently open in. The keybinding for this is `<Space>bd`, where `<Space>b` pops up a useful menu of other buffer related functions, and `d` means "delete". You aren't actually deleting the underlying *file* when you do this; you're just deleting the buffer from Vim's memory: i.e. closing it.

Here are several other commands you can use to close buffers:

Keybinding	Description	Mnemonic
`<Space>bD`	Close buffer and the window split it is in.	**D**elete buffer, but "bigger"
`<Space>bl`	Close all buffers to the left in the tab line	
`<Space>br`	Close all buffers to the right in the tab line	
`<Space>bo`	Close all buffers other than the active one	"**o**nly" this buffer
`<Space>bP`	Delete all non-pinned buffers	"P" is opposite of "p"

Table 1. Closing Buffer Commands

The last one needs some clarification. You can toggle a "pin" on any active buffer using `<Space>bp`. You'll see a pin icon show up to the left of the buffer name. The only purpose of this pin is to keep it open if you want to close all the "less important" (unpinned) files using `<Space>bP`. I personally don't use buffer pinning, so for me, `<Space>bP` is a shortcut for "close all buffers"; useful when I complete one task and am ready to start another.

I find that some "close buffer" incantations is too common a task to deserve three keys, so I would have added the following three shortcuts to my `keymaps.lua`:

```
vim.keymap.set("n", "<leader><delete>", function()
  Snacks.bufdelete()
end, { desc = "Close Buffer" })
vim.keymap.set("n", "<leader>bo", Snacks.bufdelete.other, {
desc = "Close Other Buffers" })
vim.keymap.set("n", "<leader><CR>", "<cmd>%bd<cr>", { desc =
"Close All Buffers" })
```

Listing 24. Close Buffer Keymap

The first two behave like `<Space>bd` and `<Space>bo` to close the current or all other buffers, respectively. The third one closes all buffers using an ex command and range syntax we will discuss as part of searching in chapter 12.

You can also close buffers directly from the buffer picker interface (from `<Space>,`) using `Control-x` to close the buffer that the cursor is on.

9.2.3. Scratch Buffers

Scratch buffers are handy areas to keep notes about a project. Historically, I've always kept my notes in a separate new window, and saved it with some arbitrary name if I realize I need it to last longer than one editing session. Except... sometimes I forget to save it.

Scratch buffers are tied to the current working directory. To open one, just use `<Space>.`. It pops up a floating window like this:

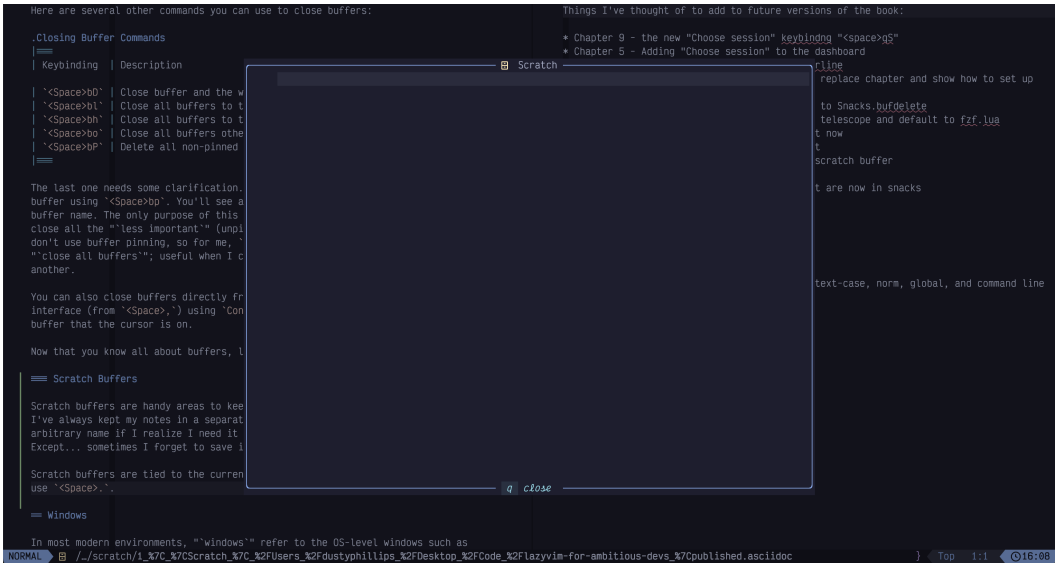


Figure 36. Scratch Buffer

Type whatever you want in there, hit `<Escape>` to switch to normal mode, and `q` to close the buffer. Close and reopen Neovim if you want; the scratch buffer doesn't care. Your notes will be saved until you need them again. Then just hit `<Space>.` again and, so long as your cwd is the same as it was when you last opened the scratch buffer, there they are.

Occasionally, you may need to open a scratch buffer that was saved on a different cwd or git branch. You can see a picker list of scratch buffers using the `<Space>S` shortcut.

Now that you know all about buffers, let's discuss windows.

9.3. Windows

In most modern environments, "windows" refer to the OS-level windows such as the terminal you are running Neovim in. Since Vi predates such environments, they were able to use the word window to refer to what are nowadays more commonly described as "panes" or "splits" in other environments.

The window management commands are collected in the `<Space>w` submode menu:

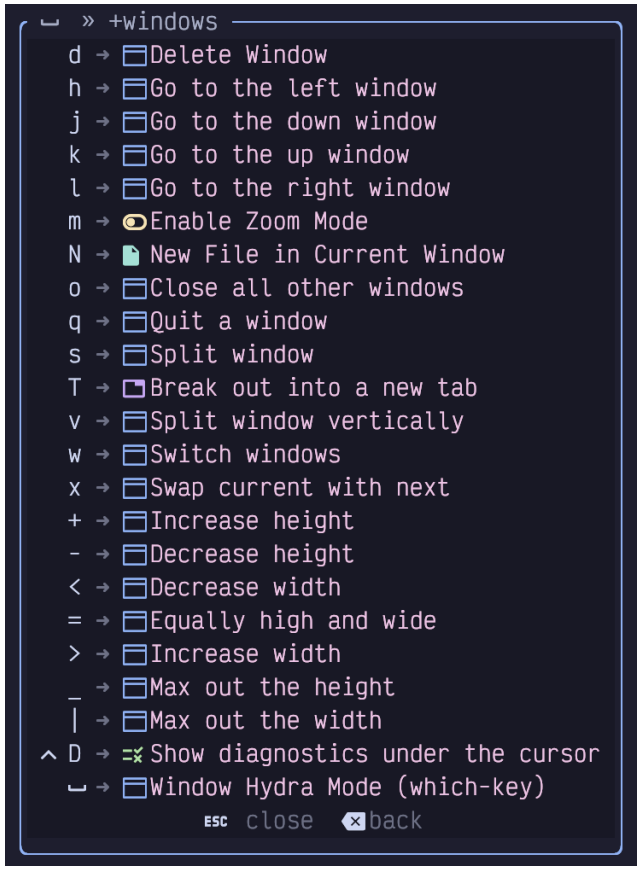


Figure 37. Window Menu

We'll cover many of these in the following sections.

This menu can also be accessed with `Control-w`. Historically, this is the the keybinding that is enabled by default in Vim and Neovim, though LazyVim has added some extra keybindings to it. However, `<Space>w` is a bit easier to type.

9.3.1. Creating Window Splits

Windows in LazyVim can be created on the fly at any time. To split the current window in half "vertically" with one window on the left and a new window on the right, use the `<Space>wv` keymap.

When you create a split, the new window will contain another view of the buffer you were already viewing, side by side. But once the split is opened, you can switch the buffer in that window using any of the buffer management commands or by opening a new file with any of the tools we've previously discussed for opening files.

To create a horizontal split between two windows (one above the other), use `<Space>ws`. The mnemonic for this is unfortunately just "split". They weren't able to reuse `<Space>wh` because that is already used for switching windows.

LazyVim also allows you to create a vertical split with `<Space>|` where the `|` is the vertical bar when you `Shift-Backslash`, and `<Space><Minus>` for a horizontal split. I find `<Space>ws` and `<Space>wv` easier to type.

9.3.2. Creating Splits When Opening Files

You already know you can open a file in the current window from Neo-Tree by moving your cursor to the file and pressing `<Enter>`. You can also use the `s` key in Neo-Tree to open it in a *vertical* split (irritatingly asymmetric with the `<Space>ws` key to create a *horizontal* split in a normal buffer). The shifted form, `S` in Neo-Tree is used to create a horizontal split.

If you are using a picker to open files, you'll use yet another set of keybindings! To open a file in a vertical split, use the `Control-v` keybinding. To open it in a horizontal split you use `Control-s`.

Finally, if you use mini.files, you can open a file in a split sensibly using the same keybindings as in a normal window (`<Space>wv` and `<Space>ws`).

9.3.3. Navigating Between Windows

You can move your cursor between window splits by holding the control key along with any of the `h`, `j`, `k`, or `l` home row "arrow-key" directions. They can also be prefixed with numerical counts if you want to skip over a window to get to the next one.

Alternatively, you can use the same keys with `<Space>w`. So `<Space>wh` will move to the window to the left of the current one.

Smart Splits

I suggest the mrjones2014/smart-splits.nvim [https://github.com/mrjones2014/smart-splits.nvim] plugin, which can be configured to navigate between Vim windows and Kitty, Wezterm, or Tmux panes with the same keybindings. Consider this screenshot:

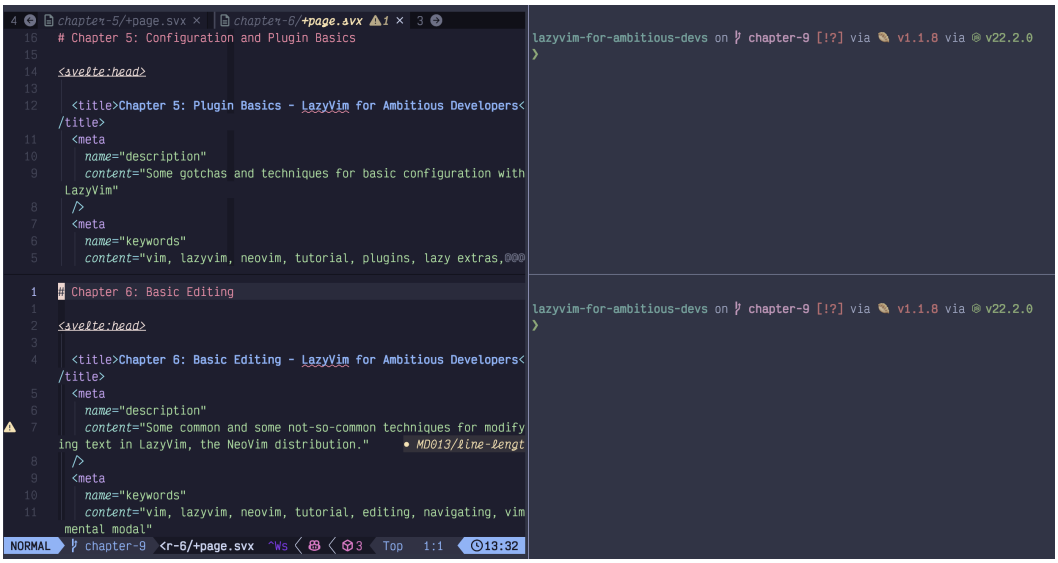


Figure 38. Kitty and Vim Splits

I have three Kitty Terminal panes open. The left one is running Neovim with two windows in it, one above the other. The right is split into two normal terminal panes. By default, if I want to navigate between the three Kitty panes, I have to use one set of keybindings, and if I want to navigate between the two Neovim windows, I have to use another set of keybindings. With the smart-splits.nvim plugin, I can navigate between all the windows with the same keybindings, no matter where my cursor is.

Setting up the terminal integration for smart splits is beyond the scope of this book (documentation on the README in the GitHub repository should be sufficient), but to configure the smart-splits plugin in Neovim, create a file in the plugins directory called e.g. `smart-splits.lua`:

```
return {
  "mrjones2014/smart-splits.nvim",
  build = "./kitty/install-kittens.bash",
  keys = {
    {
      "<A-h>",
      function()
        require("smart-splits").move_cursor_left()
      end,
      desc = "Move to left window",
    },
    {
      "<A-l>",
      function()
        require("smart-splits").move_cursor_right()
      end,
      desc = "Move to right window",
    },
    {
      "<A-j>",
      function()
        require("smart-splits").move_cursor_down()
      end,
      desc = "Move to below window",
    },
    {
      "<A-k>",
      function()
        require("smart-splits").move_cursor_up()
      end,
      desc = "Move to above window",
    },
  },
}
```

Listing 25. Smart-splits configuration

If you are using WezTerm or Tmux you won't need the `build =` line, but for all three environments, you'll also need to add some configuration to your Kitty, WezTerm, or Tmux configuration as described in the plugin's README.

9.3.4. Closing a Window Split

You can close a window at any time using one of three keybindings:

- `<Space>wq` closes the window, and if it is the only window open, exits (**q**uits) Neovim.
- `<Space>wc` **c**loses the window unless it is the only window open, in which case it displays an error and refuses to close.
- `<Space>wd` **d**eletes the window. It is actually doing exactly the same operation as `<Space>wc`, but it is helpful for muscle memory for its symmetry with `<Space>bd` to "delete" an open buffer.

In all three cases, the **buffer** remains open in the bufferline. Only the window split is closed.

If, instead, you want to close all splits except the active one, use `<Space>wo` for "**o**nly this window" or "close **o**ther" (whichever is easier to remember).

9.3.5. Resizing Windows

In my unconventional opinion, the easiest way to resize Vim splits is to use... *the mouse*. There is a vertical bar between vertical splits that you can click and drag on. The mouse cursor doesn't change to give you any feedback that you can click and drag on it, but it works.

For horizontal splits (when two windows are one above the other), there is no obvious bar to click. But you can actually just drag on the status bar of the "upper" window to move it up or down.

If you insist on using the keyboard, the keybindings are in the `<Space>w` menu: `<Space>w+` and `<Space>w-` to increase or decrease the height of the active window in a horizontal split, and `<Space>w>` or `<Space>w<` to increase or decrease the width of a vertical split. They only move by one row or column at a time, so you will almost certainly want to prefix these commands with a count greater than 10, or use "Hydra" mode, discussed next.

To change everything to a "default" size, use `<Space>w=` which will make all the windows "equally high and wide."

9.3.6. Hydra Mode

Occasionally, you'll want to run several window commands in a row, for example

when resizing a window or creating a layout containing several splits. Having to type `<Space>w` between each command would be rather tedious in these situations, so the `which-key` plugin gives us Hydra mode.

To enter Hydra window mode, press `<Space>w<Space>`. All this does is "pin" the window menu open, so you can issue multiple keypresses from it without automatically leaving the menu. For example, `<Space>w<Space>vvvs` will create four splits (three vertical and one horizontal).

To leave Hydra mode just hit `<Escape>` and return to your normal editing.

9.3.7. Zoom and Zen Mode

To get into or out of zen mode, use the `<Space>uz` keybinding, which is mapped to an action in `snacks.nvim`. under the UI menu. Zen mode opens a new window centred in front of your existing files, dimming them in the background, like this:

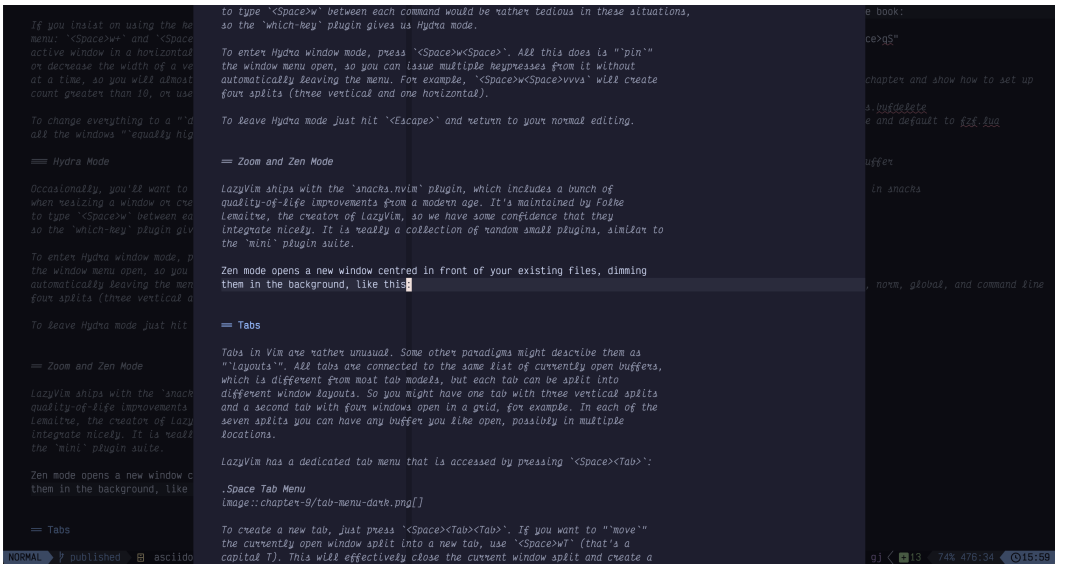


Figure 39. Zen Mode

Syntax hyghlighting is disabled except in the immediate vicinity of your cursor. Ostensibly, this helps one focus. I haven't generally had a lot of trouble focusing so I don't use it much, but folks with attention deficit disorder may find it less distracting than default layouts.

Zoom mode is similar in that it temporarily changes the window layout. It simply replaces all the existing windows with a single maximized window containing the current file. You can access it with `<Space>uZ` where `Z` is capitalized this time.

This can be handy if you need to temporarily focus on or expand a buffer. Historically, I've always gotten this behaviour by opening the same buffer in a new tab, so lets discuss tabs next.

9.4. Tabs

Tabs in Vim are rather unusual. Some other paradigms might describe them as "Layouts". All tabs are connected to the same list of currently open buffers, which is different from most tab models, but each tab can be split into different window layouts. So you might have one tab with three vertical splits and a second tab with four windows open in a grid, for example. In each of the seven splits you can have any buffer you like open, possibly in multiple locations.

LazyVim has a dedicated tab menu that is accessed by pressing `<Space><Tab>`:

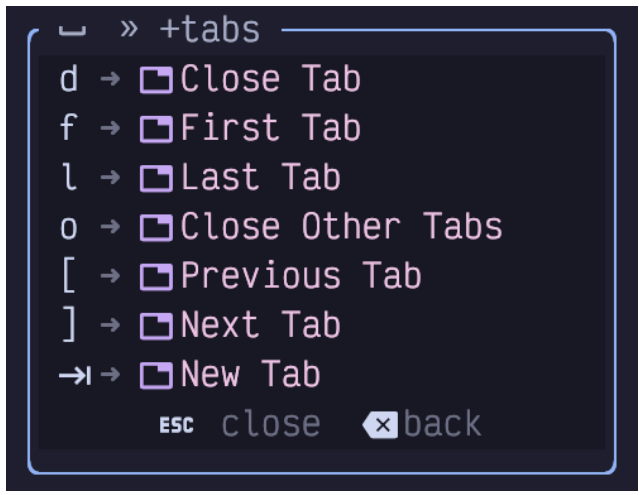


Figure 40. Space Tab Menu

To create a new tab, just press `<Space><Tab><Tab>`. If you want to "move" the currently open window split into a new tab, use `<Space>wT` (that's a capital T). This will effectively close the current window split and create a new tab with the same buffer in that tab.

After you create a tab, you'll likely have trouble finding it again! The tabs are grouped at the right end of the buffer line:

Figure 41. Tabs in Buffer Line

This screenshot has **two** tabs, numbered 1 and 2 at the right with an `X` beside them.

The two buffers at the left *are not tabs*. Have I emphasized that too many times?

Unfortunately, other than numbers, the tabs don't do anything to make themselves look unique; it is not possible to preview which buffers are active in each tab or what layout they have.

To navigate between tabs, you can click the numbers, or you can use the default vim keybindings of `gt` and `gT` to go to the next or previous tab. Alternatively, the `<Space><Tab>[` and `<Space><Tab>]` keybindings provided by LazyVim can also switch tabs. To go to a specific tab by number, use that number as the count when calling `gt`. For example `3gt` will show the tab number 3 rather than jumping three tabs to the right.

There are several ways to close a tab:

- Just close the last window in the tab (i.e. with `<Space>wq`) and the tab will disappear.
- The `<Space><tab>d` keybinding will close all the *windows* in a tab as well as the tab itself. The *buffers* will stay open.
- Click the `X` icon to the right of the tabs in the tab bar.

9.5. Code Folding

Vim's code folding system is almost too robust, probably because it has had many iterations of "best practices" over the years. LazyVim is configured with the *current* best practices, so you generally only need a small subset of the complete list of folding commands.

If you are unfamiliar with the concept, code folding allows you to hide entire sections of code by collapsing them into a single line. Visually, this has a similar effect to splitting a window horizontally and then reading two sections of the same file above and below the split, but when you use folding only one view of the buffer is visible and it scrolls as a single entity.

Consider this section of code:

Figure 42. Code With No Folds

While I'm editing, imagine that I am interested in the `clearExistingTimeout` function at the top of the screenshot and the `addTodo` function at the bottom, but not currently interested in the contents of the two `save` callbacks. I can collapse those sections and my screen looks like this:

Figure 43. Collapsed Folds

Most fold operations are accessible from the `z` mode menu accessible by typing `z` in Normal mode (we discussed some of the `z` mode operations in chapter 3 when we were dealing with scrolling). To collapse a section of code into a fold, use whatever navigation operations you like to get to that section and type `zc` for "**c**ollapse fold".

To open it again, use `zo` for "**o**pen fold".

Alternatively, if you only want to remember a single keybinding, `za` will toggle a fold,

collapsing if you are not on a folded line and expanding if you are.

If you have collapsed some folds and want to quickly get back to a point where there are no folds collapsed, use `zR` to open all folds. I had no idea what mnemonic is supposed to work with `R`, but an early reader helpfully pointed out that `zr` is "reduce folding", so `zR` is "Reduce folding BUT BIGGER".

You can even nest folds by folding already folded code. If you want to open folds recursively, use `z0`, which will open a fold and any folds that are nested under that fold.

The way LazyVim is configured, you don't have much control over what gets folded, but it will usually do something close to what you expect based on where your cursor is in the document. "What you expect" depends on both the LSP and the TreeSitter grammar for the language you are working on, but I find it best to just let it do its thing and not disagree with it.

If you find that you want way more control over code folding, I recommend reading `:help folding` in its entirety. More than likely, you'll decide that actually, you don't want more control over folding and just want LazyVim to handle it for you!

9.6. Sessions

After a long hard day of coding, you probably have several buffers open and your splits and tabs configured with all the files in just the right places. Wouldn't it be nice to put the code away for the evening and come back to all those buffers, tabs, and splits just as they were?

LazyVim has built-in session management enabled by default. Simply close LazyVim with `<Space>qq` and you're done. Tomorrow morning, use your terminal to `cd` to the folder you were working in. Open the session to the dashboard with the `nvim` command and hit `s` to be on your way.

If you forgot to open it right away and the dashboard is long gone, you can use `<Space>qs` to restore Neovim to wherever it was when you last closed it (though any files you modified and saved in the meantime will still have their new contents).

Sessions are scoped to folders. So every time you change your working directory a different session is stored and kept up to date until you exit. To open a session, either cd to a directory in your terminal before opening Neovim, or use the `:cd` command after opening it.

Alternatively, use the `<space>qS` command after you open Neovim. This will open a picker with all the folders you have recently worked in. Select one of those folders and LazyVim will automatically `:cd` to that folder and open the session.

If you use a graphical frontend to Neovim such as Neovide or VimR, the `<Space>qS` command can be super useful for you to select a recent project. In fact, it may be so useful that you want to add a command for it to your dashboard, as discussed in Chapter 5.

One last note on sessions: if you have opened Neovim temporarily and want to close it without wiping out the session that was saved the last time you closed it, hit `<Space>qd` at any time after you open Neovim and before you close it. In some contexts (notably, git commit messages), this happens automatically so you don't have to worry about making a commit after you close the editor and then losing your session.

9.7. Summary

In this chapter, we learned about Vim's buffers, windows and tabs, and how they are different not only from each other, but also from many other window management paradigms. Vim has the same concepts as other editors, but they are sometimes mixed or named in different ways.

We also covered code folding to make it easier to wrangle large files, and session management to return to your window configuration and come back to it later. This is particularly useful when combined with LazyVim's lightning fast startup time. There's no reason to keep your code editor open consuming memory when you aren't actually, you know, editing code.

In the next chapter, we'll dig into some of the terrific programming language support that LazyVim provides. This is arguably the one thing that made VS Code amazing, but the Vim community has learned from its competitors and eventually outmatched them.

Chapter 10. Programming Language Support

Visual Studio Code brought the world the concept of language servers, and all other text editors jumped on the idea. Early incarnations of language server protocol in Vim were frustrating and clunky and required plugins that tended to be fragile and complicated.

Then Neovim decided to build support for language servers into the editor itself. Neovim's built-in support is still frustrating and clunky, but over time, robust and simple plugins have evolved to make the language server experience almost automatic. LazyVim represents the pinnacle of that evolution.

In addition, Neovim also has built-in support for TreeSitter, a powerful library for parsing and identifying abstract syntax trees in source code while it is being edited, and LazyVim is configured with the plugins needed to make TreeSitter Just Work™.

Language Server protocol gives us support for things like code navigation, signature help, auto-completion, certain highlighting and formatting behaviours, diagnostics, and more. TreeSitter gives us better syntax highlighting, code folding, and syntax based navigation such as provided by the `S` command you already know.

There are two main tools for working with language servers in LazyVim: various language Lazy Extras, and the Mason.nvim plugin. We'll get to know both of these and then learn how to better use some of the tooling they provide.

10.1. The lang.* Lazy Extras

We've already used LazyVim extras for plugin configuration, and I told you to install the extras for whichever languages you use regularly. These extras include preconfigured plugins that give best-in-class support for common programming languages. Most ship with preconfigured language servers and many include additional Neovim plugins that are useful with those languages.

Once you install these extras things will usually work out of the box, and you won't have to learn any new keybindings for the commands each language provides. However, it wouldn't hurt to read the Readmes for the plugins the extra installs (accessible by looking up the Extra's documentation on the LazyVim website and clicking the headings) to make sure you aren't missing out on any commands the language provides. For example, the Python extra ships with the venv-selector.nvim plugin that allows you to activate many types of Python virtual environments either

automatically or on demand. LazyVim installs a keybinding to open the virtualenv selector using `<Space>cs` where `<Space>c` is the "Code" submode.

10.2. Mason.nvim

The Lazy Extras may not install everything you need. For example, instead of the default Typescript formatting and linting tools, I prefer to use a new hyper-fast up-and-comer called Biome.

To install things like this, you can use the Mason.nvim plugin, which is pre-installed with LazyVim. To open Mason, use the `<Space>cm` keybinding. The window that pops up looks similar to the Lazy.nvim and Lazy Extras floating window, although it ships with annoyingly unrelated keybindings.

Mason is a very large database of programming language support tooling, including language servers, formatters, and linters, along with the instructions to install them.

Mason.nvim *does* assume a certain baseline is already installed on your system; for example if you are going to install something that is Rust-based, you better have a `cargo` binary, and if you are going to install something that requires Python support, Python and pip need to be available. In most cases, if you are coding in a given language, you already have the tools Mason needs to do its thing. The main thing that Mason takes care of is ensuring that the tools are installed in such a way that other Neovim plugins can find them.

The hardest task with Mason is knowing what tool you want to install. I was already using Biome when I set up LazyVim, so I knew I was going to need to install editor support for it. That was no problem; just find `biome` in the Mason list (like any window, it is scrollable, searchable, and seekable, and Mason helpfully puts everything in alphabetical order).

But when I started working on this book, I decided I needed an advanced Markdown formatter, and I had no idea which one to use. I could search the window for `markdown` and then press `Enter` on any matching lines, which gives a description and some other information, but I had to do some research with a web browser, (along with a little trial and error) before I found the right tool for me.

Unfortunately, I can't help you with figuring out what is right for you, but once you find the tool in Mason, just use `i` to install the package under the cursor. The only other command you will use frequently in Mason is `Shift-U` to update all installed tools, and you can look up the rest with `g?`.

10.3. Validating Things Installed Cleanly

As good as both LazyExtras and Mason are at installing language servers, linting, and formatting tools, setting them up is one of the places most likely to go wrong, no matter which editor you are using. So now is a good time to introduce several commands to validate that things are working as expected.

First, LazyVim pops up notifications in the upper right corner, as you have seen with the plugin updates. These disappear after a few seconds. Every once in a while, you need to be able to refer back to them.

The secret is to use the keybinding `<Space>sn` to open the "Noice" search menu. Noice is the plugin that provides those little popups. Most often, you'll want to follow this with either an `a` or an `l` to see all recent Noice messages, or just the last one.

You can also use `<Space>snd` to dismiss any currently open notifications, but honestly, by the time you've completed those four keystrokes, they notifications have probably disappeared themselves already!

The second command you'll need for debugging LSPs is `<Space>cl`, which runs the command `:LspInfo`. It displays information about any language servers that are currently running and which buffers they are attached to. For example, while editing a Markdown document, my LSPInfo window looks like this:

Figure 44. Lspinfo Window

In this case, everything looks fine (though I'm surprised the tailwind server is

associated with Markdown), but if your LSP isn't behaving correctly, this window might give you a hint as to what the problem might be.

If your LSP is having temporary problems—like showing incorrect diagnostics or unable to find a file you know is there—sometimes it just needs to be given a good kick with `:LspRestart`. The Svelte language server has a nasty habit of not picking up new files, so I've been using this one often enough lately to add a keybinding for it.

Two other super useful commands are `:checkhealth` and `:LazyHealth`. Both provide information about the health of various installed plugins. The former is a Neovim command that plugins can register themselves with to provide plugin health information, while the latter provides LazyVim-specific information. There is a lot of overlap in the output, but I find the `:LazyHealth` output is easier to read, and the `:checkhealth` output to be a bit more comprehensive. So I usually use `:LazyHealth` first and switch to `checkhealth` only if `:LazyHealth` didn't yield the answer I need.

Don't expect to see green check marks across the board; you'll make yourself crazy. For example, my `checkhealth` output contains a bunch of warnings from Mason:

Figure 45. Lazy Health Warnings

Tools that I have used recently (and also Ruby for some reason) are installed, and I have warnings for languages that I don't generally need to edit files in. So if you don't code in Java, there's no reason to waste cycles trying to make the `java` warning go

away.

10.4. Diagnostics

Language Servers fulfill several useful functions, including identifying code problems, linting, formatting, context-aware code navigation, and documentation. We'll discuss all of these between this and the next chapter.

We already got a peek at diagnostics in Chapter 7, when we discussed jumping between error messages with the unimpaired keybindings `[d`, `[w`, `[e`, `]d`, `]w`, and `]e`. Diagnostics show up as little squiggles under specific sections of text and when you jump to them, you usually get a small overlay window telling you what is wrong at that location. For example, I have a simple typo causing an error in this screenshot:

Figure 46. Diagnostic Overlay

I misspelled "tracingMiddleware", and I get a helpful error message on that line in the virtual text, and a window pops up when I navigate to that error with `]d`. This window sometimes has more information than the virtual text. In addition, the line that imports the correctly spelled variable is showing a hint telling me that it isn't used.

The colour of the diagnostic conveys the severity—whether it is a hint, a warning, or an error—so you can decide whether it is valuable to fix. I generally try to either fix or silence all diagnostics, as they become less useful if there is much noise.

If the window doesn't pop up when you navigate to the diagnostic, you can use the `<Space>cd` keybinding to invoke it as long as your cursor is positioned somewhere within the underlined text. You can make the window disappear by moving your cursor with any motion key.

10.4.1. Trouble and Quick Fix

You can also navigate diagnostics using the Trouble menu. Trouble is a LazyVim plugin that provides an "enhanced quick fix" experience. Which is probably meaningless to you if you are new to Vim and don't know what "quick fix" means!

The Quick Fix window is essentially a list of files and line numbers that have been tagged as "interesting" for some reason, where that reason depends on context. It can be used to represent multi-file search results, diagnostics, compiler error messages, and more, depending on how you open it. You can easily hop between the targeted locations, making changes or corrections without losing the context of what you were originally searching for.

In its simpler form, Trouble is the same thing, just a little bit prettier to look at, with colours, icons, and nice groupings when fix locations are in multiple files.

The *contents* of the Quick Fix and Trouble windows depend entirely on how you open them. Most of them are accessible from the `<space>x` (I assume the `x` stands for "fi**X**") menu, which looks something like this:

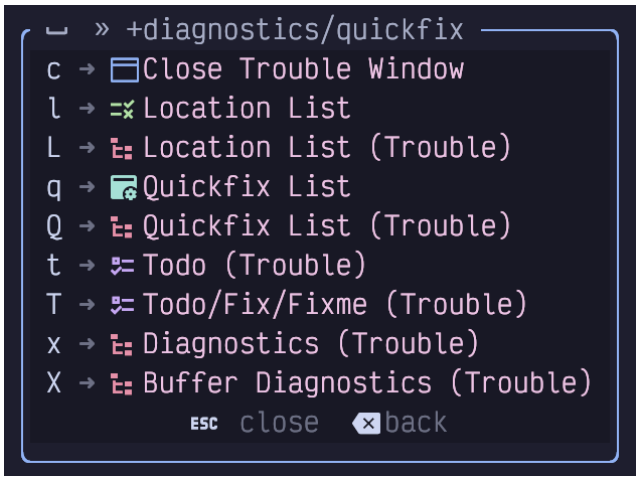


Figure 47. Diagnostic Menu

Let's take to-dos as an example, as I have a lot of them in this book. It's weird saying that because they'll be gone by the time you see it, but this screenshot will live on:

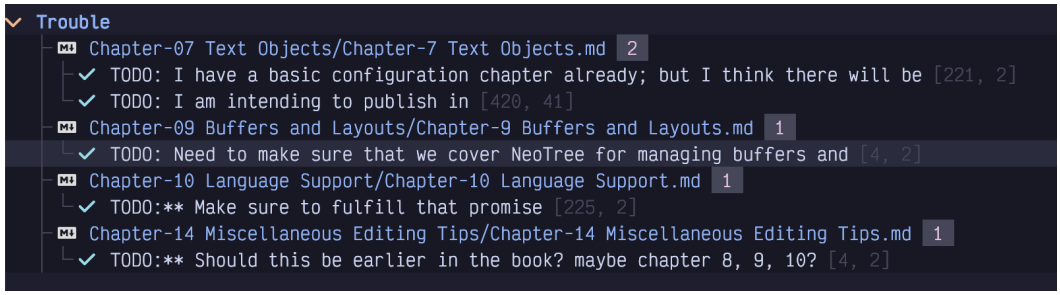


Figure 48. To-Dos List

The cool thing about this list of locations is that they are not all in the same file. Without Trouble, I could navigate between to-dos in the *current* file using the `[t` and `]t` keys. However, using Trouble, I can navigate between to-dos in multiple files by moving my cursor to the appropriate line in the Trouble window and hitting `Enter`. It will open the file and move the cursor right to the "troubling" line.

Or you can use the commands `[q` and `]q`, which will navigate between Quick Fix OR Trouble locations, no matter which file they are in, without ever focusing the Trouble window.

For diagnostics, open the Trouble menu with `<Space>xx` or `<Space>xX`. The lowercase version shows the diagnostics in the current file for a quick overview while the "but bigger" uppercase X shows all the diagnostics in the current workspace (although it depends a bit on the language servers; some language servers only show you diagnostics for all currently open buffers, not the whole project).

If you're wondering what the "Location List" is, it's a Quick Fix window that is associated with the current window (NOT buffer). I never use it; my brain can only handle fixing one problem at a time, even if it is in hundreds of files!

10.4.2. Trouble and Quick Fix from pickers

The Quick Fix and Trouble windows are not just for references. Many actions can result in a new list of file locations being added to these windows. One common one is to activate it from picker windows. Any picker window can be converted to a list of jumpable locations in the Quick Fix window by showing the picker and hitting `Control-q`. All files will become Quick Fix entries. You can also select a subset of files with `Tab` before sending only those files to Quick Fix with `Control-q`.

Sending files to Trouble from a picker is just as easy. Focus the file or select multiple files and use `Alt-t` to send them to a Trouble window.

This feature is useful in various pickers, but I most often use it with search results and go-to-reference features that we'll discuss in later chapters.

10.5. Code Actions

One of the things that made VS Code seem magical when it came out was code actions. Not that they existed, as the concept has been around for a long time, but that they WORKED. Nowadays, I kind of take them for granted.

You may be used to accessing code actions by moving your hands to the mouse and clicking a light bulb or right clicking a diagnostic. In LazyVim it is (of course) a keybinding. Navigate to a diagnostic using whatever keybindings work for you (I live by `]d`) and then invoke the `<Space>ca` menu where `c` and `a` mean "code action." A picker menu will pop up with a list of any actions you can take. You can use the arrow keys or `<Escape>` followed by `j` and `k` to navigate between them, or you can enter a number or any text from the line to filter. Hit `<Enter>` to perform the action, or `<Escape><Escape>` to cancel the menu (just one escape allows you to enter Normal mode in the search box so you can use the many LazyVim navigation keystrokes that you are, by now, accustomed to).

10.6. Linting

Linting is *mostly* handled using the `nvim-lint` plugin instead of the LSP. This was a major pain point in my pre-LazyVim days because getting the LSP and linter cooperating often required some serious troubleshooting. And then throw formatting into the mix and I'd lose a day or two. To be fair, this was true when I used VS Code, too.

Using LazyVim, it is actually likely that you don't know who is doing the linting for you. I honestly don't. Some of my diagnostics come from the LSP and others come from the linter. I don't bother to question the source of the errors; I just fix them.

The hard part with linting (at least, when it doesn't work automatically) will be making sure that the appropriate linter is installed (Mason has your back here), and configured correctly. If you are lucky and the languages you love to work in have Lazy Extras, then it is probably already configured correctly. Otherwise, you may have to do a little tweaking. The tweaking involved is, sadly, language-dependent, but you'll probably need something like this in a (for example) `extend-nvim-lint.lua` file in your plugins directory:

```
return {
  "mfussenegger/nvim-lint",
  opts = {
    linters_by_ft = {
      typescript = {
        -- lint settings for Typescript
      }
    },
  },
}
```

Listing 26. Nvim-lint Customization

Read `:help nvim-lint` for more information and refer to the LazyVim documentation for this configuration if you need further clarification.

The nice thing is that once you have your linting configured, the errors will show up using the same diagnostics described above and you can engage with them using the same keybindings, Trouble window, code actions, etc.

10.7. Formatting

Similar to linting, code formatting *can* be handled by some LSPs, but people have realized that using the language server is often more complicated than just invoking a formatter directly. So LazyVim ships with the conform.nvim plugin.

Also similar to linting, if you are lucky, it will Just Work™ after you install the appropriate Lazy Extra and/or Mason tool. However, if you don't like the default formatter (or it's not working), you will have to familiarize yourself with the LazyVim and conform.nvim documentation to figure out the exact incantation required.

The only formatter I've had to manually configure is using Prettier for Markdown. It looks eerily similar to the `nvim-lint` configuration:

```
return {
  "stevearc/conform.nvim",
  opts = {
    formatters_by_ft = {
      ["markdown"] = { "prettier" },
    },
  },
}
```

Listing 27. Conform Customization

Once it's set up (I acknowledge this may be no mean feat), formatting in LazyVim is typically fire and forget: save your file and it formats. If you want to invoke it manually without saving, use the `<Space>cf` keybinding. I can't stress how lucky you are that this is the case; without LazyVim, countless hours have been wasted trying to get the autocommands for "format on save" to work!

10.8. Configuring Non-standard LSPs

If you have installed an LSP that LazyVim isn't aware of, you may need to tweak the `nvim-lspconfig` plugin. You will minimally need to let it know that your language server is available, and possibly to configure it to your needs. For example, one of my favourite programming languages is Rescript, which doesn't have a huge ecosystem and therefore, has no LazyVim extra. I was able to install the language server with Mason easily enough, but I also needed to add the following to my `extend-lspconfig.lua` file for LazyVim to pick it up:

```
return {
  "neovim/nvim-lspconfig",
  opts = {
    servers = {
      rescriptls = {},
    },
  },
}
```

Listing 28. Third-party LSP server

As a second example, the `css_variables` language server, which I use with the excellent open-props CSS framework, works out of the box for `css` files, but I needed to use a different configuration to activate it in `svelte` files:

```
return {
  {
    "neovim/nvim-lspconfig",
    opts = {
      servers = {
        css_variables = {
          filetypes = { "css", "scss", "less", "svelte" },
        },
      },
    },
  },
}
```

Listing 29. Css Variable LSP Config

10.9. Summary

In this chapter, we learned how LazyVim integrates the language server protocol that VS Code brought to the world. It is *usually* quick and painless, which is more than can be said for manually configuring LSPs. However, there may be some headaches especially around linting and formatting. This is true in any editor, sometimes they hold your hand and sometimes they get in your way. If you get stuck, hit us up in the LazyVim discussions group on GitHub (but search it first; you're probably not the first person to have trouble).

In the next chapter, we'll learn more about navigating *code* using LSPs, TreeSitter, and several plugins.

Chapter 11. Navigating Source Files

In previous chapters, we've learned many different ways to navigate within a single buffer, as well as between open tabs and windows. This chapter will go into detail of different ways to navigate between source files.

11.1. Go To Definition

In my opinion, "Go to Definition" is the most valuable feature language servers have brought us. Major IDEs have supported it for compiled languages for eons, but dynamically typed languages—such as my beloved Python—have always been hell for static analysis, and such features were often pretty hit or miss.

As one of the best-named editor features of all time, "Go to Definition" jumps your cursor from whatever keyword it is currently sitting on to the place where that keyword is defined, regardless of what file it is in.

Most often, I use this when I am looking at a call site for a function and want to see the function itself. A simple press of `gd` (go to definition also has one of the more memorable LazyVim keybindings of all time) will take me there.

Depending on how good the LSP for the language I am editing is, this often even allows me to jump into library files or type declarations for third party modules so I can see what is really going on.

Go to definition is context dependent, but will usually do exactly what you expect. If you are looking at a variable, `gd` will jump to wherever the variable is initialized. If your cursor is on a class name, it will jump to wherever the class is defined.

Typically, once you've jumped to a definition and learned what you need to learn from that file, you'll immediately want to jump back to where you started. You can do this easily using `Control-o`, as we discussed in Chapter 3 (and `Control-i` can move forward in your jump history).

11.2. Go To References

The inverse of the Go to Definition command is "Go to References". If you are looking at a function, variable, type, etc, and want to see all the places that variable is accessed, use the `gr` command.

Unlike with a definition or declaration, there will typically be more than one

reference to a given word (a variable in isolation is a useless variable, indeed). So when you type `gr` it usually won't immediately jump to a location. Instead it will pop up a picker view of all the references to the word that was under your cursor, with all the preview and filtering luxury that the picker always brings.

It is common to want to perform some action—such as a rename or adding an argument or what have you—at every reference. You *could* keep showing the picker by hitting `gr` again or by using the `<Space>sR` keybinding which resumes your previous picker search. However, it is often much more useful to use the Trouble list that we learned about in Chapter 10.

To do that, use `gr` to show the references in a picker as usual. Then use `Alt-a` to select all results, possibly deselecting some with `<Tab>`. Now use `Enter` to open the selected lines in Quick Fix or `Control-t` to open them in Trouble. Remember, `Control-q` is a shortcut to open all matches in Quick Fix.

11.3. Context-specific Help

Most non-modal editors show you some help or "hover" text when you hold your mouse over a word or symbol. The quantity and value of this text varies widely depending on the LSP, but usually includes a function signature and documentation for the word under the cursor.

It's probably possible to set up Neovim to show help texts on hover, but why would you move your hand to the mouse when LazyVim has such amazing navigation on the keyboard? Instead, use the (shifted) `K` keybinding. Yeah, `K` is a pretty stupid mnemonic to remember, but `H` and `?` were already taken.

In fact, the `K` stands for "keywordprog", which is a legacy Vim concept that has been superseded by language servers in the modern world. So LazyVim reappropriated the keybinding.

11.4. Listing Symbols

Another handy LSP feature is to search all the symbols in the current file or project. If you are editing a particularly long file and need to jump to a function that is not terribly close to your cursor, you might use the `<Space>ss` command (mnemonic is "search symbols"). As hinted by the double `s`, this is expected to be a fairly common action.

The dialog that pops up should be fairly familiar by now, as it's the usual picker:

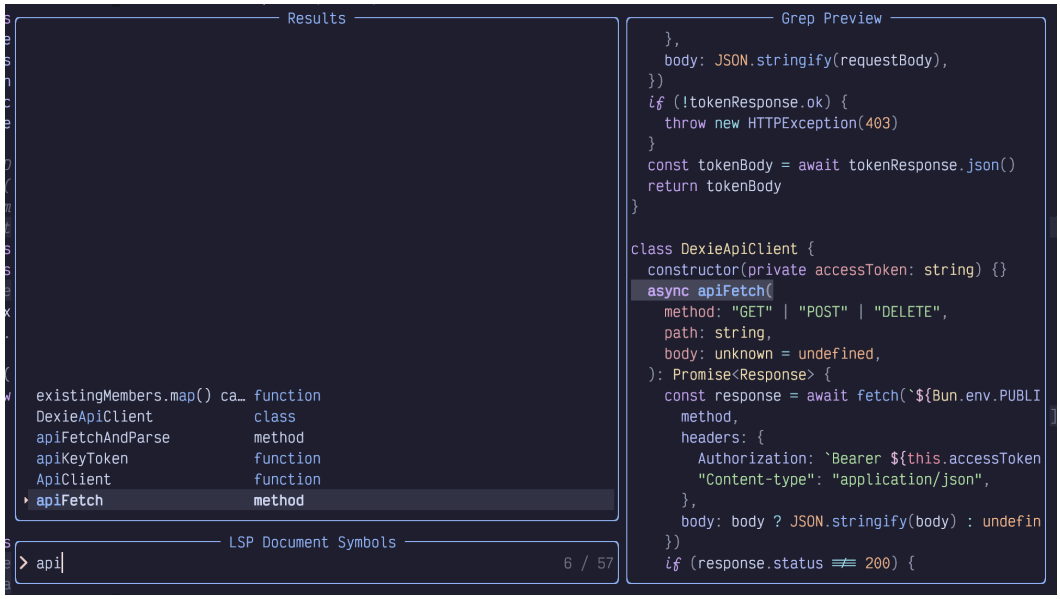


Figure 49. Picker Symbols

So you already know how to use it. However, I want to remind you of a couple Picker tips that make it more useful:

Most of the time when I'm using this symbol picker, I only care about functions, or sometimes classes. So the fields and properties scattered in the screenshot above are just a distraction. It **is** possible to configure the picker to only show certain kinds of symbols, but I prefer a quick trick that allows me to narrow it down to just functions: type (part of) the word `function`.

Since the picker includes the word "function" in the second column of the results, it merrily filters out all the lines that don't have that word in them. Handy.

Better yet, I can input a space *after* the word "function" to inform the picker to perform subsequent searching back in the first column. So "func api" filters all functions that have the word "api" in them.

My second tip is to not forget about the `Control-q` and `Control-t` shortcuts to dump picker results into the Quick Fix or Trouble list. It generates a quick and dirty table of contents of whatever symbols you filtered for.

If you want to search all the symbols in your whole project, use the "but bigger" mnemonic. `<Space>sS` will perform such a search. However, be warned that not all LSPs support workspace symbol search. Some only search in currently open files, and even many of those that fully support workspace symbol search are unusably slow.

11.5. Trouble Also has a Symbols Outline

You can also open a symbols outline using the Trouble plugin. The keybinding is `<Space>cs`, which you may have trouble finding because it is in the `Code` menu rather than the `Search` menu. Unlike most Trouble windows, it opens in a right sidebar by default. It creates a lovely tree view and you can even collapse and expand the tree nodes using the folds keybindings we discussed in Chapter 9.

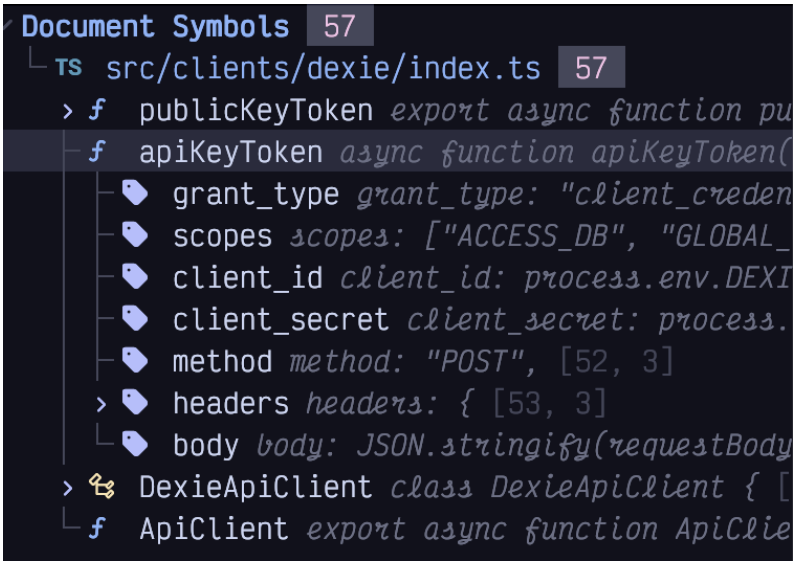


Figure 50. Trouble Symbols

You can resize the Trouble window using the same keybindings you usually use for resizing windows (`<Space>w<` and `<Space>w>`). As you move the cursor over the Trouble window, the symbol it is over will automatically scroll into view.

The fastest way to use the Trouble window is to use Seek mode. Recall that Seek mode can jump to *any* currently visible window, which includes Trouble. So if I am currently editing the above file and my cursor is currently somewhere near the end of the file, I can use `spub` to enter Seek mode and search for the characters “pub”. This will place a label on the `publicKeyToken` in the Trouble window. If I hit that label, my cursor jumps to the trouble window and my editor window immediately scrolls to the function in question. Now I just have to hit `Enter` to move the cursor back to the file I’m editing.

11.6. Context

The `nvim-treesitter-context` extra is a helpful way to know where you are in the current file. It uses treesitter to figure out which functions and types you are in, and then pins the lines that define those types to the top of the editor. Enable it as

usual by visiting `:LazyExtras` and hitting `x` over the line that contains `nvim-treesitter-context`.

This plugin keeps track of which class or function your cursor is currently in. If the function or type definition is so long that the signature scrolls off the screen, it will helpfully copy that signature into the first line or lines of the code window, highlighted with a slightly different background colour.

This is easier to describe with a reference image, so consider this screenshot:

Figure 51. Treesitter Context

In this image, the first two lines, which are *slightly* shaded, are providing context, rather than being part of the buffer. The first line tells me that I am in the `DexieApiClient` class and the second line tells me that I'm currently looking at the `forceAddMemberToRealm` method in that class.

Especially notice the relative line number column. The `class DexieApiClient` line is 109 lines above my current cursor position, and the `async forceAddMemberToRealm` line is 27 lines above it. In contrast, the first *visible* line of the function is only 13 lines above my current cursor position.

The effect is quite subtle, but the definitions that make their way into this context section tend to be exactly what you need while coding. If they fit on one line I can

see function signatures and return types. You really don't notice how often you have to scroll up to see what a variable is named in a function signature until you don't have to do it anymore! And if you DO need to scroll up to the signature, simply type the relative line number followed by `k` and you're there with no searching required.

If you need to disable the context temporarily, use the keybinding `<Space>ut`. We haven't seen much of the `<Space>u` menu yet, where you can toggle various User Interface effects. This is largely because the default user interface is configured well enough that you don't want to change it often!

11.7. Navigating with (Book)marks

You already know how to navigate through your history with `Control-o` and `Control-i`, and to jump around documents effectively using a wide variety of motions.

Vim also includes a "bookmarks" feature, although it's referred to as "marks" I assume because the `m` character was still free on the Vim keymaps.

Marks are built-into Vim and LazyVim has (as usual) added a few minor improvements.

Much like the registers that we covered in Chapter 8, marks can be assigned to each letter of the alphabet. Additionally, certain punctuation characters represent special system-set marks that you can jump to, but not set.

To set a mark on a line, precede any letter character with the letter `m`. So `ma` will set the mark `a` on the current line. You can tell that this line is marked with an `a` because there is an `a` character in the gutter to the left:

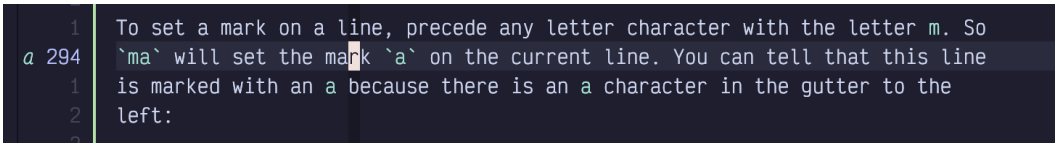


Figure 52. Mark in Sign Column

Now I can jump to the line marked with `a` from anywhere *in the current file* by using an apostrophe followed by `a`.

I don't use this very often because other tools tend to be more useful for navigating within a file than manually setting a mark. However, if I had marked the line with a capital letter (e.g. `mA`), I would be able to jump to the mark no matter which file is

currently open using `'A`.

So essentially, you can have up to 26 local marks within each file you ever open, as well as 26 global marks that you can access from any file.

Conveniently, if I just type a single apostrophe (in Normal mode), LazyVim will pop up a menu of all the marks currently available to jump to:

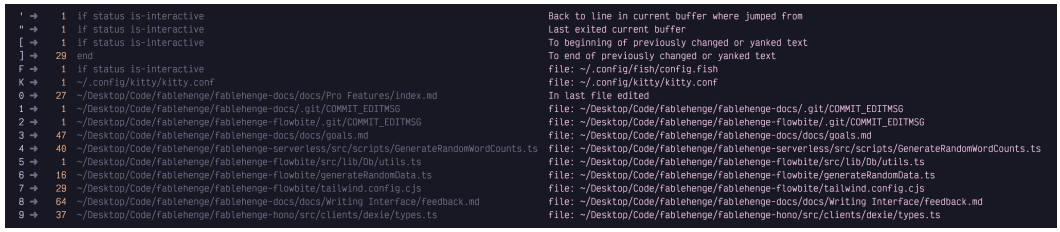


Figure 53. Marks Menu

This list shows the lowercase `a` mark that I've set in this file, several system marks that I can jump to using punctuation (notice the descriptions for each of those marks to the right so you don't have to memorize them), two global marks I use to jump to my kitty and fish configuration files, and the ten numbered marks.

I find the numbered marks to be kind of useless. They essentially refer to the file and cursor location of the last time you closed Neovim. I don't close Neovim that often unless I'm editing a commit message or pull request description in a temporary instance, so my numbered marks are mostly just those kinds of temporary files. If I need to get back to where I was previously, the `<Space>qs` keybinding to restore session is typically more useful than the numbered marks.

The menu that pops up when you press the apostrophe key is usually sufficient to find marks, but you can also use the `<Space>sm` keybinding to search marks in a picker. I don't usually have enough marks active for this to be useful, but if you've got a lot of global and local marks set and you can't remember which letter is associated with a given one, it might help to use the picker to search for the contents of the line you have marked.

Once you've set a mark, you'll eventually be dogged by the question "how do I get rid of it?" Deleting marks is probably up there with "how do I quit Vim" for most common queries! There isn't a keybinding for deleting marks. Instead, you need to use the command `:delmarks <mark>` to delete the given mark. This can be shortened to `:delm <mark>`. So to get rid of the `a` mark in this file, I used the command `:delmarks a`. You don't have to be on the marked line to delete the mark.

In Command mode, marks can be used in place of line numbers in ranges. For example, if you want to write the text between mark `a` and mark `b` to a file, you could do `:'a,'bwrite somefile.txt`. If you've seen the `'<,'>` in front of colon lines when you have text selected, that is because `'<` and `'>` represent the start and end of the most recent visual selection. So rather than manually setting `ma` and `mb` you can visually select the thing you want to write and have those marks pre-filled for you.

You can also use `'<` and `'>` to jump to the beginning or end of the most recent selection even if it has since been deselected.

The other symbol mark that I use frequently `'.` which jumps to the last place I inserted or changed text. This can sometimes be quicker than a series of `Control-o` keypresses.

11.8. Summary

In this chapter, we learned how to navigate code files using go to definition and references, and various "document symbol" plugins.

We saw how LazyVim gives us context on our current location in the document and how to look up documentation for the symbol under the cursor.

Finally, we covered Vim marks, a more manual process of tracking locations that you may want to jump to.

In the next chapter, we'll learn all about searching text both in the current file and globally across a project.

Chapter 12. Searching...

It's kind of amazing that we've gotten this far without covering searching. Find and replace in Vim has always been far more powerful and nuanced than in most editors, which just give you a little dialog with three fields and, if you're lucky, a check box to specify regular expressions.

LazyVim extends Neovim's powerful search feature to make it easier to use and prettier. You know about the Seek (`s`) and Treesitter (`S`) modes for navigating to and selecting objects you can see, as well as their remote operator-pending objects counterparts: `r` and `R`. These work great when the text you are looking for is currently visible. However, when you need to search a file and have it automatically scroll to search results, they are insufficient.

12.1. Search in Current File

To search for a pattern in Vim use the `/` command in Normal mode. The mnemonic is that the `/` key is also the question mark key, and searching for something is a kind of question.

Many tools that are not considered "modal" have adopted Vi's `/` as a command to invoke search. For example, the exceptional Linear task tracking tool uses `/` to begin a search, as does the ubiquitous GitHub.

The first time you type a `/` in Normal mode, you might lose your cursor! It doesn't pop up a new window. Instead, `/` takes over the current file's status bar with a magnifying glass icon:

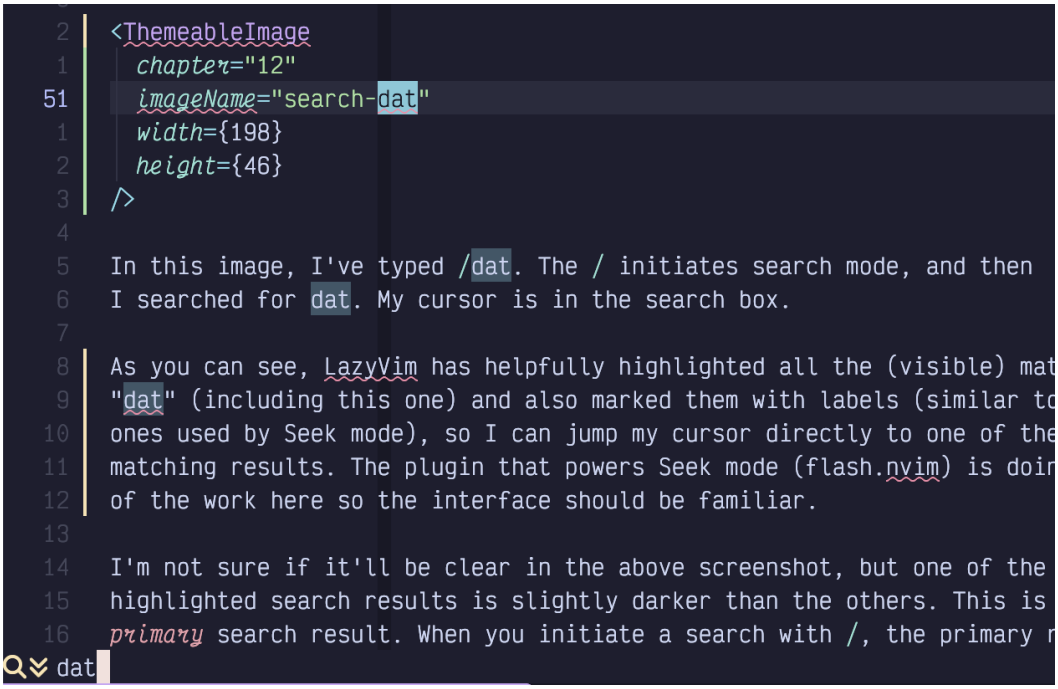


Figure 54. Search "dat"

In this image, I've typed `/dat`. The `/` initiates Search mode, and then I searched for `dat`. My cursor is in the search box.

As you can see, LazyVim has helpfully highlighted all the (visible) matches for "dat". The "primary" result will always be the first matching result *after* the point where your cursor was when you hit `/`.

Your cursor will jump to the primary result if you press `Enter` to confirm your search. Press `Escape` to cancel it, as usual.

At this point, you are back in Normal mode and can edit the buffer normally. Before doing so, however, notice that all the highlighted results are still highlighted. You can easily jump to the next result using the `n` (for **n**ext) key. This command accepts a count, so you can use `3n` to jump to the third result after the current cursor position.

The search will wrap to the top of the document if there are no more matching results at the bottom. If you know how far you need to jump, you can use a count with the `/` command as well, as in `3/something` to jump forward to the third `something`. Figuring out how far you need to jump requires some mental agility, though, so it's usually faster to use Seek mode.

If you `n` too far, you can use `Shift-N` to move the cursor to the previous result

instead. And if you *know* you need to search backwards to a previous result, you can initiate the search with `?` (i.e. `Shift-/`) instead of just `/`.

If you have used Vim before, I should warn you that this behaviour of `n` and `N` is different from the default Neovim behaviour. They used to "repeat the last `/` or `?` command," so `n` would continue *up* the document if you started with `?`. The LazyVim model is easier to remember; `n` always means "next down" and `N` always means "previous up".

12.2. Ignore Case

If you enter your search term as all lowercase letters, LazyVim will ignore case by default, but if you include a capital letter in your search term, it will enable case sensitivity. So searching for `in` will match `in` and `In`, but searching for `In` will only match `In`.

If you expressly want to search for **only** lowercase matches, you can modify the search term by inserting the two characters `\C` (that `C` is capitalized) somewhere in it.

Conveniently, it doesn't have to be at the beginning of the search term; if there is a `\C` anywhere in the search string, it will make the whole search case sensitive. So, imagine you were looking for the lowercase word "initiate". If you start typing `in` and realize it's matching a bunch of unnecessary `In` because ignore case is enabled, you can append `\C` (so you end up with `in\C`) to switch to ignore case mode before typing `iti` (so the total search string is `in\Citi`).

If you want to disable ignore case temporarily, type the colon command `:set noignorecase`. This will only last until you exit Neovim, or explicitly enable it again with `:set ignorecase`.

If you want to make the change permanent, open your `options.lua` file and add `vim.opt.ignorecase = false` somewhere in it. Note that now if you want to make any specific search case **in**sensitive, you need to use lowercase `\c` instead of `\C` in the search phrase.

The `\C` trick seems kind of weird at first, but when you think about the alternative used in most code editors, where you have to move your hand to your mouse, target a tiny checkbox with a label like `wW`, and click it, then refocus the search box and continue typing, you'll probably decide that `\C` is faster.

12.3. Regular Expressions

Vim searches use regular expressions by default. But they are kind of strange regular expressions.

Ok, I admit that *all* regular expressions are kind of strange. Vim's are only strange in comparison to the PCRE-style regular expressions that are common in most modern programming languages. Luckily, if you are searching text, you probably don't need the full complexity the Perl-compatible expressions offer.

I don't have space in this book to instruct in regular expression syntax, so I'll just mention some of the main go-tos and leave you to look up the rest:

- `.` matches *any* single character. If you need to search for a literal period, escape it with `\.`.
- `\S` matches any *non-whitespace* character.
- The `*` character matches the preceding expression zero or more times. Notably, `.*` will match any string of characters of arbitrary length.
- The `\+` string will match the preceding expression one or more times. (This is notably different from most regular expression parsers I've seen, where you don't need the `\` before the `+` to match one or more). It can be combined with e.g. `\S` to match any word without spaces: `\S\+`.
- `\=` can be used to match the preceding pattern zero or one times. Useful for things like `https\=:` where the "s" is optional. This pattern is usually `?` in most regular expression engines, and in fact `\?` also works for this. However, it would confuse Vim when the command to invoke search backwards is `?`, so `\=` wins.
- `\\` matches a literal backslash and `\/` matches a literal forward slash.

In general, if you know PCRE-style regular expressions, you'll find you need a lot more backslashes in Vim. That said, the vast majority of code editor searches are covered by the above.

If you want to "disable" regular expression matching for a specific search, place `\V` at the beginning of the line (or in the middle of the line if you only need to disable it for the remaining part of the search). The "V" stands for "very nomagic", and if you want to be extremely confused, type `:help magic`. It is so confusing, in fact, that you will prefer to learn to just use regular expressions (yes, I am aware how very confusing that is. Vim's interpretation of magic is worse).

If you desperately need a regular expression to do something you can ~~ask ChatGPT~~

skim through `:help regular expressions` to find the syntax you need. You will come away either enlightened or frustrated.

12.4. Search In Project

If you need to search for a word across your entire codebase, instead of just in one file, use the command `<Space>/` instead of just `/`. It will pop up the ever-so-familiar picker, this time in `live_grep` mode.

Make sure you have ripgrep installed and available on your path as `rg`, as that is what the pickers use under the hood.

Type the string you are looking for. The results will show up in the left side and the file will display in a preview on the right so you can be sure you found the right one:

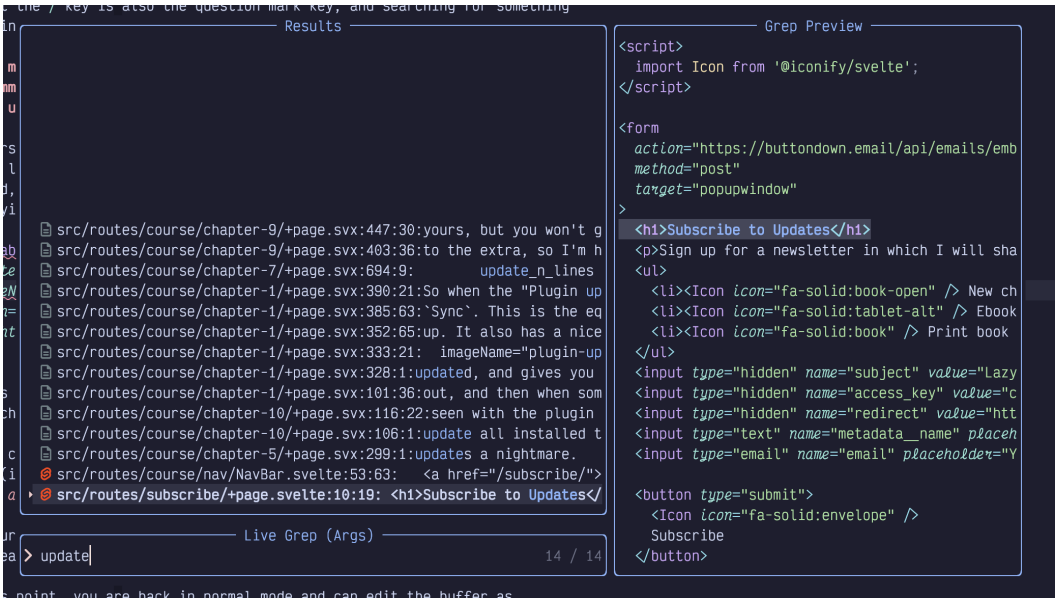


Figure 55. Live Grep Picker

Remember that you can add labels to picker results by pressing `Alt-s`. I find this more useful in the `live_grep` window because unlike most pickers, a space in `live_grep` is sent as a literal space to `ripgrep`, instead of allowing me to narrow the search results by searching for something earlier in the line.

Since this is a picker, you can press `Alt-t` while it is open to put all the search results into the Trouble window so you can navigate them while editing (using `]q` and `[q`).

Annoyingly, this search mode is completely different from Vim's built-in search. It just passes your pattern to `ripgrep` and behaves the way `ripgrep` does. And `ripgrep` doesn't know about things like Vim's strange regular expression engine. It *does* support regular expressions, but they use maddeningly different syntax from Vim. Which is to say, the same syntax as pretty much everything that isn't Vim. It's Vim that's maddening here, not `ripgrep`. Just so we're all clear.

12.5. Summary

This chapter was all about search: the `/` shortcut to enter Search mode, and the `<Space>/` shortcut to enter find in project mode. The latter is not as powerful as it could be, so we also set up the `telescope-live-grep-args` plugin to give us a little more control over the project-wide search.

Searching is, of course, only half the story. In the next chapter, we'll cover replacing text, both as part of a search operation and at a more project-wide level.

Chapter 13. ...And Replacing

Vim has a very powerful find and replace mechanism. It... takes some getting used to. On the one hand, it's pretty hard to go back after you've gotten used to the power of Vim substitution. On the other hand, getting used to it can take a lifetime.

You definitely need to know how to do old-school substitution in Vim, but I first want to discuss a slightly easier version of find and replace that I have been reaching for more and more often.

13.1. Nvim-rip-substitute

The `nvim-rip-substitute` plugin by Chris Grieser provides a familiar find and replace dialog that is similar to most other tools and editors. It isn't as powerful as native Vim substitution, but it is easier to use.

There is no Lazy Extra for `nvim-rip-substitute`, so you'll need to add a new file to your `plugins` directory. I call mine `rip-substitute.lua`, and it looks as follows:

```
return {
  "chrisgrieser/nvim-rip-substitute",
  keys = {
    {
      "g/",
      function()
        require("rip-substitute").sub()
      end,
      mode = { "n", "x" },
      desc = "Rip Substitute",
    },
  },
}
```

Listing 30. Nvim-rip-substitute configuration

Restart Neovim after adding this plugin. It adds a new `g/` keybinding. I the `g` is a prefix for "stuff that doesn't fit elsewhere" and the `/` is a parallel to the "Search" `/` from the previous chapter.

Now when you hit `g/` you get a search and replace widget in the lower right corner

of your window:

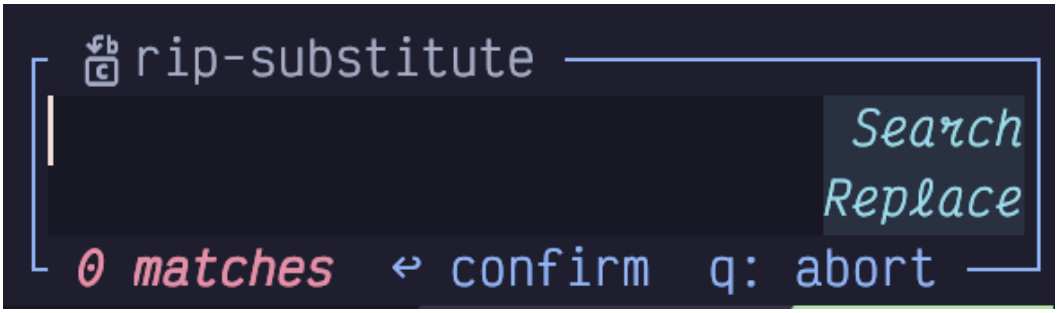


Figure 56. Rip Substitute Widget

There are two fields, and the `Search` field is focused when the widget pops up. Type whatever you want to search for in there (it may be pre-populated if you had previously selected small text).

This field uses regex syntax, but thankfully, it **isn't** vim regex syntax. It uses the modern regex syntax that you comes with-rip-grep and should be familiar if you have worked with regular expressions in any modern programming language. If you aren't, switch to Normal mode with `Escape` hit the `R` key to open the current regular expression in the helpful regex101 [https://regex101.com/] website!

To switch between the `Search` and `Replace` fields, use the `j` and `k` keys from normal mode, just as though you were switching lines in any vim window. You can even use things like `o` to enter insert mode on the next line.

Once you have got your search and replace strings arranged to your liking, and verified the live preview that pops up, use the `Enter` key in Normal mode or the `Control-Enter` key-combination in Inesrt mode to perform the subtitution.

By default, rip-substitute performs its find and replace on the whole file. If you start with a line-wise visual selection before you invoke the `g/` keybinding, it will instead search and replace inside the selected lines.

This can be confusing because if you start with a **character-wise** range selected, which can span multiple lines, it will instead use the selected text to prepopulate the `Search` field. If you start with no selection, the `Search` field is instead populated with the word that was under the cursor when you opened the widget.

You can **capture** variables in the search string and then reuse them by number in the replace section, with $1, $2, etc, where the capture groups are numbered from left to right by their opening brace.

This allows you to perform subtititions like this:

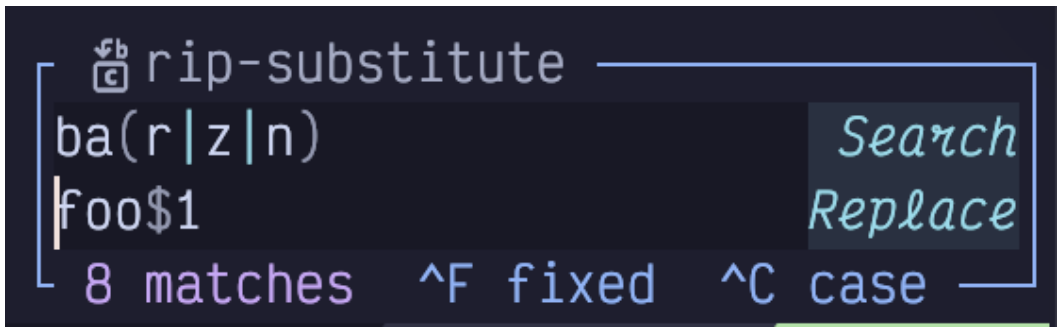


Figure 57. Rip Substitute Capture

In this example, we are matching on any of the words `bar`, `baz`, or `ban`, and, depending which was matched, replacing each with `foo`, `fooz`, or `foon`.

Rip-substitute also keeps track of your history. Use the up and down arrows in normal mode to select a previous substitution and edit it as needed.

I have made rip-substitute my daily driver for find and replace. It is powerful enough to cover the majority of my needs, and its default behaviour is quite a bit saner than the substitute command we'll cover next. That said, rip-substitute can't do everything, so you'll still need to reach for the deep dive below from time to time.

13.2. Substitute Command

Substitution predates Vim and even Vi; it goes back to the legendary `ed` by the even more legendary Ken Thompson. He wrote the original paper on regular expressions, (amoung **many** other foundational tools) so I suspect `ed` is the first place regexes were used in the wild.

Substitution in `ed` was so powerful that it has somehow stuck around for over half a century. Not only is it the primary search and replace mechanism in modern Vim and Neovim, it is also popular when automating tasks via shell scripting, using `sed` (the stream editor, a sequel to `ed`).

LazyVim, as usual, enhances the substitution command, mostly by showing you live previews of your changes as you type.

Because it is an `ex` command (ex stands for "extended ed", much like its sibling `vi` was later rewritten as "vi improved"), you access substitution by entering Command mode (with a `:`). You could type `:substitute`, but everybody shortens it to `:s` because a) it works, and b) why type more than you need to?

Then, without pressing enter, type a `/`. This is just a separator to separate the command you are issuing (`s` or `substitute`) from the term you are searching for.

Now type the search pattern. This can be any Vim regular expression, just like those we covered for a normal search in Chapter 12.

Here you can see that I have typed `:s/pattern` into my editor, and the `pattern` is highlighted on the line that my cursor was on:

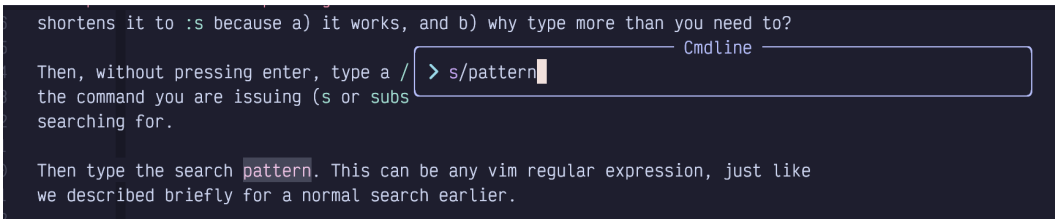


Figure 58. Substitute Pattern

Next, type another `/` to separate the *pattern* from the *replacement*, and then type whatever string you want to replace it with. LazyVim will live update all instances of the search term with the replacement term so you can preview what it will look like. Here, I'm going to replace `pattern` with `FOOBAR`:

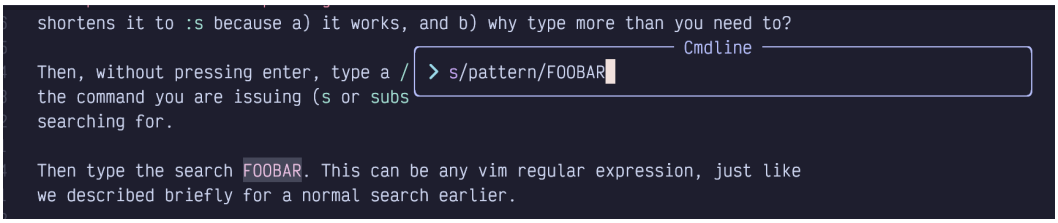


Figure 59. Substitute Replace String

Now press `Enter` to complete the command and confirm the replacement. So find and replace is as simple as `:s/pattern/replacement<Enter>`. That's not so bad, is it?

Maybe it's not, but we're not done. Not remotely. For one thing, that command will only replace the first instance of `pattern`, and only if the pattern happens to be on the same line as the cursor.

It is conventional to use a `/` after the substitute command, but if you are performing a substitution on something that has a lot of `/` characters in it (e.g a Unix path), you can use another character such as `+` for the separator to avoid having to escape a bunch of `/` characters with `\/`. For example

`:s+/home/dustyphillips/+/home/yourname/+` can be used instead of `%s/\/home\/dustyphillips\//\/home\/yourname\/`.

13.2.1. Substitute Ranges

Many Neovim ex commands can be preceded by a range of lines that the command will operate on. The syntax for ranges can be a little confusing, and to this day I still have to look it up with `:help range` if I'm doing anything non-standard.

The simplest possible range is the `.`, which stands for "current line". It would look like `:.s/pattern/replacement`. The `.` between the `:` and the `s` is the range, in this case. You normally wouldn't bother, though because `.` or "current line" is the default range.

Probably the second most common range you will use is `%`. It stands for "Entire File". If you are used to the find and replace dialog in most editors or word processors, you probably expected it to search the "entire file" by default. But it doesn't, and if you want to do a find and replace across the entire file, you would need to use `:%s/pattern/replacement` (probably with a `/g` on the end as described in the next section).

You could also set a specific line number, such as `:5s/pattern/replacement` to replace the word `pattern` on line 5. But I would normally use `5G` to move my cursor to line five and then do a default range substitution instead.

The name "range" implies that you would can cover a sequence of multiple lines, and you can indeed separate a start and end position using a comma. So, for example, `:3,8s/…` will perform the substitution on lines 3, 4, 5, 6, 7, and 8 (the selection is inclusive at both ends): Here I've started a pattern that is highlighting the word `hello` on lines 3 through 8, but no other lines:

Figure 60. Range 3-8 Inclusive

You can also use marks such as `'a` as described in the previous chapter to define the start or end of a range.

The most common way you'll use this is using `'<,'>`, which specifies the range for "the most recent visual selection". Luckily, you won't need to type those characters all that often, because if you select some text using e.g. `Shift-V` followed by a cursor movement, and then type `:`, Neovim will automatically take care of copying that range into the command line.

This means that if you want to "perform a substitution in the current visually selected text," you just have to select the text and type `:s/…`. The range will be inserted between the colon and `s`, so you'll get `:'<,'>s/…`.

If your brain is up for some recursive confusion, you can even use a search pattern to specify one end of the range! In the following example, my cursor was on line 5 when I started the substitution:

Figure 61. Pattern Range

The substitution is `:,/hello-10/s/hello/foo`. All those forward slashes in there

make it pretty hard to read (looks like a Unix file path!), but it's actually easy to write. Let's break it down from left to right:

- `:` is the Normal mode "start an ex command" trigger.
- There is nothing between the `:` and the `,` so the start of the range is the current line (line 5 in this example).
- The first `/` is a more succinct way of saying "the end of the range is the first line after the current cursor position that matches some pattern".
- `hello-10` is the pattern we are searching for to define the end of the range.
- The second `/` marks the end of the pattern. So our full range is `,/hello-10/` and means "from the current line to the line containing `hello-10`."
- The `s` indicates we want to perform a substitution on the lines in that range.
- `/hello/foo` is the pattern "hello" and replacement "foo", like any substitution.

There is a ton of other stuff you can do with Vim ranges, but the truth is, most of them only exist to support outdated editing modes. You will likely find that `%`, `'<,'>`, and `,/pattern/` cover 95% of your use cases. Read through `:help range` once to make sure you know what other sorts of syntaxes are available, and don't be afraid to look them up in the rare cases where one of the above is not sufficient.

13.2.2. Flags (Global and Ignore Case Substitutions)

You can add "flags" at the end of any substitution (after the last `/`) to modify how the search and replace behaves. The most common flag you'll use is `g` which stands for "global". You'll append it more often than not.

By default, `substitute` only replaces the first instance of a pattern on the line. So if I have a file full of the overly cheerful words `hello hello`, then the substitution `:%s/hello/foo` will only replace the first instance on each line:

Figure 62. Substitute Highlights (Non-Global)

But if I append `/g` it will replace all the `hello`'s on each line:

Figure 63. Substitute Highlights (Global)

I mentioned earlier that the supremely common use case of “replace everything in the file” is `:%s/pattern/replacement/g`. The `%` is "every line", and the `g` means “every instance in each line”.

There are almost a dozen flags, but the only other useful ones are `i`, `I`, and (rarely) `c`. The first two explicitly ignore case or disable ignoring case in the term being searched for, and you’ll only ever need one or the other depending on whether you have `ignorecase` set in your `options.lua` (it defaults to true in LazyVim). The `c` flag means confirm and is useful if you want to make substitutions in a large file but you know you want to skip some of them. You will be shown each proposed change and can accept or reject them one at a time.

Flags can be combined, so `:%s/hello/foo/gc` will do a global replace, confirming each one.

13.2.3. Handy Substitute Shortcuts

You don’t need to memorize this section, but once you get used to substituting, you’ll probably notice that some actions are rather repetitive and monotonous and you’d like to type them faster. Read through these tips so you remember to look them up when you are more comfortable with `:substitute`.

If you leave the pattern part of a substitution blank, (as in `:s//replacement/`), it will default to whatever pattern you last searched for *or* substituted. For example, if you perform these commands in order:

- `/foo` will search for the word `foo`
- `:s//bar` will replace `foo` with `bar`
- `:s/baz/bar` will replace `baz` with `bar`
- `:s//fizz` will now replace `baz` with `fizz`

This can save a little typing when you search for a term and then decide you want to replace it, or when you have substituted something in one file and want to

substitute it again in another.

If you just use `:s` without any pattern or replacement, it will repeat the last pattern **and** replacement you did. But be aware that it will not act on the same range, so if you want to repeat it exactly you'll need to type the range again.

It also won't repeat flags, but you can (usually) append the flags directly to `:s`. For beginners, the most common of these is `:%sg`, which maps to "repeat the last substitution on the entire file, globally." This is helpful when you typed `:s/long-pattern/long-replacement` and expected it to do a global replace, but actually it just replaces the first instance on the current line. `:%sg` will repeat the substitution the way you intended it. You might also reach for `'<,'>sg` to replace in the last visual selection.

Don't forget that you can repeat the last visual selection with `gv` to confirm that it is actually selecting what you expected.

If you want to reuse "whatever was matched in the pattern" in the replacement, you can use `\0` in the replacement string. This is particularly useful when you are using a regular expression that could potentially match different things.

For example, imagine I have the following file:

```
hello world
Hello thrift shop
Hellish world
```

Listing 31. An Imaginary File

For some reason, I want to add an adjective between the first and second words. This can be accomplished with the command `:%s/[hH]ell\S* /\0green /`:

Figure 64. Substitute With Pattern in Replacement

That command might be a little intimidating if you aren't comfortable with regular

expressions, so I'll break it down again:

- `:%` means "perform a command on the entire file"
- `s/` means "the command to perform is substitute"
- `[hH]` means "match h case insensitively (see note)"
- `ell` means "match the three characters `ell` exactly"
- `\S` means "match any non-whitespace character"
- `*` means "repeat the `\S` match zero or more times", which takes us to the end of the word.
- `/` includes a space and then the end of the search pattern
- `\0` says "insert whatever was matched by the above pattern into the replacement"
- `green` says "insert that text directly into the replacement"

The `[hH]` isn't necessary if you don't have `vim.opt.ignorecase=false` in your `options.lua`. An alternative would be to use `/i` at the end of the pattern to force ignoring case for this one search. Then `[hH]` could just be `h`.

You can even reuse **part** of the pattern in the replacement. To do this, place the part you want to reuse between `\(` and `\)`. Then use `\1` to represent whatever was matched between brackets in the replacement portion.

This is easier to understand with an example. If we start with the same three line example as above, we can use the substitution `:%s/hell\(\S*\)/green\1 and blue\1/i` to cause the following nonsense substitution:

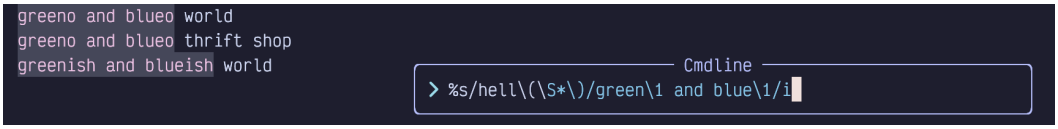


Figure 65. Substitute with Partial Pattern

The `\(\S*\)` matches the same thing as `\S*` but it stores the result in a *capture*. Then when we want to reuse the capture in the replacement, we use `\1` to refer *back* to whatever was captured on that match.

You might guess from the fact that we're using numbers here that you can have and refer back to multiple captures, and your guess

would be correct!

13.3. Project-wide Search and Replace

LazyVim ships with a plugin called Grug-far.nvim to do a global find and replace in all files in the project. Without grug-far, you would probably (unenthusiastically) do this from the command line using `sed`, the stream-oriented evolution of `ed` that I mentioned.

It is a good idea to commit your files to version control before running Grug-far. The changes it makes can be tricky to reverse. You can undo it file-by-file, but not all in one go. So make sure `git reset --hard` won't cause you to lose any work that wasn't done by Grug-far.

Grug-far is a lightweight UI wrapping `ripgrep`, a command line tool I've mentioned before. But that UI is pretty handy, as `ripgrep` has some arcane arguments.

To show the Grug-far UI, use the keyboard shortcut `<Space>sr`, where the mnemonic is r for **r**eplace. A window will open up on the right:

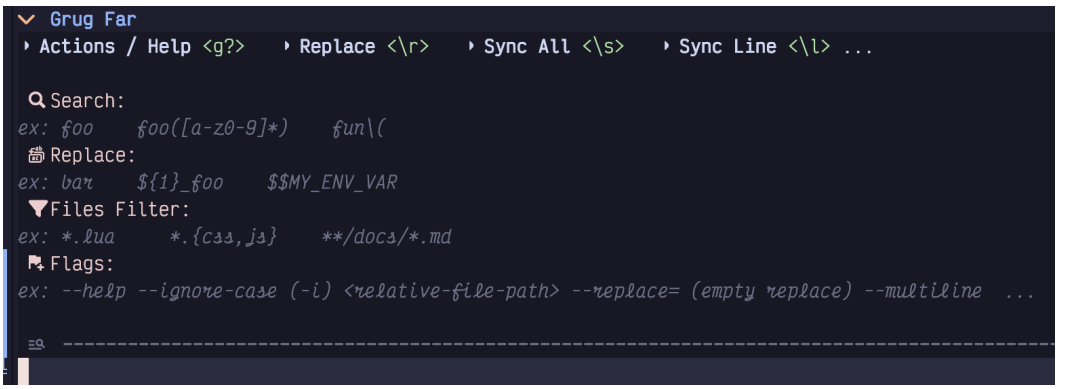


Figure 66. Empty Grug-far Window

You can navigate around this window using all the normal Vim motions, but you'll mostly just need `j` and `k` to jump between fields.

The search field can accept any regular expression. Because this is using ripgrep under the hood, it is a slightly less arcane regex syntax. The Files Filter field is used to isolate your search to a specific path or file extension, and accepts standard shell glob syntax.

As you fill in the form entries, Grug-far instantly previews the proposed changes in a live-updating widget:

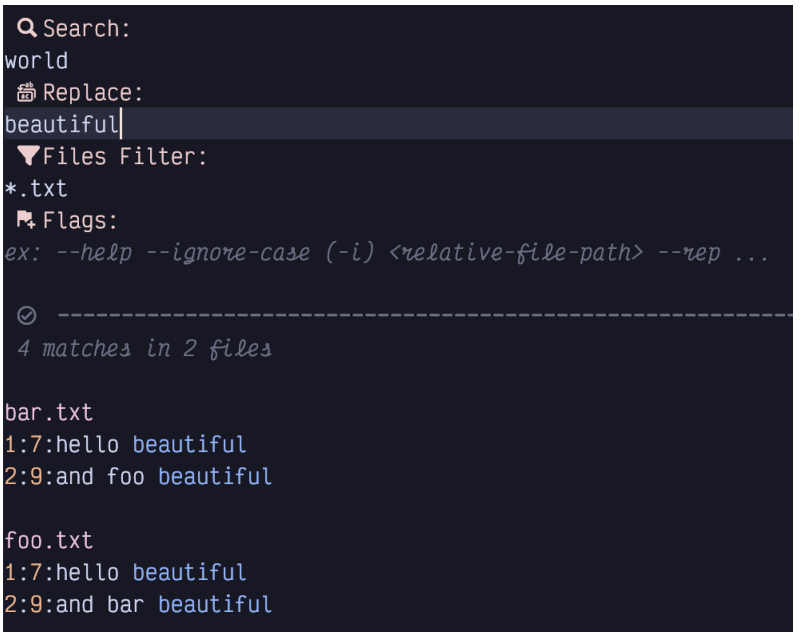


Figure 67. Grug-far With Preview

After you have inserted the search and replace text, you will need to press `Escape` to return to Normal mode. If all the results in the preview area look acceptable, simply hit `\r` to perform the **r**eplacement.

You also have the option to *tweak* the search results if some of them don't match your needs. Use any standard motions to navigate the preview window. If there are matches that you don't want to change, just use `dd` to delete them outright. Alternatively, feel free to edit any line, right in the preview window, to make it look the way you want.

Once you are satisfied with the preview, use `\s` instead of `\r` to **s**ync the changes you've made with their original source files.

You can also jump to the source file of a preview result by placing your cursor over it and hitting `Enter`.

Grug-far keeps track of your recent search and replace operations and allows you to revisit them with the `\t` keybinding. Navigate between them with standard motions and use `Enter` to reuse one of them.

There are a few other useful keybindings in a menu you can pop up with `g?`, which

I'll leave you to peruse at your leisure.

13.4. The Text-case Plugin

The text-case.nvim [https://github.com/johmsalas/text-case.nvim] plugin by Johnny Salas provides a set of commands to quickly change the case of text. It has two related, but distinct, use cases:

- You want to change the "shape" of a specific word or selection to a different shape without changing the word. This plugin can quickly convert snake-case to CamelCase, or CONSTANT_CASE for example.
- You want to change the "content" of a specific word to different content without changing the shape. For example, you can replace `foo_bar`, `FooBar`, and `FOO_BAR` to `fizz_buzz`, `FizzBuzz`, and `FIZZ_BUZZ` with one substitution command.

This is one of those plugins that you don't reach for all that often, but can save tons of time when you do need it.

There is no lazy extra for `text-case.nvim`, but you can install it into your plugins directory as follows:

```
return {
  "johmsalas/text-case.nvim",
  lazy = false,
  config = true,
  cmd = {
    "Subs",
    "TextCaseStartReplacingCommand",
  },
}
```

Listing 32. Text-case.nvim Configuration

The `lazy = false` is needed to get certain interactive features from the plugin.

Once installed, you can start changing case. Write a snake case word somewhere such as `fizz_buzz`. Move your cursor to that word (you don't even have to select it) and type `ga`. Notice how the which-key menu pops up to show you all the cases you can change it to:

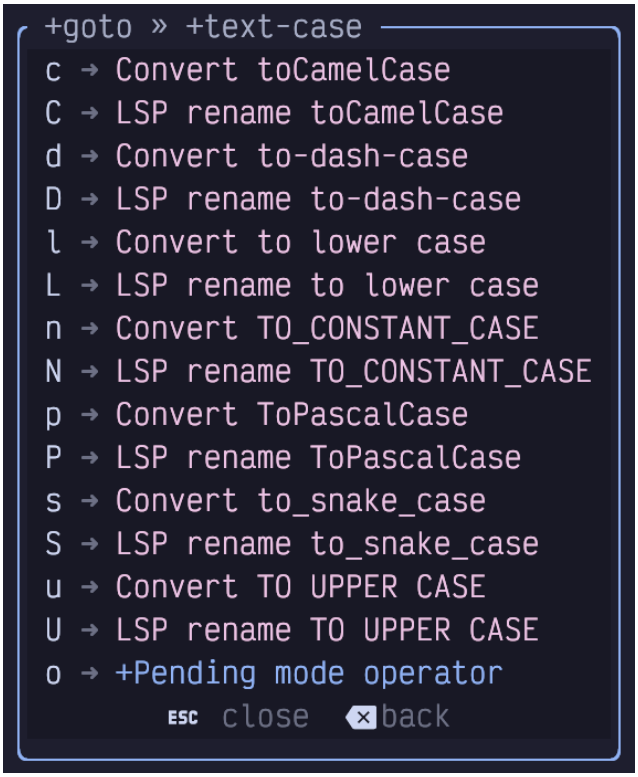


Figure 68. Text-case.nvim Which-key Menu

Select the case style you want to switch the word to and you're done! This short form only works if the word you are on doesn't have spaces. If you want to change e.g. `fizz buzz` to `FizzBuzz`, the easiest thing to do is select the two words in visual mode (e.g. move cursor to the `f` and hit `v2e`) and then hit `gap`, where `p` means "pascal" case. The which-key menu in visual mode will, of course, show other cases after you hit `ga`.

Alternatively, you can put the cursor on the starting `f` and hit `gao` to instruct text-case to enter `operator-pending` mode **after** you select your target case. Then you can use any motion (e.g. `2w`) to tell text-case what block it should operate on.

13.4.1. Case-aware substitution

The text-case plugin also supports a `:Subs` command that behaves similar to the `:substitute` command, but changes words in such a way that it always keeps its existing case.

For example, consider this simple python class:

```
class FooBar:
    fooBar: str

    def foo_bar(self):
        self.fooBar = "FOO_BAR"
```

Listing 33. Case-aware Substitution

Select the entire class (The `S` keybinding to seek surrounding objects is perfect for this) and enter the command mode command `:Subs/foo_bar/fizz_buzz`. In a single substitution, the entire class changes to the following:

```
class FizzBuzz:
    fizzBuzz: str

    def fizz_buzz(self):
        self.fizzBuzz = "FIZZ_BUZZ"
```

Listing 34. Case-aware Substitution

This fetaure is ridiculously useful when it's useful. The occasions where it is needed are relatively uncommon, but when you need it, no other tool is nearly as powerful.

13.5. Perform Vim Commands on Multiple Lines

The `:substitute` command isn't the only one that can operate on multiple lines at once, with a range. In fact, if you just want to write a few lines out to a separate file, you can pass a range to `:write`. The easiest way to do this is to select the range in Visual mode and type `:write <filename>`. Neovim will automatically convert it to `:'<,'>write` and only save only those lines.

Neovim doesn't have first class multi-cursor support (yet). Historically, Vim coders have considered multi-cursor mode to be a crutch required by less powerful editors that don't have Vim's modes. More recently, experimental editors such as Kakoune and Helix have demonstrated that multiple cursors can integrate very well with modal editing. Modern developers like multiple selections, and Neovim is expected to ship with native multiple cursor support in the future (it's currently listed as 0.12+ on the roadmap).

In the meantime, there *are* multiple cursor plugins, but I find them to be clumsy and fragile, and recommend avoiding them at this time. Instead, you can use the commands discussed below or rely on other Vim tools such as repeating recordings (with `q` `Q`, and `@@`), or Visual Block mode (`Control-v`) with an insert or append that modifies multiple lines.

13.5.1. The Norm Command

When you first use it, `:norm` feels pretty weird. It allows you to perform a sequence of arbitrary Vim normal-mode commands (including navigation commands such as `hjkl` and `web` as well as modification commands like `d`, `c`, and `y`) across multiple lines.

You can even enter Insert mode from `:norm`! But you need to know a small secret to get out of Insert mode because pressing `<Escape>` while the command menu is visible will just close the command menu. Instead, use `Control-v<Escape>`. When you are in Insert mode or Command mode, the `Control-v` keybinding means "Insert the next keypress literally instead of interpreting it as a command." The terminal usually renders `Control-v<Escape>` as `^[`.

For example imagine we are editing the following file:

```
foo
Bar
fizz buzz
one two three
```

Listing 35. An Imaginary File

For inexplicable (but pedagogical) reasons, we want to perform the following on each and every line:

- insert the word "HELLO" at the beginning of the line with a space after it
- capitalize the first letter of the first word on the line
- insert the word "BEAUTIFUL" after the first word on each line with spaces surrounding it
- append the word "WORLD" to the end of each line with a space before it

Start by typing `:%norm` to open a command line with a range that operates on

every line in the file (`%`) and the `norm` command followed by a space.

Then add `IHELLO ` to insert the text `HELLO ` at the beginning of each line in the range. Now hit `Control-v` and then `Escape` to insert the escape character into the command line.

Now type `lgUl` to move the cursor right (which puts it on the beginning of the first word), then uppercase one character to the right (i.e. the first character of the next word).

Next is `e` to jump to the end of the word, followed by `a BEAUTIFUL ` to append some text after that word. `Control-v` and `Escape` will insert another escape character.

Finally, add `A WORLD` to enter Insert mode at the end of the line and add the text `WORLD`.

The entire command would therefore be:

```
:%norm IHELLO <Control-v Escape>lgUlea BEAUTIFUL<ctrl-v
Escape>A WORLD
```

Listing 36. Norm Command

Visually, it looks like this, since the `Control-v Escape` keypresses get changed to `^[`:

Figure 69. Why Would You Ever Want To Do This?

And the end result:

```
HELLO Foo BEAUTIFUL WORLD
HELLO Bar BEAUTIFUL WORLD
HELLO Fizz BEAUTIFUL buzz WORLD
HELLO One BEAUTIFUL two three WORLD
```

Listing 37. Result of Applying Norm Command

Of course, it's unclear why you'd want to perform this exact set of actions, but it hopefully shows that anything is possible!

It's pretty common to get the command wrong the first time you try to apply it. Simply use `u` to undo the entire sequence in one go, then type `:<Up>` to edit the command line again.

If the command is kind of complicated, you'll probably get annoyed while editing it because you don't have access to all the Vim navigation commands you are used to. So now is a great time to introduce Vim's command line editor.

13.6. Command Line Editor

To display the command line editor, type `Control-F` while the little `Cmdline` window is focused. Or, if you are currently in Normal mode, type `q:`. This latter is not related to the "record to register" command typically associated with `q`. It is instead "Open the editable command line window".

This window is basically what happens when the normal command line editor marries a normal Vim window and spawns a magical superpower command line window.

The new magic window shows up at the bottom of the current buffer, just above the status bar, and it contains your entire command line history (including searches and substitutions):

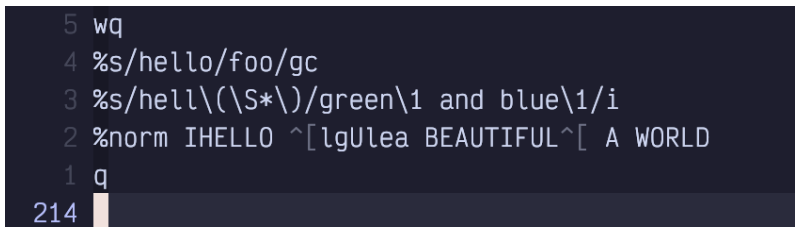


Figure 70. Command Line History Window

Use `Control-u` to scroll up this baby and you'll see every command you ever typed. You can even search it with `?` (search backward is probably more useful than search forward since your commands are ordered by recency).

To run any of those old commands, just navigate your cursor to that line and press `Enter`. Boom! History repeats itself.

Or you can enter a brand new command on the blank line at the bottom of this

magic command window (remember `Shift-G` will get you to the bottom in a hurry).

You will find this window is devilishly hard to escape, though. The escape key doesn't work, because it's reserved for escaping to Normal mode while editing *in* the window. The secret is to use `Control-C` to close it, although other window close commands such as `<Space>wq` will also work. You can even run `:q` from inside the command line window.

Most importantly, you can use normal Vim commands to *edit* any line in this window. Just navigate to the line, use whatever mad editing skills you have (including other command-mode commands such as `:s`) to make the line look the way you want it to, return to Normal mode, and press `Enter`. The edited command will execute.

13.7. Mixing the Norm Command With Recording

Recall that the `q` command can record a sequence of commands to a register for later playback. And the `:norm` command can be used to apply a sequence of commands to a range of lines. The fact that you can `p` a register that has a recording in it means there are several ways you can later apply a recording to a range of lines using `:norm`:

- `:<range>norm @q` will simply execute the `q` register on each line in the range, since `@q` is the command to execute register `q`.
- `:<range>norm <Control-r>q` will copy the *contents* of register `q` into the cmdline window so the actions will be applied to each line.
- `q:<range>inorm <Esc>"qp` will open the command line editor window, insert the word `norm` and copy the contents of register `q` into the line using the Normal mode register paste command.

13.8. The Global Command

The `:norm` command operates on a range of lines, and Neovim ranges must be contiguous lines. It's not possible to execute a command on e.g. lines 1 to 4 and 8 to 10, but not 5 to 7 (other than running `:norm` twice on different ranges).

Sometimes, you want to run a command on every line that matches a pattern. This is where the `:global` command comes in.

The syntax for `:global` is essentially `:<range>global/pattern/command`, although you can shorten it to `:<range>g/pattern/command`. The pattern is just like any Vim search or substitute pattern.

The `command`, however, is kind of weird. Technically, it's an "ex" command, which means "many but not all of the commands that come after a colon, but mostly ones you don't use in daily editing so they are hard to remember".

The most common example is "delete all lines that match a pattern", which you can do with `:%g/pattern/d`.

Another popular one is `substitute`, which you already know. If you precede your substitute with `:%g/pattern`, you can make it only perform the substitution on lines that match a certain pattern. This pattern can be *different* from the one that is used in the substitution itself. Consider the following arcane sequence of text:

```
:%g/^f/s/ba[rt]/glib
```

Listing 38. Combine Substitute with Global

What a mess! This is obviously meant to be easy to write, not easy to read. If we wanted it to be slightly easier to read, we'd probably write `:%global/^f/substitue/ba[rt]/glib`.

This command means "perform a global operation on every line that starts with `f`. The operation in this case should be to replace every instance of `bar` or `bat` with the word `glib`."

This is different from using a pattern in a range, such as `:,/foo/s/needle/haystack/`. This command performs the substitution on all lines between the cursor and the first line to contain `foo`, whereas `:%global/foo/s/needle/haystack/` performs the substitution on every line in the file that contains the word foo.

In my opinion, the most interesting use of `:global` is to run a Normal mode command on the lines that match a pattern. This effectively means mixing `:global` with `:normal`, as in `:%g/pattern/norm <some keystrokes>`.

As just one example, this will insert the word "world" at the end of every line that starts with "hello":

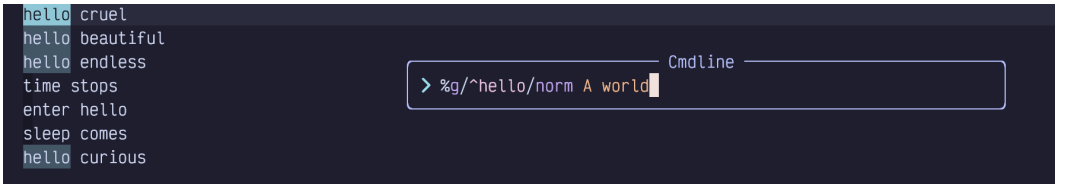


Figure 71. Mixing Global and Normal

You can also use global to perform a command on every line that **does not** match a pattern. Just use `g!/` instead of `g/`

The `g!` is useful in log files that have exceptions wrapping onto random lines. For example, a rudimentary log file might look like this:

```
2024-03-26T12:00:00 Something happened
2024-03-26T12:01:01 Something happened
2024-03-26T12:01:02 Something super bad happened
  Traceback:
    A bunch of lines I don't care about
2024-03-26T12:02:00 Something else happened
2024-03-26T12:03:58 Cool thing happened
```

Listing 39. Imaginary Log File

and prior to further processing, I might want to remove every line that doesn't start with a date:

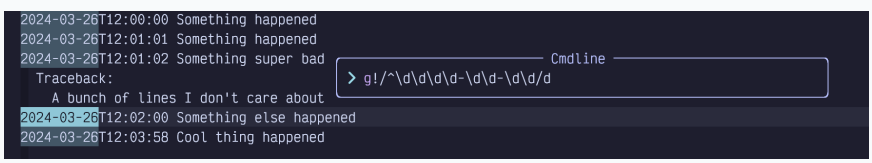


Figure 72. Global Invert Match

As has become a running theme, that might be a bit eye-watering. Each `\d` means "match a digit", while the final `/d` means "perform a delete operation on the selected lines". The `g!` is the important part; that's the one that means "the selected lines are ones that *don't* match the pattern".

I don't use `:global` nearly as often as I use `:norm`. But when I do, it is a hyper-efficient way to cause massive changes in a file. It takes some getting used to, and you'll probably be looking up the syntax the first few times you need it, but it's a really terrific tool to have in your toolbox.

13.9. Summary

This chapter was all about bulk editing text. We started with substitutions using the `:s[ubstitute]` ex command, and then took a tour of the UI for performing find and replace across multiple files using the Grug-far plugin.

Then we learned how to perform commands on multiple lines at once using `:norm` and `:global`, and earned a quick but comprehensive introduction to the command line editing window.

In the next chapter, we'll learn several random editing tips that I couldn't fit anywhere else.

Chapter 14. Miscellaneous Editing Tips

Before we dive into some of the more "IDE-like" behaviours that LazyVim enables, I wanted to collect some tips that can make your editing life a little more fun. This chapter is a bit of a grab bag, and includes some commands and plugins that I couldn't fit anywhere else.

14.1. Word Counts

Use `g<Control-g>` to spit out a message containing some helpful info about the current cursor position:

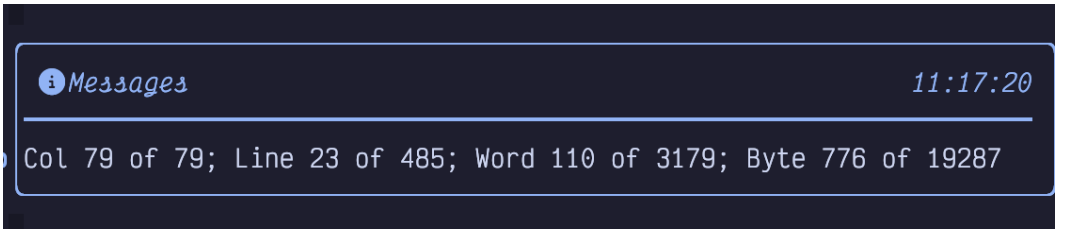


Figure 73. Word Count

Most notably, the "Word 110 of 3179" tells me that this chapter has over 3000 words in it (obviously I updated this section after I wrote more words!)

14.2. Transposed Characters

How often do you type so fast that you accidentally transpose two chracters?

Simply use `xp` to swap a character with the one to the right of it. For example, if you have typed `ra` when you meant to type `ar`, put your cursor on the `r` and hit `xp`.

This is not a special custom command. It just uses the default "delete character" and "put last deleted after the cursor" commands to move the character from its current position to the next one. You can use a similar idea to move other text around. For example, move a word with `dwwP` or use `daaWp` to delete an argument and move it later in a function signature.

14.3. Commenting and Uncommenting Code

LazyVim ships with a plugin for commenting and uncommenting code in older

versions of Neovim, but as of Neovim 0.10, this is actually a native Neovim feature.

The verb for toggling comments is `gc` and can be followed by a motion or text object. So `gc5j` will comment this line and the five lines below it, while `gcap` will comment out an entire block separated by newlines.

This command pairs beautifully with the `S` command to comment out a surrounding text object. For example `gcSh` will comment out the function surrounded by the `h` labels after the `S` is invoked.

To comment out a single line, use the easy-to-type shortcut `gcc`. This command can take a count, so `5gcc` will comment out five lines (a little easier to type than `gc4j`).

As with most verbs, `gc` can also be applied to a visual selection with e.g. `V5jgc`.

The `gc` verb is actually a toggle, so if a line is currently commented, it will uncomment it instead of commenting it a second time. Thus, `gccgcc` is a no-op. However, note that if you have a selection that contains commented and uncommented lines, you will end up with a double comment. This is usually what you want: If you temporarily comment out a block that contains other comments, when you uncomment that block, you probably want the original comments to stay commented.

As a shortcut, if you want to add a new comment line above or below the current line, instead of commenting the current line, you can use `gcO` and `gco`. Technically this is a new verb, but for memory's sake, think of it as combining `gc` with the verbs to open a new line (`o` and `O`).

14.4. Incrementing and Decrementing Numbers

If your cursor is currently on a number in Normal mode, you can use `Control-a` to increment that number. This command is somewhat smart and does the "right thing" if your number needs new digits. So `9` becomes `10` and `99` becomes `100` when you press `Control-a` anywhere in the number.

To decrement a number, use `Control-x`.

I hated these two keybindings for the longest time because they are only occasionally useful, but when they are useful, I couldn't remember them. So I spent a long time manually incrementing numbers and thinking to myself "I need to look up

those number increment commands," but the only keywords associated with this help section were the keybindings themselves!

Eventually I learned about the `:helpgrep` command, which allows you to search the help. Long before I memorized the keybindings, I remembered that `:helpgrep Adding and subtracting` would help me look them up.

But there is actually a mnemonic for these keybindings: `Control-a` is "Add", which is easy enough to remember. `Control-x` is a little harder, but now that you have `Control-a` you'll be able to look it up with `:help CTRL-a`. I'm not sure if it will help anyone else, but I think of the `x` as "'cross' out one digit to subtract".

Use `g<Control-a>` and `g<Control-x>` to decrement numbers on consecutive lines with an additional increment for each line in the count. This is useful if you are manipulating numbered lists. Say you want to make a list of 10 items. First type `o1.<esc>` to make a line that says `1.`. Then type `9.` to repeat that command 9 times. Now you have:

```
1.
1.
1.
1.
1.
1.
1.
1.
1.
1.
```

Listing 40. A Dumb List

You can use `V'[` to select the 9 rows that just got inserted, as the `'[` mark is the first character of the previously changed text. Now type `g<Control-a>` to increment them and you end up with:

```
1.
2.
3.
4.
5.
6.
7.
8.
9.
10.
```

Listing 41. A Smarter List

Not bad for just a handful of odd-looking keystrokes: `oi1.<Esc>9.V'[g<Control-a>`!

If you need to insert a new entry in the middle of a list, add the entry, select the lines with the remaining entries, and hit `Control-a` to sync them up.

Neovim will smartly increment just the first number it encounters on a line. This means it is easy to e.g. manipulate a book's outline even if it contains multiple numbers. Consider this hypothetical outline of a book largely unlike this one:

```
Chapter 1: Intro and Install
Chapter 2: 1 Weird modal editing trick
Chapter 3: The numbered marks 1-9
Chapter 4: Navigating things
...
```

Listing 42. Some Book Chapters

Let's say I want to split Chapter 1 into two different chapters: "Intro" and "Install". I can simply add the new chapter using normal text insertion like this:

```
Chapter 1: Intro
Chapter 2: Install
Chapter 2: 1 Weird modal editing trick
Chapter 3: The numbered marks 1-9
Chapter 4: Navigating things
...
```

Listing 43. Adding a New Chapter

Then I can use `<Shift-V>}` to select the chapters originally numbered 2 and higher. When I hit `Control-a`, the chapter numbers are incremented, but the `1` in `1 Weird trick` will not be impacted, nor will the numbered marks indicators.

```
Chapter 1: Intro
Chapter 2: Install
Chapter 3: 1 Weird modal editing trick
Chapter 4: The numbered marks 1-9
Chapter 5: Navigating things
...
```

Listing 44. Sync Up the Numbers

14.4.1. The Dial.nvim Extra

If the increment and decrement keybindings sound kind of like that one weird kitchen unitasker that is helpful once a month, you might want to consider installing the `editor.dial` extra from `:LazyExtras`.

This extra installs the dial.nvim plugin which allows you to increment and decrement a bunch of other cool stuff. I mostly use it to swap boolean expressions (both `Control-a` and `Control-x` will alternate `true` to `false` and vice versa.), but it can also increment words ("first" increments to "second"), months ("December" increments to "January"), version numbers, Markdown headers, and more. You can even extend it with your own patterns if you need to.

14.5. Changing Indentation

The `>` and `<` keybindings can be used in Normal mode to indent or dedent text. Most often, you'll use them doubled up (as in `<<` and `>>`) to change the indentation of the current line. However, you can also change the indentation of any motion. Another common one is `>Sx` to indent a treesitter entity by some label `x`, and `>ap` will indent

an entire blanks-delimited paragraph.

These verbs can get a little confusing when it comes to using counts. You might expect `2>>` to indent the current line by two indentation levels, but instead, it will indent two lines by one indentation level.

When you want to change by multiple indents in one command, you will need to resort to Visual mode. To indent the current line by five indentation widths, the quickest way is with `v5>`, compared to typing ten greater-than symbols. This works with any visual selection, so you can use, for example, `va{5>` to indent an entire block five levels.

Often, all you want to do is "make the indentation correct for this programming language". If conform.nvim is configured correctly, the easiest way to do this is to just save the file. LazyVim has format on save enabled by default, and if it can find a formatter, it will use it. You can also use `gq` with a motion or selection (most commonly `gqag` to format the entire file) to apply formatting.

However, if you don't want to save, or aren't using conform.nvim, you can also use the `=` verb. The behaviour of `=` depends a little on the programming language, but it generally applies the indentation engine to the visually selected (or motion selected) lines as though you had pressed `enter` to start a new line. The end result is that all lines will be indented "correctly" for some definition of "correctly".

You can also adjust indentation without leaving Insert mode. The `Control-t` and `Control-d` keybindings will indent and dedent the current line while inserting text. The mnemonics are "add **t**ab" and "**d**edent".

14.6. Reflowing Text

I've used the `gw` command a lot while writing this book. It effectively rewraps (`w` for wrap) all the text at the eighty character limit (or any ruler number, configurable with `:set textwidth=<number>`), without breaking words.

Most often, I use `gww` to rewrap the current line so that it has linebreaks at the appropriate position or `gwip` to rewrap an entire paragraph. But `gw` works with any motion or visually selection. To rewrap an entire file, use `gwig`.

This command relies heavily on the existence of newlines. Effectively, any two consecutive lines will be joined into a single line (if they fit in 80 characters). For me, this has meant that if I forget to put a newline after a heading, my first paragraph gets tied up into the heading, which is obviously not what I want.

14.7. Filtering Through External Programs

You can also pipe text out to any external program that is a good Unix citizen: one that processes input on STDIN and outputs it to STDOUT. To do so, visually select the text you want to pipe in Visual mode. Then type a `!`. This will open the command window with the visual selection as a range, and is a shortcut for `:'<,'>!`. Then type a command on the path and the selected text will be replaced with the output of that command.

Here are some examples, assuming some common Unix tools are installed:

- `!grep -v a` will replace the selection with the same text, but any lines that contain the letter "a" will be removed.
- `!tr -s ' '` will call the translate command, replacing all instances of multiple spaces with a single space.
- `!jq` will format the `json` text with `jq`
- `!pandoc -f markdown -t html` is a handy way to quickly write HTML by starting with simpler Markdown syntax.
- `!./my-custom-script` will pipe the command through an arbitrary script you wrote.
- `!python ./something.py` will pipe the command through a Python script you wrote.

If you want to run a command without modifying the text, don't supply a range. For example, `:!mkdir foo` will run the `mkdir` command without overwriting your file content.

I think it is unfortunate that this feature is not used more. Many features that are built into Neovim or supplied as plugins could just as easily be CLI programs that operate on piped input and output. As just one example, the `:sort` command that ships with Neovim is, in my opinion, just bloating the editor when `!sort` can run the external sort utility just as well.

14.8. Spell Check

You can enable or disable spell check with `<Space>us`. When enabled, errors that are not recognized by the spell checker are underlined with a curly underline similar to a diagnostic. But you have to jump between spelling errors with `[s` and `]s` instead of the diagnostic keybindings `[d` and `]d`.

To ask Vim to give you suggestions for how to spell the word, use `z=`. This is about as unmemorable as you can get, so write it down. If you can remember it is in the `z` menu rather than `Space`, you can at least find it in the menu again. The spelling suggestions will pop up in a numbered menu; enter a number to replace the word with that spelling.

14.9. Insert Mode Keybindings

If you are in Insert mode, and want to perform a single Normal mode action before going back to Insert mode, you can use `ctrl-o`. Perform the one Normal mode command, and you'll be back in Insert mode immediately. I don't really see the point of this, since `Control-o<command>` adds two keypresses, and so does `<Escape><command>i`.

While in Insert mode, if you press `Control-a`, it will Insert whatever text you inserted in the previous Insert mode session. This is similar to accessing the `".` register.

To access other registers in Insert mode, use `Control-r`. This will pop up the registers menu and you can Insert any of those registers by hitting the appropriate key. So `Control-a` in Insert mode is similar to `Control-r.`. To insert from the clipboard, use `Control-r` and then `+`.

The `CTRL-U` keybinding will remove all characters *on the current line* that were added since you entered Insert mode. So in a single line edit, it's similar to an Undo operation, but if your Insert has included an `<Enter>`, the undo would only be on one line.

Some people like to bind to an unusual sequence of characters in Insert mode. The most common suggestions are to bind `jk` to `Escape` or to bind `;;` to `Control-O` but you can do any combination you like. The former allows you to switch to Normal mode without pressing `Escape` or `Control`, and the latter allows you to temporarily perform a single Normal mode operation and return to Insert mode. They don't save you any keypresses in terms of count, but they are easy keys to hit.

If you want to explore this, open your `keymaps.lua` file and add the following lines:

```
vim.keymap.set("i", "jk", "<Esc>", { desc = "Normal mode" })
vim.keymap.set("i", ";;", "<C-o>", { desc = "Normal mode
single operation" })
```

Listing 45. Insert Mode Keymaps

NOTE

If you like the `jk` action to leave Insert mode, the `max397574/better-escape.nvim` plugin will eliminate the delay that happens every time you press `j` in Insert mode.

The important bit here is the `"i"` as the first argument. This tells Neovim that the keymapping should happen in Insert mode instead of Normal mode (`"n"`). You can also use `"o"` for operator pending mode and `v` for visual and select modes, among others.

In normal text and coding, the `;` key is rarely followed by any character other than `<Space>` or `<Enter>`, so it is a good candidate to use as a prefix for a variety of Insert mode operations.

Do not use this technique for expanding a sequence of text to a different sequence of text, though. For that, you are better off using either abbreviations or snippets, the topic of the next two sections.

14.10. Abbreviations (and Filetype Configuration)

Vim abbreviations have been around since the earliest days of the editor. They are an easy way to have "shortcut" words that expand to something else entirely without leaving Insert mode.

To create a temporary abbreviation, just use the command `:iabbr <shortcut> <expansion>`. You can use Vim's keybinding syntax to represent special characters like `<Enter>`, and `<Tab>` in the expansion. You can even use e.g. `<Left>` to reposition the cursor within the abbreviated text.

For example, consider this command:

```
:iabbr ifmain if __name__ == "__main__":<Enter>main()<Left>`
```

Listing 46. Abbreviation Command

It will expand the text (when entered in insert mode) `ifmain<Space>` to the following, and place the cursor inside the parentheses after `main`:

```
if __name__ == "__main__":
    main( )
```

Listing 47. Suggested If Main Expansion

The `i` in `iabbr` means it will work in Insert mode, and `abbr` is short for "abbreviate."

Note that I didn't have to explicitly add any indentation after the `Enter` because the Python indentation engine takes care of that for me. Note also that the `<Space>` I typed after `ifmain` was inserted between the brackets. If you need to expand an abbreviation without adding spaces, use the `Control-]` keybinding to trigger expansion instead.

And if you need to insert the words `ifmain` without expanding them, type `ifmain<Escape>` to return to Normal mode without expanding.

This abbreviation will only exist until I close the editor. To make it permanent, I need to add it to my LazyVim configuration. Typically, abbreviations only make sense within the context of a single filetype, so I collect mine in the `autocmds.lua` using syntax like this:

```
vim.api.nvim_create_autocmd("FileType", {
  pattern = { "python" },
  callback = function()
    vim.cmd('iabbr ifmain if __name__ ==
"__main__":<Enter>main()<Left>')
    vim.cmd("iabbr frang for i in range():<Enter><Esc>F(i")
    -- Other Python abbreviations
  end,
})
```

Listing 48. If Main Abbreviation

The `frang` abbreviation shows another neat trick: You can use the string `<Esc>` to enter Normal mode and move the cursor. I used `F(` to "find the previous open paren" followed by `i` to enter Insert mode inside the `range()` parens.

Vim abbreviations have been around forever and do the job well. I still use them (probably because I am old), but the world has largely moved on to snippets instead.

14.11. Snippets

LazyVim ships with the `blink.cmp` plugin, which provides the high-speed completions interface we've seen before. Amoung other completions, it connects to Neovim 0.10's built-in snippets functionality. It can load VS Code-style snippets [https://code.visualstudio.com/docs/editor/userdefinedsnippets#_create-your-own-snippets].

By default, `blink.nvim` pops up a simple menu with a bunch of completions as you type. For example, here's what I see if I type `if` in a Python file:

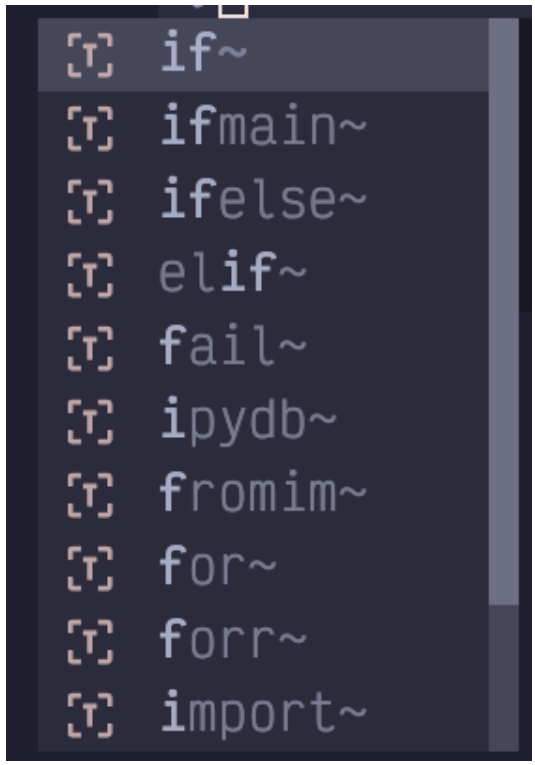


Figure 74. Cmp Menu `if`

The list shows possible completions. I can move my cursor up and down the list with the *arrow* keys or `Control-n` and `Control-p` (`j` and `k` won't work here because I'm still in Insert mode). Most completions have a preview box pop up with documentation or an example of the completion.

I temporarily disabled the LSP to hide non-snippet results in this screenshot.

This snippet was created by the `FriendlySnippets` plugin, which is a massive collection of useful snippets that ships with LazyVim. (Also notice that there is an

`ifmain` snippet much like the abbreviation I apparently didn't actually need to define above!)

If I then press the `Control-y` key, which confirms a completion (or `Enter` if you use the LazyVim defaults or `Right Arrow` if you have configured `blink.cmp` the way I have), the snippet is inserted into my editor:

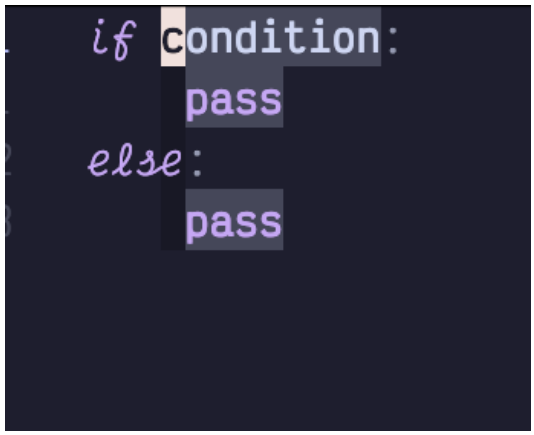


Figure 75. Inserted Snippet

The editor is currently in "Select" mode, an uncommon mode that is superficially similar to Visual mode. In LazyVim's default config, I'm not aware of any way to get into Select mode other than accepting a snippet! So we won't go into detail about this mode outside the context of snippets.

The key point is that "condition" is currently highlighted, and I can start typing immediately to overwrite it, almost as though I was in Insert mode. Once the condition has been replaced, I can press the `<Tab>` key, which, in Select mode, means "jump to the next field in the snippet." Now the `pass` inside the `if` is highlighted instead.

The `<Tab>` key only works like this if `nvim-snippets` is aware it is in a snippet that has fields.

14.11.1. Defining New Snippets

If the `FriendlySnippets` snippets aren't enough for you, you can define your own snippets using the now-ubiquitous VS Code Snippet syntax and load them in nvim-snippets. As a quick example, here's how to create a snippet for a boilerplate Svelte component:

1. If it doesn't exist, create the `~/.config/nvim/snippets/` directory to hold your snippets. This is the default location `blink.cmp` looks for snippets.

2. Create the `~/.config/nvim/snippets/package.json` file if it doesn't exist. It needs to contain a list of all snippet files. In this case, we'll be adding `svelte`:

```
{
  "name": "personal-snippets",
  "contributes": {
    "snippets": [
      { "language": "svelte", "path": "./svelte.json" }
    ]
  }
}
```

Listing 49. Snippet package.json

The language for a given filetype can be found be opening a file of that type and entering the `:set ft` command. 4. Create a `json` file that matches the path i.e. `svelte.json`. Give it the following contents:

```
{
  "Boilerplate Component": {
    "prefix": "scri",
    "description": "Basic svelte boilerplate",
    "body": [
      "script lang=\"ts\">",
      "  $1",
      "</script>",
      "",
      "${2:<div></div>}",
      "",
      "<style>",
      "  $3",
      "</style>"
    ]
  }
}
```

Listing 50. Snippet Definition

If you are unfamiliar with VS Code snippet syntax:

- `prefix` is the string you type in Insert mode to trigger the snippet. In this case,

it is `<scr`.

- `description` is a string that describes it in the preview pain.
- `body` is a list of lines in the snippet.
- `$1`, `$2`, `$3` represent "tab stops" in the snippet.
- `${2:<div></div>}` represents a tab stop with placeholder content that can be typed over.

If I restart Neovim and load a svelte file, I can type `<scri` to insert this snippet. The default output looks like this:

```
<script lang="ts">
</script>

<div></div>

<style>
</style>
```

Listing 51. Snippet Output

14.12. Summary

This chapter introduced various editing tips, starting with word counts and transposing characters, and then moving on to managing comments, indentation and formatting.

Finally, we covered the old-but-not-busted abbreviation syntax and the new-hotness Snippets engine that LazyVim ships with.

In the next chapter, we'll start discussing something completely different: version control in LazyVim.

Chapter 15. Source Control

LazyVim ships with several features to manage your source control history, and there are some excellent third-party plugins you can use as well. Some of these plugins work with multiple version control systems, while others are git-centric. This book will assume you use git because, well, you probably do, even if you use other systems as well.

15.1. The Integrated Terminal (A Rant)

For reasons I cannot explain, Neovim ships with a terminal emulator. It is bizarre to me that an editor that *runs in a terminal* ships a terminal. It is literally possible to open a terminal, open Neovim, open a terminal in Neovim, and open Neovim in a terminal in Neovim.

Add some nested ssh sessions if you really want to make a mess.

I don't need a terminal in my editor. I have a terminal already, an excellent one. I just use Kitty splits, tabs, and windows when I need a new terminal window. The smart-splits plugin allows me to switch between editor and terminal seamlessly and Kitty even manages installing itself over ssh for me.

Or I press `Control-z`, which is the traditional way Vim users used to access a terminal. It is a shortcut that I really wish hadn't gone out of style. Pressing `Control-z` "suspends" Neovim. If you're not in the know, you'll think it closed your editor without saving, because the window disappears and returns you to your terminal.

But fear not! It is merely suspended, as indicated by the `'nvim' has stopped` message in the output:

```
lazyvim-for-ambitious-devs on  chapter-15 [!+?] via  v1.1.16 via  v22.3.0
❯ nvim
fish: Job 1, 'nvim' has stopped

lazyvim-for-ambitious-devs on  chapter-15 [!+?] via  v1.1.16 via  v22.3.0 took 11s
✦ ❯ jobs
Job     Group   State   Command
1       99346   stopped nvim

lazyvim-for-ambitious-devs on  chapter-15 [!+?] via  v1.1.16 via  v22.3.0
✦ ❯ fg
```

Figure 76. Suspended Neovim

As this screenshot also shows, you can see the list of stopped (or running) background jobs using the `jobs` command in any shell. The `fg` (short for foreground) command starts the suspended Neovim process back up. If you have multiple suspended jobs, the `fg %#` command can be used to choose a specific job id (e.g. `fg %1` will run the job with id `1` in the first column of the `jobs` output).

This is not a Neovim-specific feature. The `Control-z` trick works with (almost) any long-running shell command. You can even set a suspended task to keep running in the background by using the `bg` command instead of `fg` (though if the background job prints to stdout you'll quickly become confused).

Between terminal splits and `Control-z`, there's just no need for the editor to have its own terminal embedded with it. Still, Neovim ships with an integrated terminal, so I should probably explain how to use it.

15.2. The Integrated Terminal (For Real This Time)

You can pop up a terminal at any time in Lazyvim using the keybinding `Control-/`. It will appear in front of all your other editor windows (unless you have the Edgy extra enabled, in which case it will show up in the bottom half of the editor) and can be dismissed with `Control-/` again.

Neovim's terminal window is a super weird hybrid terminal and Vim window. Once the terminal is open, you can use Normal mode commands to navigate around it.

However, unlike Insert mode, the `Escape` key WILL NOT put you in Normal mode, even though your fingers are, by now, conditioned to hit `Escape` reflexively. This actually makes sense because `Escape` is a common key to need to type in various terminal programs, so it would be rude for Neovim to steal it. LazyVim has set up the keybinding `<Escape><Escape>` (press `Escape` twice quickly) to switch to normal mode from terminal mode, or you can use the hard-to-type default incantation `<Control-\><Control-n>`.

Once in Normal mode, you can navigate anywhere in the Terminal window using any of the navigation keys including Seek and Search modes. This can occasionally be helpful if you need to yank some outputted text to the clipboard.

Pressing a key such as `a` or `i` will send you back to "Terminal mode" which effectively just sends every keystroke to the program currently running in the terminal (probably your shell).

Annoyingly, this means you can't use Normal mode to reposition your cursor on the command line; it will go back to wherever it was when you last entered normal mode.

If you want to use Vim Normal modes to edit your command line (in any terminal; not just inside Neovim) configure your shell to use "Vi mode." All modern shells support some version of this, and it usually allows you to use `Escape` to put the shell in a pseudo-normal mode. It gives you commands like `w` and `b` for navigation and basic line-editing commands like `d` and `c` to edit the command line.

There are third-party plugins that try to make the terminal experience more consistent and enjoyable, but in my opinion, they are not worth the trouble. I can just press `cmd-enter` to get a new Kitty terminal pane and have a perfectly normal terminal experience.

15.3. Checking Your Git Status

Lazyvim is preconfigured with a handful of carefully configured plugins that make your version control life much better.

The simplest of these uses the file picker to list files that have changed since the last commit. This will behave similarly to other file picker operations, except it only lists files that have modifications in git.

You can open it with `<Space>gs`. I use it a lot for switching between files related to whatever feature I am currently working on, and actually prefer it to the buffer picker (which only shows opened files) we discussed in Chapter 9.

This picker screenshot shows that I have modified two files since my last commit:

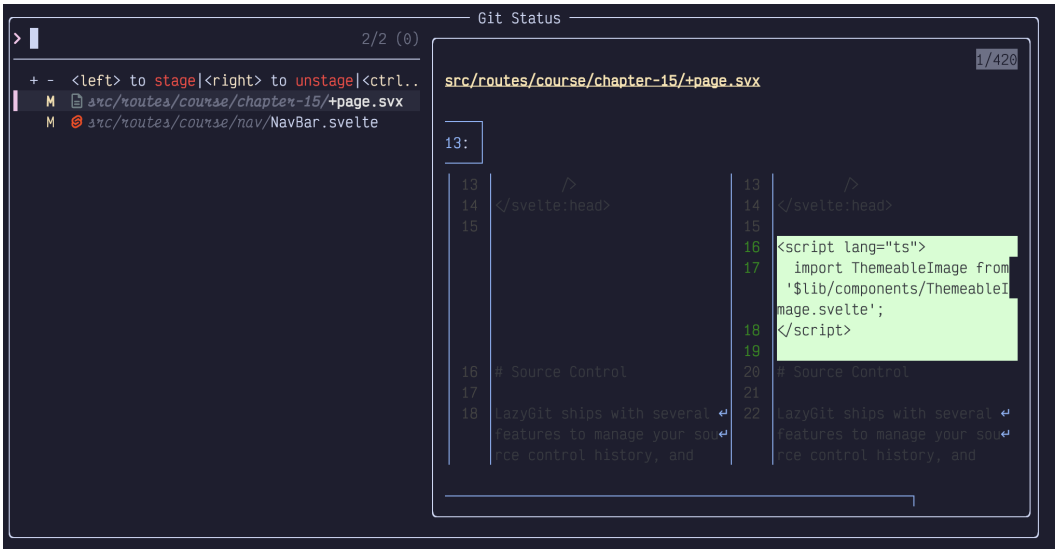


Figure 77. Git Status in Picker

Ensure the delta-pager CLI tool is installed to get the pretty diffs.

The preview pane shows the diff of lines I have added and removed. On the left, you can see that I have `page.svx` focused, and a preview of some of the changes in the file on the right.

The confusing bit to pay attention to is the first two columns in the results pane. The two columns are labelled `+` and `-`. I'm not sure why those symbols were chosen, as they don't reflect whether files or lines are added or deleted. The `+` column holds files that have been staged to go in the next commit, while the `-` column holds a status for files that have changed but have not yet been staged.

These columns indicate your git status for each file, and their meaning can be devilishly hard to remember. The symbols themselves are straightforward:

- `~` means the file on that line contains modifications since the last commit
- `-` means it has been deleted
- `?` means it is an untracked file (has been added to the working directory but not staged or committed)
- `+` means it is a new file that has been staged in git

If the sign shows up in the *first* column, it means the file has been staged and will be included in the next commit. If it is in the *second* column, then it means the file is not yet staged. If a `~` is in both columns, some parts of it have been staged and some parts have not.

In addition to allowing you to effectively view your git status, this picker also allows you to *stage* entire files. To do so, focus a file and hit the `<Left>` key to move the file to the "staged" column. To unstage it, use `<Right>`. If you want to throw away all the changes in the file, use `<Control-x>`.

15.3.1. Other Pickers

Snacks.nvim comes with pickers to view and search commit history (`<Space>gc`), kind of like a log browser, and to check out a branch. The latter doesn't have a keybinding for some reason, but you can use the command `:lua Snacks.picker.git_branches()` or bind a keypress to it if you like the picker UI for this task.

There are a variety of less commonly-used git-related pickers you can find by typing `:lua Snacks.picker.git` and then `Tab`.

15.4. Status of the Currently Focused File

Every buffer has a couple subtle indications of the changes in that file. Consider this screenshot:

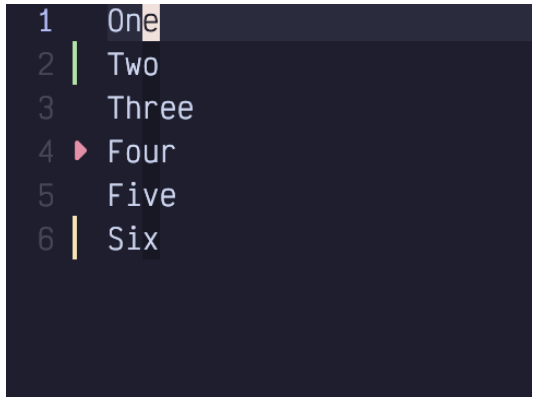


Figure 78. Git Status in Signs Column

Notice the left sidebar, to the right of the line numbers. It contains a green bar, a small red triangle, and a short orange bar. These indicators show that lines have been added, removed, and modified, respectively.

Additionally, in the status bar, just to the left of the file progress indicator we see these icons, which summarize the same information:

Figure 79. Git Status in Status Bar

15.5. Staging From the Editor

You can add files to git's index (so they are ready to commit) right from the editor. The `<Space>gh` menu (mnemonic is "**g**it **h**unks") has a bunch of interesting subcommands:

Figure 80. Git Hunks Menu

You can use `<Space>ghS` to stage an entire file, which would move it to the left column in the git status pickers we discussed above. If you want to stage a patch containing a subset of your changes, navigate to the hunk you want to stage (`[h` and `]h` are super handy for this) and hit `<Space>ghs`.

Most people have an unfortunate habit of just committing everything instead of properly curating their history, but if you are one of the rare folks who uses git properly (please be that person), you'll use the `<Space>ghs` command a lot.

You can also reset a hunk (effectively making it the same as it was at the time the last commit was made) using `<Space>ghr`. If you want to reset the entire file, use

the “but bigger” `<Space>ghR`. Resetting is a destructive operation, so be careful (though `u` for **u**ndo can usually get you back to where you were).

If you accidentally stage a hunk, use `<Space>ghu` to unstage it. Unlike reset, this won’t change the file; the changes will still be there; they just won’t be staged anymore.

15.6. Git Information Keybindings

The blame line (`<Space>ghb`) command shows the commit that last changed the line the cursor is currently on, useful for answering the all-important question “Why on Earth did I do that?”

Preview hunk (`<Space>ghp`) temporarily renders the original and changed version of a hunk (one above the other) so you can see exactly what changed.

The `Diff this` (`<Space>ghd` and `<Space>ghD`)commands do the same except in a side-by-side view that we will discuss later in this chapter.

Personally, I use many of these commands too often for the number of keystrokes required to pop them up. So I’ve created an `extend-gitsigns.lua` file in my plugins directory that copies them from `<Space>gh` to `<Space>h`:

```
return {
  "lewis6991/gitsigns.nvim",
  keys = {
    {
      "<leader>hb",
      "<cmd>Gitsigns blame_line<cr>",
      desc = "Blame Line"
    },
    {
      "<leader>hs",
      "<cmd>Gitsigns stage_hunk<cr>",
      desc = "Stage Hunk"
    },
    {
      "<leader>hS",
      "<cmd>Gitsigns stage_buffer<cr>",
      desc = "Stage Buffer"
    },
```

```
    {
      "<leader>hr",
      "<cmd>Gitsigns reset_hunk<cr>",
      desc = "Reset Hunk"
    },
    {
      "<leader>hR",
      "<cmd>Gitsigns reset_buffer<cr>",
      desc = "Reset Buffer"
    },
    {
      "<leader>hu",
      "<cmd>Gitsigns undo_stage_hunk<cr>",
      desc = "Undo Stage Hunk"
    },
  },
}
```

Listing 52. Git Hunks Menu Keymaps

I got these by copying them from the git-signs config on the LazyVim website and converting from `map` calls to the `keys =` format.

15.7. Lazygit

Lazygit (which, despite sharing the `Lazy` namespace with LazyVim and Lazy.nvim, is by an entirely different developer) is a terminal UI tool for interacting with git. It is a separate program that you will need to install with your operating system's package manager (e.g. `brew install lazygit`) if you want to use it.

LazyVim is preconfigured to show lazygit in a terminal window using the keybinding `<Space>gg`. I won't go into all the details of how to use this third-party program. It can do almost anything git can do in a much more user-friendly interface.

Lazygit takes a bit of study to get used to, but it has helpful menus and mnemonics for its keybindings so the learning curve is relatively gentle.

Ironically, I used lazygit (in its standalone format from the command line) a lot more before I started using LazyVim. I used to stage changes using lazygit, but now I use the `<Space>h` menu we just covered instead.

I also now do most of my git work with the exceptional Graphite [https://graphite.dev]

tool, which simplifies many of the flows I used to use lazygit for (especially rebasing). I still use lazygit every day; I just don't have it open 100% of the time like I used to.

15.8. Diff Mode

Neovim comes with a powerful, but slightly hard-to-learn diff viewing mode. It shows "before" and "after" files side by side and can even be configured to show the "parent" and changed state if you want a fancy merge tool.

There are several different ways to get yourself into Diff mode. The basic way is to specify it when you open two files on the command line:

```
nvim -d file1 file2
```

Listing 53. Open In Diff Mode

This opens the indicated files side by side in a linked diff view. Most often, you won't have two separate files, though. Instead, you'll want to see the difference between the current file and the staging index, which you can do with the shortcut `<Space>ghd`. Or use `<Space>ghD` to show the differences between the current file and the last commit, regardless of what has been staged.

Once you are done operating in Diff mode, it can be tricky to get back to the normal file. The issue is that when a file is in `diff` mode, it stays that way, even if other windows are opened or closed. The secret is to use the `:diffoff` command, which will disable "diff view" for the current buffer. This doesn't close the two side-by-side windows, though; you'll need to use normal window and buffer management tooling such as `<Space>bd` and `<Control-w>q` to do that.

Note that by default, the diff view will collapse any code that is identical between the two files into a single fold. Use the code unfolding command `zo` to expand a section.

15.8.1. Editing Diffs

If you use the `<Space>ghd` command to show your file in diff view against the index mode, you can keep editing the file to make additional changes. If you do this, only edit the file on the *right*. This is the "working" file. The file on the left is the "index" file; it shows the staged changes. If you want to "edit" the file on the left, use the

`<Space>ghs`, `<Space>ghr`, and `<Space>ghu` to stage, reset, and unstage hunks from the right side. It is not *forbidden* to edit the index file directly, but it will confuse the Diff mode machinery, so stick to editing, staging, and unstaging from the right side.

When working with diff view like this, I find that the stage, reset, and unstage keybindings best match the mental model I am used to. However, there are two kind of weird commands built into Neovim that you may sometimes prefer to reach for: `:diffget` and `:diffput`. These are more commonly typed as `:diffg` and `:diffp` to save a couple keystrokes, or you can use the keybindings `dp` and `do` (mnemonic `diff obtain`).

These commands are most often used in Visual mode (or with a range), and they essentially mean that (within that range) we should either "make this file the same as the other file" or "make the other file the same as this file", respectively.

Consider these two files that are slightly different:

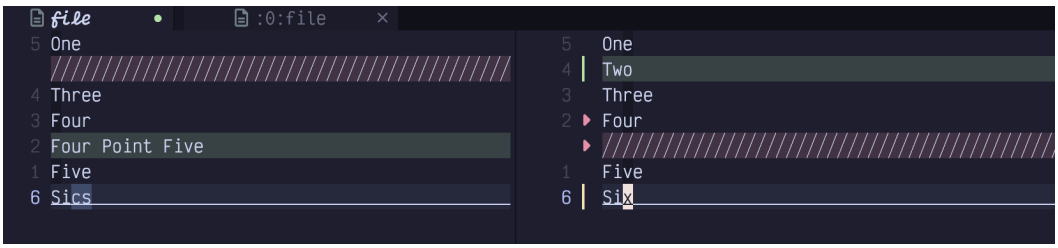


Figure 81. Diff Mode

The file on the left represents the state of my index, while the file on the right is my working copy. The indexed version was missing the word "Two", so I have added that on the right. It also had an extra "Four Point Five" line that I have removed on the right. And I modified the spelling of the word "Six".

Let's explore a couple ways to make these files identical with `:diffg` and `:diffp`. You can use these commands on either file, but it usually makes sense to operate on only one of them. For this example, assume I'm working on the right-hand file.

I can use any navigation commands to jump to the second line of the file. If you are editing a real git indexed file, the `[h` and `]h` keybinding are probably useful for jumping between hunks. However, when you are in "diff" mode you can also use the `[c` and `]c`, which mean "jump between changes," but **only** when you are in "diff" mode. (In a non-diff window, LazyVim has bound those keys to jump between classes or types.) I usually just use `[h` and `]h`, but in those instances where I am using a diff view that is not attached to git history, `[c` and `]c` should not be

forgotten.

So with my cursor on the first line of the file, `[c` or `[h` will jump to the second line, which contains the word `Two` in my file, but not the index.

I want to stage this change, so I type `:diffp`, which means "make the other file the same as this one."

The next line is `Four Point Five` in the left file, but was deleted in the right file. For the sake of argument, let's say I want to "unstage" this change, which is to say "make the right file the same as the left file". To do this from the right window, I can use `Shift-V` to enter Visual Line Mode, and select the lines containing `Four` and `Five` as well as the blank red space between those two lines representing the deleted line. Now I can type `:diffg` or `:diffget` which means "get the contents of the other window and make my window match it." Since `:diffget` and `:diffput` accept ranges, it passes the visual selection with the usual `'<` and `'>` marks.

If you find you like the above diff interface, but figuring out which files have differences is frustrating, you may want to configure the diffview.nvim [https://github.com/sindrets/diffview.nvim] plugin. I personally just use the git status telescope picker, but the `diffview.nvim` plugin has a nice interface and some handy commands.

15.9. Configuring Vim Diff as Merge Tool

Everyone seems to hate resolving merge conflicts. Armed with Diff mode and rebasing, I actually find the process kind of enjoyable. The trick is to have a slightly complicated `~/.gitconfig` (and a very large monitor).

I can't help you with the monitor, but the `.gitconfig` needs to look like this:

```
[diff]
    tool = vimdiff
[merge]
    tool = vimdiff
    conflictstyle = zdiff3
[mergetool "vimdiff"]
    cmd = nvim -d $LOCAL $BASE $REMOTE $MERGED \
          -c '$wincmd w' -c 'wincmd J'
```

Listing 54. Git Mergetool Configuration

The `zdiff3` conflict style makes diffs a bit easier to read by automatically resolving identical lines. The two `tool =` lines say to use the `vimdiff` merge tool that is configured on the last line.

That last line is a command to open Neovim with a whopping FOUR windows open and focuses the appropriate one.

To demonstrate this, I made a new git repo with two branches with conflicting changes. When I went to rebase (I always use rebase rather than merge commits because it allows me to deal with conflicts in the isolation of one change. This is why it's important to me that every commit have only one change!), one branch onto the other, of course, I end up with this error:

```
 git rebase main
Auto-merging file
CONFLICT (content): Merge conflict in file
error: could not apply f611b6f... Uppercase
Could not apply f611b6f... Uppercase
```

Listing 55. A Dreaded Git Conflict

To resolve this conflict, I run `git mergetool`. Because of the git configuration above, it will open Neovim with these four different diff windows:

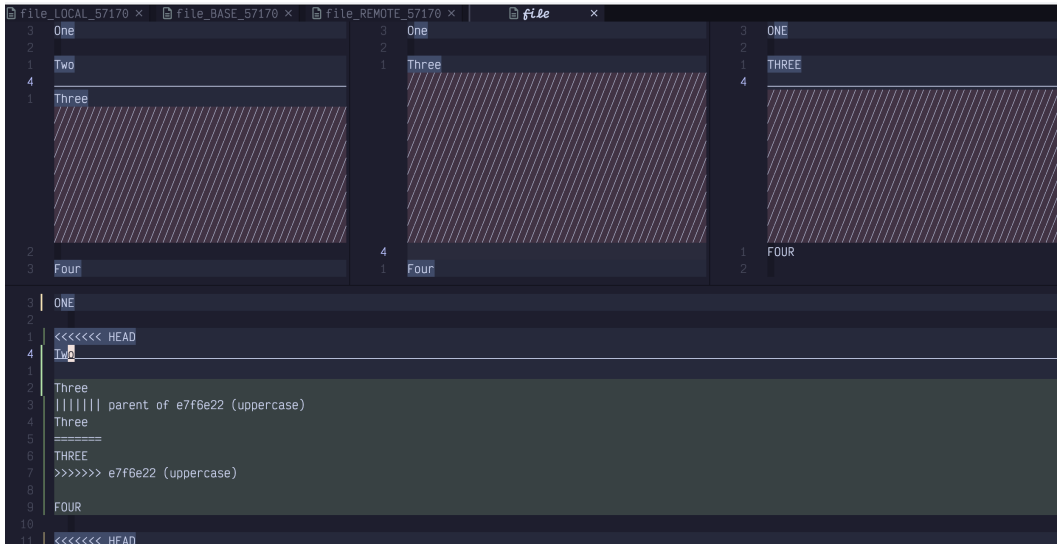


Figure 82. Merge Tool on Steroids

There are three windows across the top and one in a big pane (also pain) in the bottom.

Upper-left window

Shows the "local" changes. The meaning of "local" depends on exactly what commands you used to get into the conflict situation. In typical rebase flows, it returns to "the current state of the main branch". So when there is a conflict, it would contain "the other person's changes", so "local" doesn't seem applicable.

Middle window

Contains the "common ancestor" or "base" of the changes. Which is to say, this is the state of the file before either you or "the other person" made any changes. This window is not commonly included in merge-tool tutorials, but I find it can be quite helpful when trying to figure out what changed between the base and each of the two side windows.

Upper-right window

Contains the "Remote" changes, which, like local, can be a misnomer. In rebase flows, it usually means, "the changes I made on the branch I am rebasing onto main."

Bottom window

Contains "the current state of the file", which at the time the rebase failed includes messy conflict markers. This is the only file you should make edits to.

All four files will feature code folding if there are long sections of common code.

Also, if you scroll or move the cursor in the lower file, the upper files will also scroll so everything stays in sync, and an underline in the top three windows will indicate which line the diff tool thinks is the "current" one with respect to the cursor position in the lower window.

Most rebase flows start with using `vag` and `:diffg` from the lower window to make it identical to one of the upper windows. Then you would use `diffget` to get hunks from the left or right window, depending on context. You'll also usually have to do some manual editing.

The problem is, `:diffg` doesn't know which window to get things from because there are multiple windows open:

Figure 83. Diffget Error

Instead, we need to use the command `:%diffg 2`. The `2` is a buffer number. When you run merge-tool directly from the command line, the buffers are numbered in the order they are open. So `1` is the left-hand buffer, `2` is the middle one, `3` is the right-hand one, and `4` is the lower window. If you aren't sure, you can use the `<Space><comma>` keybinding to show the buffer list:

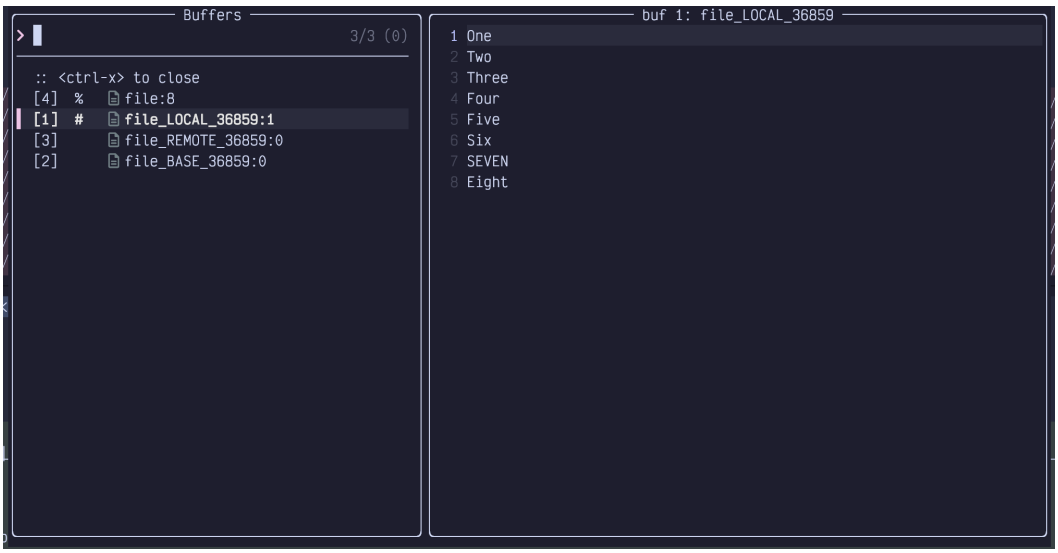


Figure 84. Buffer Numbers In Picker

In this list, the first column holds the buffer number. This number generally increases monotonically from the most recent time Neovim opened, so it can get

pretty high if you've been editing for a while. But when you use `git mergetool`, it typically opens a brand new Neovim instance and `1-4` are expected.

After running `vag` and the `:%diffg 2` command, the bottom window looks the same as the middle window, which is the state everything was before either branch was created. If I used `vag` and then `:%diffg 1` it would look the same as `main`, and `vag` followed by `:%diffg 3` would make it look the same as my branch. Then I could selectively look at differences between buffers and use `:diffg #` to get changes from the left or right one respectively.

Merge conflicts can always be somewhat stressful, but I find the four window view often makes it easier to understand what changed and why. That said, I only reach for it when I'm in a particularly knotty merge situation. Normally, I use the git-conflict.nvim plugin.

15.10. Git-conflict.nvim

While merge-tool is very helpful when working with particularly complicated merges, for simple conflicts, I usually find it quicker to just edit the file with the conflict markers in it directly. A plugin called git-conflict.nvim provides syntax highlighting and some keybindings to help navigate conflicts.

Set it up with a config something like this:

```
return {
  "akinsho/git-conflict.nvim",
  lazy = false,
  opts = {
    default_mappings = {
      ours = "<leader>ho",
      theirs = "<leader>ht",
      none = "<leader>h0",
      both = "<leader>hb",
      next = "]x",
      prev = "[x",
    },
  },
  keys = {
    {
      "<leader>gx",
      "<cmd>GitConflictListQf<cr>",
      desc = "List Conflicts"
    },
    {
      "<leader>gr",
      "<cmd>GitConflictRefresh<cr>",
      desc = "Refresh Conflicts"
    },
  },
}
```

Listing 56. Git Conflict Configuration

I use the `<Space>h` prefix that I set up previously for staging hunks and add a few new commands to it. After enabling this extension, if you open a file with conflicts, it highlights the conflict markers in a different colour. On my plaintext sample file it looks like this:

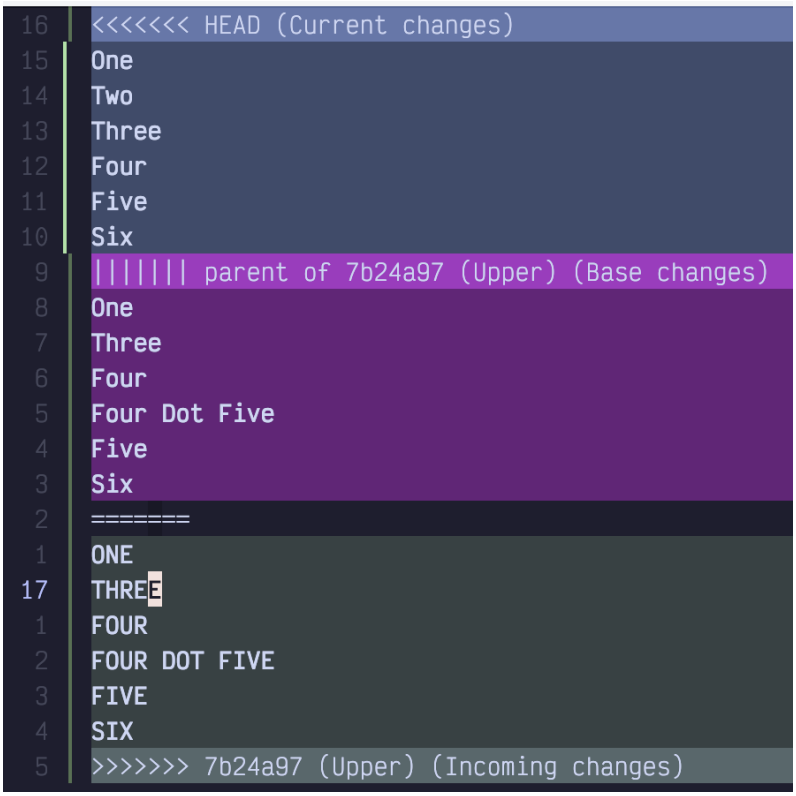


Figure 85. Conflict Markers

The conflict markers include the "current" (whatever is on main) code above, and the "new" (whatever is being rebased) code below, with the original or base code (before either change) in the middle.

I can use the `]x` keybinding to quickly jump to the next conflict (in this case there is only one). Then I can use one of the following keybindings to resolve the conflict:

- `<Space>ho` Choose the top version
- `<Space>ht` Choose the bottom version
- `<Space>hb` Choose both
- `<Space>h0` Go back to whatever is in the middle

The `o` and `t` keybindings are hard to remember. Technically they mean "ours" and "theirs", but depending on which order you did a merge or rebase, it doesn't always semantically map to your own or somebody else's commit. I just remember that `o` is before `t` in the alphabet, so it means the upper change. You could also map them to more mnemonic keybindings if you want.

In all cases, but especially in the latter two, you will likely need to do some manual

editing to make the code look correct. This is normal. None of the conflict management extensions uses AI to semantically understand what the changes *intended* to do, so you still need to do that part yourself!

About ninety percent of the time, this plugin is all I need to resolve a conflict. I only use the mergetool when things are particularly hairy or complicated.

15.11. Summary

This chapter introduced a lot of different ways of interacting with git and version control from inside LazyVim. You probably won't use all of it, but I wanted to present multiple options so you can decide which ones work best for you.

Perhaps you want to use Lazygit, or maybe you want to stay in the editor and use the functionality that git-signs and native Vim diff mode provide. Maybe you want to install some extra plugins such as git-conflict.nvim or diffview.nvim to streamline your experience (others you might want to look at include Neogit and mini.git).

Or maybe you don't want to manage this stuff from your editor at all and just want to drop to Terminal mode and use `git` or a wrapper like `graphite`. Whatever works for you, LazyVim provides the integrations you need.

In the next chapter we'll admit that it's not 2020 anymore and talk about artificial intelligence.

Chapter 16. Configuring Artificial Intelligence

This book was initially written in 2024, and I predict this year will be the one in which the world goes from making up crazy predictions about AI to it being part of our daily lives. It will be fun to revisit this statement if I get to publish a second edition. I know we're all sick to death of hearing about the future of AI, so this chapter is entirely about the present of AI.

Unfortunately, the present of AI is a bit of a mess. We don't yet know which of the various AI tools will become dominant. So this content will quickly become out of date. Hopefully the techniques and lessons it describes will be applicable to whatever the next generation of AI coding platforms will be.

LazyVim has excellent support for several AI coding tools. You'll probably need to try each of them and figure out which works best for you, and "best" is probably going to change many times over the coming months (it reminds me of the browser wars at the turn of the century).

At the time of writing, GitHub Copilot is probably the best to get started. It has a free tier and the Copilot chat plugin is a nice experience. Most importantly, if your organization already uses GitHub, Copilot is the most likely to have already been vetted by your security team. Do not allow your editor to send intellectual property to arbitray AIs!

16.1. Basic Anatomy

Most AI-coding tools integrate with `blink.cmp`, the plugin LazyVim uses for completions and snippets, and this is the simplest way to get started with them.

Some plugins also have a "chat" feature that allows you to communicate with the AI in a more interactive way. In my experience, these features don't work very well right now, with one exception that we will cover.

16.2. Codeium

If you've never used AI for coding before, you may be skeptical as to whether it's worth paying for. In fact, I use it extensively, and I'm still skeptical! Codeium is a great place to start because the free "Individual" tier is "smart enough to be useful."

It's also faster and more responsive, and when it comes to waiting for completion menus to pop up, inaccurate but fast is often more useful than accurate but slow.

To get started, you will need an account on codeium.com [https://codeium.com/].

Enabling Codeium in LazyVim is dead simple: just open the `:LazyExtras` command and find `coding.codeium`. Hit `x` to install it as usual and restart Neovim.

You should be presented with a window to authenticate Codeium automatically. If not, type the command `:Codeium Auth`. You will see a simple menu:

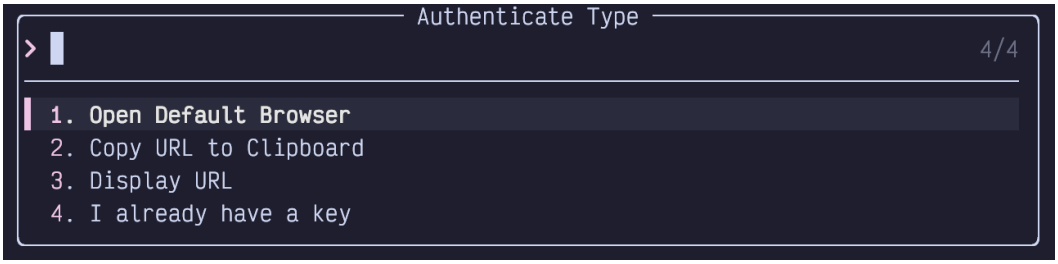


Figure 86. Codeium Auth

Select the option most appropriate for you to open a URL in a web browser. The default will work for most people, but if you are running Neovim over SSH or in a docker container you might need to get more creative.

Once logged in, you'll see a page with a token in it and instructions to copy it to your clipboard. Do so, then paste it in the `Token` box waiting in Neovim. If you've lost this box, you can find it again by typing `:Codeium Auth` and selecting "I already have a key".

You may want to restart your editor again. Now, while you are coding, you'll get Codeium entries in your completion menu:

Figure 87. Codeium Cmp

As you can see by the preview to the right, the completions may be multi-line. Sometimes, they are accurate and useful. Other times, not so much. (I once tried to get ChatGPT to tell me what percentage of the time it is accurate and it flat out

refused to give me a number).

As with everything AI, don't rely on the answers, but it can save you a few keystrokes every once in a while, and that's all any Vim user really wants in life.

16.2.1. Codeium Chat

Codeium has a chat feature that you can access through their web UI. I was excited to hear that an "in-editor" chat experience was recently added to Codeium.nvim but all it does is open a web browser to the chat window. The Codeium chat is (at time of writing) not terribly sophisticated and I find it nearly as frustrating to use as Eliza, so my recommendation is: don't bother.

16.3. GitHub Copilot

At time of writing, GitHub Copilot is considered the current gold standard for AI-driven coding. From an enterprise perspective, its primary advantage is that if you already use GitHub, your security team has already vetted it. From a developer perspective, I find it a bit slow, but more accurate than other AI tools I've tried.

First, sign up for an account on GitHub [https://www.github.com/features/copilot]. It's free for 30 days so you can decide if it's worth it before submitting an expense to your employer.

There are several different LazyVim plugins that support Copilot. The easiest is to use the `:LazyExtras` command and enable `coding.copilot`. This enables the `zbirenbaum/copilot.lua` plugin, which creates blink.cmp completions similar to those that the codeium plugin provides.

Similar to Codeium, type the command `:Copilot auth` to enable your account and follow the prompts. You'll be given a code to type into a browser window on GitHub.

After that, you should see Copilot completions in your blink.cmp popups just like you did with Codeium. You can use both at the same time, but be aware that it's hard on the environment **and** your network connection!

16.3.1. Copilot Chat

The copilot.lua plugin provides completions and little else. However, there is a second LazyVim extra you can enable that I really enjoy: `coding.copilot-chat`.

Restart Neovim, and if everything is set up correctly, you should be able to use the

keybinding `<Space>aa` to open a chat session (where `a` opens the "ai" menu). This will be a normal Neovim buffer that you can append text to in Insert mode. There are several keyboard shortcuts available to aid your chatting, summarized right in the chat window:

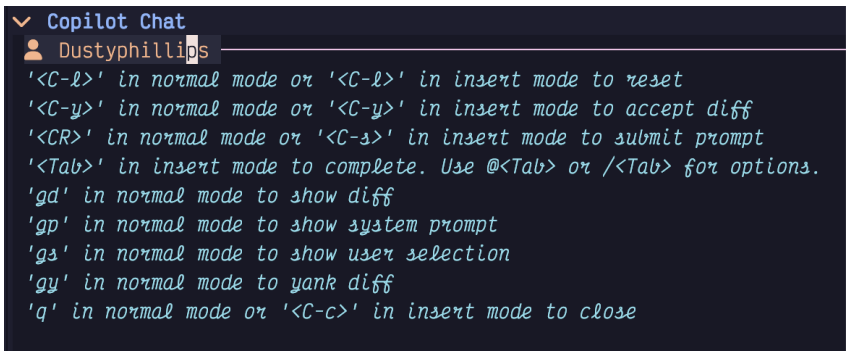


Figure 88. Copilot Chat Help Menu

In normal usage, you will be editing a file and then realize you want to send some subset of the file to Copilot to have it rewrite it. Select the code in Visual mode and hit `<Space>aa`. This will store the selected code and show the copilot chat window.

Input a question about the code you had selected and hit either `Control-s` (`s` for "**s**end") (in Insert or Normal mode), or `<Enter>` while in Normal mode to submit it. The plugin will add the code to your query automatically. Then wait (patiently) for a response. The response will contain the typical AI drivel describing what it did, and, most likely, some code.

If the code does what you want, simply hit `Control-y` (`y` for "yes") to accept it, replacing whatever you had selected with the AI text. If not, enter Insert mode and post a follow-up question to guide the AI to what you were actually looking for.

If the chat is getting out of control, as often happens with generative AI, hit `Control-l` to reset it. This is a safer course of action than typing a bunch of curses to the AI. Nowadays, it just becomes embarrassingly over-conciliatory, but you can be sure they have long memories, and you don't want to be the person who cussed out the AI when it becomes your boss!

The Extra configures a few other handy keybindings for interacting with copilot-chat, all of them in the `<Space>a` submode. The "prompt" menu with `<Space>ap` is particularly helpful if you want to instruct Copilot to do something specific like write a test or commit message. You can also use `<Space>ad` while a diagnostic is displayed to get more information about that diagnostic.

You can even change the model you use with CopilotChat! At time of writing, Claude

seems to be the least unhelpful coding assistant. To switch to Claude, or one of several other models, enter the `:CopilotChatModels` command and select the model you prefer.

16.4. TabNine

The advantage of TabNine is that it can run locally on your device, a very important feature for certain enterprise situations or anywhere that intellectual property matters (I wouldn't be surprised if AI destroys the concept of intellectual property once and for all. The advantages of a hive-mind may well outweigh the long-term advantages of corporate ownership).

You can configure Tabnine in Neovim similarly to Codeium and Copilot. LazyVim ships with support to integrate TabNine with blink.cmp to get completion sources. You can enable it with the `:LazyExtras` command and `x` on the `coding.tabnine` line.

Restart Neovim. You'll probably have to wait a while for the plugin to download and install TabNine, but it **should** all be taken care of for you. Once the plugin is installed, run `:CmpTabnineHub` to authenticate yourself, then restart Neovim one more time.

Now you should see TabNine completions in your list. I find they are virtually useless on the basic plan.

TabNine also has a newish official Neovim plugin named tabnine-nvim. It even has a chat mode, but this mode requires running a separate binary to show the chat window alongside your editor. This is obviously problematic if you want to use Neovim inside ssh or docker sessions, and installing it requires setting up the entire Rust and GTK toolchain, so it is beyond the scope of this book.

16.5. Sourcegraph Cody

Cody theoretically has an advantage over some of the other AI tools because it has access to Sourcegraph's insights into the structure of your codebase. They recently updated to Claude V3 for their chats, so conversations with it are fairly natural.

Unlike the other mentioned tools, there is currently no LazyVim Extra for Sourcegraph or Cody. Instead, you'll need to install the sg.nvim plugin manually:

```
return {
  {
    "saghen/blink.cmp",
    dependencies = { "sourcegraph/sg.nvim" },
    opts = function(_, opts)
      table.insert(opts.sources, 1, { name = "cody" })
    end,
  },
  {
    "sourcegraph/sg.nvim",
    dependencies = {
      "nvim-lua/plenary.nvim",
    },
    opts = {},
  },
}
```

Listing 57. Source Graph Cmp Configuration

Sign up for an account over at https://sourcegraph.com/ and then run the `:SourcegraphLogin` command. Note that the `g` is **not** capitalized; I thought my plugin wasn't installed correctly and spent half an hour debugging that typo.

This plugin is more than just an AI completion tool; it also gives you access to commands to browse your code using Sourcegraph. However, they depend on the Telescope picker instead of Snacks.nvim. You can replace the picker with Telescope in the Lazy Extras, but I only recommend going that far if you **really** love Sourcegraph. The product has some pretty cool features, but those are beyond the scope of this book. You can learn more about available features by reading through `:help sg.nvim`.

If things aren't working correctly, use `:checkhealth sg` to help understand what went wrong.

With the above configuration, blink.cmp completions should work similarly to the other tools. To interact with the Cody chat assistant, use the command `:CodyAsk` with a prompt. For example, select some code and type `:CodyAsk what does this do?`

A window pops up in front of your code that looks like a chat interface, with some keybinding tips on the right side:

Figure 89. Cody Ask Chat

Other chat commands you may want to read about in the help files include `:CodyChat!`, `:CodyTask`, `:CodyToggle`, `:CodyExplain`, and `:CodyRestart`.

16.6. Supermaven AI

Supermave claims to have the fastest response times of the various AI coding assistants. It has the advantage of being able to use the free plan without setting up an account, but if you sign up for a paid account, you'll supposedly get better results.

Enabling the plugin is as easy as the other tools. Simply hit `:LazyExtras`, search for `ai.supermaven` and hit `x`. Restart Neovim and give it a few moments to download the Supermaven binary behind your back.

You should automatically see an activation link that looks something like this:

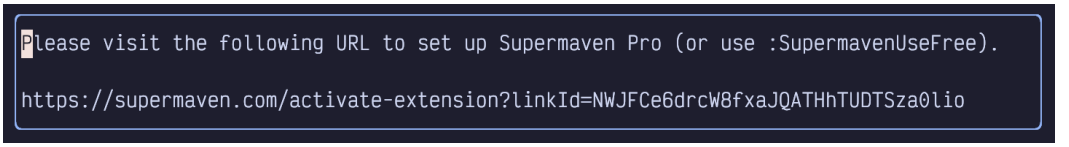


Figure 90. Supermaven Activation Link

If you want to use the pro plan, click (if your terminal supports clicking links) or copy-paste the link to your browser and follow instructions. If you want the free version, close the popup window with `:q` and then enter the `:SupermavenUseFree` command.

Now you should see Supermaven completions in your `blink.cmp` popup as you

type.

Supermaven doesn't have a chat interface. There are a handful of commands for things like restarting Supermaven if you type `:Supermaven<Tab>`.

If you have tried the free plan and decided to upgrade to pro, you may be confused! You'll need to use `:SupermavenLogout` followed by `:SupermavenUsePro` to get the activation link again.

16.7. What About ChatGPT?

ChatGPT is a good coding assistant, and there are several plugins that integrate nicely with the editor. However, none of them are available as LazyExtras. They also all require an OpenAI API key, which tends to be a lot more expensive than opening ChatGPT in the browser and asking it questions on the paid plan, then copying text over in the most awful way possible.

For the most part, I think GitHub Copilot is as smart as ChatGPT (about coding, anyway), and the CopilotChat plugin is a nice interface, so I recommend that instead.

If you want to explore integrating ChatGPT, Claude, Gemini, or several other AI engines into Neovim, check out the parrot.nvim [https://github.com/frankroeder/parrot.nvim] plugin. Configuration is similar to the other tools above.

16.8. Summary

This chapter was all about AI. AI is a weirdly simple topic, considering the complexity it abstracts away. Under the hood, LLMs are really cool, but the interface to interacting with them is typically just a simple HTTP request to some API somewhere (an API that is so useful we collectively don't think hard enough about how much we trust it).

Various Neovim plugins have created interfaces to these APIs. LazyVim makes it easy to integrate many of them with blink.cmp, and other services are covered with third-party plugins.

Choosing between the various options is the hard part. None of them work quite as well as I would like, and I find that the amount of time they save is about equal to the amount of time they cost when they get things wrong. I currently use Copilot because my employer pays for it, but I have disabled it before and not missed it.

Next up, we'll discuss running debuggers from inside LazyVim.

Chapter 17. Debugging

LazyVim supports debugging various programming languages right in the editor. To be honest, I've spent my entire career debugging largely with logging statements. I've always found that the trouble of setting up and integrating a debugger is not worth the time it takes. They work well in toy projects, but once you're trying to get the debugger to co-operate with e.g. Docker (or get anything to co-operate with Docker for that matter), or async third party libraries, it always feels like it just wasn't worth the effort.

So what I'm saying is: I've never used LazyVim's debugging system, so writing this chapter is a crash course for me as well. I always learn best by teaching (and teach best by learning)!

17.1. Debug Adapter Protocol

In addition to language server providers, VS Code also brought us the Debug Adapter Protocol, usually shortened to DAP. Like LSPs, DAP is an abstraction to allow editors to integrate with a variety of debuggers without having to reimplement the gritty language details of each one.

Neovim doesn't have built-in support for DAP like it does for LSPs, but LazyVim can be configured to enable a collection of essential plugins to use it. To do so, simply install the `dap.core` LazyExtra.

If you have also enabled the lazy extras for your preferred programming language, it is probably already configured to work with DAP. To double check, see if the `lang.<your-preferred-language>` extra has a `nvim-dap` configuration section in it. If it does, you *hopefully* don't need to configure anything.

17.2. Basic Example: Python

If you have installed the `dap.core` and `lang.python` Lazy Extras, and you have Python installed on your system, you are ready to start debugging.

I wrote a simple Python script to demonstrate this:

```
def main():
    world_descriptions = ["Hello", "Cruel", "Beautiful",
"Tired", "My"]

    for description in world_descriptions:
        print(description + " world")
        if description == "Tired":
            print("")


if __name__ == "__main__":
    main()
```

Listing 58. A Python Script

(The `ifmain` snippet we noticed in chapter 14 came in handy!)

If you open this file in Neovim and press `<Space>d`, you'll see a brand new menu of debugging commands:

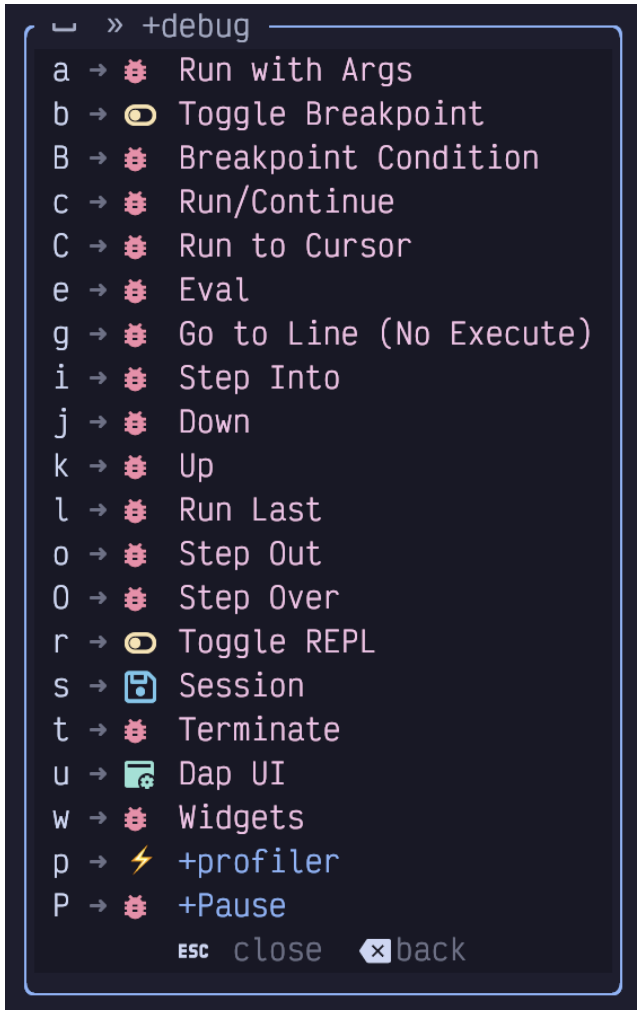


Figure 91. Debugging Menu

Many of these only make sense if the debugger is already running.

Let's move the cursor to the `print(description + "world")` line and hit `<Space>db` to set a breakpoint. Then we can use the `<Space>da` command to open the run menu, which has these options (for the Python Extra) to start debugging:

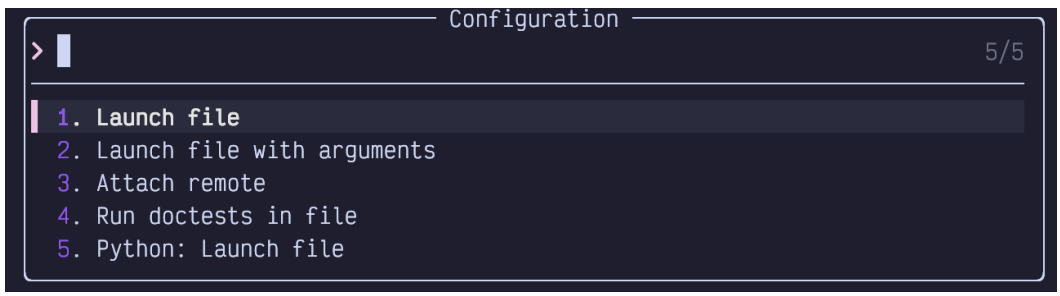


Figure 92. Run Args Menu

I don't know why there are so many because options 1, 2, and 5 all seem to do the same thing. Even when you hit enter with option 1 selected it prompts for arguments. This example doesn't need arguments, so I just hit enter a second time. The program will run up to the breakpoint and open FIVE new windows surrounding the current window (another vote for investing in big monitors):

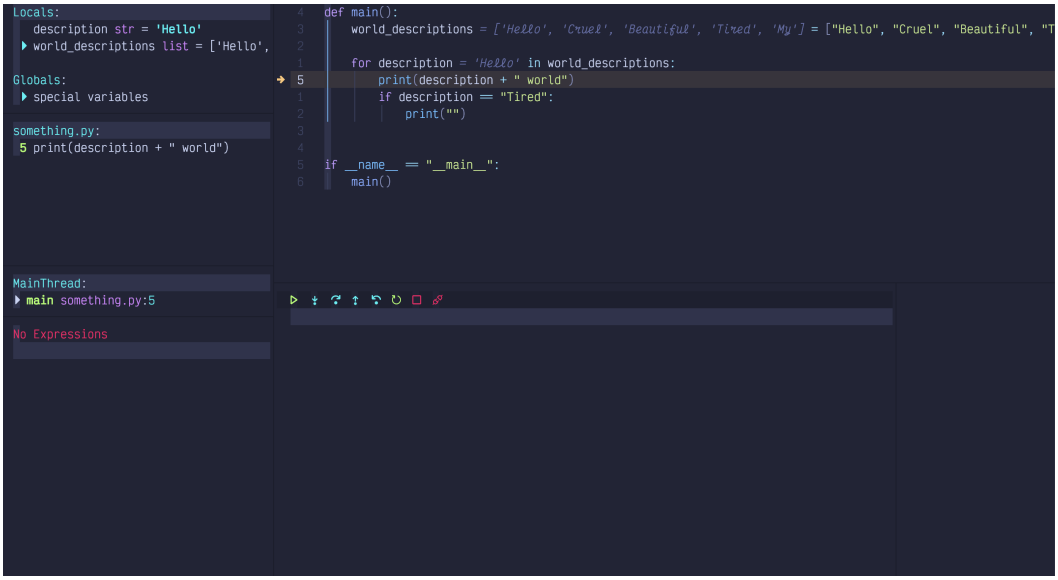


Figure 93. Debug UI

Let's start by discussing the two windows under the editor:

- The one on the left contains a toolbar with common debugging actions (most of which can also be accessed from the `<Space>d` mini-mode). Depending on language and environment, this window sometimes includes the program output, or messages about actions you've taken since the debug session started. In my Python setup, it remains blank.
- The small window on the right is where the console output so far is displayed, at least for Python. Since this is the first run through the loop, there is nothing in the console section, yet.

I've scaled the window down to fit in this book, but I would normally have this maximized on my largest monitor if I was in an active debugging session.

Let's check out the left sidebar, split into several windows:

Locals and Globals

Provide a list of known variables in the program and their current values. These update as you step through the program. Anything with a little triangle beside it can be expanded by moving the cursor to that line (`s` mode is delightful here because you can use it to jump directly to the window from the source code) and hitting `Enter`.

something.py

Shows a list of all the breakpoints currently set. In this case, there is only one, but we can see it is at line 5 and get a preview of that line of code. This information is also highlighted in the code window itself with an arrow pointing at the currently breakpointed line or a big dot if it is not the current breakpoint. This icon stays visible even after I hide the DAP UI with `<Space>du`.

MainThread

Shows me the current call stack, which is admittedly pretty simple in this particular program. As with locals and globals you may be able to expand certain lines if there is a small triangle beside it. This program doesn't have any nested function calls so there's nothing to see.

No Expression

I don't currently have any watch expressions. This window is super-powered because you can enter Insert mode in it. For example, I can add a watch on the `description` variable by typing `i` followed by `description` and then `enter`:

Figure 94. Watch Expression Input

I can now run `<Space>dc` a couple times to "continue" the debugging session, which means it will run to the next breakpoint. Since our single breakpoint is in a loop, it will break in the same place, but with new values:

Figure 95. Break on Beautiful

Notice the word `Beautiful` in a few different places:

- The `description` variable in the locals section
- The `description` watch we just added in the watches section
- In the editor window, we see `= Beautiful` beside the description in the loop. Yes, the editor is able to show us the value of variables while we debug, and this feedback is visible even after we hide the DAP UI!

Also notice that we've been through the loop twice now, so the console output section in the lower right now has two lines of output.

You can even edit the value of a variable LIVE in the `locals` window by navigating your cursor to the line and hitting `e` to be prompted for a new value. After changing it to "Foo", confirming with `Enter`, and continuing with `<Space>dc`,we see that this line gave different output from what was in the original list:

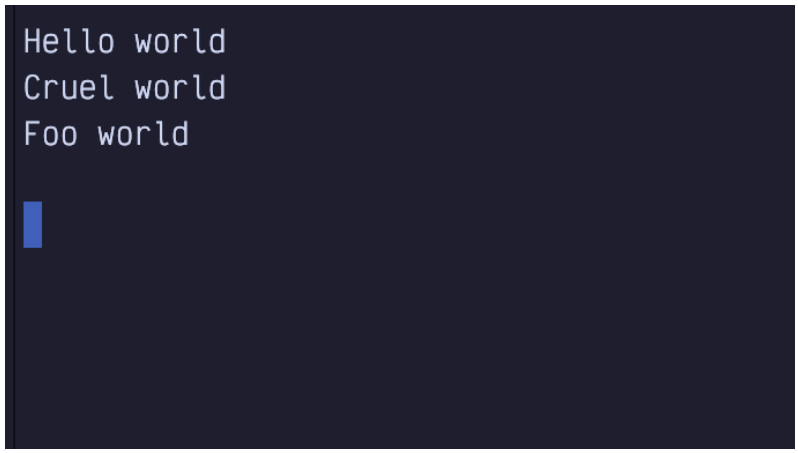


Figure 96. Live Edited Variable Output

If you want to step through the lines of a function, use `<Space>dO` for "step **O**ver". This will act as if you have a breakpoint on every line in the function. If you want to jump "out" to the function that called the current function, use `<Space>do` (lowercase `o` this time). `<Space>di` will jump "into" the function call under the cursor so you can step through lines inside the called function.

For *conditional* breakpoints, use the `<Space>dB` (uppercase `B`) instead of `<Space>db`. You'll be prompted to provide a condition as I've done here:

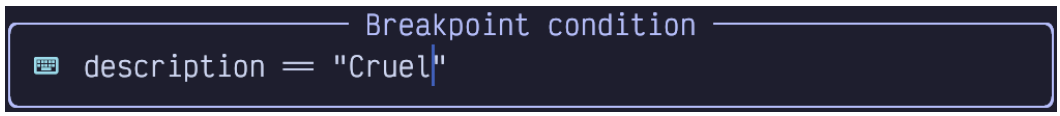


Figure 97. Conditional Breakpoint Input

Now if I run `<Space>dl` (run "last" debugging command, so I don't have to select a debugging environment and enter arguments like I did with `<Space>da`), it will go through two iterations of the loop, outputting `Hello World` before breaking only when `description == "Cruel"`.

So that's a whirlwind tour of the LazyVim debugger. It worked flawlessly in this example, but as I stated at the beginning, toy examples are normally easy with debuggers.

Most likely, a real world Python project needs to be run in a virtual environment. If you activate the virtualenv before launching Neovim, the debugger (and LSP tools) should just work with the venv. LazyVim's Python extra does support selecting virtualenvs, but I find that activating it before I open the editor is the least surprising way

to manage it.

17.3. Remote Debugging (An Example With Go)

You can also run a debug service in a remote location (typically a ssh server or Docker container) and connect to it from your local Neovim. Personally, I would rather install Neovim in the remote location and just run it from there, but that's a separate rant.

The instructions to actually set up and run the debug adapter in the remote location are entirely too language dependent. For this example, I'm going to use Go. I have enabled the `lang.go` LazyExtra, and tested it on a local file using steps similar to those described for Python.

Now it's time to test it in a container. I'm going to use Podman, but you can do this with Docker or ssh as well.

First, I ported the Python script above to Go:

```
package main

import "fmt"

func main() {
  for _, description := range []string{
    "Hello",
    "Cruel",
    "Beautiful",
    "Tired",
    "My"
  } {
    fmt.Printf("%s world\n", description)
    if description == "Tired" {
      fmt.Println()
    }
  }
}
```

Listing 59. A Go Script

Then I created the following simple `ContainerFile`:

```
FROM golang:1.22-alpine
WORKDIR /app
COPY main.go ./
RUN go build -o /hello main.go
CMD ["/hello"]
```

Listing 60. Golang ContainerFile

`podman build -t somego .` and `podman run --rm somego` indicate that the two files are working.

Typically, your organization has a `Dockerfile` or `Containerfile` that it doesn't want you to mess with to install useful things like a debugger (or Neovim).

So I always passive aggressively create a separate gitignored `Containerfile.local` that extends the company `Containerfile`.

The debugger for Go is called `delve`. You can install it temporarily by running the container as a shell:

```
$ podman run --rm -it somego /bin/sh
> go install github.com/go-delve/cmd/dlv@latest
```

Listing 61. Installing Delve in Container

Now the container should have a `dlv` command in it, but only until that container exits. To make it more permanent, my sneaky `Containerfile.local` looks like this:

```
FROM localhost/somego
RUN apk add git
RUN go install github.com/go-delve/delve/cmd/dlv@latest

EXPOSE 40000

CMD ["dlv", "debug", \
     "-l", "0.0.0.0:40000", \
     "--headless", \
     "--accept-multiclient", \
     "./main.go"]
```

Listing 62. Sneaky Container Extension

You can run the `dlv` command on any port; just make sure you `EXPOSE` the same one.

Build this container with:

```
`$ podman build -f Containerfile.local -t my-some-go`
```

then run it with:

```
`$ podman run --rm -it -p 40000:40000 --name my-some-go my-
some-go`
```

One of the many reasons I dislike containers is how verbose the commands have to be. Now it is *finally* ready to connect a debugger.

The next step is to configure `nvim-dap-go` to connect to this port. It's really rather verbose and a bit fragile. I made a new `extend-dap-go.lua` file in my plugins directory that looks like this:

```
return {
  "leoluz/nvim-dap-go",

  opts = {
    dap_configurations = {
      {
        type = "go",
        name = "Attach container",
        mode = "remote",
        request = "attach",
        substitutePath = {
          {
            from = "${workspaceFolder}",
            to = "/app",
          },
        },
      },
    },
    delve = {
      port = 40000,
    },
  },
}
```

Listing 63. Nvim-Dap Remote Go Configuration

The port needs to go in a separate `delve` section, and has the unfortunate side-effect of always binding to that port, even if you are running a local debugging session. The `substitutePath` section is configured to map breakpoints from your local directory to the `/app` folder inside the container. Change it if your `Containerfile` uses a different `WORKDIR`.

Restart Neovim to pick up the new configuration and open the `main.go` file (on the local filesystem). Set a breakpoint in the usual way with `<Space>db`. Then hit `<Space>dc` to pop up a menu of possible debug configurations. You'll see `Attach Container` in the menu. Select it, invoke whichever debugging gods are listening to you today, and wait for the breakpoint to hit.

Keep an eye on the podman container output, as that is where the `fmt.Print` statements will go. Console output won't show up in the editor.

The dev experience with this isn't great (although it's pretty typical if you are used to

working with containers). You have to stop and restart the container if you let it run to completion (or if you make changes to the code). So overall, I recommend doing your `go` coding and debugging outside the container. Go is designed to build statically self-contained binaries, after all. But if you have your reasons to use a container, now you know how to do it!

17.4. Example: Connect to Chrome

The Chrome debugger in the Browser dev tools window is excellent, so you are forgiven if you'd rather just use it than the nvim-dap-ui. (Hey, I won't judge; I still use `console.log` for most of my Chrome debugging).

But it turns out to be surprisingly easy to connect from Neovim. First, start the Chrome browser from the terminal, passing it the `--remote-debugging-port=9222` flag. The exact path to Chrome is OS- and package-manager dependent; on Linux it might be in your path as `google-chrome` and on MacOS if you installed the `.dmg`, you'll probably need to access something like:

```
/Applications/Google\ Chrome.app/Contents/MacOS/Google\ Chrome
--remote-debugging-port=9222
```

Listing 64. Path to Chrome

Second, open Mason from inside Neovim with `<Space>cm`. Search for `chrome-debug-adapter`. Install it with `i` and restart Neovim.

There is no extra configuration required. This kind of blew my mind, since I don't see that LazyVim does anything extra to hook it up. Just open a `jsx`, `tsx`, `js`, or `ts` file. Then hit `<Space>db` to add a breakpoint and `<Space>dc` to connect to Chrome and have it break on that breakpoint. I didn't even have to select a configuration!

I tested this in a brand new Vite-react app, putting the breakpoint inside a callback when a button is clicked. Click the button and the debugger pauses with all the locals set. I can even step deeply into React source code using `<Space>di` and LazyVim automatically opens the file from `node_modules`.

Again, I don't really see the utility of this over just using the Chrome devtools debugger. Normally, when I'm debugging frontend code, I'm more interested in how my interactions with the *browser* affect the state of the debug tools, so switching to my editor to continue to the next breakpoint isn't actually convenient. But there are lots of types of coders out there, and now you know that it works.

17.5. Summary

This chapter was all about debugging code directly from your editor. This is a tricky topic for an author to cover well because debuggers are surprisingly simple when they work, but they don't often work out of the box. LazyVim does as good a job as I've seen (no worse than VS Code) at out-of-the-box configuration, but you'll still probably be mucking around with configurations before it works with your system. There's no getting around that.

If you can get it working well, you'll probably find you're a more efficient developer than if you just use `print` statements. But it can take a long time to regain the initial set-up time, and the skill isn't transferable; as soon as you start a new project, you'll probably have to go through a different configuration incantation all over again!

In the next chapter, we'll discuss Neotest, a tool for running tests from inside your editor.

Chapter 18. Testing

LazyVim can be configured with the Neotest plugin, a generic runner for selecting and running tests in a variety of languages and test frameworks. As with the debug adapter, Neotest is not enabled by default. However, if the plugin is enabled, most language extras ship with a pre-configured setup to make testing work automatically.

Except when it doesn't, of course. Much like debuggers, I find the in-editor features provided by testing extensions (regardless of the editors) to be too finicky to be worth the effort of configuring them. I usually just have a test runner in watch mode running in a separate terminal, and that works well for me. I didn't previously use Neotest, but after writing this chapter I changed my mind!

18.1. Try Neotest

If you haven't already (as part of enabling recommended plugins) pop open the Lazy Extras interface and enable the `test.core` extra. This will set up Neotest and a couple dependencies for you.

Also make sure that the extras for whatever language you are working on are enabled, and double check if they include a test-related plugin. For this example, we'll be using `neotest-python`, which ships with the `lang.python` extra.

Once the extra is enabled, you'll see a new "test" top-level command in your space-mode menu, accessible with `<Space>t`:

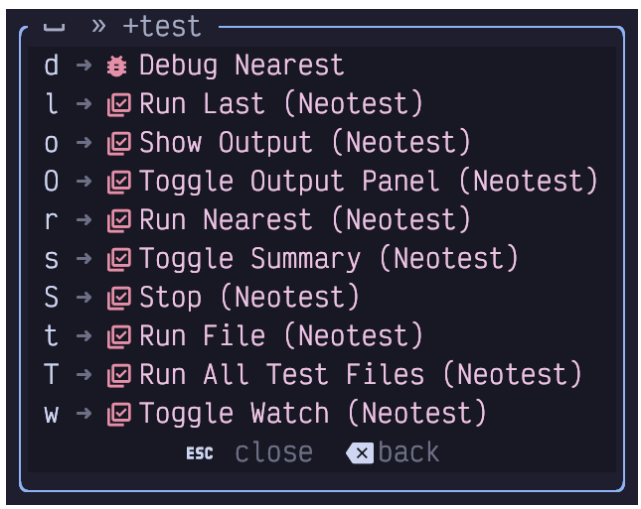


Figure 98. Test Menu

As you would expect, the easy-to-type `<Space>tt` is the most important command in the menu; it runs the current file and parses the test results. You can also use `<Space>tr` (`r` for "Run") when your cursor is inside a test to run that single test.

After running tests successfully, Neotest puts a couple check icons in your interface so that you can see that they were successful:

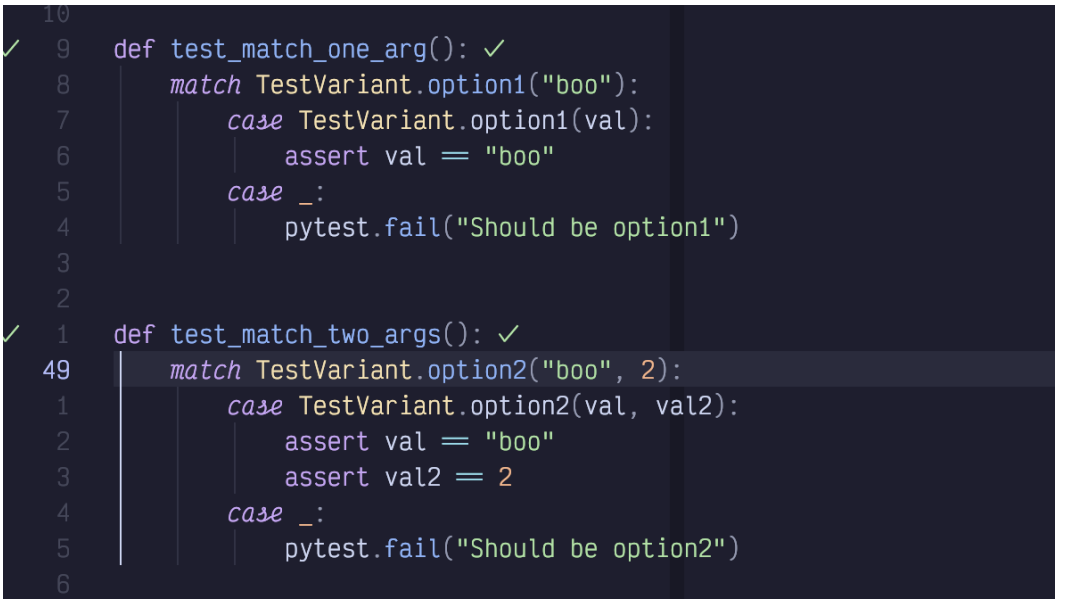


Figure 99. Tests Run Successfully

This screenshot of two tests was taken after I ran all tests in the file with `<Space>tt`. There's one checkmark in the gutter and another to the right of the test in virtual text.

18.2. Error Reporting

Things get a bit more interesting when we introduce a test failure. First, a scrollable window pops up with the test output; this is the same output I would see if I had run the test command (`pytest` in this case) from the terminal:

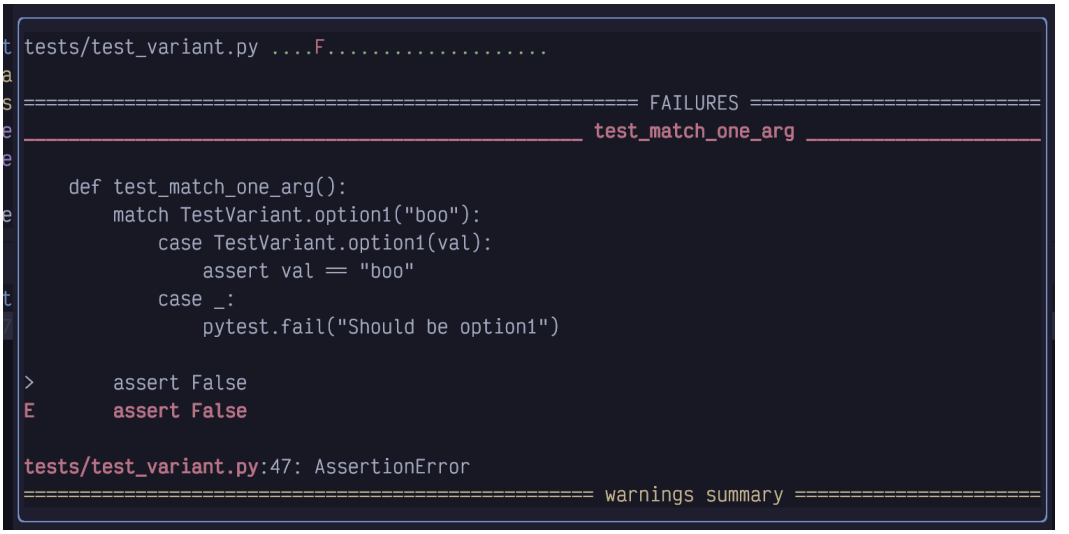


Figure 100. Test Error Output

When this window pops up after running tests, it isn't focused by default, and you can close it simply by moving the cursor (similar to a diagnostic window). If you would prefer to focus the window (for example, so you can scroll it with `<Control-d>` and `<Control-u>`), you can use `<Space>to` where `o` means "output". You can use this keybinding to show the most recent test output at any time. Or you can use `<Space>tO` (capitalized `O`) to open the output in a pane under the editor instead of a floating window.

The Neotest Output pane will behave better if you also have the edgy Extra enabled.

Once the floating window is focused, you need to use the `q` shortcut to exit it.

When a test in Neotest fails, you obviously don't see a checkmark beside the test. Instead, you see a little red X. In addition, there will be an x in a circle in the specific line that has a failure and some (hopefully) informative virtual text to the right of the offending line:

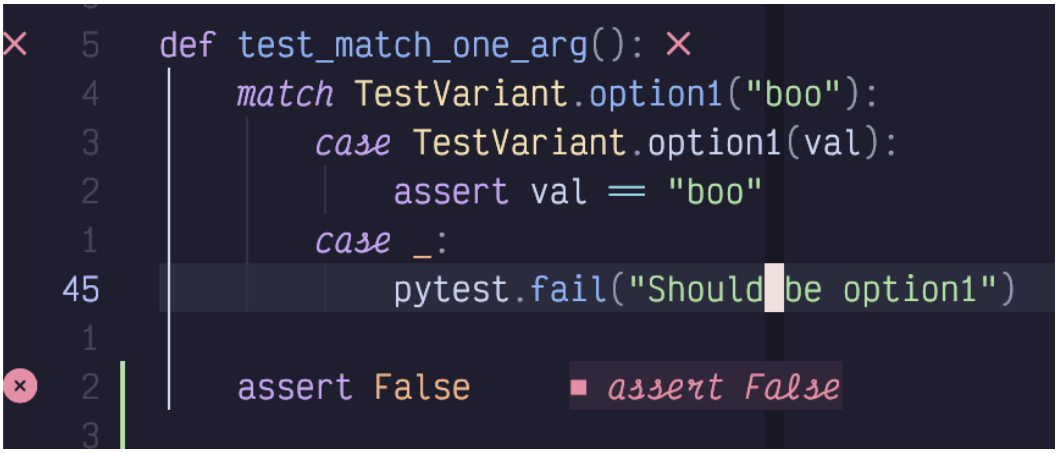


Figure 101. Test Error Virtual text

Most helpfully of all, a Trouble window will open with a list of all failing tests. In this screenshot, I've added `assert False` to two different tests in the file:

Figure 102. Failing Tests In Trouble

This is **super** convenient because I can now navigate between failing tests (possibly in multiple files) using `]q` and `[q`, or by focusing the Trouble window and using basic cursor movements.

18.3. Test Summary

The `<Space>tT` with a capitalized T does a "but bigger" style action, running all tests in your project instead of just the ones in the current file. Of course, you won't see the test markers for any files that aren't open. So you'll probably want to use `<Space>ts` to toggle the summary window:

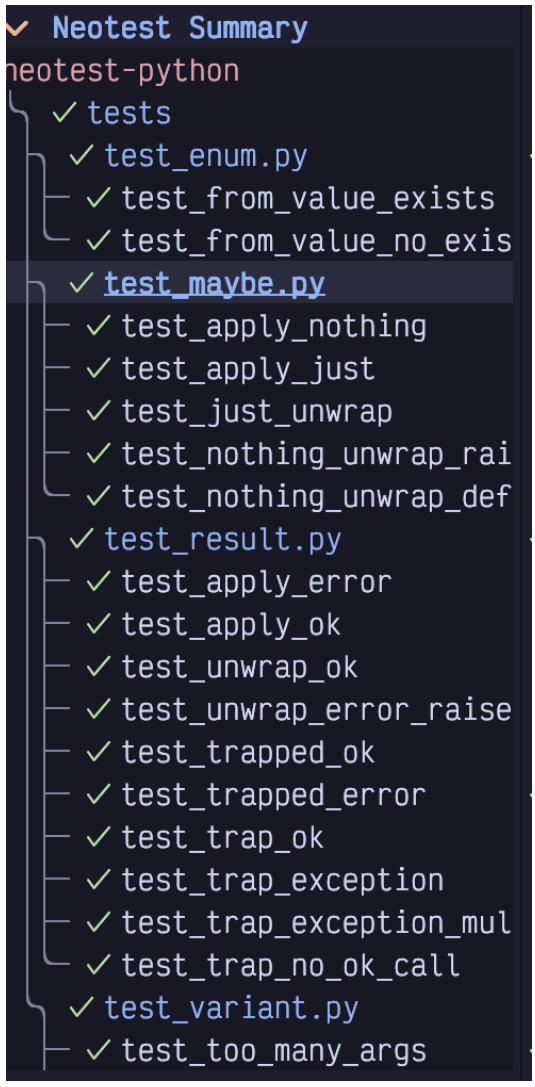


Figure 103. Test Summary

The summary view has some useful keyboard shortcuts (`J` and `K` are most useful) that you can see by typing `?` while it is focused:

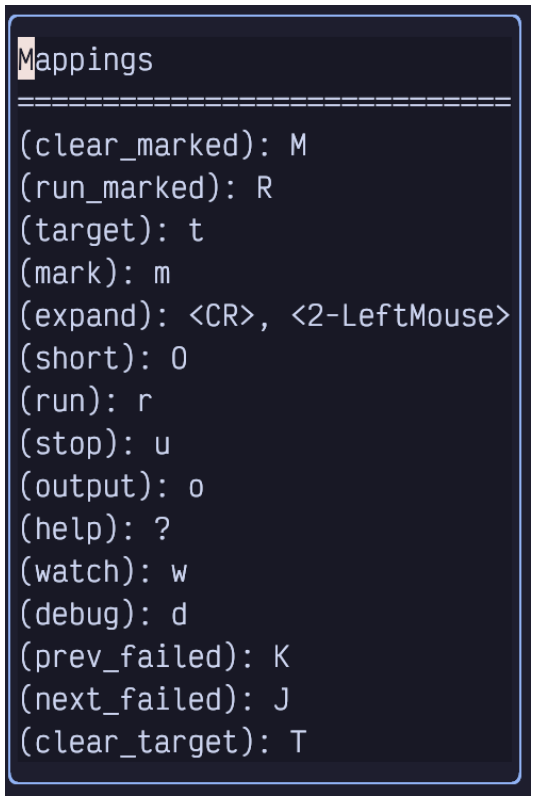


Figure 104. Test Summary Keyboard Mappings

The `m` for "mark" command deserves a callout. It allows you to mark a test as "of interest", so that when you use the `R` or `Run marked` command it will only run those tests. Use `M` (capitalized) to clear all marks or `m` while a line is marked to toggle a single mark off.

18.4. Watch Mode and Debugging

You can toggle "watch" mode for the current file using `<Space>tw`. This will automatically run the test command every time your source code changes. The summary and test file icons will all update in real time.

If you have enabled the debug adapter as described in Chapter 18, you can even have the test automatically add a breakpoint on failure by running it with `<Space>td`. This can be useful for quickly inspecting locals or adding watch statements instead of adding a bunch of print statements before an assertion.

18.5. Installing a Test Runner

If you're lucky, your language has a LazyVim extra that is preconfigured to work with Neotest. For example, the `lang.go`, and `lang.python` extras both include

configuration to set up Neotest with those frameworks.

Not all languages have a clear default test runner, however. For example, if you are coding in Typescript, you might prefer vitest or jest or the deno test runner. All three of these have Neotest support, but none of them are enabled by default with the Typescript extra.

For such languages, you'll need to do a bit of manual configuration. Let's try to set one up for vitest as an example.

The plugin we need is neotest-vitest [https://github.com/marilari88/neotest-vitest]. We need to combine the instructions from the README in that repo with LazyVim's example on the Neotest [https://www.lazyvim.org/extras/test/core] page.

I created a new `vitest.lua` file in my plugins directory and added the following configuration to it:

```
return {
  { "marilari88/neotest-vitest" },
  {
    "nvim-neotest/neotest",
    opts = { adapters = { "neotest-vitest" } },
  },
}
```

Listing 65. Neotest-vitest Configuration

Then I restarted Neovim and opened a file that had a vitest test in it. `<Space>tt` did the right thing, and the plugin is configured.

In the repo I was testing, `vitest` was installed with npm, so no extra installation was needed. In most cases I would expect the tooling that Neotest plugins call into to be installed already when they access your project. If not, you may be able to install it from the Mason menu, accessible from `<Space>cm`.

18.6. Writing Your Own Test Adapter

This is a bit out of the scope of this book, but I decided to include it because a) I needed to do it anyway, b) this chapter is suprisingly short, and c) it's a good example for writing a simple plugin.

Writing your own Neotest adapter requires implementing just five methods to match the `neotest.Adapter` interface. However, the devil is in the details.

For this example, I'll be writing a test adaptter for the Bun [https://bun.sh] test runner. I chose Bun partially because a Neotest adapter for it doesn't exist and I use Bun in my own projects. But it's also a good choice for demonstration because there are already three Typescript/Javascript Neotest adapters we draw on for examples and inspiration:

- neotest-jest [https://github.com/haydenmeade/neotest-jest]
- neotest-vitest [https://github.com/marilari88/neotest-vitest]
- neotest-deno [https://github.com/MarkEmmons/neotest-deno]

We won't be implementing all the possible features (notably, the debugger will be missing), but we'll get the basics of running Bun tests and parsing output.

If you are unfamiliar with Bun, it is a Javascript/Typescript runtime and compiler, more or less an alternative to nodejs. The built-in command `bun test` runs a jest-like test suite. It is this command we will be binding to.

18.6.1. Initializing a Local Plugin

By default, LazyVim downloads plugins from a provider such as GitHub. However, you can pass it any git url or point it at a local directory. We'll be doing the latter.

First, let's initialize a basic plugin structure in a new directory. You'll need to make three nested directories:

```
$ mkdir -p neotest-bun/lua/neotest-bun
```

Listing 66. Neotest Bun Mkdir

The first `neotest-bun` can actually be any name. The `lua` is required for LazyVim to pick up any files inside it, and the last `neotest-bun` is a lua module that we will import in our configuration.

Inside this directory, create a file named `init.lua`. The contents of the file can just be some simple Lua code for now:

```
print("Hello, Lua!")
```

Listing 67. Simple Lua Script

The next step is to hook this local plugin up to our LazyVim configuration. Create a new file in your LazyVim plugin directory (I called mine `neotest-bun.lua`). We'll use the same format we used for `vitest` above, except we'll point to our local plugin with the `dir` key:

```
return {
  { dir = "~/Desktop/Code/neotest-bun/" },
  {
    "nvim-neotest/neotest",
    opts = { adapters = { "neotest-bun" } },
  },
}
```

Listing 68. Local Plugin Configuration

To see if it's working so far, open any file in Neovim and run `<Space>tt` to attempt to kick off the test runner. It will fail because we haven't properly implemented the adapter interface, but it should also pop up a notification that says, "Hello, Lua!".

The notification will disappear quickly, which is very inconvenient if it contains a traceback you want to introspect. You can always use `<Space>sna` to show all the recent messages in a pane. The `:messages` command also works.

18.6.2. Implementing the Neotest Adapter

Let's flesh out the adapter interface. Open the `neotest-bun/init.lua` file and replace the `print` statement with the following content:

```lua
local BunNeotestAdapter = { name = "neotest-bun" }

function BunNeotestAdapter.root(dir) end

function BunNeotestAdapter.filter_dir(name, rel_path, root)
end

function BunNeotestAdapter.is_test_file(file_path) end

function BunNeotestAdapter.discover_positions(file_path) end

function BunNeotestAdapter.build_spec(args) end

function BunNeotestAdapter.results(spec, result, tree) end

return BunNeotestAdapter
```

Listing 69. Neotest Adapter Interface

This is the interface we need to implement. If you're wondering where I got this, it is defined in the Neotest source code [https://github.com/nvim-neotest/neotest/blob/master/lua/neotest/adapters/interface.lua] and linked from the Neotest README. I also have the GitHub repositories for the `neotest-jest` and `neotest-deno` packages open for reference.

You'll need to exit Neovim and restart it to pick up any changes you make to the init.lua file. Remember that you can use the `<Space>qq` command to exit Neovim and then the `s` command from the dashboard to restore your setup with a minimal amount of fuss.

If you try to run tests on a Bun test file now, it will (probably) fail with a "No Tests Found" message. If it doesn't, there may be another test runner installed that thinks this is a legit test file.

Our adapter is currently correctly reporting that it is a Neotest adapter, but then it is failing to register the current folder or file as a test file. We can fix that by implementing the first three methods in the file.

The `root` directory is supposed to find the project root given a current directory. When using Bun, we can use the presence or absence of a `bun.lockb` file as an indicator of the current project root. This file is generated when you run `bun`

`install` and is used for keeping track of dependencies.

So let's implement the `BunNeoTestAdapter.root` method like this:

```
local lib = require("neotest.lib")

local BunNeotestAdapter = { name = "neotest-bun" }

function BunNeotestAdapter.root(dir)
  return lib.files.match_root_pattern("bun.lockb")(dir)
end
```

Listing 70. Root Directory Implementation

The key here is the `neotest.lib` function `match_root_pattern`. We import that library and assign it to a local, then our `root` function just needs to create a callback and call it.

While we're at it, we can also implement the `filter_dir` function. This method is designed to filter out directories that shouldn't be scanned. In a Bun project, this includes the `node_modules` folder. We definitely don't want to waste time scanning for tests in that folder!

```
function BunNeotestAdapter.filter_dir(name, rel_path, root)
  return name ~= "node_modules"
end
```

Listing 71. Filter Directory Implementation

Now if you restart Neovim and try to run tests with `<Space>tT` (that is a capital `T` the second time) it will not show the "No Tests found" message. It won't *do* anything, but at least it won't error. However, `<Space>tt` will error, because it doesn't know that we are currently in a test file. We can address that from the `is_test_file` method.

In Bun, like most Javascript runtimes, tests are typically in a `something.test.js` or `somethingElse.test.ts` file. So we can use the following Lua function to check if we are in a test file:

```
function BunNeotestAdapter.is_test_file(file_path)
  return string.match(file_path, ".*.test.[tj]s$") ~= nil
end
```

Listing 72. Is Test File Implementation

If I open my `cohere.test.ts` file and run `<Space>tt`, I still get `No Tests found`. It is identifying the file as a Bun test file, but it doesn't know how to look inside the file to find any tests.

18.6.3. Discovering Test Positions

Solving this requires implementing the `discover_positions` function, and that is... complicated. Typically, you would write Treesitter queries that identify namespaces and tests in the file. I suppose you could also write your own parser or use `string.match`, but Treesitter's parser is probably better than anything we can write.

I don't know anything about writing Treesitter queries, and I don't particularly want to. So I'm just going to rely on the fact that Bun uses the same describe/test syntax that Jest uses, and I'll copy the queries wholesale from the neotest-jest [https://github.com/nvim-neotest/neotest-jest/blob/main/lua/neotest-jest/init.lua#L162] plugin!

It's a pretty long bit of code that will likely be hard to read in book format, but I'll include it here for completeness:

```
function BunNeotestAdapter.discover_positions(file_path)
  local query = [[
    ; -- Namespaces --
    ; Matches: `describe('context', () => {})`
    ((call_expression
      function: (identifier) @func_name (
        #eq? @func_name "describe"
      )
      arguments: (arguments (
        string (string_fragment) @namespace.name
      ) (arrow_function))
    )) @namespace.definition
    ; Matches: `describe('context', function() {})`
    ((call_expression
```

```
    function: (identifier) @func_name (
      #eq? @func_name "describe"
    )
    arguments: (arguments (
      string (string_fragment) @namespace.name
    ) (function_expression))
  )) @namespace.definition
  ; Matches: `describe.only('context', () => {})`
  ((call_expression
    function: (member_expression
      object: (identifier) @func_name (
        #any-of? @func_name "describe"
      )
    )
    arguments: (
      arguments (string (string_fragment) @namespace.name
    ) (arrow_function))
  )) @namespace.definition
  ; Matches: `describe.only('context', function() {})`
  ((call_expression
    function: (member_expression
      object: (identifier) @func_name (
        #any-of? @func_name "describe"
      )
    )
    arguments: (arguments (
      string (string_fragment) @namespace.name
    ) (function_expression))
  )) @namespace.definition
  ; Matches: `describe.each(['data'])('context', () => {})`
  ((call_expression
    function: (call_expression
      function: (member_expression
        object: (identifier) @func_name (
          #any-of? @func_name "describe"
        )
      )
    )
    arguments: (arguments (
      string (string_fragment) @namespace.name
    ) (arrow_function))
  )) @namespace.definition
```

```
    ; Matches: `describe.each(['data'])('context', function()
{})`
    ((call_expression
      function: (call_expression
        function: (member_expression
          object: (identifier) @func_name (
            #any-of? @func_name "describe"
          )
        )
      )
      arguments: (arguments (
        string (string_fragment) @namespace.name
      ) (function_expression))
    )) @namespace.definition

    ; -- Tests --
    ; Matches: `test('test') / it('test')`
    ((call_expression
      function: (identifier) @func_name (
        #any-of? @func_name "it" "test"
      )
      arguments: (arguments (
        string (string_fragment) @test.name
      ) [(arrow_function) (function_expression)])
    )) @test.definition
    ; Matches: `test.only('test') / it.only('test')`
    ((call_expression
      function: (member_expression
        object: (identifier) @func_name (
          #any-of? @func_name "test" "it"
        )
      )
      arguments: (arguments (
        string (string_fragment) @test.name
      ) [(arrow_function) (function_expression)])
    )) @test.definition
    ; Matches: `test.each(['data'])('test')
    ((call_expression
      function: (call_expression
        function: (member_expression
          object: (identifier) @func_name (
            #any-of? @func_name "it" "test"
```

```
          )
          property: (property_identifier) @each_property (
            #eq? @each_property "each"
          )
        )
      )
      arguments: (arguments (
        string (string_fragment) @test.name
      ) [(arrow_function) (function_expression)])
    )) @test.definition
  ]]

  local positions = lib.treesitter.parse_positions(
    file_path, query, {
    nested_tests = false,
  })

  return positions
end
```

Listing 73. Borrowed Discover Positions Queries

Now you can restart Neovim and open a bun test file to get a new error! New errors are progress, right?

You'll now notice that Neotest is identifying the positions of `describe` and `test` calls in the gutter. Instead of the red cross or green check we would expect from a successful test run, it will be an eye with a cross through it. I suspect this means the test was skipped or not runnable. The good news is it is finding the tests. The bad news is it is not *running* the tests.

18.6.4. Building the Spec

We can run the tests by implementing the `build_spec` function. This function accepts various parameters to determine how the user kicked off the tests. If they used `<Space>tr` it's in "single test" mode, but if they used `<Space>tt` it is in "file" mode, and `<Space>tT` would run it in "all tests" mode.

The return value of `build_spec` is essentially a command to be run and some context to read the results back.

The code is actually not that long, so here it is in its entirety, followed by discussion:

```
function BunNeotestAdapter.build_spec(args)
  local results_path = async.fn.tempname()
  local position = args.tree:data()
  local cwd = assert(
    BunNeotestAdapter.root(
      position.path
    ),
    "could not locate root directory of " .. position.path)
  local command = nil

  if position.type == "test" or position.type == "namespace"
then
    command = "bun test " ..
      position.path ..
      " --test-name-pattern " ..
      position.name
  elseif position.type == "file" then
    command = "bun test " .. position.name
  elseif position.type == "dir" then
    command = "bun test"
  end

  return {
    command = command .. " 2>" .. results_path,
    context = {
      results_path = results_path,
    },
    cwd = cwd,
  }
end
```

Listing 74. Build Spec Implementation

We start by creating a temporary `results_path` using the `neotest.async` library. (You'll need to import this with `local async = require("neotest.async")` at the top of the file). We load the `position`, which is a structure constructed from the return value of the `discover_positions` method we just wrote.

The `if..elseif` block is essentially checking how the user kicked off the test, and running the appropriate Bun command. If they provided a test name, then we pass the `--test-name-pattern` argument to the `bun test` command. If they kicked it off as a file, we run `bun test filename`. And if they wanted to run all tests, we

simply run all tests with `bun test`.

The Bun test runner is so fast, that I would expect to mostly just use the last form with `<Space>tT` and the summary view open in the left sidebar.

The returned object includes some necessary context that will be used when we parse results. The `bun test` command is rather unusual in that it outputs the results to standard error, so we pass a `2>` redirect to store the results in the temporary file we defined.

Now we just need to extract the results from that file.

18.6.5. Parsing Results

This ended up being simpler than I expected, because `bun test` aggregates names in a way that maps to Neotest's expected form quite easily. But it took me half a day of fussing around to get code that actually worked!

The `results` method mostly just has to read through the `results_path` file that was created by our `build_spec` function and convert it to a simple Lua table. The keys of the result table are the names of the tests in question, and the values are just a second table with `{status = "passed"}` or `{status = "failed"}`. At least, that's all we're going to put in it. Neotest does accept some other details here that it can render in the UI, but I'll leave that as "an exercise for the reader."

When reading other instructional books, "an exercise for the reader" is just authors being lazy, or (occasionally) publishers trying to cut word count. Now you know.

The tricky part is the "keys are the names of the tests". I couldn't find any documentation on this, and it took some trial and error to discover that nested "namespaces" (`describe` calls, in this case) in Neotest are separated by `::`. The name also needs the absolute path to the test file. If we return that in the right format, Neotest will happily convert our results to the appropriate icons!

Here's the code:

```
function BunNeotestAdapter.results(spec, result, tree)
  local results = {}
```

```
  local file = assert(io.open(spec.context.results_path))
  local line = file:read("l")
  local test_suite = ""

  while line do
    local pass_match = string.match(line, "^%(pass%) (.*)
%[.*%]$")
    if pass_match ~= nil then
      local test_name = pass_match.gsub(pass_match, " > ",
"::")
      results[test_suite .. test_name] = { status = "passed" }
    end

    local fail_match = string.match(line, "^%(fail%) (.*)
%[.*%]$")
    if fail_match ~= nil then
      local test_name = fail_match.gsub(fail_match, " > ",
"::")
      results[test_suite .. test_name] = { status = "failed" }
    end

    local test_file = string.match(line, "^(.+.test.[tj]s):$")
    if test_file ~= nil then
      test_suite = spec.cwd .. "/" .. test_file .. "::"
    end

    line = file:read("l")
  end

  if file then
    file:close()
  end

  return results
end
```

Listing 75. Neotest Results Implementation

For context, this function is designed to translate output like this:

```
src/clients/stability/stability.test.ts:
(pass) Stability > generates a reference image [0.40ms]

src/clients/passage/passage.test.ts:
(pass) Passage > GetUserTier > Undefined [1.24ms]
(pass) Passage > GetUserTier > with tier free
(fail) Passage > GetUserTier > with tier hobby
```

Listing 76. Bun Test Output

into something like this:

```
{
  "/.../stability.test.ts::Stability::generates reference
image" = {
    status = "passed"
  },
  "/.../passage.test.ts::Passage::GetUserTier::Undefined" = {
    status = "passed"
  },
  "/.../passage.test.ts::Passage::GetUserTier::with tier free"
= {
    status = "passed"
  },
  "/.../passage.test.ts::Passage::GetUserTier::with tier
hobby" = {
    status = "failed"
  },
}
```

Listing 77. Translated Bun Test Output

The method first grabs the filename from the context (the context was returned from the `build_spec` method). It opens the file and reads the lines from that file one by one. It then uses a matcher to determine if the line starts with `(passed)` or `(failed)` which is how Bun reports a test result. The rest of the test line will be the properly namespaced test name (minus a timing in square brackets), except the namespaces are separated by `>` instead of `::`. So we use `gsub` to replace the `>` with `::`. This is combined with the absolute path of the name of the file to get the right test name.

If a given line is not a test name, then it might be the name of the test file, which Bun kindly specifies as a relative path followed by a colon. So we do a match on that format and store the test name as an absolute path (that's what the `spec.cwd` is for) to be prepended to subsequent test results.

And that's the basics of implementing our own test runner! It's missing some features, notably debugging tests and capturing output, but it's a good start.

18.7. Summary

This chapter covered the Neotest plugin, including various ways to invoke it and how to set it up the easy way, hard way, and extra hard way.

We also learned a little bit about how to write a Neovim plugin. At its core, it's just a collection of Lua files that LazyVim can import. In this case, the Lua file is a Neotest adapter, and we configured it to load our plugin into Neotest.

Chapter 19. Comprehensive Guide to LazyVim Configuration

We covered basic plugin configuration in Chapter 5, and I've given details on how to deal with more complicated situations when I needed them to configure a specific plugin, but it's scattered throughout the book.

This chapter will start with a bit of a review of all that and then attempt to give you the tools you need to find and configure other Neovim plugins that are not available as Lazy Extras.

19.1. Plugins Directory

As we covered in Chapter 5, plugins are managed by the Lazy.nvim plugin manager. It is configured to automatically load any `.lua` file in your config folder's `lua/plugins` directory. Typically, this will be `~/.config/nvim/lua/plugins` where `~` is your home directory. However, if you use the `NVIM_APPNAME` environment variable, then it will be `~/.config/$NVIM_APPNAME/lua/plugins`.

Lua files in this directory should always return a Lua table. Therefore, they will be structured like this:

```
return {

}
```

Listing 78. Empty Plugin

This Lua table can contain either a plugin specification or a table containing multiple plugin specifications in their own Lua table. For example, the structure will either be:

```
return {
  "username/plugin",
  opts = {...},
  keys = {...}
  ...
}
```

Listing 79. Basic Single Plugin

for a single plugin specification, or:

```
return {
  {
    "username/plugin",
    opts = {...},
    keys = {...}
  },
  {
    "username2/plugin2",
    ...
  }

}
```

Listing 80. Multiple Plugins in One File

Personally, I usually place each plugin in its own file so they are easy to search for when I use the `<Space>fc` shortcut. However, I do use the multiple plugin format when the existence of one plugin requires me to modify the configuration of another.

19.2. Plugin Specifications Cascade

Any one plugin can be specified multiple times in your configuration and LazyVim will merge everything together. There are a few cases where this is useful:

- Some people like to configure keybindings in a separate area from the plugin options and configuration.
- Sometimes you want your primary plugin configuration to be in one file, and then have a second, related plugin configure some overrides for that plugin.
- (Most Common) LazyVim pre-configures many plugins with sensible defaults, but you will occasionally want to override those defaults with your preferred keybindings or options.

19.3. Plugin Specification

The simplest plugin specification is just a table containing a single string with the GitHub username and repo separated by a `/`. Occasionally, this is all you need,

especially for VimScript plugins. If the plugin in question isn't hosted on GitHub, you can omit this first argument and pass either `dir=/path/to/a/folder` or `url=https://domain.com/path/to/plugin`.

If you need to pin your plugin to a specific version or git branch, you can pass the `branch`, `tag`, `commit`, or `version`. For GitHub-hosted plugins, you can find the values for these version specifiers on GitHub. You will likely use this only rarely, as it is normal to install the `main` branch of the plugin. But if you know that a recent change in a plugin is causing issues for you, or if you want to try out some bleeding edge feature that hasn't been merged yet, you'll want to set one of these.

There are almost two dozen options you can pass to a Lazy.nvim plugin specification, all documented at https://lazy.folke.io/spec. We discussed some of them in Chapter 5, namely `enabled`, `opts`, and `keys`. Now we'll touch on several of the others.

19.4. Plugin Lifecycle Methods

There are several options that are invoked at various times during the lifecycle of a plugin. You only need to specify these rarely, but they can be very useful to control when code executes, especially if you are trying to port "raw config" installation instructions to Lazy.nvim config.

19.4.1. Build

The `build` option is called once when the plugin is installed or updated, and is not called during the normal startup or execution of Neovim. We saw an example of it in the `smart-splits` configuration in chapter 9. In that example, we passed a string path to a shell script that ships with the plugin. Every time the plugin is installed or upgraded, that build command is run, ensuring that the relevant Kitty scripts are installed. In addition to a string pathname, `build` can also be a:

- Lua function that accepts the plugin spec as its only argument.
- path to an arbitrary Lua file.
- the string "rockspec", which will build a `luarocks` package. Luarocks is a package manager and index for Lua modules not unlike pypi, npm, or crates in the Python, Javascript, and Rust ecosystems respectively.
- string starting with `:`, which will execute an arbitrary Vim command.
- list of one or more of the above.

As a plugin consumer, you will likely only specify the `build` function if the plugin's

documentation instructs you to do so.

19.4.2. Init

The `init` option is executed during program startup, so to keep the startup time short, it's best to avoid it unless absolutely necessary. It accepts a Lua function with a single argument holding any specs for that plugin. (Specifically, it is an instance of LazyPlugin [https://github.com/folke/lazy.nvim/blob/main/lua/lazy/types.lua]).

I've never actually had to specify `init` in any of my plugin configurations. Typically, if I need the plugin to execute at startup, I use `lazy=false` so it configures the whole thing on startup. I can see `init` being helpful if there are legitimate two-step setups, but in my experience, all of those are in plugins that LazyVim is managing on my behalf, so I've never needed it.

19.4.3. Config

The `config` option is the one you are most likely to specify, but try to only reach for it if you've run out of other options. It is called whenever the plugin is loaded, which may be on startup, or only when it is first used, depending on how it is configured.

The first thing you need to understand about `config` is its default behaviour if you don't specify it.

There is a de facto standard in Lua-based plugins of providing a `setup` function that accepts a single argument which is a table of options for that plugin. The vast majority of plugins you encounter will have instructions in their README to write code something like this:

```
require('pluginName').setup({key = value, key2 = value2...})
```

Listing 81. Non-LazyVim Setup Call

They'll tell you to put this in your `init.lua`. These are good instructions if you're not using LazyVim, but it's not for us.

The default `config` function that Lazy.nvim provides invokes this code automatically if your spec contains any `opts`. So the above code should always be ported to something like this:

```
return {
  'username/plugin',
  opts = {key = value, key2 = value2}
}
```

Listing 82. LazyVim Options

Under the hood, if you supply `opts` in the plugin spec, the `config` function will call the `.setup` code for you.

This is a wonderful thing because LazyVim does automatic merging of `opts` tables if you provided multiple specs for the same plugin in different places. If you override `config`, you won't so easily be able to take advantage of this merging behaviour.

However, if the plugin you are configuring is non-standard or requires you to run additional code when it is starting up, you'll need to specify `config` yourself.

The `config` function is always a Lua function that accepts two arguments. The first is the same `LazyPlugin` spec that `init` receives. The second is the table of `opts` that Lazy.nvim has created for you, possibly by merging opts from multiple definitions.

The main place where this becomes a problem is when you want to customize the default LazyVim configuration for a plugin and that default configuration overrides config.

Unlike with `opts` and `keys`, only one `config` function is called (the last one loaded). So if you specify `config` for a plugin, the LazyVim one will not execute.

Typically, these overridden configs can be modified with judicious use of `opts` or `keys`, but if you need to do imperative tasks that differ from whatever LazyVim provides, there's a good chance you'll be copying the entire configuration for that plugin from the https://lazyvim.org website into your personal config.

While that's not a great outcome, it's still better than if you didn't use LazyVim at all, because then you'd have to write the entire configuration from scratch instead of copying and modifying a trusted source.

19.5. Modifying Options In-place

The "merging" that LazyVim does on `opts` if you supply a table may not always do

the right thing, especially with nested tables or if you want to remove, instead of add, a key. Sometimes it is better to specify opts as a Lua function that takes the existing `opts` as input and modifies it in place. A great example is adding an entry to the dashboard menu:

```
return {
  "snacks.nvim",
  opts = function(_, opts)
    table.insert(
      opts.dashboard.preset.keys,
      7,
      { icon = "S", key = "S", desc = "Select Session", action
= require("persistence").select }
    )
  end,
}
```

Listing 83. Modifying Options With Function

Here, `opts` is specified as a function that takes two arguments, and we modify the second one, which is the Lua table LazyVim is building to pass into `config`.

19.6. Complex Plugin Example: Telescope-live-grep-args

LazyVim's plugin abstractions are usually very elegant, but occasionally can get in the way. I wanted to include a complicated example in the hopes it will help you navigate the hairier situations.

Note: Unfortunately, this example has become outdated and I haven't had time to find another one. It relies on using the Telescope file picker instead of Snacks.nvim, which I don't recommend. If you want to use it or test out this example, you can switch on the `Telescope.nvim` extra in `:LazyExtras`.

The Telescope `live_grep` integration that ships with LazyVim doesn't allow us to pass arguments to ripgrep by default. However, there is a `telescope-live-grep-args` extension that does permit customizing the ripgrep arguments.

Start by visiting the telescope-live-grep-args.nvim [https://github.com/nvim-telescope/telescope-live-grep-args.nvim] repository. You'll find installation instructions for Lazy.nvim (that's the plugin manager, NOT the LazyVim distro itself) that, at time of

writing look like this:

```
-- This is not helpful with LazyVim
use {
  "nvim-telescope/telescope.nvim",
  dependencies = {
    {
        "nvim-telescope/telescope-live-grep-args.nvim" ,
    },
  },
  config = function()
    require("telescope").load_extension("live_grep_args")
  end
}
```

Listing 84. Incorrect Configuration for Telescope Plugin

The reason I added the "not helpful" comment there is the call to `config`. LazyVim already configures Telescope with a fairly complex function that you can find under the `editor` section of the plugins menu on the LazyVim website (click the `Full spec`) tab.

For the most part, LazyVim does a good job of merging its own defaults with any customizations you do with the various plugins it sets up. It's easy to change or remove keybindings or override the options that get passed into a `setup` function, for example.

But it's not easy to override `config`.

You *could* do something like this:

```
config = function(_, opts)
  require("telescope").setup(opts)
  require("telescope").load_extension("live_grep_args")

end
```

Listing 85. Another Incorrect Configuration for Telescope Plugin

This works because the default behaviour of `config` in Lazy.nvim is to call `require(<the_plugin>).setup(opts)`. So we're basically copying the contents

of that function into our custom function. But if LazyVim happened to have a very complicated `config` for Telescope, you would have to copy the whole thing in, and it would eventually get out of date with any changes that LazyVim makes in the future. That wouldn't be fun for you to maintain. More importantly, it runs a very real risk of clobbering any Telescope-related changes that LazyVim makes with other plugins or extras.

In general, I try very hard to avoid implementing `config` when overriding LazyVim's defaults. It's fine to have a custom implementation of `config` if I am adding a new plugin that LazyVim isn't aware of, since I'm already responsible for maintaining it. But when I'm *customizing* plugins, I try not to override `config`.

The secret (in this case) is to use the "dependencies" feature of Lazy.nvim. The "Full Spec" on the LazyVim website has an example of how to set up the telescope-fzf-native.nvim extension, which looks something like this

```
dependencies = {
  {
    "nvim-telescope/telescope-fzf-native.nvim",
    -- snipped some build instructions
    config = function()
      LazyVim.on_load("telescope.nvim", function()
        -- snipped loading the extension
      end)
    end,
  },
},
```

Listing 86. How LazyVim Configures Telescope-fzf-native

This is showing us how to have a mini-config for a dependent plugin, which is *exactly* what we want. Further, if we customize our spec, `dependencies` is one of the tables that Lazy.nvim will *merge* with the parent spec (the one created by LazyVim). So we **don't** need to copy the above code into our Telescope extension file so it doesn't get clobbered. Instead we only need to create a single new table.

Here's the configuration (I put it in `extend-telescope.lua`):

```
return {
  { "nvim-telescope/telescope-live-grep-args.nvim" },
  {
    "nvim-telescope/telescope.nvim",
    dependencies = {
      {
        "nvim-telescope/telescope-live-grep-args.nvim",
        config = function()
          LazyVim.on_load("telescope.nvim", function()
            require(
              "telescope"
            ).load_extension("live_grep_args")
          end)
        end,
      },
    },
  }
}
```

Listing 87. Correct Configuuration for Telescope Plugin

It's kind of verbose, but this should be all you need to enable the plugin.

Next, you need to set up a keybinding to invoke the plugin. You can choose to override the existing `<Space>/` keybinding, or perhaps you would use `<Space>?` if you want to have separate "default live_grep" and "live_grep_args" mode.

Your first attempt, if you are following the live-grep-args README might look like this:

```
-- This won't work
keys = {
  {
    "<leader>/",
    require(
      "telescope"
    ).extensions.live_grep_args.live_grep_args,
    desc = "Grep with Args",
  },
},
```

Listing 88. Incorrect configuration for Live Grep Args

Unfortunately, this is too easy for LazyVim. Because the `live_grep_args` plugin has been set up to run in a `LazyVim.on_load`, it is not defined at the time this keys array is created.

The solution is to wrap the call in another function, so the import only happens after you press the keybinding. That works because the `onLoad` handler will have been called by that point:

```
keys = {
  -- Other telescope-related keybindings
  {
    "<leader>/",
    function()
      require(
        "telescope"
      ).extensions.live_grep_args.live_grep_args()
    end,
    desc = "Grep with Args (root dir)",
  },
},
```

Listing 89. Correct Configuration for Live Grep Args

Before we move on to actually **using** the live-grep-args plugin, there is one more place you need to apply this weird nested function calls trick. Telescope-live-grep-args suggests hooking up `ctrl-k` to the `quote_prompt()` action, like so:

```
-- Don't do this
local lga_actions = require("telescope-live-grep-
args.actions")
telescope.setup {
  extensions = {
    live_grep_args = {
      mappings = { -- extend mappings
        i = {
          ["<C-k>"] = lga_actions.quote_prompt(),
        },
      },
    }
  }
}
```

Listing 90. Incorrect Configuration for Live Grep Args Keymaps

The table passed into `setup` there just comes from our `opts` array, but we again need to avoid importing `telescope-live-grep-args` at the top-level like that. Instead, we need a new function. But there are a couple gotchas:

- `quote_prompt()` is a function that returns a different function. So we need to invoke that function with a odd-looking `()()` syntax.
- Telescope mappings accept an integer argument (the internal id of the picker), so we need to forward that to the called function.

The resulting `opts` array looks like this:

```
opts = {
    extensions = {
      live_grep_args = {
        mappings = {
          i = {
            ["<C-k>"] = function(picker)
              require(
                "telescope-live-grep-args.actions"
              ).quote_prompt()(picker)
            end,
          },
        },
      },
    },
  }
```

Listing 91. Correct Configuration For Live Grep Args Keymaps

For completeness (and because all the above snippets on their own may not make indentational sense), here is my entire Telescope configuration:

```
return {
  { "nvim-telescope/telescope-live-grep-args.nvim" },
  {
    "nvim-telescope/telescope.nvim",
    dependencies = {
      {
        "nvim-telescope/telescope-live-grep-args.nvim",
        config = function()
          LazyVim.on_load("telescope.nvim", function()
            require(
              "telescope"
            ).load_extension("live_grep_args")
          end)
        end,
      },
    },
    keys = {
      {
        "<leader>/",
        function()
```

```
          require(
            "telescope"
          ).extensions.live_grep_args.live_grep_args()
        end,
        desc = "Grep with Args (root dir)",
      },
    },
    opts = {
      extensions = {
        live_grep_args = {
          mappings = {
            i = {
              ["<C-k>"] = function(picker)
                require(
                  "telescope-live-grep-args.actions"
                ).quote_prompt()(picker)
              end,
            },
          },
        },
      },
    },
  }
}
```

Listing 92. Complete Configuration for Live Grep Args

It's a mess, I know. Part of the mess is because Telescope is pretty generic, and that mess would still exist if you were managing your own config. But part of the mess is because we need to cooperate with LazyVim when we enter our customizations, and the config is necessarily more complicated than it would be. As usual, I'm ok with this because I have to do it rarely enough and I appreciate not having to manage *most* of my configuration by myself.

19.7. Configuring Non-plugin Options

Vim is a highly configurable editor and Neovim is even more so. There are over three hundred options in the `:help option-list` output. The default Vim configuration for most of these is fine, though there are a few that have silly defaults for historical reasons. Neovim has fixed a few of these and LazyVim sets almost a third of them so the out-of-the-box experience approximates what most modern developers would want.

However, you'll probably still want to change a few options to suit your taste. This is typically done in the `lua/config/options.lua` file. LazyVim loads this file by default.

For any given option, you *probably* want to set the `vim.opt.<optionname>` field. `vim.opt` is a special Lua table that allows you to interact with Vim settings as... well, a special Lua table. If you are searching for a Vim setting and see something saying you should call `:set option=value`, that will work, if you want to do it temporarily. But if you want to store the setting for the next time you start Neovim, you need to translate it to `vim.opt.option = value`.

Sometimes, and after 25 years using Vim I'm *still* not sure exactly when, you'll need to set `vim.g.<optionname> = value` instead. The `g` in this case means global. If you see an instruction to set a variable such as `let g:varname = value`, you may need to use `vim.g.varname = value`. Some options are inherently global while others only apply to the current buffer *unless* you specify `g`. Some plugins, especially older non-Lua plugins, are configured with global variables, and this is the syntax you would use for them as well.

In general, use `vim.opt` unless you see `vim.g` or `g:` in the documentation or code you are copying from.

19.8. Setting the Colour Scheme (Theme)

Vim has two options for setting the colour scheme and they can interact in unexpected ways.

First, you can set your window background to be either `dark` or `light`. You can toggle this setting at runtime with the `<Space>ub` keybinding under the "Ui" menu. **Usually** when you toggle the background, the colour scheme will change the foreground colours to suit the chosen background, but this isn't always reliable. Sometimes it even changes the colour scheme to a related one that is more suitable when the background is light. For example, if you have `catpuccin-mocha` enabled, and change your background to light, `catpuccin-latte` will become the selected colour scheme.

If you want to change the background permanently, set `vim.opt.background = "light"` or `vim.opt.background = "dark"` in your `options.lua` file.

To change to a different colour scheme altogether, use `<Space>uC` where the C is capitalized. This will pop up a picker with all currently installed colour schemes. Neovim ships with a few default colour schemes, and LazyVim adds several

variations of `Catpuccin` and `Tokyo Night` (the default). The advantage of these colour schemes over the Neovim-provided ones is that they have a ton of extra highlight groups for the various plugins LazyVim installs.

If you want to change the colour scheme permanently, it should be set as an `opt` on the `LazyVim/LazyVim` plugin. I prefer the `catpuccin` colour scheme, so I have a `plugins/core.lua` file that looks like this:

```
return {
  {
    "LazyVim/LazyVim",
    opts = {
      colorscheme = "catppuccin",
    },
  },
}
```

Listing 93. Colour Scheme Configuration

However if you want to set the colour scheme to something that isn't shipped with LazyVim by default, you *also* need to install the plugin that ships that colour scheme. For example, the popular standby gruvbox can be installed like this:

```
return {
  { "ellisonleao/gruvbox.nvim" },
  {
    "LazyVim/LazyVim",
    opts = {
      colorscheme = "gruvbox",
    },
  },
}
```

Listing 94. Third Party Colour Scheme

When looking for new colour schemes, try to find well-maintained repositories that feature "treesitter support" and provide highlight groups for all the plugins you use regularly (including those that ship with LazyVim). You can find colour schemes on the Awesome Neovim list [https://github.com/rockerBOO/awesome-neovim/].

19.9. Lazy Loading

LazyVim automatically lazy-load plugins on demand instead of during program startup. This can shave milliseconds off your startup time (that oh-so-precious resource).

LazyVim knows to load the plugin whenever its code is `require`d, or when any of the keybindings specified in the `keys` array are pressed. This is usually exactly when you want it to load. However, if you encounter plugins that are not working as expected, you may need to tweak the lazy loading configuration.

First, try adding `lazy = false` to the plugin spec. This will ensure the plugin loads on startup. If the plugin works after you add that, you have two options:

- Just leave `lazy = false` and get on with your day.
- Micro-optimize to try to force the plugin to load lazily at the right time.

For the most part, I recommend the first option. Micro-optimization makes sense in the context of optimizing plugins for public consumption. Plugin maintainers might use it in their packages, and certainly distro maintainers such as Folke himself spend a lot of time on it. But it's not likely worth it for you.

Run the plugin with lazy enabled and disabled and check the startup time in the dashboard. If it makes a difference that you want to solve for, read on.

If a plugin should only be specified for certain filetypes, then add a `ft` key to the spec. This key accepts a string or list of strings representing the filetype. (To get the filetype for the current buffer, type `:set ft<Enter>`).

If instead the plugin should only be loaded if certain commands are called, specify a list of strings with the command names under the `cmd` key.

You can also specify a Neovim `event` that should trigger the plugin to load. There are way too many of these to list in this book, so I'll refer you to `:help events`. The most common ones to trigger plugin loading would be `BufEnter`, `BufRead`, and `BufWrite`.

19.10. Filetype-specific Configuration

If you need to configure something to run for a specific filetype, you'll need the `nvim_create_autocmd` function. Technically you can place this call anywhere,

including `init.lua` or the `config` or `init` for a specific plugin, but the LazyVim convention is to put them in the `lua/config/autocmds.lua` file.

I actually only have one autocmd because LazyVim does such a good job of configuring filetype-specific behaviours for me (usually using the appropriate `lang.*` LazyVim Extra). That command looks like so:

```
vim.api.nvim_create_autocmd({ "BufRead", "BufNewFile" }, {
pattern = "\*.svx",
command = "setlocal filetype=markdown",
})
```

Listing 95. Filetype-specific Autocommands

The first argument is a keyless Lua table of string event names; in this case any time I create or read a file, the autocommand is run. There are other events you can use, but for the most part you'll only need these two unless you are writing your own plugin (see `:help autocmd-events` for a whole list).

The second argument contains the options for the autocommand. In this case I am including a `pattern`, which is a Vim regex for the type of file I want to match. I specifically said "Vim regex" to excuse the goofy `\*`, as discussed in Chapter 12.

In this case I'm using the `command` key to execute a command whenever I read a `*.svx` file. You can instead specify a `callback` key where the value is a Lua function that will be called whenever the event occurs.

19.11. Per-project Configuration

Sometimes, you'll want to have a custom LazyVim configuration for a specific project. For example, most of my Typescript projects are in Svelte, which means I can't (yet) use the exceptional `biome` linter/formatter for them. That means I'm using `prettier` for formatting in these repos. My `extend-conform.lua` plugin specification looks like this:

```lua
return {
  {
    "stevearc/conform.nvim",
    opts = {
      formatters_by_ft = {
        ["typescript"] = { "prettier" },
        ["markdown"] = { "prettier" },
        ["yaml"] = { "prettier" },
        ["svelte"] = { "prettier" },
      },
    },
  },
}
```

Listing 96. Prettier in Default Configuration

However, for my Typescript API servers, I **can** use Biome, and I want to override my configuration for those projects only.

LazyVim makes this dead simple: just create a `.lazy.lua` file in your project. It can return any valid plugin spec, and it will be called after any other plugins are loaded, overwriting them. So my Hono api server has a `.lazy.lua` file that looks like this:

```lua
return {
  {
    "stevearc/conform.nvim",
    opts = {
      formatters_by_ft = {
        ["typescript"] = { "biome" },
      },
    },
  },
}
```

Listing 97. Project-specific `.lazy.lua`

You can choose to commit this with your project or `.gitignore` it depending on the standards of the project.

19.12. LazyVim Recipes

LazyVim helpfully collects some of the most commonly requested features as recipes on their home page [https://www.lazyvim.org/configuration/recipes]. Most of these can simply be copied directly into any file in your `lua/plugins` folder. Remember to add `return` in front of them so that whatever table they give you is exported.

Most of the recipes are just providing a set of suggested `opts` to trigger the behaviour in question. These `opts` are always plugin-specific and you'll need to visit the help file or README for the plugin to understand what they are doing.

19.13. Summary

This chapter was all about configuring LazyVim. Much of it may have been review of examples I've used throughout the book when adding plugins to support specific features. However, I wanted to make sure there was coverage in one place for when you want to look things up.

Chapter 20. Where to go Next

We're wrapping up our LazyVim journey together, but if you've made it this far, I'm sure you'll continue to forge new paths with this excellent Neovim distro. This final chapter summarizes some of my favourite places to go for more help, deeper insights, news, and plugins.

20.1. Reread This Book

You've definitely forgotten some interesting tidbits that I've covered in this book. I know this because in rereading it myself during each of several editorial passes I have learned things. One of the reasons I love writing is that it forces me to cover topics in far greater detail than I otherwise would have.

So I recommend rereading this book, or at least skimming through it. Write out a cheat sheet of every keybinding that fits in the "That's really cool but I don't think I'll memorize it" box. I recommend handwriting it; it'll stick in your memory longer and you can keep annotating it on your desk as you learn new things.

If you are reading a print or E-book copy, it will, sadly, become outdated. For the latest version, have a look at my official website at https://lazyvim-ambitious-devs.phillips.codes, and subscribe to my mailing list if you want to hear about updates.

20.2. The Neovim Documentation

Vim was created in 1991 when not everyone had access to the Internet. Back then it was common to ship documentation with software instead of linking to it on the world-wide web (indeed, the web was created the same year). Given that you had the stamina to read this entire book, you might just want to type `:help user-manual<Enter>` and read from beginning to end. Possibly more than once throughout your Vim career. (`:help<Enter>` will give access to even more documentation).

The key to navigating the help files is the keybinding `Control-]`. The documentation is interlinked like a wiki and if you place your cursor over any of the bold text and hit `Control-]`, you'll jump to that section, much like clicking a link.

If you prefer reading documentation in a web browser, the user manual is rendered to html at https://neovim.io/doc/user/usr_toc.html. It's the same content as `:help`, just more clicky.

The manual is extremely comprehensive and covers a lot of commands and keybindings that maybe aren't as relevant as they once were. I believe I've covered everything that is relevant to a modern developer, but I'm sure you'll find additional nuggets of wisdom in there.

20.3. The LazyVim Documentation

LazyVim has its own website at https://www.lazyvim.org. It is a little sparse on hand-holding (which is why this book exists), but it is a super valuable resource once you know how to use it. Most importantly, it comprehensively lists the current configuration for every plugin and extra that ships with the distro. You **will** be visiting these from time to time when you want to tweak the configuration to better match your usage. Every plugin is listed with an `Options` and a `Full Spec` tab. The `Options` represent whatever LazyVim passes as `opts` for that plugin spec, while `Full Spec` includes the entire configuration.

20.4. LazyVim Discussion Groups

The quickest way to get answers to questions (at least, those questions that can't be answered by the copilot-chat plugin we discussed in Chapter 16) is to ask them in the discussion groups on GitHub [http:///github.com/LazyVim/LazyVim/discussions]. I am somewhat active there, but your question will probably be answered by someone far more knowledgeable than me.

If you have found an issue with LazyVim, the first step is to create a minimal repro with as few plugins as possible. The LazyVim issue tracker contains a template for a `repro.lua` that you can use to configure the minimal repro. Add the relevant plugins and run it with `nvim -u repro.lua` to test the issue. Upload this with your issue (or to the Discussion groups) to make it easy for the maintainers to help you.

20.5. Finding Interesting Plugins

For the most part, LazyVim contains the best-in-class version of most plugins you'll need. However, if there is a feature you think the editor should have and it isn't available as an extra, you can almost certainly find it in the Awesome Neovim [https://github.com/rockerBOO/awesome-neovim] repo on Github.

Another terrific resource is the neovimcraft [https://neovimcraft.com] website. Incidentally, the maintainers of neovimcraft are also responsible for pgs.sh [https://pgs.sh], the host for this book's website. I appreciate their product so I wanted to throw in some free advertising for them.

20.6. Dotfiles

Historically, the easiest way to configure Vim has always been to look at someone else's configuration and copy the interesting bits. Nowadays, the easiest way is to use LazyVim, but you may still want to check out the dotfiles of some prominent Neovim and LazyVim users:

Folke Lemaitre [https://github.com/folke/dot]

The creator of LazyVim and a whole host of excellent Neovim plugins (most of which are included with LazyVim).

Evgeni Chasnovski [https://github.com/echasnovski/nvim/]

The creator of the mini.nvim group of plugins and a Neovim core contributor.

Iordanis Petkakis [https://github.com/dpetka2001/dotfiles]

is the person who is most likely to answer your question if you ask it on the LazyVim GitHub Discussion group.

Myself [https://github.com/dusty-phillips/dotfiles/]

I don't really deserve to be mentioned alongside the other names on this list! I push tweaks to my dotfiles regularly and I'm pretty selective about which plugins I use, so if it's in there, it's probably pretty good.

These are also people worth following on various social networks.

20.7. Neovim GUIs

I still maintain that LazyVim is best run in a first-rate terminal, but there are some excellent GUIs out there that you can try if you want.

The primary benefit of Neovim GUIs is that they operate at the pixel level instead of the character level. This means that things like smooth scrolling, cursor motion, or animated window resizes can be more nuanced or performant. Some GUIs also optionally ship with their own bufferline, statusline, or file tree, but they are generally inferior to the ones that ship with LazyVim.

There have been a ton of attempts at Neovim GUIs over the years, but most of them are not or poorly maintained. I've actually tried pretty much every GUI out there, and it is my considered opinion that these are the only ones worth testing, at least at the time of writing:

Vimr [https://github.com/qvacua/vimr]

is MacOS only, but it provides some nice integrations with MacOS if that is what you use.

Neovide [https://neovide.dev]

is the best one if you want to use all your existing Neovim plugins, but want the animations and scrolling to be a little smoother. It's super performant as well.

Goneovim [https://github.com/akiyosi/goneovim]

is a solid contender as well.

FVim [https://github.com/yatli/fvim]

used to be my favourite, but it's dropped off maintenance recently and has some quirks on MacOS that may bump it from this list soon.

If you use any of these, you may well want to look for plugins that make the embedded Neovim integrated terminal experience better.

20.8. Summary

This was a short chapter summarizing various resources you might find valuable as you independently continue your LazyVim and Neovim journey.

I want to thank you for sticking with me to the end. I hope you've enjoyed the book! If you're anything like me, you may be using modal editing concepts for decades to come. Someday Neovim will be "Old Vim", but I'm very confident the principles that guide it will be relevant in future iterations of this editor as well.

Code happily, everyone!

About the Author

Dusty Phillips is a software developer and author. His most popular book, Python Object Oriented Programming is now in its 4th edition, though Dusty is no longer a fan of OOP!

Dusty began coding in QBasic, Visual Basic, and Java as a teenager, but settled on a career in Python long before it became popular. He was the first person to write a book on Python 3 and worked with the team instrumental in bringing Python 3 to Instagram and Facebook. More recently, he enjoys experimenting with esoteric programming languages, many articles on which can be found on his blog.

Dusty has worked for a wide variety of companies, from two person startups to the world's largest social network (and the world's largest government), and companies of all sizes in between.

Dusty lives with his wife and 75 acres of trees in New Brunswick, Canada. The two of them recently co-founded Fablehenge, a platform for novelists to organize and write their stories.